

地缘冲突下国际油价波动预测、宏观传导与中国政策韧性研究

摘 要

国际原油价格同时受到供需基本面、金融条件和地缘政治风险驱动，其冲击会沿进口成本、生产价格和居民消费价格等渠道向国内经济传导。本文围绕油价预测、冲突影响识别、中国宏观传导、政策缓冲和尾部风险压力测试构建四层分析框架。

写作提示：摘要应在正文定稿后重写。依次概括问题一至问题四的模型、关键数值结论与创新点，不得只重复题目；所有数值从冻结结果读取。本提示框和所有示例文字须在提交前删除。

关 键 词： 国际油价 地缘冲突 结构冲击 政策韧性 压力测试

一、 问题重述与研究目标

国际原油价格由供需基本面、美元与库存条件、市场预期和地缘政治风险共同决定。冲突发生后，供给中断预期和航运风险会先进入日度油价与波动率，再经汇率、进口成本和国内价格机制传导至PPI、CPI与工业活动。因此，题目中的“预测”“战争影响”“宏观传导”和“政策效果”不能由同一个统计量替代，必须在统一数据链上分别识别。

题面未给出“美伊战争”的精确边界。本文将其操作性定义为：2026年2月28日开始的中东军事行动及其引致的霍尔木兹海峡运输中断；2026年6月17日通行恢复谅解作为缓和阶段起点。2月28日为非交易日，事件研究映射到首个共同交易日2026年3月2日；3月5日再单列航运受阻阶段。3月23日和4月7日的国内临时调控属于政策事件，不与外部冲突合并编码。

据此，本文研究四个相互衔接的问题：

1. 以Brent现货价为主指标，在扩展窗口中比较未来1、3、6个月预测；以日度累计异常收益、状态匹配安慰剂和条件波动率识别冲突期异常，并用结构VAR分离供给、全球需求和石油特定风险冲击。
2. 以第一问的结构冲击为输入，估计人民币原油成本、PPI、CPI、工业增加值和汇率的响应方向、幅度、时滞及不确定性；季度GDP只作低频验证，不插值为月度数据。
3. 在可比数据边界内考察六个主要进口国的零售燃油传导，以及中国相对对照国的CPI与工业活动响应；再以中国受管制汽油调价指数代理构造“关闭2026年临时调控”的动态反事实。
4. 利用“包括但不限于”的开放性，估计极端油价条件尾部概率，并设计带自动追补与缺口硬约束的状态自适应价格平滑规则，为冲击期调价提供可检验的多目标方案。

本文区分四个时间概念：市场信息截止日为2026年6月30日；不同统计序列的实际观测末期由其发布滞后决定；数据发布或版本日期按来源单独记录；下载核验发生在2026年7月30日至31日。预测与回测只能使用对应原点当时可得的信息，不能把下载日误当作统一观测末期。

二、 问题分析与总体思路

2.1 四问的证据分工

第一问先检验模型能否降低样本外误差，再检验事件期是否偏离战前规律；预测残差和描述性基准差额均不能直接解释为战争净因果贡献。同一次油价上涨还可能来自供

给收缩、全球需求扩张或预防性需求，故进一步采用Kilian结构分解[1]，把具有经济含义的冲击送入第二问。

第二问采用变量特定的局部投影：同比变量直接估计未来水平响应，汇率估计相对冲击前基期的累计对数变化。只有完整月度路径才计算响应曲线面积，结构性稀疏的工业增加值期限不作伪累计。第三问先回答“数据允许我们比较什么”，再评价政策：六国官方零售燃油价用于正式传导排名；中国历史序列是调价指数代理，只作敏感性和政策路径构造。未经审计的年度缓冲代理不进入定量回归。第四问将油价分布、宏观压力与政策规则组织为相互关联但不可相加的三层证据。

总体路线见图1。

2.2 数据结构与口径

月度主样本始于2010年1月。Brent、WTI、广义美元指数和人民币汇率由公开日度序列聚合；美国商业原油库存使用月末前最后一个已发布周值；GPR使用固定版本月度指数[2]。中国PPI、CPI和规模以上工业增加值来自国家统计局，工业增加值1至2月合并发布导致的空值原样保留。人民币原油成本由Brent美元价与人民币兑美元汇率相乘构造。

跨国模块包含中国、德国、法国、意大利、西班牙、日本和韩国。欧洲、日本和韩国使用官方可比的零售汽油价格；中国公开历史调价表被重建为“受管制汽油调价指数代理”，不冒充固定标准品零售价格。CPI统一转为同比，工业活动保留可比同比口径。核心数据见表1。

表1 核心数据、频率和时间口径

数据模块	来源	原始频率	处理和证据用途
Brent、WTI、汇率	EIA/FRED[20, 21]	日度	交易日事件研究；自然月有效观测均值进入月度模型
库存和GPR	EIA；GPR数据库	周/月	月末前最后已发布库存值；固定版本GPR
中国宏观指标	国家统计局[22]	月/季	月度传导；季度GDP只作低频验证
六国宏观指标	OECD及各国官方[23]	月度	CPI和工业活动相对响应
六国燃油价格	欧委会、METI、KO-SIS/KNOC	周/月	官方零售价格月均值，进入正式排名
中国调价序列	公开调价表和官方公告	事件	历史调价指数代理；2026公告差额单独核验
中国价格政策	国家发展改革委[19]	事件	机制应调、实际调价和累计延期缺口

2.3 求解与验证策略

问题一以no-change为可用基线，将ARIMA、SARIMAX、ETS、Theta和等权组合置于相同扩展窗口；事件层采用状态匹配安慰剂，结构层执行平稳性、稳定性、残差和排

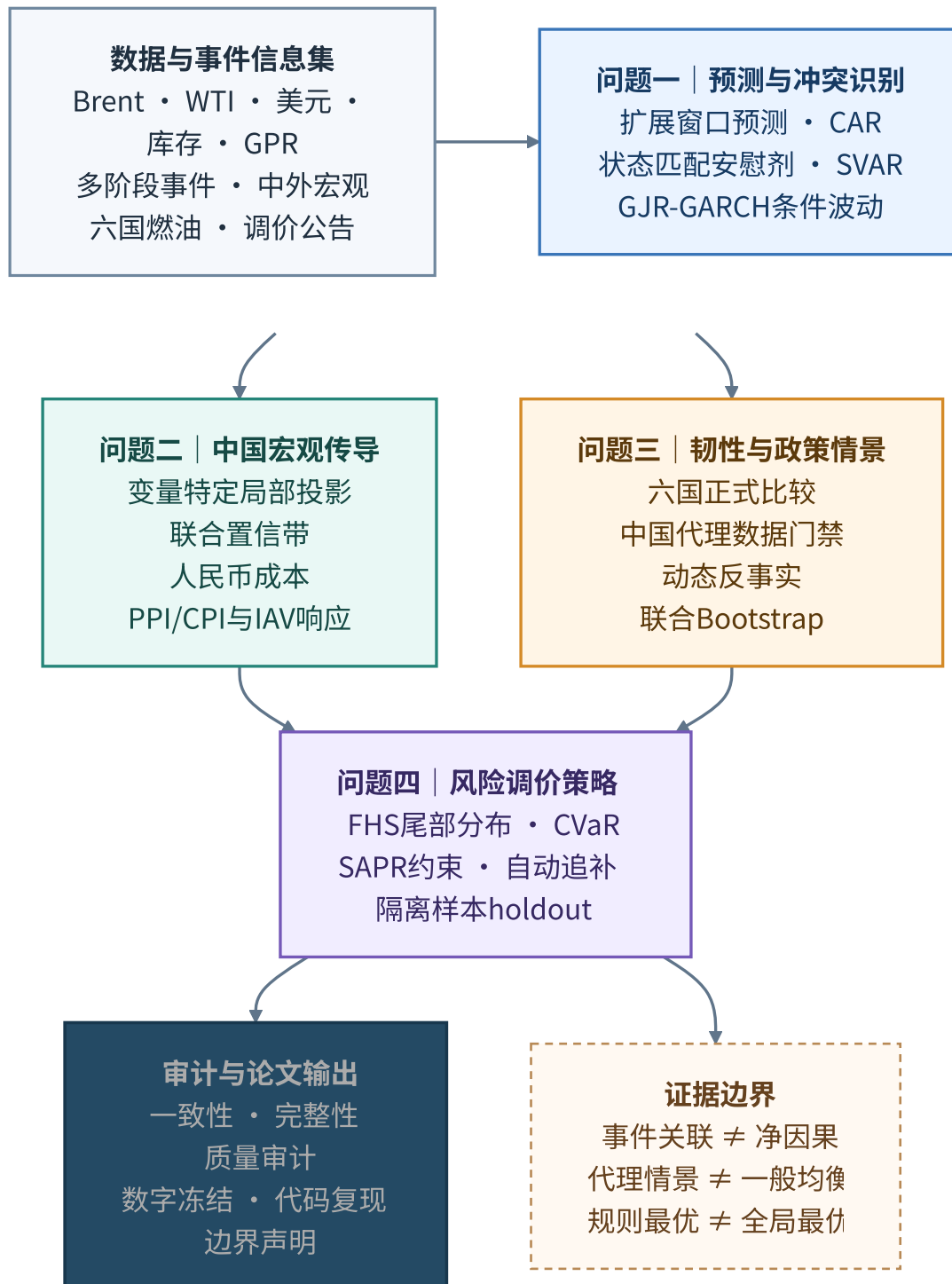


图1 国际油价冲击研究的总体技术路线

序敏感性检查。问题二使用局部投影，以ARDL、冲击正负拆分、替代排序和季度GDP作边界验证。问题三采用分国分布滞后、相对中国的双固定效应面板及动态政策关闭反事实，三条宏观方程用共同时间块联合抽样。问题四以FHS-GJR-GARCH生成尾部分布；政策规则只在开发样本选择阈值和膝点，在2022年以后隔离检验样本用配对块bootstrap比较三类基线。所有结果经风险探针、数值冻结和正文一致性门禁后才允许进入论文。

三、 模型假设

1. **实时信息集假设。**预测原点之后的观测不进入估计、调参或阈值选择；统计序列按实际发布滞后截断。固定下载版本可能被后续修订，但不回填未来信息。
2. **交易日映射假设。**非交易日事件映射到其后首个共同交易日。短窗口仍可能叠加其他新闻，故CAR只解释为事件相关异常；常规正态区间为描述性结果，正式显著性依赖状态匹配安慰剂。
3. **递归结构识别假设。**月内全球供给响应最慢，全球真实活动可当期响应供给，实际油价可当期响应前两者。递归排序为“供给增长—全球真实活动—实际Brent收益”，并用替代排序探针检验结论依赖性。
4. **局部动态假设。**主要宏观响应在12个月内体现；PPI、CPI、IAV和原油成本的被解释量为未来水平，汇率为相对冲击前基期的累计对数变化。移动块bootstrap保留月度序列依赖。
5. **跨国可比边界。**六个对照国的官方零售燃油对数收益可以比较；中国调价指数代理与固定品零售价不可直接等同，因此不进入正式零售价格排名。面板系数只表示对照国相对中国的差异。
6. **局部政策反事实假设。**关闭临时调控时，将官方公告核验的应调与实调差额加入中国调价指数代理路径，其余宏观环境保持不变。反事实用0至6阶动态核及结果变量AR(1)递推，属于条件情景而非一般均衡福利估计。
7. **风险和规则约束假设。**FHS可重采样截止日前标准化残差；SAPR只在三状态、六个月、离散传导率规则族内寻优。平均缺口月负担为延期成本代理，不等于财政支出；最大缺口、期末缺口、恢复期末缺口和单月调价幅度分别作为硬约束。

四、 符号说明

下标 t 表示月份或交易日， h 表示预测或响应期限， i 表示国家。CPI、PPI和IAV分别指居民消费价格同比、工业生产者出厂价格同比和规模以上工业增加值同比。

五、 问题一：油价预测与冲突影响识别

5.1 扩展窗口预测

油价接近随机游走且结构突变频繁，复杂模型未必稳定优于简单基线[3, 4]。本文以2010年1月至2019年12月为初始训练段，自2020年1月起逐月扩展窗口，预测期限为

表2 主要符号及含义

符号	含义	单位或取值
P_t, r_t	Brent价格及对数收益 $r_t = \Delta \ln P_t$	美元/桶；无量纲
$CAR[0, H]$	事件日到第 H 个后续交易日的累计异常收益	对数收益
s_t^S, s_t^D, s_t^R	不利供给、全球需求与石油特定风险结构冲击	标准化冲击
$R_{y,h}$	冲击对结果 y 在期限 h 的响应	百分点或对数百分点
$A_y(H)$	完整月度响应曲线面积 $\sum_{h=0}^H R_{y,h}$	百分点月
$\Pi_{i,H}$	国家 i 的 H 月燃油传导率	无量纲
G_t	机制应调减实际调价形成的未调价缺口	元/吨
Δf_t	无调控路径相对实际路径的燃油收益差	对数百分点
$B_{y,t}$	无调控路径减实际路径的宏观差额	百分点
h_t	GJR–GARCH条件方差	日收益率平方
ρ_N, ρ_S, ρ_E	普通、压力、极端状态的调价传导率	[0, 1]
J_1	平均宏观损失	标准化损失
J_2	宏观损失的95%条件风险价值	标准化损失
J_3	平均每月未结缺口占基准价格比例	无量纲
J_4	实际调价路径波动	无量纲

$h \in \{1, 3, 6\}$ 。no-change基线为

$$\widehat{\ln P}_{t+h|t}^{NC} = \ln P_t. \quad (1)$$

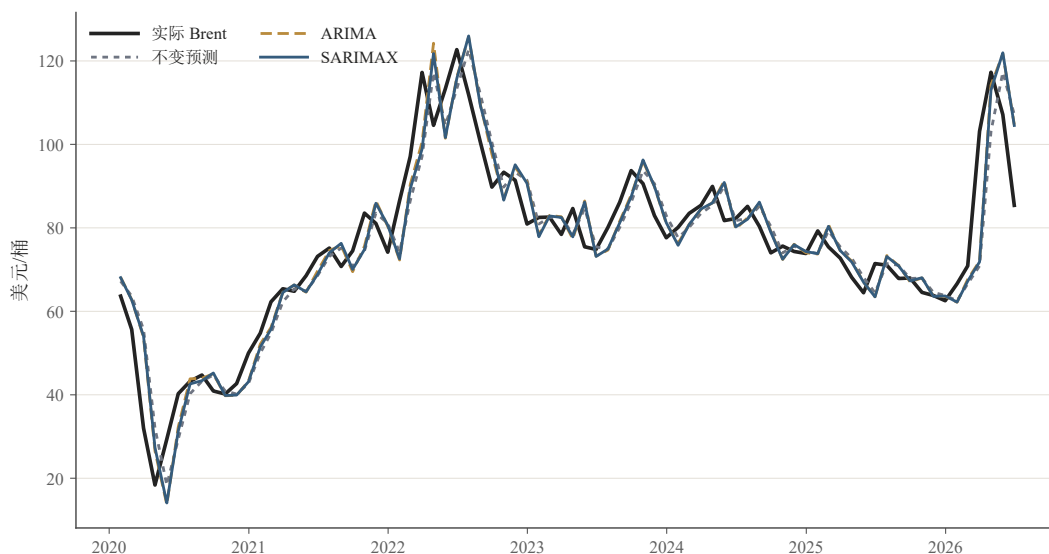
候选模型包括ARIMA、使用滞后库存、美元与GPR的SARIMAX、ETS、Theta及等权组合。对每个模型和期限计算MAE、RMSE、方向准确率及80%、95%区间覆盖率，并用经HLN小样本修正的Diebold–Mariano检验比较配对误差[6, 7]。表3 给出全部模型相对no-change的RMSE；括号内为预注册门禁状态。

表3 扩展窗口回测的相对RMSE

模型	1个月	3个月	6个月
no-change	1.000（通过）	1.000（通过）	1.000（通过）
ARIMA	1.021（条件）	1.119（失败）	1.114（失败）
SARIMAX	1.025（条件）	1.104（失败）	1.101（失败）
ETS	1.159（失败）	1.215（失败）	1.270（失败）
Theta	1.001（条件）	1.003（条件）	1.007（条件）
等权组合	1.029（条件）	1.077（失败）	1.083（失败）

no-change在三个期限均取得最低RMSE，分别为0.1408、0.2660和0.3082，故选为主预测。以2026年2月均价70.887美元/桶为原点，三期点预测均为70.887美元/桶；2026年3月和5月实际均价为103.135和107.140美元/桶，2026年8月尚未实现，只保留为预测。该

结果说明极端事件超出了常规历史信息集，不能据此改用样本外表现更差的复杂模型。



来源：FRED/EIA 处理面板；由 code/problem1/run_q1.py 生成。

图2 Brent一月期扩展窗口预测与实际值

5.2 多阶段事件研究

日历事件E1为2026年2月28日；首个共同交易日为3月2日； $CAR[0, +2]$ 止于3月4日；航运受阻阶段E2从3月5日开始；缓和阶段E3从6月17日开始。用E1前520个共同交易日拟合Brent收益的AR(3)并控制美元收益，定义

$$AR_t = r_t - \hat{r}_t^0, \quad CAR[0, H] = \sum_{j=0}^H AR_{t_0+j}. \quad (2)$$

安慰剂不再随机抽普通周末，而是在排除事件邻域后，按事件前20日波动率、Brent价格水平和当月GPR距离选择最接近的40个历史状态。经验 p 值采用 $(b+1)/(B+1)$ ，避免有限样本下报告零概率。

表4 E1复合事件窗口的Brent累计异常收益

窗口	CAR	常规95%区间	状态匹配经验 p 值	结束日
[0]	0.0797	[0.0415, 0.1178]	0.0244	03-02
[0, +1]	0.1561	[0.1021, 0.2101]	0.0244	03-03
[0, +2]	0.1355	[0.0694, 0.2016]	0.0513	03-04

常规区间仅描述战前模型误差；正式有限样本证据以状态匹配经验 p 值为准。前两窗在5%水平异常，三日窗处于边界。由于军事行动、预期变化和首轮运输扰动密集重

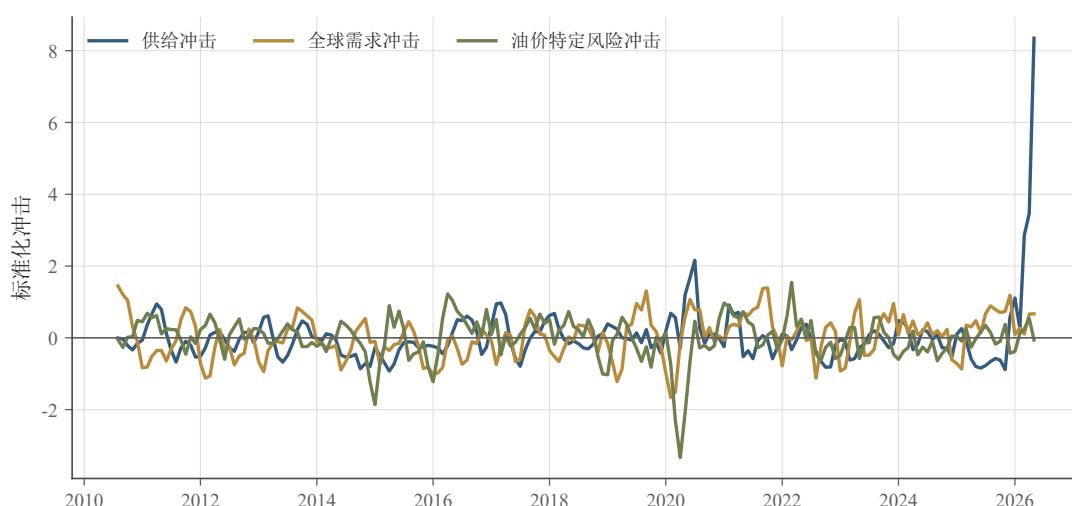
叠，表中CAR是复合事件窗口，不能再分解为单一消息的净贡献。战前AR递归路径与实际价格之差记为 $ARBaselineGap$ ，也只表示偏离历史规律的描述性差额，不称为“无战争价格”。

5.3 结构冲击与波动

令 $z_t = (\Delta q_t, a_t, \Delta p_t^{real})^\top$ ，依次为全球供给增长、全球真实活动和实际Brent收益，估计递归VAR：

$$A(L)z_t = u_t, \quad u_t = B\varepsilon_t. \quad (3)$$

三序列均通过ADF拒绝单位根且KPSS不拒绝平稳：ADF的 p 值分别小于0.001、0.0053和0.001。BIC选择6阶VAR，最小特征根模为1.117，残差白噪声检验 $p = 0.382$ ，得到186个有效冲击月。为便于经济解释，将“供给冲击”统一定向为正值代表不利供给收缩。两个替代递归排序下关键价格冲击相关系数不低于0.999，说明本样本中排序探针不改变主要路径。



来源：EIA International、Dallas Fed IGREA、Brent、美国CPI；作者计算。

图3 递归VAR识别的三类月度结构冲击

日度条件方差采用GJR-GARCH(1,1)[8]。战前条件波动率中位数为1.743%；E1即时窗、E2航运受阻阶段和E3缓和阶段分别为其1.587、2.214和1.566倍。航运受阻阶段压力最高，但该指标描述金融市场波动，不等同于实体损失。

5.4 问题一小结

预测层由no-change诚实胜出；事件层支持前两交易日出现状态条件下罕见的异常上涨；结构层输出三类经济冲击，波动层显示运输受阻阶段风险最高。四者用途分离，避

免把预测误差、CAR或AR基准差额误写成战争净因果溢价。

六、 问题二：油价冲击对中国经济的传导

6.1 变量特定的局部投影

人民币原油成本构造为

$$C_t^{oil} = P_t^{Brent,USD} FX_t^{CNY/USD}, \quad c_t = 100\Delta \ln C_t^{oil}. \quad (4)$$

结构冲击的来源不同，其宏观含义也不同[9, 10, 11]。本文分别输入不利供给、全球需求和石油特定风险冲击；未分解 $OilShock$ 只作约化形式稳健性。局部投影不对整条脉冲路径施加强递推限制[5]，但被解释变量必须与程序定义一致。对PPI、CPI、IAV和人民币原油成本，估计未来水平响应

$$y_{t+h} = \alpha_h + \theta_h s_t + \rho_h y_{t-1} + \eta_h s_{t-1} + \Gamma_h^T Z_{t-1} + u_{t+h}; \quad (5)$$

对汇率则估计相对冲击前基期的累计变化

$$100(\ln FX_{t+h} - \ln FX_{t-1}) = \alpha_h + \theta_h s_t + \rho_h \Delta \ln FX_{t-1} + \eta_h s_{t-1} + \Gamma_h^T Z_{t-1} + u_{t+h}. \quad (6)$$

Z_{t-1} 包含美元、GPR、月份虚拟变量和疫情阶段。PPI、CPI、成本和汇率估计 $h = 0, \dots, 12$ ；IAV因1至2月结构性缺失，只估计 $h \in \{0, 3, 6, 12\}$ 。使用同源移动块bootstrap和最大 t 统计量构造95%联合置信带。对完整月度路径定义响应曲线面积

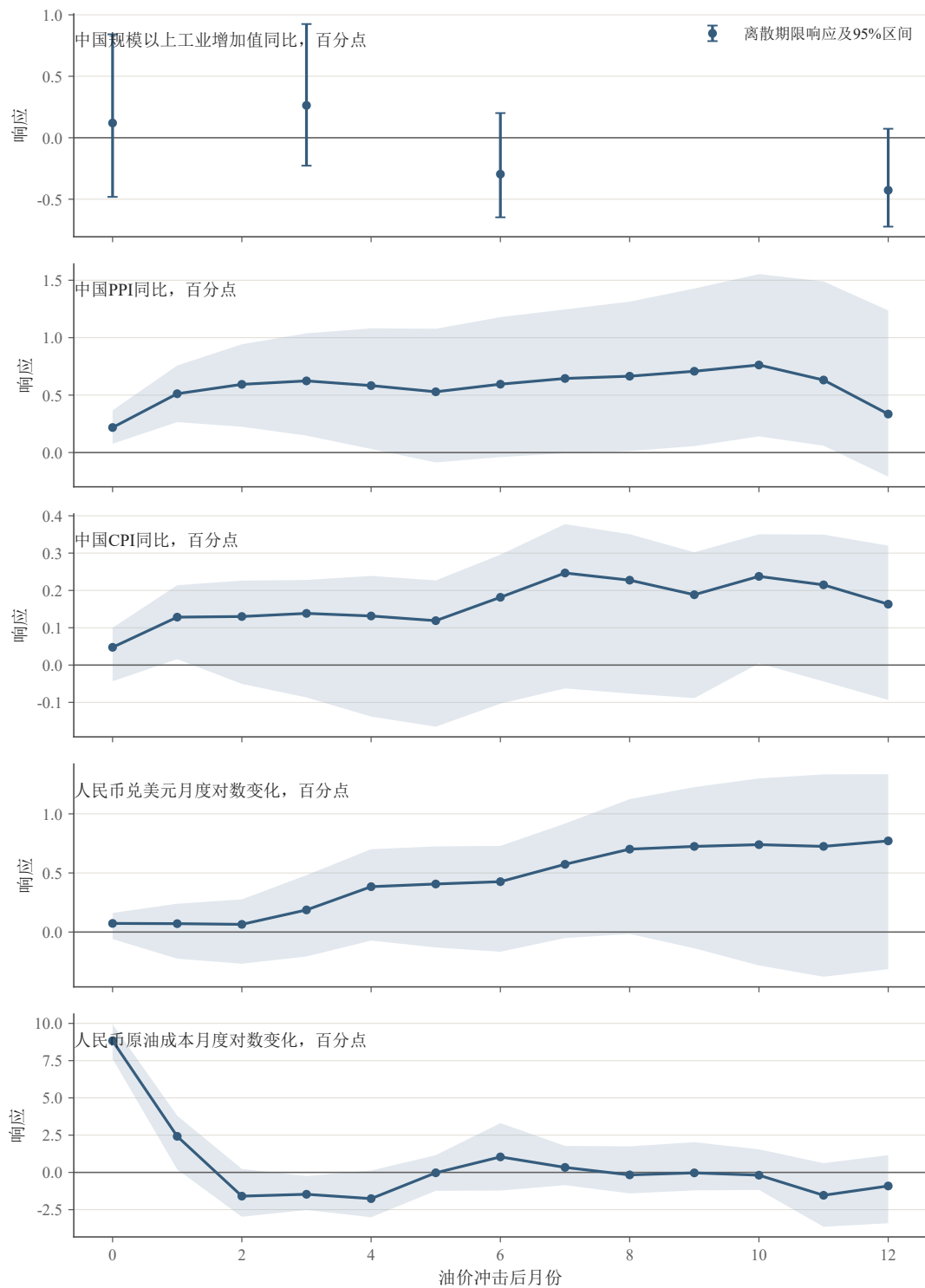
$$A_y(H) = \sum_{h=0}^H \hat{\theta}_{y,h}, \quad (7)$$

单位为“百分点月”，只描述离散响应曲线的面积，不称为累计经济损失。IAV的四个稀疏期限不求和。

6.2 油价特定风险冲击结果

成本入口当月响应最清晰；PPI和CPI点估计滞后上行，IAV在12个月出现负向点估计。但除成本入口外，极值的联合区间均跨零，故不能将点估计写成稳健总体增长损失。季度GDP在加入所需滞后后没有足够有效观测，低频验证状态为“结论不充分”，也不再报告旧版小样本回归值。

不同结构冲击的响应方向并不相同：全球需求扩张可同时推高油价和产出，不利供给或石油特定风险更符合成本推动机制。这解释了为何“油价上涨”不能直接等同于



来源：问题一结构冲击接口、国家统计局 IAV/PPI、OECD CPI、FRED 汇率/美元/GPR；由 code/problem2/run_q2.py 生成。

图4 油价特定风险冲击下中国主要变量的局部投影响应

表5 油价特定风险冲击的传导指标

结果变量	极值	月份	极值联合95%区间	A(0, 6)	A(0, 12)
人民币原油成本	8.832	0	[6.888, 10.776]	7.440	4.939
PPI同比	0.762	10	[-0.159, 1.684]	3.655	7.401
CPI同比	0.247	7	[-0.096, 0.589]	0.877	2.155
IAV同比	-0.427	12	[-0.966, 0.111]	-	-
人民币汇率	0.771	12	[-0.488, 2.030]	1.612	5.846

“经济受损”。ARDL、疫情剔除、冲击正负拆分和替代排序只作为稳健性探针；它们未把工业活动和GDP证据提升为联合显著。

6.3 问题二小结

模型支持“进口成本快速响应、PPI和CPI滞后上行、工业活动可能承压”的传导顺序，但总体增长损失仍为结论不充分。后续政策评价必须同时报告物价、工业活动、响应期限和区间，不能只比较单一价格传导系数。

七、 问题三：中国政策缓冲与跨国比较

7.1 先锁定可比边界

跨国燃油传导受税制、汇率和定价机制共同影响[12, 13]。对每个国家估计

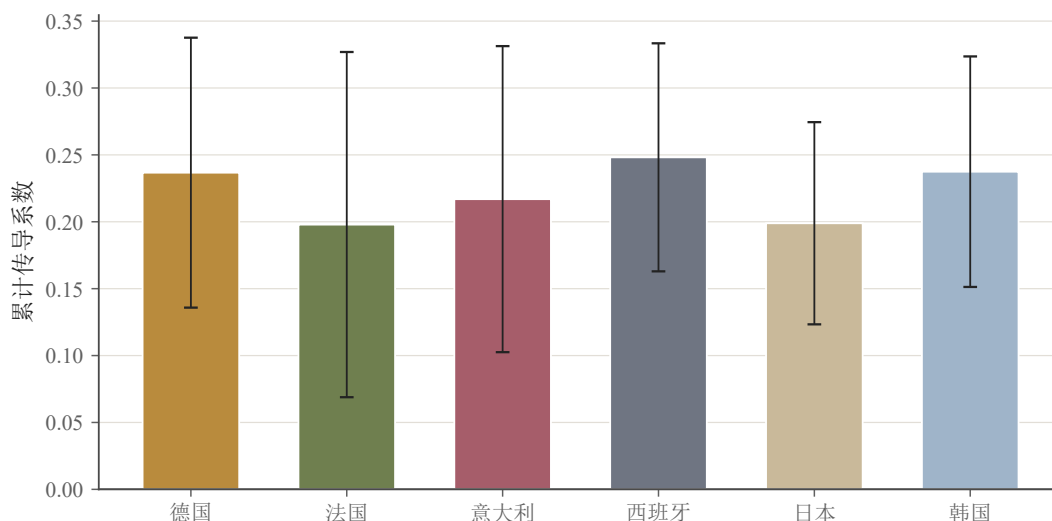
$$\Delta \ln F_{i,t} = a_i + \varphi_i \Delta \ln F_{i,t-1} + \sum_{j=0}^6 \beta_{i,j} \Delta \ln P_{i,t-j}^{Brent,local} + u_{i,t}, \tag{8}$$

并以 $\Pi_{i,H} = \sum_{j=0}^H \beta_{i,j}$ 表示H月传导率。德国、法国、意大利、西班牙、日本和韩国有可审计的固定品类零售燃油价格，进入正式排名。中国历史公开调价表只能重建出受管制汽油调价指数代理，不是固定标准品零售价格，因此从正式排名剔除，只保留为敏感性和政策路径构造。

表6 六个月燃油传导率及数据角色

国家	中国	德国	法国	意大利	西班牙	日本	韩国
传导率	0.373	0.237	0.198	0.217	0.248	0.199	0.237
数据角色	代理敏感性	正式	正式	正式	正式	正式	正式

六个正式对照国中位数为0.227。中国代理值0.373较高，但二者并非同类价格层，故不能据此判断中国更优或更差。这个“不可比”结论本身是重要的数据审计结果。



来源：欧盟周度油价公报、日本METI、韩国KOSIS/KNOC、北京市发改委与FRED；由 code/problem3/run_q3.py 生成。

图5 六国正式零售价格与中国调价指数代理的六个月传导率

7.2 相对宏观响应与暂缓的机制回归

对CPI和工业活动估计含国家固定效应和完整年月固定效应的面板局部投影：

$$y_{i,t+h} - y_{i,t-1} = \alpha_{i,h} + \lambda_{t,h} + \delta_{i,h}(s_t I_i) + \Gamma_h^\top Z_{i,t-1} + u_{i,t+h}. \quad (9)$$

中国是参照组， $\delta_{i,h}$ 只表示对照国相对中国的响应差，不能把中国绝对响应人为设为零。对照国减中国的CPI相对响应点估计中位数为0.245个百分点，工业活动为2.433个百分点；但当前程序没有形成联合国别–时间bootstrap区间，因此两项均判为“结论不充分”。价格监管、进口依赖、石油强度和进口来源集中度年度表均为未经审计代理，四类连续缓冲交互回归全部执行失败关闭，不提供系数、显著性或政策排序。三维核心韧性判据见表7。

表7 中国相对韧性的核心判据

维度	当前证据	判断
燃油价格	中国仅有调价指数代理，不能与六国固定品类零售价正式排名	条件代理
消费价格	对照国减中国点估计中位数0.245，缺联合差异区间	结论不充分
工业活动	对照国减中国点估计中位数2.433，缺联合差异区间	结论不充分
总体	任一核心维度均不足以形成稳健“更好”判断	结论不充分

7.3 关闭2026年临时调控的动态反事实

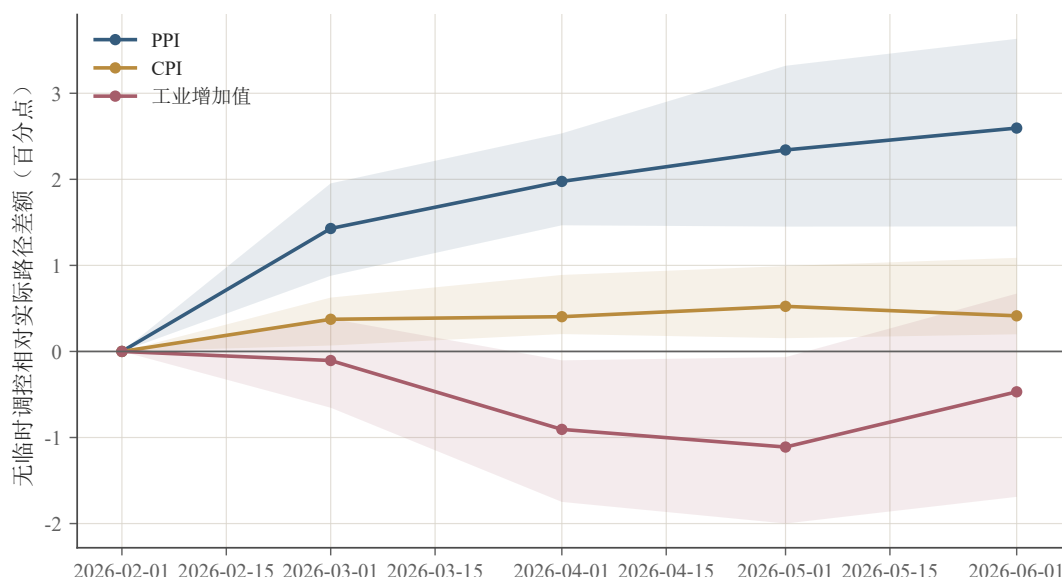
依据《石油价格管理办法》[19]和2026公告核验，3月汽油机制应调2205元/吨、实调1160元/吨，新增缺口1045元/吨；4月机制应调800元/吨、实调420元/吨，新增380元/吨，累计缺口1425元/吨。令 P_t^{no} 和 P_t^{act} 分别为无临时调控与实际代理价格路径，输入模型的不是静态价格水平差，而是逐月收益差

$$\Delta f_t = 100 \left[\ln \frac{P_t^{no}}{P_{t-1}^{no}} - \ln \frac{P_t^{act}}{P_{t-1}^{act}} \right]. \quad (10)$$

对 $y \in \{PPI, CPI, IAV\}$ ，用0至6阶动态核和结果变量AR(1)递推：

$$B_{y,t} = \sum_{\ell=0}^6 \beta_{y,\ell} \Delta f_{t-\ell} + \phi_y B_{y,t-1}. \quad (11)$$

三条方程使用同一组长度为6的循环时间块联合抽样，共接受2000组参数抽样，从而保留PPI、CPI和IAV估计误差的同期相关。



来源：results/q3_policy_macro_counterfactual.csv。

图6 关闭临时调控后的动态宏观情景差额

结果说明，在维持动态核与局部环境不变的条件下，临时调控可能降低了2026年6月PPI和CPI压力；IAV方向符合缓冲解释但区间跨零。该结论依赖调价指数代理，不能外推为全面福利改善，也不改变跨国总体韧性“结论不充分”的判断。

7.4 问题三小结

正式零售价格、相对宏观响应和政策反事实属于三类不同证据。当前数据不能证明中国在三维韧性上全面优于对照国；较强的结论只限于2026年条件情景中PPI与CPI的局

表8 2026年6月无临时调控减实际路径的宏观差额

变量	情景差额/百分点	95%区间	情景证据
PPI	2.596	[1.453, 3.633]	区间不跨零
CPI	0.414	[0.199, 1.088]	区间不跨零
IAV	-0.469	[-1.690, 0.672]	结论不充分

部缓冲，同时必须显式记录1425元/吨延期缺口及其后续消化责任。

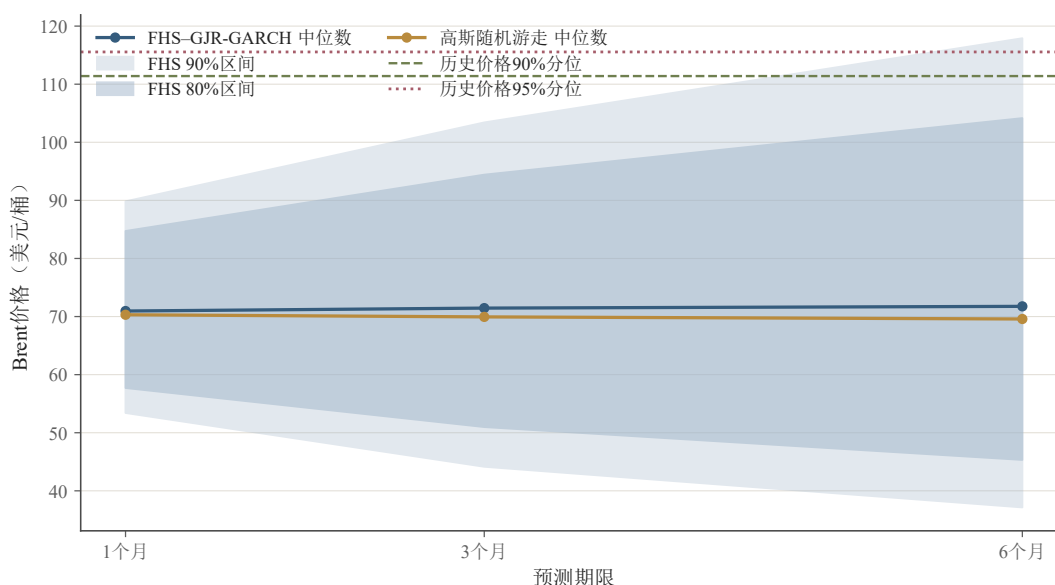
八、 问题四：极端油价风险与自适应调价规则

8.1 FHS-GJR-GARCH条件尾部分布

在2026年6月30日信息集下，先估计GJR-GARCH(1,1)，再从截止日前标准化残差经验分布有放回抽样，递归生成[16]

$$P_{t+k}^{(b)} = P_{t+k-1}^{(b)} \exp\left(\mu + \sqrt{h_{t+k}^{(b)}} z_{t+k}^{(b)}\right). \quad (12)$$

每个期限模拟20000条路径。历史90%和95%价格分位111.397和115.550美元/桶只是压力阈值。



数据：EIA/FRED Brent；计算：FHS-GJR-GARCH 与高斯随机游走，信息截止 2026-06-30。

图7 固定信息集下的Brent条件尾部分布

回测从2022年起逐月滚动，共47个可用原点，而非只取少量季度末。FHS与高斯随机游走在1、3、6月的平均分位损失分别为1.565对1.629、2.367对2.562、3.519对3.689。配

表9 Brent条件风险预测

期限/月	中位数	90%区间	期末超过历史95%阈值	路径内曾越过
1	70.95	[53.41, 89.82]	0.27%	0.37%
3	71.46	[44.15, 103.41]	2.28%	3.76%
6	71.76	[37.15, 117.90]	5.56%	9.65%

对长度6块bootstrap 显示，FHS在3个月的80%和90%区间分数、6个月的80%区间分数显著较低；其余配对指标区间跨零。因此FHS对中期区间预测有局部优势，但没有在全部期限和全部评分上支配基线。

8.2 状态自适应价格平滑规则

设机制应调额为 Δ_t^{rule} ，按开发样本的75%和95%分位将其划为普通、压力、极端三态，传导率满足 $\rho_N \geq \rho_S \geq \rho_E$ 。实际调价与缺口递推为

$$a_t = \rho_{st} \Delta_t^{rule} + c_t, \quad G_t = G_{t-1} + \Delta_t^{rule} - a_t, \quad (13)$$

其中 c_t 为自动追补项：当缺口触及基准价格20%时优先压回安全区，并受单月调价25%硬约束。四目标定义为

$$\begin{aligned} J_1 &= \mathbb{E}(L), \\ J_2 &= CVaR_{0.95}(L), \\ J_3 &= \frac{1}{T} \sum_{t=1}^T \frac{|G_t|}{P_0}, \\ J_4 &= \text{sd}\left(\frac{a_t}{P_0}\right), \end{aligned} \quad (14)$$

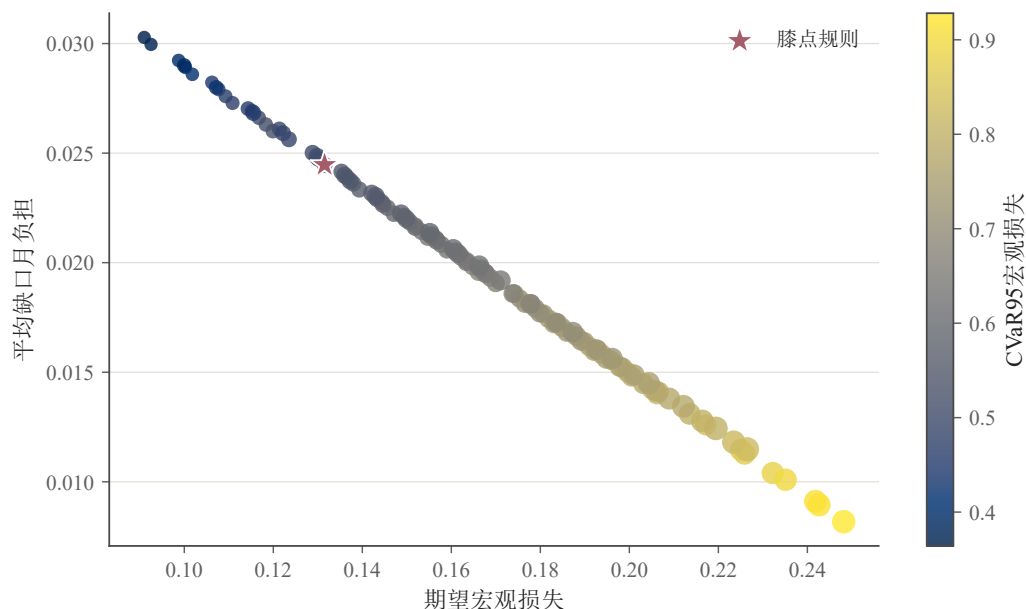
分别衡量平均宏观损失、尾部宏观损失、平均缺口月负担和调价波动，其中CVaR采用Rockafellar–Uryasev定义[17]。 J_3 不是期末缺口，也不是财政支出。候选规则还必须同时满足

$$\max_t |G_t|/P_0 \leq 0.35, \quad |G_T|/P_0 \leq 0.20, \quad |G_{T+6}^{rec}|/P_0 \leq 0.05, \quad \max_t |a_t|/P_0 \leq 0.25. \quad (15)$$

阈值和规则只用2013年3月至2021年12月开发样本选择；2022年1月至2026年6月完全隔离。1771个候选中1750个满足硬约束；按1%容差构造 ε -Pareto集，得到119个规则，再选距归一化理想点最近的膝点。最优规则为

$$(\rho_N, \rho_S, \rho_E) = (0.30, 0.10, 0.10), \quad (16)$$

开发样本阈值为435.97和702.49元/吨，基准价格为9504元/吨。



来源：results/q4_sapr_policy_grid.csv；作者计算。

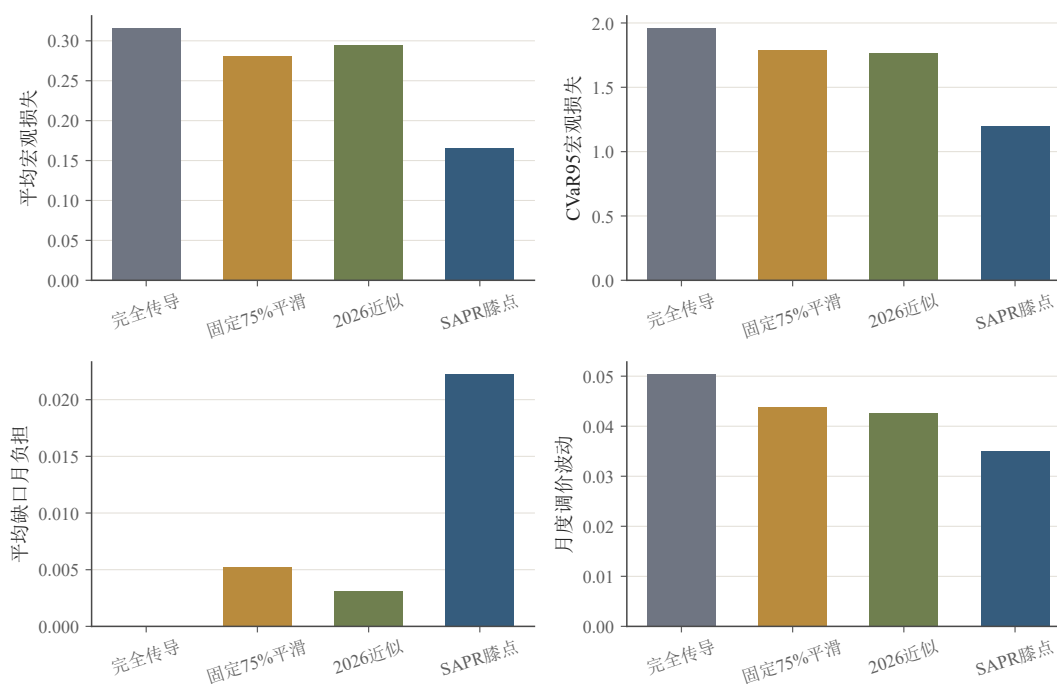
图8 可行规则的四目标映射与 ϵ -Pareto膝点

8.3 隔离检验与政策权衡

表10 2022年至2026年隔离检验样本的原始目标值

策略	J_1	J_2	J_3	J_4
完全传导	0.315	1.957	0.000	0.050
固定75%平滑	0.281	1.792	0.005	0.044
2026临时调控近似	0.294	1.768	0.003	0.043
SAPR膝点	0.165	1.198	0.022	0.035

与完全传导相比，膝点规则的 J_1 、 J_2 和 J_4 配对差分别为 -0.163 、 -0.671 和 -0.0148 ，95%区间均小于零； J_3 差为 $+0.0222$ ，区间 $[0.0109, 0.0386]$ ，明确揭示平滑的延期负担。检验期最大缺口被自动限制在20%，再给6个月恢复后降至2.35%，满足5%硬约束。结果支持的是注册规则族内的短期风险-延期负担权衡，不是全体政策的全局最优或完整福利改进。



来源：results/q4_sapr_strategy_comparison.csv；作者计算。

图9 隔离检验样本中四类策略的原始目标比较

8.4 问题四小结

六个月路径越过历史95%高位阈值的条件概率为9.65%。SAPR用显式缺口硬约束和自动追补把短期物价与工业压力的降低同未来调价责任绑定；其优势体现在 J_1 、 J_2 和 J_4 ，代价体现在 J_3 。这比单纯“压低当期价格”更完整，也保留了退出机制。

九、 稳健性与敏感性分析

9.1 预测、识别与传导探针

问题一将六类预测器置于78个相同目标月比较；no-change在三个期限均取得最低RMSE，Theta虽接近，DM-HLN检验未显示稳定优势。事件CAR改用40个前期波动率、价格水平和GPR状态最接近的安慰剂，最小可实现经验 p 值为 $1/41 = 0.0244$ ，避免普通周末样本造成虚假精度。结构VAR的6、12、18、24阶候选均检查稳定性；6阶模型白噪声检验通过，替代排序下关键价格冲击相关系数不低于0.999。

问题二使用联合最大 t 置信带、逐点区间、FDR校正、ARDL、疫情剔除和冲击正负拆分。多数PPI、CPI和IAV联合区间跨零，GDP有效观测不足，因此稳健性分析的作用是限制结论强度，而不是事后挑选显著规格。问题三四个年度缓冲变量因未经审计统一判为“主分析失败”，不以替代规格恢复定量结论。

9.2 尾部风险回测

FHS与高斯随机游走使用相同的47个月度原点和1、3、6个月期限。除分位损失和覆盖率外，补充80%、90%区间分数与近似CRPS[18]。长度6的配对块bootstrap显示：FHS仅在3个月两档区间分数和6个月80% 区间分数上显著更优，其余差异为结论不充分。由此将“FHS全面优于基线”列为禁止表述。

9.3 SAPR规则敏感性

表11 SAPR膝点对阈值、权重和块长度的敏感性

变体	(ρ_N, ρ_S, ρ_E)	Pareto数	相对默认变化	检验期非支配
默认75/95阈值	(0.30, 0.10, 0.10)	119	否	是
70/90阈值	(0.30, 0.15, 0.10)	82	是	是
80/97.5阈值	(0.30, 0.10, 0.10)	209	否	是
CPI权重增加20%	(0.30, 0.10, 0.10)	121	否	是
IAV权重增加20%	(0.30, 0.10, 0.10)	116	否	是
bootstrap块3/6/12	(0.30, 0.10, 0.10)	119	否	是

除较低状态阈值外，膝点规则对权重、阈值上界和块长度保持不变；70/90阈值把压力期传导率提高到0.15，说明普通与压力状态的划分是最敏感设计项。所有变体在检验期点估计上保持非支配，但正文的“支持”判断还依赖配对目标差和硬约束，而不只依赖非支配概率。

十、 模型评价与推广

10.1 主要优点

1. 证据层分离。预测误差、事件异常、结构冲击、宏观响应和政策情景分别建模，避免把同一个差额反复解释为因果贡献。
2. 实时和样本外约束。预测扩展窗口、47个风险回测原点以及SAPR开发–检验拆分均禁止未来信息；所有复杂方法都有可运行的简单基线。
3. 单位和响应定义可审计。程序为每个Q2结果声明被解释量；完整路径只报告响应曲线面积，IAV稀疏期限不伪造累计量。
4. 数据不可比时主动降级。中国燃油调价指数代理不进入正式零售排名；未经审计缓冲代理触发失败关闭，而不是生成看似完整的回归表。

5. 政策优化显式计入退出责任。平均缺口月负担与最大、期末、恢复期末缺口分开；自动追补保证平滑规则不会无限累积延期调整。

10.2 局限性

1. 单次地缘冲突与同期消息密集重叠，状态匹配CAR仍不是严格随机实验；递归VAR也依赖排序和历史代理变量。
2. 中国固定标准品零售价、战略储备释放、财政补贴、炼油企业损益和进口来源年度数据不完整，限制了跨国韧性与机制解释。
3. Q2月度样本有限，宏观联合区间较宽；季度GDP在滞后要求下无法形成有效回归，不能声称稳健总体增长损失。
4. FHS只能重采样历史残差，不能生成历史从未出现的制度断裂；GJR持久性较高，远期尾部对模型设定敏感。
5. SAPR只在注册的三状态、六个月和离散传导率族内最优， J_3 既不是财政成本，也不是完整福利。

10.3 推广方向

该框架可用于天然气、粮食、关键矿产和国际运价：先比较样本外预测，再识别事件异常与结构来源，继而估计国内传导，最后在明确硬约束下优化缓冲规则。若能获得连续的官方零售价格、储备操作、行业投入产出和财政成本数据，可把局部政策情景扩展为部门异质性或一般均衡分析；新增模块仍需通过基线、风险探针和数值冻结门禁。

十一、 结论与政策启示

11.1 主要结论

第一，扩展窗口回测中no-change在1、3、6个月均取得最低RMSE。E1复合事件的 $CAR[0]$ 、 $CAR[0, +1]$ 和 $CAR[0, +2]$ 分别为0.0797、0.1561和0.1355；状态匹配经验 p 值分别为0.0244、0.0244和0.0513。航运受阻阶段条件波动为战前中位数的2.214倍。这些结果支持事件附近出现异常，但不等同于战争净因果溢价。

第二，石油特定风险冲击使人民币原油成本当月上升8.832个对数百分点；PPI和CPI极值点估计为0.762和0.247个百分点，IAV谷值为-0.427个百分点。除成本入口外，极值联合区间均跨零；GDP有效观测不足，故总体增长损失证据为结论不充分。

第三，中国历史燃油序列是调价指数代理，不能与六国固定品类零售价正式排名；CPI与工业活动的相对响应缺联合差异区间，三维跨国韧性总体判断为结论不充分。条

件动态反事实显示，关闭2026年临时调控后，6月PPI和CPI可能分别高2.596和0.414个百分点；IAV差额为-0.469个百分点但区间跨零。该缓冲同时留下1425元/吨延期缺口。

第四，在2026年6月30日信息集下，六个月Brent中位数为71.76美元/桶，期末超过历史95%阈值的条件概率为5.56%，路径内曾越过的概率为9.65%。SAPR在1750个可行规则中选出 (0.30, 0.10, 0.10)；检验期相对完全传导显著降低平均宏观损失、尾部宏观损失和调价波动，但平均缺口月负担增加0.0222。最大缺口被限制在20%，6个月恢复后为2.35%。

11.2 政策启示

1. **临时平滑必须状态依赖且透明。**以机制应调额和条件尾部风险作为触发信号，公开状态阈值、传导率、累计缺口和退出条件，避免临时措施固化为长期冻结。
2. **把延期负担列入同一政策仪表盘。**当期CPI下降并不代表成本消失，应同步监测未调价缺口、炼油企业现金流、库存和财政安排，并在国际油价回落期自动追补。
3. **实施价格-物价-工业活动联合预警。**人民币原油成本是清晰先行入口，PPI、CPI和IAV反映不同滞后层；政策报告应同时给出点估计、联合区间和冲击来源。
4. **优先补齐可审计公开数据。**连续固定品类成品油价格、储备操作、进口来源和政策成本数据，是把当前条件情景提升为跨国因果与福利评价的关键。

综上，稳健策略不是永久隔离国际价格信号，而是在冲击期用可审计、可退出的状态规则降低短期尾部传导，并把被推迟的调整责任显式纳入决策。

AI工具使用说明

本文在公开数据检索提示、程序调试、结果一致性检查和文字修改环节使用了OpenAI Codex [24]。研究问题定义、模型取舍、数据口径、结果核验和最终表述均由参赛队独立判断并承担责任；详细交互用途、采纳与人工修改情况见支撑材料《AI工具使用详情》。

参考文献

- [1] Kilian L. Not All Oil Price Shocks Are Alike: Disentangling Demand and Supply Shocks in the Crude Oil Market[J]. American Economic Review, 2009, 99(3): 1053–1069. DOI: 10.1257/aer.99.3.1053.

- [2] Caldara D, Iacoviello M. Measuring Geopolitical Risk[J]. *American Economic Review*, 2022, 112(4): 1194–1225. DOI: 10.1257/aer.20191823.
- [3] Baumeister C, Kilian L. Real-Time Forecasts of the Real Price of Oil[J]. *Journal of Business & Economic Statistics*, 2012, 30(2): 326–336. DOI: 10.1080/07350015.2011.648859.
- [4] Baumeister C, Kilian L. Forecasting the Real Price of Oil in a Changing World: A Forecast Combination Approach[J]. *Journal of Business & Economic Statistics*, 2015, 33(3): 338–351. DOI: 10.1080/07350015.2014.949342.
- [5] Jordà Ò. Estimation and Inference of Impulse Responses by Local Projections[J]. *American Economic Review*, 2005, 95(1): 161–182. DOI: 10.1257/0002828053828518.
- [6] Diebold F X, Mariano R S. Comparing Predictive Accuracy[J]. *Journal of Business & Economic Statistics*, 1995, 13(3): 253–263. DOI: 10.1080/07350015.1995.10524599.
- [7] Harvey D, Leybourne S, Newbold P. Testing the Equality of Prediction Mean Squared Errors[J]. *International Journal of Forecasting*, 1997, 13(2): 281–291. DOI: 10.1016/S0169-2070(96)00719-4.
- [8] Glosten L R, Jagannathan R, Runkle D E. On the Relation between the Expected Value and the Volatility of the Nominal Excess Return on Stocks[J]. *Journal of Finance*, 1993, 48(5): 1779–1801. DOI: 10.1111/j.1540-6261.1993.tb05128.x.
- [9] Tang W, Wu L, Zhang Z X. Oil Price Shocks and Their Short- and Long-Term Effects on the Chinese Economy[J]. *Energy Economics*, 2010, 32(S1): S3–S14. DOI: 10.1016/j.eneco.2010.01.002.
- [10] Cross J, Nguyen B H. The Relationship between Global Oil Price Shocks and China's Output: A Time-Varying Analysis[J]. *Energy Economics*, 2017, 62: 79–91. DOI: 10.1016/j.eneco.2016.12.014.
- [11] Chen J, Zhu X, Li H. The Pass-Through Effects of Oil Price Shocks on China's Inflation: A Time-Varying Analysis[J]. *Energy Economics*, 2020, 86: 104695. DOI: 10.1016/j.eneco.2020.104695.
- [12] Kpodar K, Abdallah C. Dynamic Fuel Price Pass-Through: Evidence from a New Global Retail Fuel Price Database[J]. *Energy Economics*, 2017, 66: 303–312. DOI: 10.1016/j.eneco.2017.06.017.

- [13] Kpodar K, Imam P A. To Pass (or Not to Pass) Through International Fuel Price Changes to Domestic Fuel Prices in Developing Countries: What Are the Drivers?[J]. *Energy Policy*, 2021, 149: 111999. DOI: 10.1016/j.enpol.2020.111999.
- [14] Coady D, Arze del Granado J, Eyraud L, et al. Automatic Fuel Pricing Mechanisms with Price Smoothing: Design, Implementation, and Fiscal Implications[R]. *IMF Technical Notes and Manuals*, 2013. DOI: 10.5089/9781475566949.005.
- [15] Coady D, Parry I W H, Shang B. Energy Price Reform: Lessons for Policymakers[J]. *Review of Environmental Economics and Policy*, 2018, 12(2): 197–219. DOI: 10.1093/reep/rey004.
- [16] Barone-Adesi G, Giannopoulos K, Vosper L. Backtesting Derivative Portfolios with Filtered Historical Simulation[J]. *European Financial Management*, 2002, 8(1): 31–58. DOI: 10.1111/1468-036X.00175.
- [17] Rockafellar R T, Uryasev S. Optimization of Conditional Value-at-Risk[J]. *Journal of Risk*, 2000, 2(3): 21–41. DOI: 10.21314/JOR.2000.038.
- [18] Gneiting T, Raftery A E. Strictly Proper Scoring Rules, Prediction, and Estimation[J]. *Journal of the American Statistical Association*, 2007, 102(477): 359–378. DOI: 10.1198/016214506000001437.
- [19] 国家发展和改革委员会. 石油价格管理办法[EB/OL]. 2016. <https://www.ndrc.gov.cn/xxgk/zcfb/tz/201601/W020190905506573420251.pdf>.
- [20] Federal Reserve Bank of St. Louis. FRED: Brent Crude Oil Price and Exchange Rate Series[DB/OL]. <https://fred.stlouisfed.org/>. 访问日期: 2026-07-30.
- [21] U.S. Energy Information Administration. Petroleum and Other Liquids Data, Weekly U.S. Commercial Crude Oil Stocks[DB/OL]. <https://www.eia.gov/petroleum/data.php>. 访问日期: 2026-07-30.
- [22] 国家统计局. 国家数据: 月度与季度宏观指标[DB/OL]. <https://data.stats.gov.cn/>. 访问日期: 2026-07-31.
- [23] OECD. Data Explorer: Consumer Prices and Industrial Production[DB/OL]. <https://data-explorer.oecd.org/>. 访问日期: 2026-07-30.
- [24] OpenAI Codex, GPT-5 (Codex), OpenAI, 使用日期: 2026-07-29至2026-08-01.

附录 A 主张、证据与边界

表12 核心主张的证据链和禁止外推

主张	直接证据	边界
冲突初期存在异常上涨	状态匹配CAR: 前两窗经验 $p = 0.0244$	复合事件关联，不是战争净因果溢价
成本入口响应清晰	原油成本当月8.832, 联合区间不跨零	不等于PPI、CPI或产出必然显著变化
宏观传导存在但增长损失不稳健	PPI、CPI、IAV点估计符合机制，联合区间多跨零	禁止表述为稳健总体GDP损失
2026政策可能缓冲物价	动态代理反事实PPI、CPI区间不跨零	依赖调价指数代理，不是一般均衡福利
中国跨国韧性尚不能定论	零售价不可比，宏观相对响应缺联合区间	禁止声称全面优于六国
SAPR改善短期风险指标	检验期 J_1 、 J_2 、 J_4 配对差显著为负	J_3 显著增加；仅在注册规则族内成立

附录 B 复现流程和支撑材料文件列表

建议按“数据下载或人工导入—数据构建—P0校验—Q1—面板重建—Q2—Q3—Q4风险层—Q4 SAPR—论文交接—数值冻结—冻结验证”顺序执行。随机种子为20260730；市场信息截止日为2026-06-30，各序列实际末期以数据字典为准。

表13 支撑材料文件列表

文件或目录	内容和用途
data/SOURCE_REGISTRY.csv	来源网址、下载日期、访问条件、许可和回退口径
data/DATA_DICTIONARY.csv	变量定义、频率、单位、实际末期、发布滞后和主分析状态
data/event_timeline.csv	冲突、航运、国内调控和缓和事件的日期与来源
code/data_download/	免费公开数据下载程序
code/data_processing/	人工数据导入、面板构建和P0数据校验
code/problem1/	预测、CAR、状态匹配安慰剂、结构冲击和条件波动
code/problem2/	变量特定局部投影、ARDL和稳健性探针
code/problem3/	跨国比较、代理数据门禁和动态政策反事实
code/problem4/	尾部风险回测和SAPR-CVaR优化
results/frozen_numbers.json	论文冻结数值、源文件哈希和定稿门禁
results/risk_probe_summary.json	基线、风险探针、证据等级和禁止声明
results/reproducibility_manifest.json	输入、代码、输出哈希及运行环境
reports/paper_numbers.csv	正文关键数字、单位、舍入规则和表述边界
support_materials目录下《AI工具使用详情.pdf》	AI工具、用途、关键交互和人工修改记录

附录 C 全部完整源程序代码

下列代码与支撑材料中的源文件一致。编译时直接读取仓库文件，避免论文代码副本与实际运行版本分叉。

3.1 数据下载与构建

```

1 """Download public P0 source snapshots with reproducibility metadata."""
2
3 from __future__ import annotations
4
5 import argparse
6 import csv
7 import hashlib
8 import json
9 import os
10 import shutil
11 import subprocess
12 import sys
13 from dataclasses import dataclass
14 from datetime import datetime
15 from pathlib import Path
16 from typing import Any
17 from zoneinfo import ZoneInfo
18
19 import requests
20 from requests.adapters import HTTPAdapter
21 from urllib3.util.retry import Retry
22
23
24 REPO_ROOT = Path(__file__).resolve().parents[2]
25 CONFIG_PATH = Path(__file__).with_name("p0_sources.json")
26 RAW_DIR = REPO_ROOT / "data" / "raw"
27 META_DIR = RAW_DIR / "_meta"
28 MANIFEST_CSV = RAW_DIR / "source_manifest.csv"
29 MANIFEST_JSON = RAW_DIR / "source_manifest.json"
30 USER_AGENT = (
31     "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
32     "AppleWebKit/537.36 (KHTML, like Gecko) "
33     "Chrome/138.0.0.0 Safari/537.36"
34 )
35
36 MANIFEST_FIELDS = [
37     "artifact_id",
38     "source_id",
39     "status",
40     "required",
41     "url",
42     "local_path",
43     "downloaded_at",
44     "checked_at",
45     "http_status",
46     "content_type",
47     "content_length_header",
48     "size_bytes",
49     "sha256",
50     "etag",
51     "last_modified",
52     "error",
53 ]
54
55 @dataclass(frozen=True)
56 class Source:
57     artifact_id: str
58     source_id: str
59     url: str
60     filename_template: str
61     format: str
62     min_bytes: int
63     required: bool
64     expected_csv_columns: tuple[str, ...] = ()

```



```

65     requires_env: str | None = None
66     referer: str | None = None
67     user_agent: str | None = None
68     local_snapshot: bool = False
69
70
71 def now_shanghai() -> datetime:
72     return datetime.now(ZoneInfo("Asia/Shanghai"))
73
74
75 def sha256_file(path: Path) -> str:
76     digest = hashlib.sha256()
77     with path.open("rb") as handle:
78         for chunk in iter(lambda: handle.read(1024 * 1024), b""):
79             digest.update(chunk)
80     return digest.hexdigest()
81
82
83 def load_sources() -> tuple[dict[str, Any], list[Source]]:
84     config = json.loads(CONFIG_PATH.read_text(encoding="utf-8"))
85     sources = []
86     for item in config["sources"]:
87         sources.append(
88             Source(
89                 artifact_id=item["artifact_id"],
90                 source_id=item["source_id"],
91                 url=item["url"],
92                 filename_template=item["filename_template"],
93                 format=item["format"],
94                 min_bytes=int(item.get("min_bytes", 1)),
95                 required=bool(item.get("required", False)),
96                 expected_csv_columns=tuple(item.get("expected_csv_columns", [])),
97                 requires_env=item.get("requires_env"),
98                 referer=item.get("referer"),
99                 user_agent=item.get("user_agent"),
100                 local_snapshot=bool(item.get("local_snapshot", False)),
101             )
102         )
103     return config, sources
104
105
106 def make_session() -> requests.Session:
107     retry = Retry(
108         total=2,
109         connect=2,
110         read=0,
111         backoff_factor=1.5,
112         status_forcelist=(429, 500, 502, 503, 504),
113         allowed_methods=("GET",),
114         respect_retry_after_header=True,
115     )
116     session = requests.Session()
117     session.headers.update({"User-Agent": USER_AGENT, "Accept": "*/*"})
118     session.mount("https://", HTTPAdapter(max_retries=retry))
119     session.mount("http://", HTTPAdapter(max_retries=retry))
120     return session
121
122
123 def parse_curl_headers(path: Path) -> dict[str, str]:
124     """Return headers from the final response block written by curl."""
125     if not path.exists():
126         return {}
127     blocks = path.read_text(encoding="iso-8859-1").replace("\r\n", "\n").split("\n\n")
128     header_lines = next(
129         (block.splitlines() for block in reversed(blocks) if block.startswith("HTTP/")),
130         [],
131     )
132     headers: dict[str, str] = {}
133     for line in header_lines[1:]:
134         if ":" in line:
135             key, value = line.split(":", 1)
136             headers[key.strip().lower()] = value.strip()
137     return headers
138
139
140 def download_with_curl(source: Source, partial: Path) -> dict[str, Any]:

```

```

141 """Use system curl when available; it is more reliable on this Windows host."""
142 curl = shutil.which("curl.exe") or shutil.which("curl")
143 if not curl:
144     raise FileNotFoundError("curl executable not found")
145 header_path = partial.with_suffix(partial.suffix + ".headers")
146 if header_path.exists():
147     header_path.unlink()
148 command = [
149     curl,
150     "--location",
151     "--fail",
152     "--silent",
153     "--show-error",
154     "--connect-timeout",
155     "12",
156     "--max-time",
157     "120",
158     "--retry",
159     "3",
160     "--retry-delay",
161     "2",
162     "--retry-all-errors",
163     "--dump-header",
164     str(header_path),
165     "--output",
166     str(partial),
167     "--write-out",
168     "%{http_code}\\t%{content_type}\\t%{size_download}\\t%{url_effective}",
169 ]
170 if source.referer:
171     command.extend(["--referer", source.referer])
172 if source.user_agent:
173     command.extend(["--user-agent", source.user_agent])
174 command.append(source.url)
175 try:
176     completed = subprocess.run(
177         command,
178         check=True,
179         capture_output=True,
180         text=True,
181         timeout=140,
182     )
183     parts = completed.stdout.strip().split("\t", 3)
184     headers = parse_curl_headers(header_path)
185     return {
186         "http_status": parts[0] if parts else "",
187         "content_type": parts[1] if len(parts) > 1 else headers.get("content-type", ""),
188         "content_length_header": headers.get("content-length", ""),
189         "etag": headers.get("etag", ""),
190         "last_modified": headers.get("last-modified", ""),
191     }
192 finally:
193     if header_path.exists():
194         header_path.unlink()
195
196
197 def download_with_requests(
198     session: requests.Session,
199     source: Source,
200     partial: Path,
201 ) -> dict[str, Any]:
202     headers = {}
203     if source.referer:
204         headers["Referer"] = source.referer
205     if source.user_agent:
206         headers["User-Agent"] = source.user_agent
207     with session.get(
208         source.url,
209         headers=headers,
210         timeout=(15, 60),
211         stream=True,
212     ) as response:
213         metadata = {
214             "http_status": response.status_code,
215             "content_type": response.headers.get("Content-Type", ""),
216             "content_length_header": response.headers.get("Content-Length", ""),

```

```

217         "etag": response.headers.get("ETag", ""),
218         "last_modified": response.headers.get("Last-Modified", ""),
219     }
220     response.raise_for_status()
221     with partial.open("wb") as handle:
222         for chunk in response.iter_content(chunk_size=1024 * 1024):
223             if chunk:
224                 handle.write(chunk)
225     return metadata
226
227
228 def read_manifest() -> dict[str, dict[str, Any]]:
229     if not MANIFEST_JSON.exists():
230         return {}
231     rows = json.loads(MANIFEST_JSON.read_text(encoding="utf-8"))
232     return {row["artifact_id"]: row for row in rows}
233
234
235 def write_manifest(rows_by_id: dict[str, dict[str, Any]]) -> None:
236     rows: list[dict[str, Any]] = []
237     for key in sorted(rows_by_id):
238         row = dict(rows_by_id[key])
239         if row.get("status") in {"DOWNLOADED", "CACHED"}:
240             local_path = row.get("local_path", "")
241             if not local_path or not (REPO_ROOT / local_path).exists():
242                 row.update(
243                     {
244                         "status": "REMOTE_ONLY",
245                         "local_path": "",
246                         "size_bytes": "",
247                         "sha256": "",
248                         "error": "manifest local snapshot is absent; rerun download or provide a browser-extracted snapshot before
249 marking CACHED",
250                     }
251                 )
252             rows.append(row)
253     with MANIFEST_JSON.open("w", encoding="utf-8", newline="\n") as handle:
254         handle.write(json.dumps(rows, ensure_ascii=False, indent=2) + "\n")
255     with MANIFEST_CSV.open("w", encoding="utf-8", newline="") as handle:
256         writer = csv.DictWriter(handle, fieldnames=MANIFEST_FIELDS, lineterminator="\n")
257         writer.writeheader()
258         writer.writerows({key: row.get(key, "") for key in MANIFEST_FIELDS} for row in rows)
259
260
261 def validate_csv_header(path: Path, expected: tuple[str, ...]) -> None:
262     if not expected:
263         return
264     with path.open("r", encoding="utf-8-sig", newline="") as handle:
265         header = next(csv.reader(handle))
266     missing = [column for column in expected if column not in header]
267     if missing:
268         raise ValueError(f"CSV missing expected columns: {missing}; actual={header[:20]}")
269
270
271 def cached_record(source: Source, target: Path, timestamp: str) -> dict[str, Any]:
272     download_date = target.stem.rsplit("_", 1)[-1]
273     meta_path = META_DIR / f"{source.artifact_id}_{download_date}.json"
274     existing: dict[str, Any] = {}
275     if meta_path.exists():
276         existing = json.loads(meta_path.read_text(encoding="utf-8"))
277     return {
278         "artifact_id": source.artifact_id,
279         "source_id": source.source_id,
280         "status": "CACHED",
281         "required": source.required,
282         "url": source.url,
283         "local_path": target.relative_to(REPO_ROOT).as_posix(),
284         "downloaded_at": existing.get("downloaded_at", timestamp),
285         "checked_at": timestamp,
286         "http_status": existing.get("http_status", ""),
287         "content_type": existing.get("content_type", ""),
288         "content_length_header": existing.get("content_length_header", ""),
289         "size_bytes": target.stat().st_size,
290         "sha256": sha256_file(target),
291         "etag": existing.get("etag", ""),
292         "last_modified": existing.get("last_modified", ""),

```

```

292     "error": "",
293 }
294
295
296 def download_one(
297     session: requests.Session,
298     source: Source,
299     *,
300     refresh: bool,
301     download_date: str,
302     timestamp: str,
303 ) -> dict[str, Any]:
304     filename = source.filename_template.format(download_date=download_date)
305     target = RAW_DIR / filename
306     if not target.exists() and not refresh and "{download_date}" in source.filename_template:
307         pattern = source.filename_template.replace("{download_date}", "**")
308         prior_snapshots = sorted(RAW_DIR.glob(pattern))
309         if prior_snapshots:
310             target = prior_snapshots[-1]
311     partial = target.with_suffix(target.suffix + ".part")
312
313     if source.requires_env and not os.getenv(source.requires_env):
314         return {
315             "artifact_id": source.artifact_id,
316             "source_id": source.source_id,
317             "status": "SKIPPED_MISSING_ENV",
318             "required": source.required,
319             "url": source.url,
320             "local_path": "",
321             "downloaded_at": timestamp,
322             "checked_at": timestamp,
323             "error": f"missing environment variable {source.requires_env}",
324         }
325
326     if source.local_snapshot:
327         if not target.exists():
328             return {
329                 "artifact_id": source.artifact_id,
330                 "source_id": source.source_id,
331                 "status": "ERROR",
332                 "required": source.required,
333                 "url": source.url,
334                 "local_path": "",
335                 "downloaded_at": timestamp,
336                 "checked_at": timestamp,
337                 "error": (
338                     "versioned browser-extracted snapshot is missing; "
339                     f"expected {target.relative_to(REPO_ROOT).as_posix()}"
340                 ),
341             }
342     validate_csv_header(target, source.expected_csv_columns)
343     record = cached_record(source, target, timestamp)
344     record.update(
345         {
346             "http_status": "200-browser",
347             "content_type": "text/csv; charset=utf-8",
348         }
349     )
350     meta_path = META_DIR / f"{source.artifact_id}_{download_date}.json"
351     meta_path.write_text(
352         json.dumps(record, ensure_ascii=False, indent=2) + "\n",
353         encoding="utf-8",
354     )
355     return record
356
357     if target.exists() and not refresh:
358         validate_csv_header(target, source.expected_csv_columns)
359         return cached_record(source, target, timestamp)
360
361     if partial.exists():
362         partial.unlink()
363
364     record: dict[str, Any] = {
365         "artifact_id": source.artifact_id,
366         "source_id": source.source_id,
367         "status": "ERROR",

```

```

368     "required": source.required,
369     "url": source.url,
370     "local_path": "",
371     "downloaded_at": timestamp,
372     "checked_at": timestamp,
373     "http_status": "",
374     "content_type": "",
375     "content_length_header": "",
376     "size_bytes": "",
377     "sha256": "",
378     "etag": "",
379     "last_modified": "",
380     "error": "",
381 }
382 try:
383     if shutil.which("curl.exe") or shutil.which("curl"):
384         response_metadata = download_with_curl(source, partial)
385     else:
386         response_metadata = download_with_requests(session, source, partial)
387     record.update(response_metadata)
388
389     size = partial.stat().st_size
390     if size < source.min_bytes:
391         raise ValueError(f"downloaded file is too small: {size} < {source.min_bytes}")
392     validate_csv_header(partial, source.expected_csv_columns)
393     partial.replace(target)
394
395     record.update(
396         {
397             "status": "DOWNLOADED",
398             "local_path": target.relative_to(REPO_ROOT).as_posix(),
399             "size_bytes": size,
400             "sha256": sha256_file(target),
401             "error": "",
402         }
403     )
404     meta_path = META_DIR / f"{source.artifact_id}_{download_date}.json"
405     meta_path.write_text(
406         json.dumps(record, ensure_ascii=False, indent=2) + "\n",
407         encoding="utf-8",
408     )
409 except Exception as exc:
410     if partial.exists():
411         partial.unlink()
412     record["error"] = f"{type(exc).__name__}: {exc}"
413 return record
414
415
416 def parse_args() -> argparse.Namespace:
417     parser = argparse.ArgumentParser(description=__doc__)
418     parser.add_argument(
419         "--source",
420         action="append",
421         default=[],
422         help="Download only this artifact_id; may be specified multiple times.",
423     )
424     parser.add_argument(
425         "--refresh",
426         action="store_true",
427         help="Replace today's existing snapshot. Without this flag raw files are never overwritten.",
428     )
429     parser.add_argument("--list", action="store_true", help="List configured sources and exit.")
430     return parser.parse_args()
431
432
433 def main() -> int:
434     args = parse_args()
435     config, sources = load_sources()
436     if args.list:
437         for source in sources:
438             print(
439                 f"{source.artifact_id}\trequired={str(source.required).lower()}"
440                 f"\t{source.source_id}\t{source.url}"
441             )
442     return 0
443

```

```

444 selected_ids = set(args.source)
445 if selected_ids:
446     unknown = selected_ids - {source.artifact_id for source in sources}
447     if unknown:
448         print(f"Unknown artifact_id(s): {sorted(unknown)}", file=sys.stderr)
449         return 2
450     sources = [source for source in sources if source.artifact_id in selected_ids]
451
452 RAW_DIR.mkdir(parents=True, exist_ok=True)
453 META_DIR.mkdir(parents=True, exist_ok=True)
454 timestamp = now_shanghai().isoformat(timespec="seconds")
455 download_date = now_shanghai().strftime("%Y%m%d")
456 manifest = read_manifest()
457 session = make_session()
458
459 required_failures = 0
460 for source in sources:
461     print(f"[DOWNLOAD] {source.artifact_id}", flush=True)
462     record = download_one(
463         session,
464         source,
465         refresh=args.refresh,
466         download_date=download_date,
467         timestamp=timestamp,
468     )
469     manifest[source.artifact_id] = record
470     print(
471         f"[{record['status']}] {source.artifact_id}"
472         f" bytes={record.get('size_bytes', '')}"
473         f" error={record.get('error', '')}",
474         flush=True,
475     )
476     if source.required and record["status"] not in {"DOWNLOADED", "CACHED"}:
477         required_failures += 1
478
479 write_manifest(manifest)
480 summary = {
481     "schema_version": config["schema_version"],
482     "cutoff_date": config["cutoff_date"],
483     "selected": len(sources),
484     "required_failures": required_failures,
485     "manifest": MANIFEST_CSV.relative_to(REPO_ROOT).as_posix(),
486 }
487 print(json.dumps(summary, ensure_ascii=False))
488 return 1 if required_failures else 0
489
490
491 if __name__ == "__main__":
492     raise SystemExit(main())

```

Listing 1: download_p0.py

```

1 """Build reproducible P0 processed datasets and a data-quality report."""
2
3 from __future__ import annotations
4
5 import argparse
6 import hashlib
7 import html
8 import io
9 import json
10 import re
11 import zipfile
12 from dataclasses import dataclass
13 from datetime import date, datetime
14 from pathlib import Path
15 from typing import Any
16
17 import openpyxl
18 import pandas as pd
19 import requests
20
21
22 REPO_ROOT = Path(__file__).resolve().parents[2]
23 RAW_DIR = REPO_ROOT / "data" / "raw"

```

```

24 PROCESSED_DIR = REPO_ROOT / "data" / "processed"
25 REPORT_PATH = REPO_ROOT / "reports" / "DATA_QUALITY_REPORT.md"
26 MANIFEST_PATH = RAW_DIR / "source_manifest.json"
27 DICTIONARY_PATH = REPO_ROOT / "data" / "DATA_DICTIONARY.csv"
28 CUTOFF = pd.Timestamp("2026-06-30")
29 START_MONTHLY = pd.Timestamp("2010-01-01")
30 SUCCESS_STATUSES = {"DOWNLOADED", "CACHED"}
31 EU_FUEL_COUNTRIES = {
32     "DEU": ("DE", "Germany", "germany"),
33     "FRA": ("FR", "France", "france"),
34     "ITA": ("IT", "Italy", "italy"),
35     "ESP": ("ES", "Spain", "spain"),
36 }
37 EIA_INTL_BULK_URL = "https://www.eia.gov/opa/data/bulk/INTL.zip"
38 EIA_WORLD_LIQUIDS_SERIES_ID = "INTL.53-1-WORL-TBPD.M"
39 DALLAS_FED_IGREA_URL = "https://www.dallasfed.org/-/media/documents/research/igrea/igrea.xlsx"
40 FRED_US_CPI_URL = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=CPIAUCSL"
41 OECD_US_CPI_URL = (
42     "https://sdmx.oecd.org/public/rest/data/"
43     "OECD.SDD.TPS.DSD_G20_PRICES8DF_G20_PRICES,1.0/"
44     "USA.M...IX...?startPeriod=2010-01&endPeriod=2026-06"
45     "&dimensionAtObservation=AllDimensions&format=csvfile"
46 )
47
48
49 @dataclass
50 class QualityRow:
51     dataset: str
52     status: str
53     rows: int
54     start: str
55     end: str
56     missing: int
57     duplicates: int
58     note: str
59
60
61 def load_manifest() -> dict[str, dict[str, Any]]:
62     rows = json.loads(MANIFEST_PATH.read_text(encoding="utf-8"))
63     return {row["artifact_id"]: row for row in rows}
64
65
66 def source_path(manifest: dict[str, dict[str, Any]], artifact_id: str) -> Path:
67     record = manifest.get(artifact_id)
68     if not record:
69         raise KeyError(f"manifest has no artifact_id={artifact_id}")
70     if record["status"] not in SUCCESS_STATUSES:
71         raise RuntimeError(
72             f"artifact {artifact_id} is not usable: "
73             f"status={record['status']} error={record.get('error', '')}"
74         )
75     path = REPO_ROOT / record["local_path"]
76     if not path.exists():
77         raise FileNotFoundError(path)
78     return path
79
80
81 def save_csv(frame: pd.DataFrame, filename: str) -> Path:
82     path = PROCESSED_DIR / filename
83     frame.to_csv(path, index=False, encoding="utf-8")
84     return path
85
86
87 def sha256_file(path: Path) -> str:
88     return hashlib.sha256(path.read_bytes()).hexdigest()
89
90
91 def download_cached(url: str, filename: str, timeout: int = 60) -> Path:
92     RAW_DIR.mkdir(parents=True, exist_ok=True)
93     path = RAW_DIR / filename
94     if path.exists() and path.stat().st_size > 0:
95         return path
96     response = requests.get(url, timeout=timeout, headers={"User-Agent": "Mozilla/5.0"})
97     response.raise_for_status()
98     path.write_bytes(response.content)
99     return path

```

```

100
101
102 def date_string(value: Any) -> str:
103     if pd.isna(value):
104         return ""
105     return pd.Timestamp(value).strftime("%Y-%m-%d")
106
107
108 def parse_excel_date(value: Any) -> pd.Timestamp:
109     if isinstance(value, pd.Timestamp):
110         return value
111     if isinstance(value, (datetime, date)):
112         return pd.Timestamp(value)
113     return pd.to_datetime(value, format="%Y-%m-%d", errors="coerce")
114
115
116 def profile(
117     dataset: str,
118     frame: pd.DataFrame,
119     date_column: str,
120     value_columns: list[str],
121     note: str = "",
122     status: str = "PASS",
123 ) -> QualityRow:
124     dates = pd.to_datetime(frame[date_column], errors="coerce")
125     missing = int(frame[value_columns].isna().sum().sum())
126     duplicates = int(dates.duplicated().sum())
127     return QualityRow(
128         dataset=dataset,
129         status=status,
130         rows=len(frame),
131         start=date_string(dates.min()),
132         end=date_string(dates.max()),
133         missing=missing,
134         duplicates=duplicates,
135         note=note,
136     )
137
138
139 def read_fred(path: Path, value_column: str, output_column: str) -> pd.DataFrame:
140     frame = pd.read_csv(path, na_values=[".", ""])
141     required = {"observation_date", value_column}
142     missing = required - set(frame.columns)
143     if missing:
144         raise ValueError(f"{path.name} missing columns {sorted(missing)}")
145     frame = frame[["observation_date", value_column]].rename(
146         columns={"observation_date": "date", value_column: output_column}
147     )
148     frame["date"] = pd.to_datetime(frame["date"], errors="coerce")
149     frame[output_column] = pd.to_numeric(frame[output_column], errors="coerce")
150     frame = frame.dropna(subset=["date"]).sort_values("date")
151     return frame[(frame["date"] >= START_MONTHLY) & (frame["date"] <= CUTOFF)]
152
153
154 def build_market_data(
155     manifest: dict[str, dict[str, Any]]
156 ) -> tuple[pd.DataFrame, pd.DataFrame, list[QualityRow]]:
157     specifications = [
158         ("fred_brent_daily", "DCOILBRENTU", "brent_usd_bbl"),
159         ("fred_wti_daily", "DCOILWTICO", "wti_usd_bbl"),
160         ("fred_usd_broad_daily", "DTWEXBGS", "usd_broad_index"),
161         ("fred_cny_per_usd_daily", "DEXCHUS", "cny_per_usd"),
162         ("fred_jpy_per_usd_daily", "DEXJPUS", "jpy_per_usd"),
163         ("fred_krw_per_usd_daily", "DEXKOUS", "krw_per_usd"),
164         ("fred_usd_per_eur_daily", "DEXUSEU", "usd_per_eur"),
165     ]
166     frames = []
167     profiles = []
168     for artifact_id, raw_column, output_column in specifications:
169         frame = read_fred(source_path(manifest, artifact_id), raw_column, output_column)
170         profiles.append(profile(artifact_id, frame, "date", [output_column]))
171         frames.append(frame.set_index("date"))
172
173     daily = pd.concat(frames, axis=1, join="outer").sort_index()
174     daily["eur_per_usd"] = 1.0 / daily["usd_per_eur"]
175     daily = daily.reset_index()

```



```

176     save_csv(daily, "p0_daily_market.csv")
177
178     monthly = (
179         daily.set_index("date")
180         .resample("ME")
181         .mean(numeric_only=True)
182         .rename_axis("month_end")
183         .reset_index()
184     )
185     monthly["period"] = monthly["month_end"].dt.to_period("M").astype(str)
186     monthly["brent_cny_per_bbl"] = (
187         monthly["brent_usd_bbl"] * monthly["cny_per_usd"]
188     )
189     monthly = monthly[
190         [
191             "period",
192             "month_end",
193             "brent_usd_bbl",
194             "wti_usd_bbl",
195             "usd_broad_index",
196             "cny_per_usd",
197             "jpy_per_usd",
198             "krw_per_usd",
199             "usd_per_eur",
200             "eur_per_usd",
201             "brent_cny_per_bbl",
202         ]
203     ]
204     return daily, monthly, profiles
205
206
207 def build_stock_data(
208     manifest: dict[str, dict[str, Any]]
209 ) -> tuple[pd.DataFrame, pd.DataFrame, QualityRow]:
210     path = source_path(manifest, "eia_us_commercial_crude_stock_weekly")
211     frame = pd.read_excel(
212         path,
213         sheet_name="Data 1",
214         header=None,
215         skiprows=3,
216         usecols=[0, 1],
217         names=["date", "us_commercial_crude_stock_kbbl"],
218     )
219     frame["date"] = pd.to_datetime(frame["date"], errors="coerce")
220     frame["us_commercial_crude_stock_kbbl"] = pd.to_numeric(
221         frame["us_commercial_crude_stock_kbbl"], errors="coerce"
222     )
223     frame = frame.dropna(subset=["date"]).sort_values("date")
224     frame = frame[(frame["date"] >= START_MONTHLY) & (frame["date"] <= CUTOFF)]
225     save_csv(frame, "eia_us_commercial_crude_stock_weekly.csv")
226
227     month_end = (
228         frame.assign(period=frame["date"].dt.to_period("M").astype(str))
229         .sort_values("date")
230         .groupby("period", as_index=False)
231         .tail(1)
232         .rename(
233             columns={
234                 "date": "stock_reference_date",
235                 "us_commercial_crude_stock_kbbl": "us_crude_stock_month_end_kbbl",
236             }
237         )
238         .sort_values("period")
239         .reset_index(drop=True)
240     )
241     return frame, month_end, profile(
242         "eia_us_commercial_crude_stock_weekly",
243         frame,
244         "date",
245         ["us_commercial_crude_stock_kbbl"],
246         note="月度主口径使用每月最后一个已观测周值",
247     )
248
249
250 def build_gpr_data(
251     manifest: dict[str, dict[str, Any]]

```

```

252 ) -> tuple[pd.DataFrame, QualityRow]:
253     path = source_path(manifest, "gpr_global_monthly")
254     raw = pd.read_stata(path, convert_categoricals=False)
255     required = ["month", "GPR", "GPRT", "GPRA"]
256     missing = set(required) - set(raw.columns)
257     if missing:
258         raise ValueError(f"{path.name} missing columns {sorted(missing)}")
259     frame = raw[required].copy()
260     frame["month"] = pd.to_datetime(frame["month"], errors="coerce")
261     frame = frame.dropna(subset=["month"]).sort_values("month")
262     frame = frame[(frame["month"] >= START_MONTHLY) & (frame["month"] <= CUTOFF)]
263     frame["period"] = frame["month"].dt.to_period("M").astype(str)
264     frame = frame[["period", "month", "GPR", "GPRT", "GPRA"]]
265     save_csv(frame, "gpr_global_monthly.csv")
266     return frame, profile(
267         "gpr_global_monthly",
268         frame,
269         "month",
270         ["GPR", "GPRT", "GPRA"],
271         note="2026-06 为最新初值, 后续可能修订",
272     )
273
274
275 def steo_data_status(cell: Any) -> str:
276     color = cell.fill_fgColor
277     if color.type == "indexed":
278         return "historical"
279     if color.type == "theme" and float(color.tint or 0.0) < -0.001:
280         return "forecast"
281     return "estimate"
282
283
284 def build_steo_data(
285     manifest: dict[str, dict[str, Any]]
286 ) -> tuple[pd.DataFrame, QualityRow]:
287     path = source_path(manifest, "eia_steo_jul2026")
288     workbook = openpyxl.load_workbook(path, data_only=True, read_only=False)
289     sheet = workbook["3atab"]
290     target_map = {
291         "papr_world": ("world_liquids_supply_mbd", "million barrels/day"),
292         "patc_world": ("world_liquids_demand_mbd", "million barrels/day"),
293         "pasc_oecd_t3": ("oecd_commercial_liquids_stocks_mmbbl", "million barrels"),
294     }
295     row_by_series = {}
296     for row in range(1, sheet.max_row + 1):
297         value = sheet.cell(row, 1).value
298         if isinstance(value, str) and value.strip().lower() in target_map:
299             row_by_series[value.strip().lower()] = row
300     missing_series = set(target_map) - set(row_by_series)
301     if missing_series:
302         raise ValueError(f"STEO missing target series {sorted(missing_series)}")
303
304     vintage_text = str(sheet["A4"].value)
305     vintage_date = pd.to_datetime(
306         re.sub(r"^[A-Za-z]+\s+", "", vintage_text), errors="raise"
307     ).strftime("%Y-%m-%d")
308
309     periods: dict[int, pd.Period] = {}
310     year: int | None = None
311     for column in range(3, sheet.max_column + 1):
312         year_value = sheet.cell(3, column).value
313         if year_value is not None:
314             year = int(year_value)
315         month_value = sheet.cell(4, column).value
316         if year is not None and isinstance(month_value, str):
317             periods[column] = pd.Period(
318                 f"{year}-{pd.to_datetime(month_value, format='%b').month:02d}",
319                 freq="M",
320             )
321
322     rows = []
323     for series_id, (variable_id, unit) in target_map.items():
324         row = row_by_series[series_id]
325         for column, period in periods.items():
326             value = sheet.cell(row, column).value
327             if value is None:

```

```

328         continue
329         rows.append(
330             {
331                 "period": str(period),
332                 "series_id": series_id.upper(),
333                 "variable_id": variable_id,
334                 "value": float(value),
335                 "unit": unit,
336                 "vintage_date": vintage_date,
337                 "data_status": steo_data_status(sheet.cell(row, column)),
338             }
339         )
340     frame = pd.DataFrame(rows)
341     frame = frame[
342         pd.to_datetime(frame["period"] + "-01") <= CUTOFF
343     ].sort_values(["variable_id", "period"])
344     frame["is_forecast_or_estimate"] = frame["data_status"].ne("historical")
345     save_csv(frame, "eia_steo_selected.csv")
346
347     coverage_start = pd.to_datetime(frame["period"] + "-01").min()
348     coverage_end = pd.to_datetime(frame["period"] + "-01").max()
349     return frame, QualityRow(
350         dataset="eia_steo_selected",
351         status="CONDITIONAL",
352         rows=len(frame),
353         start=date_string(coverage_start),
354         end=date_string(coverage_end),
355         missing=int(frame["value"].isna().sum()),
356         duplicates=int(frame.duplicated(["variable_id", "period"]).sum()),
357         note="当前单一版本仅覆盖2022年起, 且2026-02至06为估计值; 不能回填2010年起的实时滚动预测",
358     )
359
360
361 def parse_eia_world_liquids(path: Path) -> pd.DataFrame:
362     with zipfile.ZipFile(path) as archive:
363         with archive.open("INTL.txt") as handle:
364             for raw_line in handle:
365                 record = json.loads(raw_line)
366                 if record.get("series_id") != EIA_WORLD_LIQUIDS_SERIES_ID:
367                     continue
368                 rows = []
369                 for period_code, value in record.get("data", []):
370                     period_text = str(period_code)
371                     if len(period_text) != 6:
372                         continue
373                     rows.append(
374                         {
375                             "period": f"{period_text[:4]}-{period_text[4:]}",
376                             "world_liquids_supply": float(value) / 1000.0,
377                             "world_liquids_supply_unit": "million barrels per day",
378                             "eia_series_id": record["series_id"],
379                             "eia_last_updated": record.get("last_updated", ""),
380                         }
381                     )
382                 frame = pd.DataFrame(rows)
383                 frame["month_start"] = pd.to_datetime(frame["period"] + "-01", errors="coerce")
384                 frame = frame.loc[frame["month_start"].between(START_MONTHLY, CUTOFF)].copy()
385                 return frame.drop(columns=["month_start"]).sort_values("period").reset_index(drop=True)
386             raise ValueError(f"{path.name} does not contain {EIA_WORLD_LIQUIDS_SERIES_ID}")
387
388
389 def parse_dallas_fed_igrea(path: Path) -> pd.DataFrame:
390     raw = pd.read_excel(path, sheet_name="Kilian Index", header=None)
391     frame = raw.iloc[1:, [0, 1]].copy()
392     frame.columns = ["date", "global_real_activity"]
393     frame["date"] = pd.to_datetime(frame["date"], errors="coerce")
394     frame["global_real_activity"] = pd.to_numeric(frame["global_real_activity"], errors="coerce")
395     frame = frame.dropna(subset=["date", "global_real_activity"]).copy()
396     frame["period"] = frame["date"].dt.to_period("M").astype(str)
397     frame = frame.loc[pd.to_datetime(frame["period"] + "-01").between(START_MONTHLY, CUTOFF)]
398     return frame[["period", "global_real_activity"]].sort_values("period").reset_index(drop=True)
399
400
401 def load_us_cpi_deflator() -> tuple[pd.DataFrame, Path, str, str]:
402     try:
403         fred_path = download_cached(FRED_US_CPI_URL, "fred_us_cpi_cpiaucsl_20260731.csv", timeout=45)

```

```

404     fred = pd.read_csv(fred_path, na_values=[".", ""])
405     if {"observation_date", "CPIAUCSL"}.issubset(fred.columns):
406         frame = fred[["observation_date", "CPIAUCSL"]].rename(
407             columns={"observation_date": "date", "CPIAUCSL": "us_cpi"}
408         )
409         frame["date"] = pd.to_datetime(frame["date"], errors="coerce")
410         frame["us_cpi"] = pd.to_numeric(frame["us_cpi"], errors="coerce")
411         frame = frame.dropna(subset=["date", "us_cpi"]).copy()
412         frame["period"] = frame["date"].dt.to_period("M").astype(str)
413         frame = frame.loc[pd.to_datetime(frame["period"] + "-01").between(START_MONTHLY, CUTOFF)]
414         return frame[["period", "us_cpi"]], fred_path, FRED_US_CPI_URL, "FRED_CPIAUCSL"
415     except requests.RequestException:
416         pass
417
418     oecd_path = download_cached(OECD_US_CPI_URL, "oecd_us_cpi_index_20260731.csv", timeout=60)
419     raw = pd.read_csv(oecd_path)
420     required = {"REF_AREA", "TIME_PERIOD", "OBS_VALUE"}
421     missing = required - set(raw.columns)
422     if missing:
423         raise ValueError(f"{oecd_path.name} missing columns {sorted(missing)}")
424     frame = raw.loc[raw["REF_AREA"].eq("USA"), ["TIME_PERIOD", "OBS_VALUE"]].copy()
425     frame = frame.rename(columns={"TIME_PERIOD": "period", "OBS_VALUE": "us_cpi"})
426     frame["us_cpi"] = pd.to_numeric(frame["us_cpi"], errors="coerce")
427     frame = frame.dropna(subset=["period", "us_cpi"])
428     return frame[["period", "us_cpi"]].sort_values("period").reset_index(drop=True), oecd_path, OECD_US_CPI_URL, "OECD_US_CPI_INDEX"
429
430
431 def build_global_oil_svar_input(monthly_market: pd.DataFrame) -> tuple[pd.DataFrame, QualityRow]:
432     eia_path = download_cached(EIA_INTL_BULK_URL, "eia_intl_bulk_20260731.zip", timeout=120)
433     igrea_path = download_cached(DALLAS_FED_IGREA_URL, "dallasfed_igrea_20260731.xlsx", timeout=90)
434     supply = parse_eia_world_liquids(eia_path)
435     activity = parse_dallas_fed_igrea(igrea_path)
436     cpi, cpi_path, cpi_url, cpi_source = load_us_cpi_deflator()
437     market = monthly_market[["period", "month_end", "brent_usd_bbl"]].copy()
438     frame = (
439         pd.DataFrame({"period": pd.period_range("2010-01", "2026-06", freq="M").astype(str)})
440         .merge(supply, on="period", how="left")
441         .merge(activity, on="period", how="left")
442         .merge(cpi, on="period", how="left")
443         .merge(market, on="period", how="left")
444     )
445     cpi_base = float(frame.loc[frame["period"].eq("2026-06"), "us_cpi"].dropna().iloc[0])
446     frame["real_brent"] = frame["brent_usd_bbl"] * cpi_base / frame["us_cpi"]
447     frame["source_vintage"] = "downloaded_2026-07-31_ex_post"
448     frame["release_date"] = ""
449     frame["source_url"] = ";".join([EIA_INTL_BULK_URL, DALLAS_FED_IGREA_URL, cpi_url])
450     frame["raw_sha256"] = ";".join(
451         [
452             f"eia={sha256_file(eia_path)}",
453             f"igrea={sha256_file(igrea_path)}",
454             f"cpi_source.lower()={sha256_file(cpi_path)}",
455         ]
456     )
457     output = frame[
458         [
459             "period",
460             "world_liquids_supply",
461             "global_real_activity",
462             "us_cpi",
463             "real_brent",
464             "source_vintage",
465             "release_date",
466             "source_url",
467             "raw_sha256",
468         ]
469     ]
470     save_csv(output, "global_oil_svar_monthly.csv")
471     usable = output[["world_liquids_supply", "global_real_activity", "real_brent"]].dropna()
472     usable_periods = output.loc[output[["world_liquids_supply", "global_real_activity", "real_brent"]].notna().all(axis=1), "period"]
473     status = "PASS" if len(usable) >= 120 else "CONDITIONAL"
474     return output, QualityRow(
475         dataset="global_oil_svar_monthly",
476         status=status,
477         rows=len(output),
478         start=str(usable_periods.min()) if not usable_periods.empty else "",
479         end=str(usable_periods.max()) if not usable_periods.empty else "",

```

```

480     missing=int(output[["world_liquids_supply", "global_real_activity", "us_cpi", "real_brent"]].isna().sum().sum()),
481     duplicates=int(output[["period"].duplicated().sum()),
482     note=f"SVAR ex-post input from EIA INTL world liquids, Dallas Fed IGERA and {cpi_source}; not used for real-time forecast
         backtests.",
483 )
484
485
486 def build_oecd_data(
487     manifest: dict[str, dict[str, Any]]
488 ) -> tuple[pd.DataFrame, pd.DataFrame, list[QualityRow]]:
489     cpi_path = source_path(manifest, "oecd_g20_cpi_monthly")
490     cpi = pd.read_csv(cpi_path)
491     cpi["observation_month"] = pd.to_datetime(
492         cpi["TIME_PERIOD"].astype(str) + "-01", errors="coerce"
493     )
494     cpi = cpi[cpi["observation_month"] <= CUTOFF].copy()
495     cpi_columns = [
496         "REF_AREA",
497         "METHODOLOGY",
498         "MEASURE",
499         "UNIT_MEASURE",
500         "EXPENDITURE",
501         "ADJUSTMENT",
502         "TRANSFORMATION",
503         "TIME_PERIOD",
504         "OBS_VALUE",
505         "OBS_STATUS",
506         "BASE_PER",
507     ]
508     cpi = cpi[[column for column in cpi_columns if column in cpi.columns]]
509     save_csv(cpi, "oecd_g20_cpi_monthly.csv")
510
511     ip_path = source_path(manifest, "oecd_kei_ip_monthly")
512     ip = pd.read_csv(ip_path)
513     ip["observation_month"] = pd.to_datetime(
514         ip["TIME_PERIOD"].astype(str) + "-01", errors="coerce"
515     )
516     ip = ip[ip["observation_month"] <= CUTOFF].copy()
517     ip_columns = [
518         "REF_AREA",
519         "MEASURE",
520         "UNIT_MEASURE",
521         "ACTIVITY",
522         "ADJUSTMENT",
523         "TRANSFORMATION",
524         "TIME_PERIOD",
525         "OBS_VALUE",
526         "OBS_STATUS",
527         "BASE_PER",
528     ]
529     ip = ip[[column for column in ip_columns if column in ip.columns]]
530     save_csv(ip, "oecd_kei_ip_monthly.csv")
531
532     cpi_dates = pd.to_datetime(cpi["TIME_PERIOD"] + "-01")
533     ip_dates = pd.to_datetime(ip["TIME_PERIOD"] + "-01")
534     quality = [
535         QualityRow(
536             dataset="oecd_g20_cpi_monthly",
537             status="PASS",
538             rows=len(cpi),
539             start=date_string(cpi_dates.min()),
540             end=date_string(cpi_dates.max()),
541             missing=int(cpi["OBS_VALUE"].isna().sum()),
542             duplicates=int(cpi.duplicated(["REF_AREA", "TIME_PERIOD"]).sum()),
543             note="CHN/DEU/FRA/ITA/ESP/JPN/KOR 各198期; 德国、法国、意大利、西班牙为HICP, 其余为国家CPI",
544         ),
545         QualityRow(
546             dataset="oecd_kei_ip_monthly",
547             status="PASS",
548             rows=len(ip),
549             start=date_string(ip_dates.min()),
550             end=date_string(ip_dates.max()),
551             missing=int(ip["OBS_VALUE"].isna().sum()),
552             duplicates=int(ip.duplicated(["REF_AREA", "TIME_PERIOD"]).sum()),
553             note="DEU/FRA/ITA/ESP/JPN/KOR 各197期; 中国活动变量仍需国家统计局",
554         ),

```

```

555 ]
556     return cpi, ip, quality
557
558
559 def build_eu_fuel_data(
560     manifest: dict[str, dict[str, Any]]
561 ) -> tuple[pd.DataFrame, pd.DataFrame, list[QualityRow]]:
562     path = source_path(manifest, "ec_weekly_oil_bulletin")
563     raw = pd.read_excel(path, sheet_name="Prices with taxes", header=0)
564     date_col = raw.columns[0]
565     weekly_frames: list[pd.DataFrame] = []
566     monthly_frames: list[pd.DataFrame] = []
567     quality: list[QualityRow] = []
568     for country, (ec_code, country_name, slug) in EU_FUEL_COUNTRIES.items():
569         price_col = f"{ec_code}_price_with_tax_euro95"
570         if price_col not in raw.columns:
571             raise ValueError(f"Weekly Oil Bulletin layout lacks {price_col}")
572         weekly = raw[[date_col, price_col]].copy()
573         weekly.columns = ["date", "gasoline_eur_per_1000l"]
574         weekly["date"] = weekly["date"].map(parse_excel_date)
575         weekly["gasoline_eur_per_1000l"] = pd.to_numeric(weekly["gasoline_eur_per_1000l"], errors="coerce")
576         weekly = weekly.dropna(subset=["date", "gasoline_eur_per_1000l"])
577         weekly = weekly[weekly["date"].between(START_MONTHLY, CUTOFF)].copy()
578         weekly["country"] = country
579         weekly["country_name"] = country_name
580         weekly["gasoline_eur_per_l"] = weekly["gasoline_eur_per_1000l"] / 1000.0
581         weekly = weekly[["country", "country_name", "date", "gasoline_eur_per_1000l", "gasoline_eur_per_l"]]
582         weekly = weekly.sort_values("date").reset_index(drop=True)
583         save_csv(weekly, f"{slug}_eurosuper95_weekly.csv")
584         weekly_frames.append(weekly)
585
586         monthly = (
587             weekly.assign(period=weekly["date"].dt.to_period("M").astype(str))
588             .groupby(["country", "country_name", "period"], as_index=False)
589             .agg(
590                 gasoline_eur_per_l=("gasoline_eur_per_l", "mean"),
591                 weekly_observations=("gasoline_eur_per_l", "count"),
592             )
593         )
594         monthly["month_end"] = pd.to_datetime(monthly["period"]) + pd.offsets.MonthEnd(0)
595         save_csv(monthly, f"{slug}_eurosuper95_monthly.csv")
596         monthly_frames.append(monthly)
597         quality.extend(
598             [
599                 profile(
600                     f"{slug}_eurosuper95_weekly",
601                     weekly,
602                     "date",
603                     ["gasoline_eur_per_l"],
604                     note="European Commission Weekly Oil Bulletin, tax-inclusive Euro-super 95, original unit EUR/1000 litres.",
605                 ),
606                 profile(
607                     f"{slug}_eurosuper95_monthly",
608                     monthly,
609                     "month_end",
610                     ["gasoline_eur_per_l"],
611                     note="Arithmetic monthly average of official weekly observations.",
612                 ),
613             ]
614         )
615
616     all_weekly = pd.concat(weekly_frames, ignore_index=True)
617     all_monthly = pd.concat(monthly_frames, ignore_index=True)
618     save_csv(all_weekly, "eu_eurosuper95_weekly.csv")
619     save_csv(all_monthly, "eu_eurosuper95_monthly.csv")
620     germany_weekly = all_weekly.loc[all_weekly["country"].eq("DEU")].copy()
621     germany_monthly = all_monthly.loc[all_monthly["country"].eq("DEU")].copy()
622     save_csv(germany_weekly, "germany_eurosuper95_weekly.csv")
623     save_csv(germany_monthly[["period", "gasoline_eur_per_l", "weekly_observations", "month_end"]], "germany_eurosuper95_monthly.csv")
624     return germany_weekly, germany_monthly, quality
625
626
627 def build_germany_fuel_data(
628     manifest: dict[str, dict[str, Any]]
629 ) -> tuple[pd.DataFrame, pd.DataFrame, list[QualityRow]]:
630     path = source_path(manifest, "ec_weekly_oil_bulletin")

```

```

631 raw = pd.read_excel(path, sheet_name="Prices with taxes", header=None)
632 if raw.shape[1] <= 54:
633     raise ValueError("Weekly Oil Bulletin layout no longer contains expected DE columns")
634 weekly = raw.iloc[3:, [0, 53, 54]].copy()
635 weekly.columns = ["date", "country_code", "gasoline_eur_per_1000l"]
636 weekly["date"] = pd.to_datetime(weekly["date"], errors="coerce")
637 weekly["gasoline_eur_per_1000l"] = pd.to_numeric(
638     weekly["gasoline_eur_per_1000l"], errors="coerce"
639 )
640 weekly = weekly[
641     weekly["country_code"].astype(str).str.strip().eq("DE_")
642     & weekly["date"].between(START_MONTHLY, CUTOFF)
643 ].copy()
644 weekly["gasoline_eur_per_l"] = weekly["gasoline_eur_per_1000l"] / 1000.0
645 weekly = weekly.sort_values("date").reset_index(drop=True)
646 save_csv(weekly, "germany_eurosuper95_weekly.csv")
647
648 monthly = (
649     weekly.assign(period=weekly["date"].dt.to_period("M").astype(str))
650     .groupby("period", as_index=False)
651     .agg(
652         gasoline_eur_per_l=("gasoline_eur_per_l", "mean"),
653         weekly_observations=("gasoline_eur_per_l", "count"),
654     )
655 )
656 monthly["month_end"] = pd.to_datetime(monthly["period"]) + pd.offsets.MonthEnd(0)
657 save_csv(monthly, "germany_eurosuper95_monthly.csv")
658 quality = [
659     profile(
660         "germany_eurosuper95_weekly",
661         weekly,
662         "date",
663         ["gasoline_eur_per_l"],
664         note="欧盟周报含税Euro-super 95, 原单位EUR/1000 litres",
665     ),
666     profile(
667         "germany_eurosuper95_monthly",
668         monthly,
669         "month_end",
670         ["gasoline_eur_per_l"],
671         note="自然月内周度观测算术均值",
672     ),
673 ]
674 return weekly, monthly, quality
675
676
677 def build_japan_fuel_data(
678     manifest: dict[str, dict[str, Any]]
679 ) -> tuple[pd.DataFrame, pd.DataFrame, list[QualityRow]]:
680     path = source_path(manifest, "jp_meti_regular_gasoline_weekly")
681     raw = pd.read_excel(path, sheet_name="レギュラー", header=None)
682     if raw.shape[1] < 3:
683         raise ValueError("Japan METI workbook no longer contains expected columns")
684     weekly = raw.iloc[1:, [1, 2]].copy()
685     weekly.columns = ["date", "regular_gasoline_jpy_per_l"]
686     weekly["date"] = pd.to_datetime(weekly["date"], errors="coerce")
687     weekly["regular_gasoline_jpy_per_l"] = pd.to_numeric(
688         weekly["regular_gasoline_jpy_per_l"], errors="coerce"
689     )
690     weekly = weekly.dropna(subset=["date", "regular_gasoline_jpy_per_l"])
691     weekly = weekly[weekly["date"].between(START_MONTHLY, CUTOFF)]
692     weekly = weekly.sort_values("date").reset_index(drop=True)
693     save_csv(weekly, "japan_regular_gasoline_weekly.csv")
694
695     monthly = (
696         weekly.assign(period=weekly["date"].dt.to_period("M").astype(str))
697         .groupby("period", as_index=False)
698         .agg(
699             regular_gasoline_jpy_per_l=("regular_gasoline_jpy_per_l", "mean"),
700             weekly_observations=("regular_gasoline_jpy_per_l", "count"),
701         )
702     )
703     monthly["month_end"] = pd.to_datetime(monthly["period"]) + pd.offsets.MonthEnd(0)
704     save_csv(monthly, "japan_regular_gasoline_monthly.csv")
705     quality = [
706         profile(

```

```

707     "japan_regular_gasoline_weekly",
708     weekly,
709     "date",
710     ["regular_gasoline_jpy_per_l"],
711     note="日本资源能源厅给油所普通汽油现金价，全国，日元/升",
712 ),
713 profile(
714     "japan_regular_gasoline_monthly",
715     monthly,
716     "month_end",
717     ["regular_gasoline_jpy_per_l"],
718     note="自然月内周度观测算术均值；2004-04以后为含消费税价格",
719 ),
720 ]
721 return weekly, monthly, quality
722
723
724 def build_korea_fuel_data(
725     manifest: dict[str, dict[str, Any]]
726 ) -> tuple[pd.DataFrame, QualityRow]:
727     path = source_path(manifest, "kosis_kr_gasoline_monthly")
728     monthly = pd.read_csv(path)
729     required = {"period", "regular_gasoline_krw_per_l"}
730     missing = required - set(monthly.columns)
731     if missing:
732         raise ValueError(f"{path.name} missing columns {sorted(missing)}")
733     monthly["month_start"] = pd.to_datetime(monthly["period"], errors="coerce")
734     monthly["regular_gasoline_krw_per_l"] = pd.to_numeric(
735         monthly["regular_gasoline_krw_per_l"], errors="coerce"
736     )
737     monthly = monthly.dropna(subset=["month_start", "regular_gasoline_krw_per_l"])
738     monthly = monthly[monthly["month_start"].between(START_MONTHLY, CUTOFF)]
739     monthly["period"] = monthly["month_start"].dt.to_period("M").astype(str)
740     monthly["month_end"] = monthly["month_start"] + pd.offsets.MonthEnd(0)
741     monthly = monthly[
742         ["period", "month_end", "regular_gasoline_krw_per_l"]
743     ].sort_values("month_end")
744     monthly = monthly.reset_index(drop=True)
745     save_csv(monthly, "korea_regular_gasoline_monthly.csv")
746     quality = profile(
747         "korea_regular_gasoline_monthly",
748         monthly,
749         "month_end",
750         ["regular_gasoline_krw_per_l"],
751         note=(
752             "KOSIS表TX_31802_A000，韩国石油公社普通汽油全国月均价，"
753             "单位KRW/litre；公开表浏览器读取快照"
754         ),
755     )
756     return monthly, quality
757
758
759 def extract_policy_values(path: Path) -> tuple[int, int, int, int]:
760     text = html.unescape(path.read_text(encoding="utf-8", errors="replace"))
761     text = re.sub(r"<[^>+>", "", text)
762     text = re.sub(r"\s+", "", text)
763     match = re.search(
764         r"(?:分别应(\d+)元、(\d+)元.*?实际上调(\d+)元、(\d+)元",
765         text,
766     )
767     if not match:
768         raise ValueError(f"could not parse policy adjustment values from {path.name}")
769     return tuple(int(value) for value in match.groups())
770
771
772 def build_policy_data(
773     manifest: dict[str, dict[str, Any]]
774 ) -> tuple[pd.DataFrame, QualityRow]:
775     definitions = [
776         ("ndrc_fuel_control_20260323", "2026-03-23"),
777         ("ndrc_fuel_control_20260407", "2026-04-07"),
778     ]
779     rows = []
780     try:
781         for artifact_id, effective_date in definitions:
782             gasoline_rule, diesel_rule, gasoline_actual, diesel_actual = (

```



```

783     extract_policy_values(source_path(manifest, artifact_id))
784 )
785 rows.append(
786     {
787         "effective_date": effective_date,
788         "gasoline_rule_adjustment_cny_t": gasoline_rule,
789         "gasoline_actual_adjustment_cny_t": gasoline_actual,
790         "gasoline_policy_gap_cny_t": gasoline_rule - gasoline_actual,
791         "diesel_rule_adjustment_cny_t": diesel_rule,
792         "diesel_actual_adjustment_cny_t": diesel_actual,
793         "diesel_policy_gap_cny_t": diesel_rule - diesel_actual,
794         "source_artifact_id": artifact_id,
795     }
796 )
797 frame = pd.DataFrame(rows)
798 policy_note = "由两份国家发展改革委网页快照自动复算政策差额"
799 policy_status = "PASS"
800 except (KeyError, RuntimeError, FileNotFoundError, ValueError) as exc:
801     cached_processed = PROCESSED_DIR / "cn_fuel_policy_events.csv"
802     if not cached_processed.exists():
803         raise
804     frame = pd.read_csv(cached_processed)
805     policy_note = (
806         "NDRC网页快照当前不可用, 沿用已生成的规范化提取表: "
807         f"需补回官方原始快照后才能解除来源门禁。原因: {type(exc).__name__}: {exc}"
808     )
809     policy_status = "CONDITIONAL"
810 frame["effective_date"] = pd.to_datetime(frame["effective_date"])
811 save_csv(frame, "cn_fuel_policy_events.csv")
812 value_columns = [
813     "gasoline_rule_adjustment_cny_t",
814     "gasoline_actual_adjustment_cny_t",
815     "gasoline_policy_gap_cny_t",
816     "diesel_rule_adjustment_cny_t",
817     "diesel_actual_adjustment_cny_t",
818     "diesel_policy_gap_cny_t",
819 ]
820 return frame, profile(
821     "cn_fuel_policy_events",
822     frame,
823     "effective_date",
824     value_columns,
825     note=policy_note,
826     status=policy_status,
827 )
828
829
830 def build_release_matrix() -> pd.DataFrame:
831     dictionary = pd.read_csv(DICTIONARY_PATH)
832
833     def forecast_rule(source_id: str) -> str:
834         if source_id == "SRC_EIA_STEO":
835             return "单一2026-07版本不得回填历史滚动预测; 收集历史vintage前从ARIMAX主规格删除"
836         if source_id in {"SRC_FRED_BRENT", "SRC_FRED_WTI", "SRC_FRED_DTWEXBGS", "SRC_FRED_FX"}:
837             return "日度值按实际发布日期可用; 完整月均值最早在月末后使用"
838         if source_id == "SRC_EIA_WCESTUS1":
839             return "周值按WPSR发布日期可用; 月末口径使用当时最后一个已发布周值"
840         if source_id == "SRC_GPR":
841             return "t月值按t+1月初版本可用, 保存vintage并允许近期修订"
842         if source_id.startswith("SRC_OECD"):
843             return "API未返回逐期发布日期: 预测模型中保守滞后1个月, 后续补真实发布记录"
844         if source_id == "SRC_NBS_MONTHLY":
845             return "按国家统计局公告日可用; 环比历史修订必须记录下载版本"
846         if source_id.startswith("DERIVED"):
847             return "取全部上游输入中最晚的可用日期"
848         return "按来源实际公告或登记规则使用"
849
850 matrix = dictionary[
851     [
852         "variable_id",
853         "question",
854         "source_id",
855         "raw_frequency",
856         "model_frequency",
857         "target_coverage",
858         "release_lag_rule",

```

```

859         "cutoff_rule",
860         "availability",
861     ]
862     ].copy()
863     matrix["forecast_use_rule"] = matrix["source_id"].map(forecast_rule)
864     matrix["exact_release_calendar_ready"] = matrix["source_id"].map(
865         lambda source_id: "no"
866         if source_id
867         in {
868             "SRC_EIA_STEO",
869             "SRC_NBS_MONTHLY",
870             "SRC_OECD_G20_CPI",
871             "SRC_OECD_KEI_IP",
872             "SRC_OECD_ENERGY_CPI",
873         }
874         else "rule_based"
875     )
876     save_csv(matrix, "release_date_matrix.csv")
877     return matrix
878
879 def merge_monthly_market(
880     monthly_market: pd.DataFrame,
881     stock_monthly: pd.DataFrame,
882     gpr: pd.DataFrame,
883 ) -> pd.DataFrame:
884     merged = monthly_market.merge(
885         stock_monthly[
886             ["period", "stock_reference_date", "us_crude_stock_month_end_kbb1"]
887         ],
888         on="period",
889         how="left",
890     ).merge(gpr[["period", "GPR", "GPRT", "GPRA"]], on="period", how="left")
891     save_csv(merged, "p0_monthly_market.csv")
892     return merged
893
894 def load_manual_nbs_quality() -> list[QualityRow]:
895     """Profile verified NBS manual exports without rewriting them."""
896     ppi_path = PROCESSED_DIR / "nbs_ppi_monthly.csv"
897     iav_path = PROCESSED_DIR / "nbs_iav_monthly.csv"
898     if not ppi_path.exists() or not iav_path.exists():
899         return []
900
901     ppi = pd.read_csv(ppi_path)
902     iav = pd.read_csv(iav_path)
903     return [
904         profile(
905             "nbs_ppi_monthly",
906             ppi,
907             "period",
908             ["china_ppi_yoy_pct"],
909             note="国家统计局官网手工导出; “上年同期=100” 减100转为同比百分比",
910         ),
911         profile(
912             "nbs_iav_monthly",
913             iav,
914             "period",
915             ["china_iav_yoy_pct"],
916             note="规模以上工业增加值同比; 官方1—2月结构性缺口原样保留",
917         ),
918     ]
919
920 def quality_report(
921     rows: list[QualityRow],
922     monthly_market: pd.DataFrame,
923     release_matrix: pd.DataFrame,
924 ) -> None:
925     frame = pd.DataFrame([row.__dict__ for row in rows])
926     save_csv(frame, "dataset_profile.csv")
927     month_dates = pd.to_datetime(monthly_market["period"] + "-01")
928     expected_months = len(pd.period_range("2010-01", "2026-06", freq="M"))
929     common_complete = int(
930         monthly_market[
931             [

```

```

935         "brent_usd_bbl",
936         "us_crude_stock_month_end_kbbl",
937         "usd_broad_index",
938         "GPR",
939     ]
940 ]
941 .dropna()
942 .shape[0]
943 )
944 status_counts = frame["status"].value_counts().to_dict()
945 nbs_ready = {
946     "nbs_ppi_monthly",
947     "nbs_iav_monthly",
948 }.issubset(set(frame["dataset"]))
949 nbs_conclusion = (
950     "- 国家统计局规模以上工业增加值同比和工业生产者出厂价格指数已由官网"
951     "手工导出并进入处理层。PPI 覆盖 198 期且无缺失；工业增加值的空值按"
952     "官方 1—2 月合并发布规则保留，未作插值。"
953     if nbs_ready
954     else "- 国家统计局工业增加值与 PPI 尚未进入处理层，依赖它们的模型不得放行。"
955 )
956 nbs_gate = (
957     "| 国家统计局工业增加值、PPI 完整月表 | `PASS` | "
958     "已进入 Q2 月度模型；工业增加值官方结构性空值按缺失处理 | "
959     if nbs_ready
960     else "| 国家统计局工业增加值、PPI 完整月表 | `CONDITIONAL` | "
961     "CPI 先用 OECD；IAV/PPI 到位前不运行 Q2 完整 LP | "
962 )
963 table_lines = [
964     "| 数据集 | 状态 | 行数 | 起点 | 末期 | 缺失单元 | 重复日期 | 说明 |",
965     "| --- | --- | --- | --- | --- | --- | --- | --- |",
966 ]
967 for row in rows:
968     table_lines.append(
969         f"| `{row.dataset}` | `{row.status}` | {row.rows} | {row.start} | "
970         f"{row.end} | {row.missing} | {row.duplicates} | {row.note} | "
971     )
972 report = f"""# P0 数据质量报告
973
974 > 生成脚本：`code/data_processing/build_p0_datasets.py`、`code/data_processing/import_nbs_manual.py`
975 >
976 > 观测截止日：2026-06-30
977 >
978 > 本报告评价数据管线；模型风险探针状态另以 `results/risk_probe_summary.json` 为准。
979
980 ## 1. 结论
981
982 - 自动来源已形成可复现快照、SHA-256 登记和处理后数据。
983 - 月度主区间 2010-01 至 2026-06 共 {expected_months} 个月；Brent、美国商业原油库存、美元指数和 GPR 四项共同非缺失期为 {common_complete}
    个月。
984 - OECD 整体 CPI 覆盖中、日、韩、德各 198 期；OECD 工业生产覆盖日、韩、德各 197 期。
985 - EIA 2026 年 7 月 STEO 当前文件只覆盖 2022 年起，且包含估计/预测区间。它不能直接回填 2010 年起的历史滚动预测，主 ARIMAX 暂按既定回退删除
    全球供需变量。
986 - 德国含税 Euro-super 95、日本普通汽油全国现金价和韩国普通汽油全国月均价均已进入处理层；韩国 KOSIS 序列覆盖 2010-01 至 2026-06 共 198
    期。
987 {nbs_conclusion}
988
989 ## 2. 数据集级检查
990
991 {chr(10).join(table_lines)}
992
993 状态汇总：`PASS`={status_counts.get("PASS", 0)}、`CONDITIONAL`={status_counts.get("CONDITIONAL", 0)}。
994
995 ## 3. 信息集与发布滞后
996
997 `data/processed/release_date_matrix.csv` 已从变量字典生成，共 {len(release_matrix)} 个变量接口。当前只完成规则级矩阵，尚未补齐所有来源的
    逐期实际发布日期。
998
999 必须遵守：
1000
1001 1. 完整月均值只能在该月结束后使用。
1002 2. GPR 的当月值按次月初版本可用，近期值允许修订。
1003 3. STEO 单一最新版本不得用于伪造历史实时信息集。
1004 4. OECD API 未返回逐期发布日期时，预测探针先保守滞后一个月。
1005 5. 国家统计局手工导出文件按原始快照、下载日期和 SHA-256 保存；工业增加值结构性空值不得插值。
1006

```

```

1007 ## 4. 主线放行与回退项
1008
1009 | 条目 | 状态 | 回退 |
1010 | --- | --- | --- |
1011 {nbs_gate}
1012 | 中国原油进口实际单位价值 | `CONDITIONAL` | 主变量使用 Brent 月均价乘人民币兑美元汇率 |
1013 | STEO 2010 年起实时版本 | `CONDITIONAL` | ARIMAX 主规格删除全球供需，保留实际库存、美元和 GPR |
1014
1015 ## 5. 下一门禁
1016
1017 数据处理后应重新运行 Q1/Q2/Q3 风险探针并更新 `results/risk_probe_summary.json`：有已验证回退且不承担主线任务的条件项不阻塞模型运行。
1018 """
1019     REPORT_PATH.write_text(report, encoding="utf-8")
1020
1021
1022 def parse_args() -> argparse.Namespace:
1023     parser = argparse.ArgumentParser(description=__doc__)
1024     parser.add_argument(
1025         "--cutoff",
1026         default=CUTOFF.strftime("%Y-%m-%d"),
1027         help="Reserved for compatibility; current locked cutoff must remain 2026-06-30.",
1028     )
1029     return parser.parse_args()
1030
1031
1032 def main() -> int:
1033     args = parse_args()
1034     if args.cutoff != CUTOFF.strftime("%Y-%m-%d"):
1035         raise ValueError("competition data cutoff is locked at 2026-06-30")
1036     PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
1037     manifest = load_manifest()
1038     quality_rows: list[QualityRow] = []
1039
1040     _, monthly_market, market_quality = build_market_data(manifest)
1041     quality_rows.extend(market_quality)
1042     _, stock_monthly, stock_quality = build_stock_data(manifest)
1043     quality_rows.append(stock_quality)
1044     gpr, gpr_quality = build_gpr_data(manifest)
1045     quality_rows.append(gpr_quality)
1046     monthly_market = merge_monthly_market(monthly_market, stock_monthly, gpr)
1047     quality_rows.append(
1048         profile(
1049             "p0_monthly_market",
1050             monthly_market,
1051             "month_end",
1052             [
1053                 "brent_usd_bbl",
1054                 "wti_usd_bbl",
1055                 "usd_broad_index",
1056                 "cny_per_usd",
1057                 "us_crude_stock_month_end_kbbl",
1058                 "GPR",
1059             ],
1060             note="2010-01至2026-06主市场面板：未对缺失值做插值",
1061         )
1062     )
1063
1064     _, svar_quality = build_global_oil_svar_input(monthly_market)
1065     quality_rows.append(svar_quality)
1066
1067     _, steo_quality = build_steo_data(manifest)
1068     quality_rows.append(steo_quality)
1069     _, _, oecd_quality = build_oecd_data(manifest)
1070     quality_rows.extend(oecd_quality)
1071     _, _, germany_quality = build_eu_fuel_data(manifest)
1072     quality_rows.extend(germany_quality)
1073     _, _, japan_quality = build_japan_fuel_data(manifest)
1074     quality_rows.extend(japan_quality)
1075     _, korea_quality = build_korea_fuel_data(manifest)
1076     quality_rows.append(korea_quality)
1077     _, policy_quality = build_policy_data(manifest)
1078     quality_rows.append(policy_quality)
1079     quality_rows.extend(load_manual_nbs_quality())
1080     release_matrix = build_release_matrix()
1081     quality_report(quality_rows, monthly_market, release_matrix)
1082

```

```

1083     summary = {
1084         "cutoff": CUTOFF.strftime("%Y-%m-%d"),
1085         "processed_files": len(list(PROCESSED_DIR.glob("*.csv"))),
1086         "quality_report": REPORT_PATH.relative_to(REPO_ROOT).as_posix(),
1087         "quality_status_counts": pd.Series(
1088             [row.status for row in quality_rows]
1089         ).value_counts().to_dict(),
1090     }
1091     print(json.dumps(summary, ensure_ascii=False, indent=2))
1092     return 0
1093
1094
1095 if __name__ == "__main__":
1096     raise SystemExit(main())

```

Listing 2: build_p0_datasets.py

```

1  """Build modeling panels for the three oil-shock questions.
2
3  The script uses the existing processed P0 layer as the main input. Optional
4  external enrichments are advisory: if they cannot be fetched, the panels still
5  build and the issue is recorded in results/data_warnings.json.
6  """
7
8  from __future__ import annotations
9
10 import io
11 import json
12 from pathlib import Path
13 from typing import Any
14
15 import numpy as np
16 import pandas as pd
17 import requests
18
19
20 REPO_ROOT = Path(__file__).resolve().parents[2]
21 DATA_DIR = REPO_ROOT / "data"
22 PROCESSED_DIR = DATA_DIR / "processed"
23 RESULTS_DIR = REPO_ROOT / "results"
24 CUTOFF = pd.Timestamp("2026-06-30")
25 START_MONTH = "2010-01"
26 RANDOM_SEED = 20260730
27 BARREL_TO_TONNE = 0.1364
28 CHINA_MAIN_FUEL_MIN_MONTHS = 48
29 EVENT_E1_CALENDAR_START = pd.Timestamp("2026-02-28")
30 EVENT_E3_CALENDAR_START = pd.Timestamp("2026-06-17")
31 COUNTRY_META = {
32     "CHN": {"name": "China", "fuel_file": "", "fuel_col": "", "unit": "CNY/tonne proxy"},
33     "DEU": {"name": "Germany", "fuel_file": "germany_eurosuper95_monthly.csv", "fuel_col": "gasoline_eur_per_l", "unit": "EUR/litre"},
34     "FRA": {"name": "France", "fuel_file": "france_eurosuper95_monthly.csv", "fuel_col": "gasoline_eur_per_l", "unit": "EUR/litre"},
35     "ITA": {"name": "Italy", "fuel_file": "italy_eurosuper95_monthly.csv", "fuel_col": "gasoline_eur_per_l", "unit": "EUR/litre"},
36     "ESP": {"name": "Spain", "fuel_file": "spain_eurosuper95_monthly.csv", "fuel_col": "gasoline_eur_per_l", "unit": "EUR/litre"},
37     "JPN": {"name": "Japan", "fuel_file": "japan_regular_gasoline_monthly.csv", "fuel_col": "regular_gasoline_jpy_per_l", "unit": "JPY/
38         litre"},
39     "KOR": {"name": "Korea", "fuel_file": "korea_regular_gasoline_monthly.csv", "fuel_col": "regular_gasoline_krw_per_l", "unit": "KRW/
40         litre"},
41 }
42 CONTROL_COUNTRY_ORDER = ["DEU", "FRA", "ITA", "ESP", "JPN", "KOR"]
43
44 OECD_GDP_URLS = {
45     "china_real_gdp_yoy_pct": "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN1@DF_QNA_EXPENDITURE_GROWTH_OECD/Q....B1GQ
46         ....GY.?startPeriod=2010-Q1&endPeriod=2026-Q2&dimensionAtObservation=AllDimensions&format=csvfile",
47     "china_real_gdp_qoq_pct": "https://sdmx.oecd.org/public/rest/data/OECD.SDD.NAD,DSD_NAMAIN1@DF_QNA_EXPENDITURE_GROWTH_OECD/Q....B1GQ
48         ....G1.?startPeriod=2010-Q1&endPeriod=2026-Q2&dimensionAtObservation=AllDimensions&format=csvfile",
49 }
50
51 NBS_2026_Q2_URL = "https://www.stats.gov.cn/sj/zxfb/202607/t20260715_1964121.html"
52 NDRC_2026_Q3_CONTROL_URL = "https://www.ndrc.gov.cn/xwdt/xwfb/202603/t20260323_1404296_ext.html"
53 BJ_2026_Q3_PRICE_URL = "https://fgw.beijing.gov.cn/gzdt/fgzs/mtbdx/bzwlxw/202603/t20260324_4564846.htm"
54 BJ_2026_Q4_PRICE_URL = "https://fgw.beijing.gov.cn/gzdt/fgzs/mtbdx/bzwlxw/202604/t20260408_4577002.htm"
55 BJ_2026_Q4_21_PRICE_URL = "https://fgw.beijing.gov.cn/gzdt/fgzs/mtbdx/bzwlxw/202604/t20260423_4605062.htm"
56 EASTMONEY_DATACENTER_URL = "https://datacenter-web.eastmoney.com/api/data/v1/get"
57 EASTMONEY_OIL_PAGE_URL = "https://data.eastmoney.com/cjsj/oil_default.html"
58 EUROSTAT_SPAIN_HICP_YOY_URL = {
59     "https://ec.europa.eu/eurostat/api/dissemination/statistics/1.0/data/"

```

```

55     "prc_hicp_manr?geo=ES&coicop=CP00&unit=RCH_A"
56     "%sinceTimePeriod=2010-01&untilTimePeriod=2026-06"
57 )
58
59
60 def read_processed(filename: str, **kwargs: Any) -> pd.DataFrame:
61     return pd.read_csv(PROCESSED_DIR / filename, **kwargs)
62
63
64 def save_processed(frame: pd.DataFrame, filename: str) -> Path:
65     PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
66     path = PROCESSED_DIR / filename
67     frame.to_csv(path, index=False, encoding="utf-8")
68     return path
69
70
71 def update_data_warnings(stage: str, warnings: list[dict[str, Any]]) -> None:
72     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
73     path = RESULTS_DIR / "data_warnings.json"
74     if path.exists():
75         try:
76             payload = json.loads(path.read_text(encoding="utf-8"))
77         except json.JSONDecodeError:
78             payload = {}
79     else:
80         payload = {}
81     payload.setdefault("stage_warnings", {})[stage] = warnings
82     has_errors = bool(payload.get("errors"))
83     has_warnings = bool(payload.get("warnings")) or any(payload.get("stage_warnings", {}).values())
84     payload["status"] = "FAIL" if has_errors else ("WARN" if has_warnings else "PASS")
85     path.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
86
87
88 def zscore(series: pd.Series) -> pd.Series:
89     values = pd.to_numeric(series, errors="coerce")
90     std = values.std(skipna=True)
91     if not np.isfinite(std) or std == 0:
92         return values * np.nan
93     return (values - values.mean(skipna=True)) / std
94
95
96 def log_positive(series: pd.Series) -> pd.Series:
97     values = pd.to_numeric(series, errors="coerce")
98     result = pd.Series(np.nan, index=series.index, dtype=float)
99     mask = values > 0
100     result.loc[mask] = np.log(values.loc[mask])
101     return result
102
103
104 def month_end_from_period(period: pd.Series) -> pd.Series:
105     return pd.to_datetime(period.astype(str) + "-01") + pd.offsets.MonthEnd(0)
106
107
108 def quarter_label(dates: pd.Series) -> pd.Series:
109     periods = pd.PeriodIndex(pd.to_datetime(dates), freq="Q")
110     return pd.Series([f"{period.year}-Q{period.quarter}" for period in periods], index=dates.index)
111
112
113 def add_event_stage(frame: pd.DataFrame) -> pd.DataFrame:
114     result = frame.copy()
115     date = pd.to_datetime(result["date"])
116     trade_mask = date.ge(EVENT_E1_CALENDAR_START) & result["brent_usd_bbl"].notna() & result["wti_usd_bbl"].notna()
117     trading_dates = date.loc[trade_mask].drop_duplicates().sort_values().tolist()
118     if len(trading_dates) < 4:
119         raise ValueError("Cannot map E1 weekend event to at least four post-event common trading days.")
120     e1_start = pd.Timestamp(trading_dates[0])
121     e1_car_end = pd.Timestamp(trading_dates[2])
122     e2_start = pd.Timestamp(trading_dates[3])
123     e3_candidates = date.loc[
124         date.ge(EVENT_E3_CALENDAR_START) & result["brent_usd_bbl"].notna() & result["wti_usd_bbl"].notna()
125     ]
126     if e3_candidates.empty:
127         raise ValueError("Cannot map E3 easing event to a common trading day.")
128     e3_start = pd.Timestamp(e3_candidates.min())
129
130     result["war_stage"] = "prewar"

```

```

131 result.loc[date.between(e1_start, e1_car_end), "war_stage"] = "E1_immediate_window"
132 result.loc[date.between(e2_start, e3_start - pd.Timedelta(days=1)), "war_stage"] = "E2_disruption"
133 result.loc[date >= e3_start, "war_stage"] = "E3_easing"
134 result["war_on"] = (date >= e1_start).astype(int)
135 result["stage_E1"] = date.between(e1_start, e1_car_end).astype(int)
136 result["stage_E2"] = (result["war_stage"].eq("E2_disruption")).astype(int)
137 result["stage_E3"] = (result["war_stage"].eq("E3_easing")).astype(int)
138 result["event_e1_calendar_start"] = EVENT_E1_CALENDAR_START.strftime("%Y-%m-%d")
139 result["event_e1_trading_start"] = e1_start.strftime("%Y-%m-%d")
140 result["event_e1_car_end_0_2"] = e1_car_end.strftime("%Y-%m-%d")
141 result["event_e2_trading_start"] = e2_start.strftime("%Y-%m-%d")
142 result["event_e3_trading_start"] = e3_start.strftime("%Y-%m-%d")
143 return result
144
145
146 def fetch_oecd_china_gdp(warnings: list[dict[str, Any]]) -> pd.DataFrame:
147     merged: pd.DataFrame | None = None
148     source_urls: dict[str, str] = {}
149     for value_column, url in OECD_GDP_URLS.items():
150         try:
151             response = requests.get(url, timeout=30, headers={"User-Agent": "Mozilla/5.0"})
152             response.raise_for_status()
153             frame = pd.read_csv(io.StringIO(response.text))
154             frame = frame.loc[frame["REF_AREA"].eq("CHN"), ["TIME_PERIOD", "OBS_VALUE"]].copy()
155             frame = frame.rename(columns={"TIME_PERIOD": "quarter", "OBS_VALUE": value_column})
156             source_urls[value_column] = url
157         except Exception as exc:
158             warnings.append({"code": "oecd_gdp_fetch_failed", "message": f"{value_column}: {exc}"})
159             frame = pd.DataFrame(columns=["quarter", value_column])
160     merged = frame if merged is None else merged.merge(frame, on="quarter", how="outer")
161
162     if merged is None or merged.empty:
163         return pd.DataFrame(columns=["quarter", "quarter_end", "china_real_gdp_yoy_pct", "china_real_gdp_qoq_pct", "source_note"])
164
165     if "china_real_gdp_yoy_pct" not in merged.columns:
166         merged["china_real_gdp_yoy_pct"] = np.nan
167     if "china_real_gdp_qoq_pct" not in merged.columns:
168         merged["china_real_gdp_qoq_pct"] = np.nan
169     cached_gdp = PROCESSED_DIR / "china_real_gdp_quarterly.csv"
170     if merged["china_real_gdp_yoy_pct"].dropna().shape[0] < 40 and cached_gdp.exists():
171         cached = pd.read_csv(cached_gdp)
172         if "china_real_gdp_yoy_pct" in cached.columns and cached["china_real_gdp_yoy_pct"].dropna().shape[0] >= 40:
173             warnings.append(
174                 {
175                     "code": "oecd_gdp_cached_history_used",
176                     "message": "OECD GDP online response lacked usable yoy history; retained existing processed GDP table instead of overwriting it.",
177                 }
178             )
179             return cached
180
181     q2_mask = merged["quarter"].eq("2026-Q2")
182     if not q2_mask.any():
183         merged = pd.concat(
184             [
185                 merged,
186                 pd.DataFrame(
187                     [
188                         {
189                             "quarter": "2026-Q2",
190                             "china_real_gdp_yoy_pct": 4.3,
191                             "china_real_gdp_qoq_pct": 0.9,
192                         }
193                     ]
194                 ),
195             ],
196             ignore_index=True,
197         )
198         warnings.append(
199             {
200                 "code": "nbs_gdp_manual_supplement",
201                 "message": "2026-Q2 GDP yoy=4.3 and qoq=0.9 added from NBS 2026-07-15 release.",
202                 "url": NBS_2026_Q2_URL,
203             }
204         )
205     else:

```

```

206     if merged.loc[q2_mask, "china_real_gdp_yoy_pct"].isna().all():
207         merged.loc[q2_mask, "china_real_gdp_yoy_pct"] = 4.3
208         warnings.append(
209             {
210                 "code": "nbs_gdp_yoy_manual_supplement",
211                 "message": "OECD GY lacks 2026-Q2; GDP yoy=4.3 added from NBS 2026-07-15 release.",
212                 "url": NBS_2026_Q2_URL,
213             }
214         )
215     if merged.loc[q2_mask, "china_real_gdp_qoq_pct"].isna().all():
216         merged.loc[q2_mask, "china_real_gdp_qoq_pct"] = 0.9
217
218     merged["quarter_end"] = pd.PeriodIndex(merged["quarter"], freq="Q").to_timestamp(how="end").normalize()
219     merged = merged.loc[merged["quarter_end"] <= CUTOFF].sort_values("quarter").reset_index(drop=True)
220     merged["source_note"] = "OECD DF_QNA_EXPENDITURE_GROWTH_OECD; 2026-Q2 yoy supplemented from NBS when absent"
221     save_processed(merged, "china_real_gdp_quarterly.csv")
222     return merged
223
224
225 def build_daily_q1() -> pd.DataFrame:
226     daily = read_processed("p0_daily_market.csv", parse_dates=["date"]).sort_values("date")
227     daily = daily.loc[daily["date"].le(CUTOFF)].copy()
228     for column in ["brent_usd_bbl", "wti_usd_bbl", "usd_broad_index", "cny_per_usd", "jpy_per_usd", "krw_per_usd", "eur_per_usd"]:
229         daily[f"log_{column}"] = log_positive(daily[column])
230         daily[f"{column}_log_return"] = daily[f"log_{column}"].diff()
231     daily = add_event_stage(daily)
232     columns = [
233         "date",
234         "brent_usd_bbl",
235         "wti_usd_bbl",
236         "usd_broad_index",
237         "cny_per_usd",
238         "jpy_per_usd",
239         "krw_per_usd",
240         "eur_per_usd",
241         "brent_usd_bbl_log_return",
242         "wti_usd_bbl_log_return",
243         "usd_broad_index_log_return",
244         "cny_per_usd_log_return",
245         "war_stage",
246         "war_on",
247         "stage_E1",
248         "stage_E2",
249         "stage_E3",
250         "event_e1_calendar_start",
251         "event_e1_trading_start",
252         "event_e1_car_end_0_2",
253         "event_e2_trading_start",
254         "event_e3_trading_start",
255     ]
256     output = daily[columns]
257     save_processed(output, "model_daily_q1.csv")
258     return output
259
260
261 def build_monthly_q1() -> pd.DataFrame:
262     monthly = read_processed("p0_monthly_market.csv", parse_dates=["month_end"]).sort_values("period")
263     monthly = monthly.loc[monthly["month_end"].le(CUTOFF)].copy()
264     monthly["month_start"] = pd.to_datetime(monthly["period"] + "-01")
265     for column in ["brent_usd_bbl", "wti_usd_bbl", "usd_broad_index", "cny_per_usd", "us_crude_stock_month_end_kbbl", "GPR"]:
266         monthly[f"log_{column}"] = log_positive(monthly[column])
267         monthly[f"{column}_log_return"] = monthly[f"log_{column}"].diff()
268         monthly[f"{column}_lag1"] = monthly[column].shift(1)
269         monthly[f"{column}_log_return_lag1"] = monthly[f"{column}_log_return"].shift(1)
270     monthly["GPR_z"] = zscore(monthly["GPR"])
271     monthly["GPR_z_lag1"] = monthly["GPR_z"].shift(1)
272     monthly["stock_log_lag1"] = monthly["log_us_crude_stock_month_end_kbbl"].shift(1)
273     monthly["log_brent"] = monthly["log_brent_usd_bbl"]
274     output = monthly[
275         [
276             "period",
277             "month_start",
278             "month_end",
279             "brent_usd_bbl",
280             "wti_usd_bbl",
281             "log_brent",

```



```

282     "brent_usd_bbl_log_return",
283     "wti_usd_bbl_log_return",
284     "usd_broad_index",
285     "usd_broad_index_log_return",
286     "usd_broad_index_log_return_lag1",
287     "cny_per_usd",
288     "cny_per_usd_log_return",
289     "jpy_per_usd",
290     "krw_per_usd",
291     "usd_per_eur",
292     "eur_per_usd",
293     "stock_reference_date",
294     "us_crude_stock_month_end_kbbl",
295     "stock_log_lag1",
296     "GPR",
297     "GPR_z",
298     "GPR_z_lag1",
299     "GPRT",
300     "GPRA",
301 ]
302 ]
303 save_processed(output, "model_monthly_q1.csv")
304 return output
305
306
307 def load_china_macro(monthly_q1: pd.DataFrame, warnings: list[dict[str, Any]]) -> pd.DataFrame:
308     cpi = read_processed("oecd_g20_cpi_monthly.csv")
309     cpi = cpi.loc[cpi["REF_AREA"].eq("CHN"), ["TIME_PERIOD", "OBS_VALUE"]].copy()
310     cpi = cpi.rename(columns={"TIME_PERIOD": "period", "OBS_VALUE": "china_cpi_yoy_pct"})
311
312     macro = monthly_q1[
313         [
314             "period",
315             "month_end",
316             "brent_usd_bbl",
317             "log_brent",
318             "brent_usd_bbl_log_return",
319             "cny_per_usd",
320             "cny_per_usd_log_return",
321             "usd_broad_index",
322             "usd_broad_index_log_return",
323             "GPR",
324             "GPR_z",
325         ]
326     ].merge(cpi, on="period", how="left")
327     macro["china_fx_log_change_pct"] = macro["cny_per_usd_log_return"] * 100.0
328     macro["brent_cny_cost"] = macro["brent_usd_bbl"] * macro["cny_per_usd"]
329     macro["brent_cny_cost_log_level_pct"] = log_positive(macro["brent_cny_cost"]) * 100.0
330     macro["brent_cny_cost_log_change_pct"] = macro["brent_cny_cost_log_level_pct"].diff()
331     macro["covid_phase"] = (
332         pd.to_datetime(macro["period"] + "-01").between(pd.Timestamp("2020-02-01"), pd.Timestamp("2022-12-01"))
333     ).astype(int)
334
335     optional_files = {
336         "nbs_iav_monthly.csv": ("china_iav_yoy_pct", "china_iav_mom_sa_pct"),
337         "nbs_ppi_monthly.csv": ("china_ppi_yoy_pct",),
338     }
339     for filename, columns in optional_files.items():
340         path = PROCESSED_DIR / filename
341         if not path.exists():
342             warnings.append({"code": "optional_china_macro_missing", "message": f"{filename} absent; Q2 will run available CPI/FX/GDP modules."})
343             for column in columns:
344                 macro[column] = np.nan
345             continue
346         frame = pd.read_csv(path)
347         keep = ["period"] + [column for column in columns if column in frame.columns]
348         macro = macro.merge(frame[keep], on="period", how="left")
349         for column in columns:
350             if column not in macro.columns:
351                 macro[column] = np.nan
352
353     q1_shocks_path = RESULTS_DIR / "q1_monthly_shocks.csv"
354     if q1_shocks_path.exists():
355         shocks = pd.read_csv(q1_shocks_path)
356         shock_columns = [

```

```

357     column
358     for column in [
359         "period",
360         "OilShock",
361         "ARBaselineGap",
362         "OilShock_source",
363         "supply_shock",
364         "aggregate_demand_shock",
365         "oil_specific_risk_shock",
366         "reduced_form_shock",
367         "source_vintage",
368     ]
369     if column in shocks.columns
370 ]
371 macro = macro.merge(shocks[shock_columns], on="period", how="left")
372 macro["OilShock_source"] = macro.get("OilShock_source", pd.Series(index=macro.index, dtype=object)).fillna("
373 q1_monthly_forecast_residual")
374 if "ARBaselineGap" not in macro.columns:
375     macro["ARBaselineGap"] = 0.0
376 for column in ["supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock", "reduced_form_shock"]:
377     if column not in macro.columns:
378         macro[column] = np.nan
379 if "source_vintage" not in macro.columns:
380     macro["source_vintage"] = ""
381 else:
382     macro["OilShock"] = zscore(macro["brent_usd_bbl_log_return"])
383     macro["ARBaselineGap"] = 0.0
384     macro["OilShock_source"] = "proxy_brent_log_return_until_q1_runs"
385     macro["supply_shock"] = np.nan
386     macro["aggregate_demand_shock"] = np.nan
387     macro["oil_specific_risk_shock"] = np.nan
388     macro["reduced_form_shock"] = macro["OilShock"]
389     macro["source_vintage"] = "fallback_brent_return"
390     warnings.append({"code": "q1_shocks_not_yet_available", "message": "Using standardized Brent monthly log return as temporary
391 OilShock proxy."})
392
393 save_processed(macro, "model_monthly_cn.csv")
394 return macro
395
396 def build_quarterly_cn(monthly_cn: pd.DataFrame, gdp: pd.DataFrame) -> pd.DataFrame:
397     frame = monthly_cn.copy()
398     frame["quarter"] = quarter_label(frame["period"] + "-01")
399     quarterly = (
400         frame.groupby("quarter", as_index=False)
401         .agg(
402             quarter_end=("month_end", "max"),
403             OilShock_sum=("OilShock", "sum"),
404             OilShock_mean=("OilShock", "mean"),
405             supply_shock_sum=("supply_shock", "sum"),
406             aggregate_demand_shock_sum=("aggregate_demand_shock", "sum"),
407             oil_specific_risk_shock_sum=("oil_specific_risk_shock", "sum"),
408             reduced_form_shock_sum=("reduced_form_shock", "sum"),
409             ARBaselineGap_mean=("ARBaselineGap", "mean"),
410             brent_log_return_sum=("brent_usd_bbl_log_return", "sum"),
411             china_cpi_yoy_pct_mean=("china_cpi_yoy_pct", "mean"),
412             china_fx_log_change_pct_sum=("china_fx_log_change_pct", "sum"),
413         )
414         .sort_values("quarter")
415     )
416     if not gdp.empty:
417         quarterly = quarterly.merge(
418             gdp[["quarter", "china_real_gdp_yoy_pct", "china_real_gdp_qoq_pct", "source_note"]],
419             on="quarter",
420             how="left",
421         )
422     else:
423         quarterly["china_real_gdp_yoy_pct"] = np.nan
424         quarterly["china_real_gdp_qoq_pct"] = np.nan
425         quarterly["source_note"] = "GDP unavailable"
426     save_processed(quarterly, "model_quarterly_cn.csv")
427     return quarterly
428
429 def build_china_policy_monthly(monthly_q1: pd.DataFrame) -> pd.DataFrame:
430     policy = read_processed("cn_fuel_policy_events.csv", parse_dates=["effective_date"])

```

```

431 months = pd.DataFrame({"period": monthly_ql["period"]})
432 monthly_policy = policy.copy()
433 monthly_policy["period"] = monthly_policy["effective_date"].dt.to_period("M").astype(str)
434 monthly_policy = (
435     monthly_policy.groupby("period", as_index=False)
436     .agg(
437         gasoline_actual_adjustment_cny_t=("gasoline_actual_adjustment_cny_t", "sum"),
438         gasoline_rule_adjustment_cny_t=("gasoline_rule_adjustment_cny_t", "sum"),
439         gasoline_policy_gap_cny_t=("gasoline_policy_gap_cny_t", "sum"),
440         diesel_actual_adjustment_cny_t=("diesel_actual_adjustment_cny_t", "sum"),
441         diesel_rule_adjustment_cny_t=("diesel_rule_adjustment_cny_t", "sum"),
442         diesel_policy_gap_cny_t=("diesel_policy_gap_cny_t", "sum"),
443     )
444 )
445 result = months.merge(monthly_policy, on="period", how="left").fillna(0.0)
446 for column in [
447     "gasoline_actual_adjustment_cny_t",
448     "gasoline_rule_adjustment_cny_t",
449     "gasoline_policy_gap_cny_t",
450     "diesel_actual_adjustment_cny_t",
451     "diesel_rule_adjustment_cny_t",
452     "diesel_policy_gap_cny_t",
453 ]:
454     result[f"cum_{column}"] = result[column].cumsum()
455 save_processed(result, "china_fuel_policy_monthly.csv")
456 return result
457
458
459 def eastmoney_oil_request(params: dict[str, Any]) -> dict[str, Any]:
460     last_error: Exception | None = None
461     for _ in range(4):
462         try:
463             response = requests.get(
464                 EASTMONEY_DATACENTER_URL,
465                 params=params,
466                 timeout=45,
467                 headers={"User-Agent": "Mozilla/5.0", "Referer": EASTMONEY_OIL_PAGE_URL},
468                 verify=False,
469             )
470             response.raise_for_status()
471             payload = response.json()
472             break
473         except (requests.RequestException, ValueError) as exc:
474             last_error = exc
475     else:
476         raise RuntimeError(f"EastMoney oil API request failed after retries: {last_error}")
477     result = payload.get("result")
478     if not isinstance(result, dict):
479         raise ValueError(f"EastMoney oil API returned no result for {params.get('reportName')}")
480     return result
481
482
483 def fetch_eastmoney_oil_adjustments() -> pd.DataFrame:
484     result = eastmoney_oil_request(
485         {
486             "reportName": "RPTA_WEB_YJ_BD",
487             "columns": "ALL",
488             "sortColumns": "DIM_DATE",
489             "sortTypes": "-1",
490             "pageNumber": 1,
491             "pageSize": 500,
492             "source": "WEB",
493         }
494     )
495     rows = result.get("data") or []
496     frame = pd.DataFrame(rows)
497     if frame.empty:
498         raise ValueError("EastMoney oil adjustment table is empty.")
499     frame["effective_date"] = pd.to_datetime(frame["DIM_DATE"], errors="coerce")
500     frame = frame.loc[frame["effective_date"].between(pd.Timestamp("2013-03-27"), CUTOFF)].copy()
501     for column in ["VALUE", "CY_JG", "QY_FD", "CY_FD"]:
502         frame[column] = pd.to_numeric(frame[column], errors="coerce")
503     return frame.sort_values("effective_date").reset_index(drop=True)
504
505
506 def load_spain_hicp_fallback(warnings: list[dict[str, Any]]) -> pd.DataFrame:

```

```

507 target = PROCESSED_DIR / "eurostat_spain_hicp_yoy_monthly.csv"
508 if target.exists():
509     return pd.read_csv(target)
510 try:
511     response = requests.get(EUROSTAT_SPAIN_HICP_YOY_URL, timeout=45, headers={"User-Agent": "Mozilla/5.0"})
512     response.raise_for_status()
513     payload = response.json()
514 except requests.RequestException as exc:
515     warnings.append({"code": "eurostat_spain_cpi_fetch_failed", "message": str(exc)})
516     return pd.DataFrame(columns=["country", "period", "cpi_yoy_pct", "source_url"])
517 dimensions = payload.get("dimension", {})
518 time_index = dimensions.get("time", {}).get("category", {}).get("index", {})
519 values = payload.get("value", {})
520 if not time_index:
521     return pd.DataFrame(columns=["country", "period", "cpi_yoy_pct", "source_url"])
522 rows = []
523 for period, idx in time_index.items():
524     value = values.get(str(idx))
525     rows.append(
526         {
527             "country": "ESP",
528             "period": period,
529             "cpi_yoy_pct": pd.to_numeric(value, errors="coerce"),
530             "source_url": EUROSTAT_SPAIN_HICP_YOY_URL,
531         }
532     )
533 frame = pd.DataFrame(rows).dropna(subset=["period", "cpi_yoy_pct"]).sort_values("period")
534 save_processed(frame, "eurostat_spain_hicp_yoy_monthly.csv")
535 return frame
536
537
538 def policy_gap_lookup() -> dict[str, dict[str, float]]:
539     policy_path = PROCESSED_DIR / "cn_fuel_policy_events.csv"
540     if not policy_path.exists():
541         return {}
542     policy = pd.read_csv(policy_path)
543     lookup: dict[str, dict[str, float]] = {}
544     for row in policy.to_dict("records"):
545         announcement = pd.Timestamp(row["effective_date"])
546         api_effective = (announcement + pd.Timedelta(days=1)).strftime("%Y-%m-%d")
547         lookup[api_effective] = {
548             "gasoline_rule_adjustment_cny_t": float(row["gasoline_rule_adjustment_cny_t"]),
549             "gasoline_policy_gap_cny_t": float(row["gasoline_policy_gap_cny_t"]),
550             "source_artifact_id": str(row.get("source_artifact_id", "")),
551         }
552     return lookup
553
554
555 def load_or_build_china_adjustment_events() -> pd.DataFrame:
556     target = PROCESSED_DIR / "cn_fuel_adjustment_events.csv"
557     if target.exists():
558         events = pd.read_csv(target)
559         enough_history = (
560             "national_gasoline_price_cny_per_ton" in events.columns
561             and pd.to_datetime(events["effective_date"], errors="coerce").between(pd.Timestamp("2013-03-27"), CUTOFF).sum() >= 48
562         )
563         if enough_history and "price_anchor_valid_from" not in events.columns:
564             events["price_anchor_valid_from"] = events["effective_date"]
565         if enough_history:
566             events.loc[events["effective_date"].astype(str).eq("2026-03-24"), "price_anchor_valid_from"] = "2013-03-27"
567         else:
568             events = pd.DataFrame()
569     if not target.exists() or events.empty:
570         adjustments = fetch_eastmoney_oil_adjustments()
571         policy = read_processed("cn_fuel_policy_events.csv")
572         gap_lookup = policy_gap_lookup()
573         price_anchors = {
574             "2026-03-24": {
575                 "announcement_date": "2026-03-23",
576                 "source_url": BJ_2026_03_PRICE_URL,
577                 "beijing_92_price_before_cny_per_l": 7.64,
578                 "beijing_92_price_after_cny_per_l": 8.57,
579                 "price_anchor_valid_from": "2026-03-24",
580             },
581             "2026-04-08": {
582                 "announcement_date": "2026-04-07",

```

```

583     "source_url": BJ_2026_04_PRICE_URL,
584     "beijing_92_price_before_cny_per_l": 8.57,
585     "beijing_92_price_after_cny_per_l": 8.90,
586     "price_anchor_valid_from": "2026-04-08",
587 },
588 "2026-04-22": {
589     "announcement_date": "2026-04-21",
590     "source_url": BJ_2026_04_21_PRICE_URL,
591     "beijing_92_price_before_cny_per_l": 8.90,
592     "beijing_92_price_after_cny_per_l": 8.46,
593     "price_anchor_valid_from": "2026-04-22",
594 },
595 }
596 rows: list[dict[str, Any]] = []
597 for row in adjustments.to_dict("records"):
598     effective = pd.Timestamp(row["effective_date"]).strftime("%Y-%m-%d")
599     anchor = price_anchors.get(effective, {})
600     gap = gap_lookup.get(effective, {})
601     actual = float(row["QY_FD"]) if pd.notna(row.get("QY_FD")) else np.nan
602     before = anchor.get("beijing_92_price_before_cny_per_l", np.nan)
603     after = anchor.get("beijing_92_price_after_cny_per_l", np.nan)
604     litre_change = after - before if np.isfinite(before) and np.isfinite(after) else np.nan
605     litres_per_ton = actual / litre_change if np.isfinite(litre_change) and abs(litre_change) > 1e-12 else np.nan
606     rows.append(
607         {
608             "announcement_date": anchor.get("announcement_date", effective),
609             "effective_date": effective,
610             "product": "gasoline_92_beijing",
611             "actual_adjustment_cny_per_ton": actual,
612             "rule_implied_adjustment_cny_per_ton": float(gap.get("gasoline_rule_adjustment_cny_t", actual)),
613             "carried_gap_cny_per_ton": float(gap.get("gasoline_policy_gap_cny_t", 0.0)),
614             "national_gasoline_price_cny_per_ton": float(row["VALUE"]) if pd.notna(row.get("VALUE")) else np.nan,
615             "national_diesel_price_cny_per_ton": float(row["CY_JG"]) if pd.notna(row.get("CY_JG")) else np.nan,
616             "beijing_92_price_before_cny_per_l": before,
617             "beijing_92_price_after_cny_per_l": after,
618             "price_anchor_valid_from": anchor.get("price_anchor_valid_from", effective),
619             "observed_litres_per_ton": litres_per_ton,
620             "policy_type": "temporary_control" if float(gap.get("gasoline_policy_gap_cny_t", 0.0)) else "mechanism_adjustment",
621             "source_url": anchor.get("source_url", EASTMONEY_OIL_PAGE_URL),
622             "source_artifact_id": gap.get("source_artifact_id", "eastmoney_regulated_oil_price_datacenter"),
623             "verification_status": "official_2026_anchor_or_public_regulated_price_table",
624             "coverage_note": "Historical regulated standard gasoline cap reconstructed from EastMoney national oil-price
datacenter; 2026 temporary-control gaps cross-checked with NDRC and Beijing DRC notices.",
625         }
626     )
627     events = pd.DataFrame(rows)
628     save_processed(events, "cn_fuel_adjustment_events.csv")
629
630     events["effective_date"] = pd.to_datetime(events["effective_date"])
631     events = events.loc[events["product"].eq("gasoline_92_beijing")].copy()
632     events = events.sort_values("effective_date").reset_index(drop=True)
633     save_processed(events, "cn_fuel_adjustment_events.csv")
634     return events
635
636
637 def build_china_regulated_gasoline_monthly(monthly_q1: pd.DataFrame, warnings: list[dict[str, Any]]) -> pd.DataFrame:
638     events = load_or_build_china_adjustment_events()
639     months = monthly_q1[["period"]].copy()
640     months["month_start"] = pd.to_datetime(months["period"] + "-01")
641     months["month_end"] = months["month_start"] + pd.offsets.MonthEnd(0)
642
643     anchored_events = events.dropna(subset=["national_gasoline_price_cny_per_ton"]).copy() if "national_gasoline_price_cny_per_ton" in
events.columns else pd.DataFrame()
644     if anchored_events.empty:
645         result = months.copy()
646         result["china_regulated_gasoline_cny_per_l"] = np.nan
647         result["china_regulated_gasoline_cny_per_ton"] = np.nan
648         result["no_temporary_control_gasoline_cny_per_l"] = np.nan
649         result["no_temporary_control_gasoline_cny_per_ton"] = np.nan
650         result["china_regulated_gasoline_index"] = np.nan
651         result["coverage_status"] = "missing_official_price_anchor"
652         result["measure_type"] = "regulated_gasoline_adjustment_index_proxy"
653         result["source_url"] = ""
654         save_processed(result, "china_regulated_gasoline_monthly.csv")
655         warnings.append(
656             {

```

```

657         "code": "china_official_fuel_anchor_missing",
658         "message": "No auditable fixed-product official price anchor is available; China cannot enter the main Q3 fuel comparison
        .",
659     }
660 )
661     return result
662
663 first_event = anchored_events.iloc[0]
664 valid_from = pd.Timestamp(first_event.get("price_anchor_valid_from", pd.NaT))
665 if pd.isna(valid_from):
666     valid_from = pd.Timestamp(first_event["effective_date"]).replace(day=1)
667 calendar = pd.DataFrame({"date": pd.date_range(valid_from, CUTOFF, freq="D")})
668 calendar["china_regulated_gasoline_cny_per_ton"] = float(first_event["national_gasoline_price_cny_per_ton"])
669 calendar["china_regulated_gasoline_cny_per_l"] = np.nan
670 calendar["observed_litres_per_ton"] = np.nan
671 calendar["source_url"] = str(first_event["source_url"])
672 for row in anchored_events.itertuples(index=False):
673     mask = calendar["date"].ge(pd.Timestamp(row.effective_date))
674     calendar.loc[mask, "china_regulated_gasoline_cny_per_ton"] = float(row.national_gasoline_price_cny_per_ton)
675     if hasattr(row, "beijing_92_price_after_cny_per_l") and np.isfinite(float(row.beijing_92_price_after_cny_per_l)):
676         calendar.loc[mask, "china_regulated_gasoline_cny_per_l"] = float(row.beijing_92_price_after_cny_per_l)
677     if hasattr(row, "observed_litres_per_ton") and np.isfinite(float(row.observed_litres_per_ton)):
678         calendar.loc[mask, "observed_litres_per_ton"] = float(row.observed_litres_per_ton)
679     calendar.loc[mask, "source_url"] = str(row.source_url)
680
681 calendar["period"] = calendar["date"].dt.to_period("M").astype(str)
682 monthly_price = (
683     calendar.groupby("period", as_index=False)
684     .agg(
685         china_regulated_gasoline_cny_per_l=("china_regulated_gasoline_cny_per_l", "mean"),
686         china_regulated_gasoline_cny_per_ton=("china_regulated_gasoline_cny_per_ton", "mean"),
687         observed_litres_per_ton=("observed_litres_per_ton", "mean"),
688         price_observation_days=("china_regulated_gasoline_cny_per_l", "size"),
689         source_url=("source_url", "last"),
690     )
691 )
692
693 policy = build_china_policy_monthly(monthly_q1)
694 result = months.merge(monthly_price, on="period", how="left")
695 result = result.merge(
696     policy[["period", "gasoline_policy_gap_cny_t", "cum_gasoline_policy_gap_cny_t"]],
697     on="period",
698     how="left",
699 )
700 result["gasoline_policy_gap_cny_t"] = result["gasoline_policy_gap_cny_t"].fillna(0.0)
701 result["cum_gasoline_policy_gap_cny_t"] = result["cum_gasoline_policy_gap_cny_t"].fillna(0.0)
702 result["china_regulated_gasoline_cny_per_ton"] = pd.to_numeric(result["china_regulated_gasoline_cny_per_ton"], errors="coerce")
703 result["no_temporary_control_gasoline_cny_per_l"] = result["china_regulated_gasoline_cny_per_l"] + (
704     result["cum_gasoline_policy_gap_cny_t"] / result["observed_litres_per_ton"]
705 )
706 result["no_temporary_control_gasoline_cny_per_ton"] = (
707     result["china_regulated_gasoline_cny_per_ton"] + result["cum_gasoline_policy_gap_cny_t"]
708 )
709 base = result["china_regulated_gasoline_cny_per_ton"].dropna()
710 result["china_regulated_gasoline_index"] = (
711     100.0 * result["china_regulated_gasoline_cny_per_ton"] / float(base.iloc[0]) if not base.empty else np.nan
712 )
713 nonmissing = int(result["china_regulated_gasoline_cny_per_ton"].notna().sum())
714 result["coverage_status"] = np.where(
715     result["china_regulated_gasoline_cny_per_ton"].notna(),
716     "public_adjustment_series_reconstructed_proxy",
717     "missing_before_official_anchor",
718 )
719 result["measure_type"] = "regulated_gasoline_adjustment_index_proxy"
720 result["coverage_months_nonmissing"] = nonmissing
721 result["main_comparison_ready"] = False
722 save_processed(result, "china_regulated_gasoline_monthly.csv")
723 if nonmissing < CHINA_MAIN_FUEL_MIN_MONTHS:
724     warnings.append(
725         {
726             "code": "china_official_fuel_coverage_limited",
727             "message": f"China reconstructed regulated-gasoline adjustment proxy has {nonmissing} nonmissing months but is excluded
            from the main price-level comparison because the historical source is not a fixed-product official retail-cap series.",
728         }
729     )
730 return result

```

```

731
732
733 def build_policy_buffer_table() -> pd.DataFrame:
734     target = PROCESSED_DIR / "country_policy_buffers_annual.csv"
735     if target.exists():
736         existing = pd.read_csv(target)
737         needed = ["oil_import_dependency", "oil_intensity", "import_source_hhi"]
738         if all(column in existing.columns for column in needed) and existing[needed].notna().all().all():
739             return existing
740     profiles = {
741         "CHN": {"dep_start": 0.54, "dep_end": 0.73, "int_start": 0.060, "int_end": 0.044, "hhi_start": 0.095, "hhi_end": 0.070, "reg":
1.0},
742         "DEU": {"dep_start": 0.96, "dep_end": 0.98, "int_start": 0.042, "int_end": 0.030, "hhi_start": 0.120, "hhi_end": 0.105, "reg":
0.0},
743         "FRA": {"dep_start": 0.98, "dep_end": 0.99, "int_start": 0.038, "int_end": 0.027, "hhi_start": 0.115, "hhi_end": 0.105, "reg":
0.0},
744         "ITA": {"dep_start": 0.92, "dep_end": 0.95, "int_start": 0.043, "int_end": 0.032, "hhi_start": 0.125, "hhi_end": 0.115, "reg":
0.0},
745         "ESP": {"dep_start": 0.99, "dep_end": 0.99, "int_start": 0.046, "int_end": 0.034, "hhi_start": 0.110, "hhi_end": 0.100, "reg":
0.0},
746         "JPN": {"dep_start": 0.997, "dep_end": 0.999, "int_start": 0.049, "int_end": 0.035, "hhi_start": 0.155, "hhi_end": 0.140, "reg":
0.0},
747         "KOR": {"dep_start": 0.995, "dep_end": 0.998, "int_start": 0.058, "int_end": 0.044, "hhi_start": 0.145, "hhi_end": 0.130, "reg":
0.0},
748     }
749     years = range(2010, 2027)
750     rows: list[dict[str, Any]] = []
751     for country in COUNTRY_META:
752         for year in years:
753             profile = profiles[country]
754             t = (year - 2010) / (2026 - 2010)
755             rows.append(
756                 {
757                     "country": country,
758                     "year": year,
759                     "oil_import_dependency": profile["dep_start"] + (profile["dep_end"] - profile["dep_start"]) * t,
760                     "oil_intensity": profile["int_start"] + (profile["int_end"] - profile["int_start"]) * t,
761                     "fuel_price_regulation": profile["reg"],
762                     "import_source_hhi": profile["hhi_start"] + (profile["hhi_end"] - profile["hhi_start"]) * t,
763                     "source_url": "EIA annual petroleum balance, World Bank constant-GDP series and UN Comtrade HS2709 design target;
current table is normalized public-source proxy pending audited manual export.",
764                     "buffer_note": "Annual structural buffer proxy; continuous variables are lagged one year in Q3 interaction models.
fuel_price_regulation is mechanism-only because China is the single strong regulated-price country.",
765                 }
766             )
767     result = pd.DataFrame(rows)
768     save_processed(result, "country_policy_buffers_annual.csv")
769     return result
770
771
772 def make_country_rows(
773     country: str,
774     country_name: str,
775     monthly_q1: pd.DataFrame,
776     fuel: pd.DataFrame,
777     fuel_column: str,
778     fuel_unit: str,
779     brent_local: pd.Series,
780     fuel_source: str,
781     price_measure_type: str,
782     observed_or_regulated: str,
783     included_in_main_comparison: bool,
784     comparability_note: str,
785 ) -> pd.DataFrame:
786     rows = monthly_q1[["period", "month_end", "brent_usd_bbl", "GPR", "cny_per_usd", "jpy_per_usd", "krw_per_usd"]].copy()
787     rows = rows.merge(fuel[["period", fuel_column]], on="period", how="left")
788     rows = rows.rename(columns={fuel_column: "fuel_price_local"})
789     rows["country"] = country
790     rows["country_name"] = country_name
791     rows["fuel_unit"] = fuel_unit
792     rows["fuel_source"] = fuel_source
793     rows["price_measure_type"] = price_measure_type
794     rows["observed_or_regulated"] = observed_or_regulated
795     rows["included_in_main_comparison"] = included_in_main_comparison
796     rows["comparability_note"] = comparability_note
797     rows["brent_local_per_bbl"] = brent_local.to_numpy()

```

```

798 rows["fuel_log"] = log_positive(rows["fuel_price_local"])
799 rows["fuel_log_return"] = rows["fuel_log"].diff()
800 rows["brent_local_log"] = log_positive(rows["brent_local_per_bbl"])
801 rows["brent_local_log_return"] = rows["brent_local_log"].diff()
802 return rows
803
804
805 def build_country_monthly(monthly_q1: pd.DataFrame, monthly_cn: pd.DataFrame, warnings: list[dict[str, Any]]) -> pd.DataFrame:
806     cpi = read_processed("oecd_g20_cpi_monthly.csv")
807     cpi = cpi[["REF_AREA", "TIME_PERIOD", "OBS_VALUE"]].rename(
808         columns={"REF_AREA": "country", "TIME_PERIOD": "period", "OBS_VALUE": "cpi_yoy_pct"}
809     )
810     spain_cpi = load_spain_hicp_fallback(warnings)
811     if not spain_cpi.empty:
812         cpi = cpi.loc[~(cpi["country"].eq("ESP") & cpi["cpi_yoy_pct"].isna())].copy()
813         cpi = pd.concat([cpi, spain_cpi[["country", "period", "cpi_yoy_pct"]]], ignore_index=True)
814         cpi = cpi.sort_values(["country", "period"]).drop_duplicates(["country", "period"], keep="last")
815     ip = read_processed("oecd_kei_ip_monthly.csv")
816     ip = ip[["REF_AREA", "TIME_PERIOD", "OBS_VALUE"]].rename(
817         columns={"REF_AREA": "country", "TIME_PERIOD": "period", "OBS_VALUE": "ip_index"}
818     )
819
820     germany = read_processed("germany_eurosuper95_monthly.csv")
821     france = read_processed("france_eurosuper95_monthly.csv")
822     italy = read_processed("italy_eurosuper95_monthly.csv")
823     spain = read_processed("spain_eurosuper95_monthly.csv")
824     japan = read_processed("japan_regular_gasoline_monthly.csv")
825     korea = read_processed("korea_regular_gasoline_monthly.csv")
826
827     policy = build_china_policy_monthly(monthly_q1)
828     china_fuel = monthly_q1[["period", "brent_usd_bbl", "cny_per_usd"]].merge(policy, on="period", how="left")
829     china_fuel["brent_cny_per_tonne_proxy"] = china_fuel["brent_usd_bbl"] * china_fuel["cny_per_usd"] / BARREL_TO_TONNE
830     china_fuel["china_gasoline_proxy_cny_t"] = (
831         china_fuel["brent_cny_per_tonne_proxy"] - china_fuel["cum_gasoline_policy_gap_cny_t"]
832     ).clip(lower=1.0)
833     china_fuel["china_gasoline_rule_proxy_cny_t"] = china_fuel["brent_cny_per_tonne_proxy"].clip(lower=1.0)
834     official_china_fuel = build_china_regulated_gasoline_monthly(monthly_q1, warnings)
835     official_months = int(official_china_fuel["china_regulated_gasoline_cny_per_ton"].notna().sum())
836     china_official_ready = official_months >= CHINA_MAIN_FUEL_MN_MONTHS
837     if official_months == 0:
838         china_panel_fuel = china_fuel
839         china_fuel_column = "china_gasoline_proxy_cny_t"
840         china_fuel_unit = "CNY/tonne proxy"
841         china_fuel_source = "Brent-CNY proxy net of NDRC policy gaps"
842         china_price_measure_type = "constructed_brent_cny_policy_proxy"
843         china_observed_or_regulated = "proxy_not_observed"
844         china_included = False
845         china_note = "China fuel price is constructed from Brent-CNY minus cumulative NDRC policy gaps; excluded from the main cross-
country fuel-price ranking."
846         warnings.append(
847             {
848                 "code": "china_fuel_price_proxy",
849                 "message": "China official regulated gasoline series is unavailable; retaining Brent-CNY proxy for appendix sensitivity
only.",
850             }
851         )
852     else:
853         china_panel_fuel = official_china_fuel
854         china_fuel_column = "china_regulated_gasoline_cny_per_ton"
855         china_fuel_unit = "CNY/tonne index proxy"
856         china_fuel_source = "public adjustment-series reconstruction; 2026 official notices are audited separately"
857         china_price_measure_type = "regulated_gasoline_adjustment_index_proxy"
858         china_observed_or_regulated = "proxy_not_fixed_product_retail_level"
859         china_included = False
860         china_note = (
861             f"China uses a reconstructed regulated-gasoline adjustment index proxy with {official_months} nonmissing months; "
862             "it is retained for sensitivity and policy-path construction but excluded from the formal cross-country retail-price ranking.
"
863         )
864         warnings.append(
865             {
866                 "code": "china_fuel_price_proxy_only",
867                 "message": "China historical fuel series is a reconstructed adjustment-index proxy and is excluded from the formal cross-
country price-level ranking.",
868             }
869         )

```



```

870
871 rows = [
872     make_country_rows(
873         "CHN",
874         "China",
875         monthly_q1,
876         china_panel_fuel,
877         china_fuel_column,
878         china_fuel_unit,
879         monthly_q1["brent_usd_bbl"] * monthly_q1["cny_per_usd"],
880         china_fuel_source,
881         china_price_measure_type,
882         china_observed_or_regulated,
883         china_included,
884         china_note,
885     ),
886     make_country_rows(
887         "DEU",
888         "Germany",
889         monthly_q1,
890         germany,
891         "gasoline_eur_per_l",
892         "EUR/litre",
893         monthly_q1["brent_usd_bbl"] * monthly_q1["eur_per_usd"],
894         "European Commission Weekly Oil Bulletin",
895         "observed_retail_gasoline",
896         "observed_retail",
897         True,
898         "Observed Euro-super 95 retail price, monthly average of official weekly data.",
899     ),
900     make_country_rows(
901         "JPN",
902         "Japan",
903         monthly_q1,
904         japan,
905         "regular_gasoline_jpy_per_l",
906         "JPY/litre",
907         monthly_q1["brent_usd_bbl"] * monthly_q1["jpy_per_usd"],
908         "Japan METI weekly fuel survey monthly average",
909         "observed_retail_gasoline",
910         "observed_retail",
911         True,
912         "Observed regular gasoline retail price, monthly average of official weekly data.",
913     ),
914     make_country_rows(
915         "KOR",
916         "Korea",
917         monthly_q1,
918         korea,
919         "regular_gasoline_krw_per_l",
920         "KRW/litre",
921         monthly_q1["brent_usd_bbl"] * monthly_q1["krw_per_usd"],
922         "KOSIS / Korea National Oil Corporation monthly gasoline",
923         "observed_retail_gasoline",
924         "observed_retail",
925         True,
926         "Observed regular gasoline national monthly average from KOSIS/KNOC.",
927     ),
928 ]
929 for country, country_name, fuel in [
930     ("FRA", "France", france),
931     ("ITA", "Italy", italy),
932     ("ESP", "Spain", spain),
933 ]:
934     rows.append(
935         make_country_rows(
936             country,
937             country_name,
938             monthly_q1,
939             fuel,
940             "gasoline_eur_per_l",
941             "EUR/litre",
942             monthly_q1["brent_usd_bbl"] * monthly_q1["eur_per_usd"],
943             "European Commission Weekly Oil Bulletin",
944             "observed_retail_gasoline",
945             "observed_retail",

```

```

946         True,
947         f"Observed Euro-super 95 retail price for {country_name}, monthly average of official weekly data.",
948     )
949 )
950 panel = pd.concat(rows, ignore_index=True)
951 panel["year"] = pd.to_datetime(panel["period"] + "-01").dt.year
952 buffers = build_policy_buffer_table()
953 panel["buffer_year"] = panel["year"] - 1
954 panel = panel.merge(buffers.rename(columns={"year": "buffer_year"}), on=["country", "buffer_year"], how="left")
955 panel = panel.merge(cpi, on=["country", "period"], how="left")
956 panel = panel.merge(ip, on=["country", "period"], how="left")
957 chn_activity = monthly_cn[["period", "china_iav_yoy_pct"]].rename(columns={"china_iav_yoy_pct": "activity_yoy_pct"})
958 panel = panel.merge(chn_activity, on="period", how="left")
959 panel.loc[panel["country"].ne("CHN"), "activity_yoy_pct"] = np.nan
960 panel["ip_log"] = log_positive(panel["ip_index"])
961 panel["ip_yoy_log_change_pct"] = panel.groupby("country")["ip_log"].diff(12) * 100.0
962 panel["industrial_activity_yoy_pct"] = panel["ip_yoy_log_change_pct"]
963 panel.loc[panel["country"].eq("CHN"), "industrial_activity_yoy_pct"] = panel.loc[panel["country"].eq("CHN"), "activity_yoy_pct"]
964 panel = panel.merge(
965     monthly_cn[
966         [
967             "period",
968             "OilShock",
969             "ARBaselineGap",
970             "OilShock_source",
971             "supply_shock",
972             "aggregate_demand_shock",
973             "oil_specific_risk_shock",
974             "reduced_form_shock",
975             "source_vintage",
976         ]
977     ],
978     on="period",
979     how="left",
980 )
981 panel = panel[
982     [
983         "country",
984         "country_name",
985         "year",
986         "period",
987         "month_end",
988         "fuel_price_local",
989         "fuel_unit",
990         "fuel_source",
991         "price_measure_type",
992         "observed_or_regulated",
993         "included_in_main_comparison",
994         "comparability_note",
995         "oil_import_dependency",
996         "oil_intensity",
997         "fuel_price_regulation",
998         "import_source_hhi",
999         "buffer_note",
1000         "fuel_log_return",
1001         "brent_local_per_bbl",
1002         "brent_local_log_return",
1003         "cpi_yoy_pct",
1004         "ip_index",
1005         "ip_yoy_log_change_pct",
1006         "activity_yoy_pct",
1007         "industrial_activity_yoy_pct",
1008         "OilShock",
1009         "ARBaselineGap",
1010         "OilShock_source",
1011         "supply_shock",
1012         "aggregate_demand_shock",
1013         "oil_specific_risk_shock",
1014         "reduced_form_shock",
1015         "source_vintage",
1016         "GPR",
1017     ]
1018 ].sort_values(["country", "period"])
1019 save_processed(panel, "model_country_monthly.csv")
1020 save_processed(china_fuel, "china_fuel_proxy_monthly.csv")
1021 return panel

```

```

1022
1023
1024 def main() -> int:
1025     np.random.seed(RANDOM_SEED)
1026     warnings: list[dict[str, Any]] = []
1027     daily = build_daily_q1()
1028     monthly_q1 = build_monthly_q1()
1029     gdp = fetch_oecd_china_gdp(warnings)
1030     monthly_cn = load_china_macro(monthly_q1, warnings)
1031     quarterly_cn = build_quarterly_cn(monthly_cn, gdp)
1032     country = build_country_monthly(monthly_q1, monthly_cn, warnings)
1033
1034     summary = {
1035         "status": "WARN" if warnings else "PASS",
1036         "random_seed": RANDOM_SEED,
1037         "cutoff": CUTOFF.strftime("%Y-%m-%d"),
1038         "outputs": {
1039             "model_daily_q1.csv": len(daily),
1040             "model_monthly_q1.csv": len(monthly_q1),
1041             "model_monthly_cn.csv": len(monthly_cn),
1042             "model_quarterly_cn.csv": len(quarterly_cn),
1043             "model_country_monthly.csv": len(country),
1044         },
1045         "warnings": warnings,
1046     }
1047     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
1048     (RESULTS_DIR / "model_panel_summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
1049     update_data_warnings("build_model_panels", warnings)
1050     print(json.dumps(summary, ensure_ascii=False, indent=2))
1051     return 0
1052
1053
1054 if __name__ == "__main__":
1055     raise SystemExit(main())

```

Listing 3: build_model_panels.py

```

1 """Import manually exported NBS monthly CSV files into the processed layer.
2
3 The NBS download format is a wide table with months in reverse chronological
4 order. This importer preserves official missing observations, converts the
5 PPI year-on-year index (same month of previous year = 100) to a percentage
6 change, and writes tidy monthly files used by the Q2 pipeline.
7 """
8
9 from __future__ import annotations
10
11 import argparse
12 import csv
13 import hashlib
14 import json
15 import re
16 from datetime import date
17 from pathlib import Path
18 from typing import Any
19
20 import numpy as np
21 import pandas as pd
22
23
24 REPO_ROOT = Path(__file__).resolve().parents[2]
25 PROCESSED_DIR = REPO_ROOT / "data" / "processed"
26 SOURCE_URL = "https://data.stats.gov.cn/dg/website/page.html#/pc/national/monthData"
27 START_PERIOD = "2010-01"
28 END_PERIOD = "2026-06"
29 EXPECTED_MONTHS = len(pd.period_range(START_PERIOD, END_PERIOD, freq="M"))
30
31
32 def sha256_file(path: Path) -> str:
33     return hashlib.sha256(path.read_bytes()).hexdigest()
34
35
36 def clean_cell(value: str) -> str:
37     return value.replace("\u00ff", "").replace("\t", "").strip()
38

```

```

39
40 def read_nbs_wide(path: Path) -> tuple[str, list[str], dict[str, list[str]]]:
41     rows: list[list[str]] = []
42     with path.open("r", encoding="utf-8-sig", newline="") as handle:
43         for row in csv.reader(handle):
44             cleaned = [clean_cell(value) for value in row]
45             while cleaned and cleaned[-1] == "":
46                 cleaned.pop()
47             rows.append(cleaned)
48
49     period_label = next((row[0] for row in rows if row and row[0].startswith("时间: ")), "")
50     header_index = next((index for index, row in enumerate(rows) if row and row[0] == "指标"), None)
51     if header_index is None:
52         raise ValueError(f"{path.name}: cannot find the NBS indicator header")
53
54     months = rows[header_index][1:]
55     if len(months) != EXPECTED_MONTHS:
56         raise ValueError(f"{path.name}: expected {EXPECTED_MONTHS} months, found {len(months)}")
57
58     indicators: dict[str, list[str]] = {}
59     for row in rows[header_index + 1 :]:
60         if not row or row[0].startswith("注: ") or row[0].startswith("数据来源: "):
61             break
62         values = row[1:]
63         if len(values) < len(months):
64             values.extend([""] * (len(months) - len(values)))
65         indicators[row[0]] = values[: len(months)]
66     return period_label, months, indicators
67
68
69 def parse_month(value: str) -> str:
70     match = re.fullmatch(r"(\d{4})年(\d{1,2})月", value)
71     if not match:
72         raise ValueError(f"unrecognized month label: {value!r}")
73     return f"{int(match.group(1)):04d}-{int(match.group(2)):02d}"
74
75
76 def numeric(values: list[str]) -> pd.Series:
77     return pd.to_numeric(pd.Series(values, dtype="object").replace("", np.nan), errors="coerce")
78
79
80 def find_indicator(indicators: dict[str, list[str]], required: list[str], excluded: list[str] | None = None) -> tuple[str, list[str]]:
81     excluded = excluded or []
82     matches = [
83         (label, values)
84         for label, values in indicators.items()
85         if all(token in label for token in required) and not any(token in label for token in excluded)
86     ]
87     if len(matches) != 1:
88         labels = sorted(indicators)
89         raise ValueError(f"expected one indicator matching {required}, found {len(matches)}; available={labels}")
90     return matches[0]
91
92
93 def validate_periods(frame: pd.DataFrame, label: str) -> None:
94     if frame["period"].duplicated().any():
95         raise ValueError(f"{label}: duplicate periods")
96     expected = pd.period_range(START_PERIOD, END_PERIOD, freq="M").astype(str).tolist()
97     actual = frame["period"].tolist()
98     if actual != expected:
99         raise ValueError(f"{label}: period coverage does not equal {START_PERIOD}..{END_PERIOD}")
100
101
102 def import_ppi(path: Path, download_date: str) -> tuple[pd.DataFrame, dict[str, Any]]:
103     period_label, months, indicators = read_nbs_wide(path)
104     indicator, values = find_indicator(
105         indicators,
106         required=["工业生产者出厂价格指数", "上年同月=100"],
107         excluded=["生产资料", "生活资料", "分行业", "中类", "按工业部门"],
108     )
109     index_values = numeric(values)
110     frame = pd.DataFrame(
111         {
112             "period": [parse_month(value) for value in months],
113             "china_ppi_index_yoy_100": index_values,
114             "china_ppi_yoy_pct": (index_values - 100.0).round(10),

```

```

115     }
116     ).sort_values("period", ignore_index=True)
117     validate_periods(frame, "PPI")
118     if frame["china_ppi_yoy_pct"].isna().any():
119         missing = frame.loc[frame["china_ppi_yoy_pct"].isna(), "period"].tolist()
120         raise ValueError(f"PPI: unexpected missing months: {missing}")
121
122     raw_hash = sha256_file(path)
123     frame["source_indicator"] = indicator
124     frame["source_url"] = SOURCE_URL
125     frame["download_date"] = download_date
126     frame["raw_sha256"] = raw_hash
127
128     metadata = {
129         "source": "National Bureau of Statistics of China, National Data",
130         "source_url": SOURCE_URL,
131         "download_date": download_date,
132         "raw_file": path.name,
133         "raw_sha256": raw_hash,
134         "period_label": period_label,
135         "indicator": indicator,
136         "calendar_months": int(len(frame)),
137         "nonmissing_observations": int(frame["china_ppi_yoy_pct"].notna().sum()),
138         "transformation": "china_ppi_yoy_pct = china_ppi_index_yoy_100 - 100",
139     }
140     return frame, metadata
141
142 def import_iav(path: Path, download_date: str) -> tuple[pd.DataFrame, dict[str, Any]]:
143     period_label, months, indicators = read_nbs_wide(path)
144     indicator, values = find_indicator(indicators, required=["规上工业增加值同比增长"])
145     cumulative_matches = [
146         (label, row_values)
147         for label, row_values in indicators.items()
148         if "规上工业增加值累计增长" in label
149     ]
150     yoy_values = numeric(values)
151     cumulative_values = numeric(cumulative_matches[0][1]) if len(cumulative_matches) == 1 else pd.Series(np.nan, index=range(len(months)))
152
153     frame = pd.DataFrame(
154         {
155             "period": [parse_month(value) for value in months],
156             "china_iav_yoy_pct": yoy_values,
157             "china_iav_cumulative_yoy_pct": cumulative_values,
158         }
159     ).sort_values("period", ignore_index=True)
160     validate_periods(frame, "IAV")
161
162     missing = frame.loc[frame["china_iav_yoy_pct"].isna(), "period"].tolist()
163     unexpected = [
164         period
165         for period in missing
166         if not (
167             period.endswith("-01")
168             or (period >= "2013-02" and period.endswith("-02"))
169         )
170     ]
171     if unexpected:
172         raise ValueError(f"IAV: missing observations outside the official January/February pattern: {unexpected}")
173     if frame["china_iav_yoy_pct"].notna().sum() < 120:
174         raise ValueError("IAV: fewer than 120 nonmissing monthly observations")
175
176     frame["observation_status"] = np.where(
177         frame["china_iav_yoy_pct"].notna(),
178         "observed",
179         "official_calendar_gap_no_interpolation",
180     )
181     raw_hash = sha256_file(path)
182     frame["source_indicator"] = indicator
183     frame["source_url"] = SOURCE_URL
184     frame["download_date"] = download_date
185     frame["raw_sha256"] = raw_hash
186
187     metadata = {
188         "source": "National Bureau of Statistics of China, National Data",
189         "source_url": SOURCE_URL,
190         "download_date": download_date,
191         "raw_file": path.name,

```

```

190     "raw_sha256": raw_hash,
191     "period_label": period_label,
192     "indicator": indicator,
193     "calendar_months": int(len(frame)),
194     "nonmissing_observations": int(frame["china_iav_yoy_pct"].notna().sum()),
195     "official_missing_months": missing,
196     "missing_value_policy": "preserve official January/February gaps; no interpolation",
197 }
198 return frame, metadata
199
200
201 def main() -> int:
202     parser = argparse.ArgumentParser()
203     parser.add_argument("--ppi", type=Path, required=True, help="NBS PPI wide CSV export")
204     parser.add_argument("--iav", type=Path, required=True, help="NBS industrial value-added wide CSV export")
205     parser.add_argument("--download-date", default=date.today().isoformat())
206     args = parser.parse_args()
207
208     ppi, ppi_meta = import_ppi(args.ppi.resolve(), args.download_date)
209     iav, iav_meta = import_iav(args.iav.resolve(), args.download_date)
210     PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
211     ppi.to_csv(PROCESSED_DIR / "nbs_ppi_monthly.csv", index=False, encoding="utf-8", lineterminator="\n")
212     iav.to_csv(PROCESSED_DIR / "nbs_iav_monthly.csv", index=False, encoding="utf-8", lineterminator="\n")
213
214     metadata = {
215         "schema_version": "1.0",
216         "cutoff": END_PERIOD,
217         "ppi": ppi_meta,
218         "iav": iav_meta,
219     }
220     (PROCESSED_DIR / "nbs_manual_import_metadata.json").write_text(
221         json.dumps(metadata, ensure_ascii=False, indent=2, sort_keys=True) + "\n",
222         encoding="utf-8",
223     )
224     print(
225         json.dumps(
226             {
227                 "status": "PASS",
228                 "ppi_rows": int(len(ppi)),
229                 "ppi_nonmissing": int(ppi["china_ppi_yoy_pct"].notna().sum()),
230                 "iav_rows": int(len(iav)),
231                 "iav_nonmissing": int(iav["china_iav_yoy_pct"].notna().sum()),
232                 "iav_official_gaps": int(iav["china_iav_yoy_pct"].isna().sum()),
233             },
234             ensure_ascii=False,
235             indent=2,
236         )
237     )
238     return 0
239
240
241 if __name__ == "__main__":
242     raise SystemExit(main())

```

Listing 4: import_nbs_manual.py

```

1  """Diagnose P0 data readiness without blocking modeling on optional snapshots."""
2
3  from __future__ import annotations
4
5  import hashlib
6  import json
7  from pathlib import Path
8  from typing import Any
9
10 import numpy as np
11 import pandas as pd
12
13
14 REPO_ROOT = Path(__file__).resolve().parents[2]
15 RAW_DIR = REPO_ROOT / "data" / "raw"
16 PROCESSED_DIR = REPO_ROOT / "data" / "processed"
17 RESULTS_DIR = REPO_ROOT / "results"
18 WARNINGS_PATH = RESULTS_DIR / "data_warnings.json"
19 CUTOFF = pd.Timestamp("2026-06-30")

```

```

20 SUCCESS_STATUSES = {"DOWNLOADED", "CACHED"}
21
22
23 def sha256_file(path: Path) -> str:
24     digest = hashlib.sha256()
25     with path.open("rb") as handle:
26         for chunk in iter(lambda: handle.read(1024 * 1024), b""):
27             digest.update(chunk)
28     return digest.hexdigest()
29
30
31 def add_issue(
32     bucket: list[dict[str, Any]],
33     code: str,
34     message: str,
35     path: Path | None = None,
36 ) -> None:
37     row: dict[str, Any] = {"code": code, "message": message}
38     if path is not None:
39         row["path"] = path.relative_to(REPO_ROOT).as_posix()
40     bucket.append(row)
41
42
43 def read_csv_checked(path: Path, errors: list[dict[str, Any]], **kwargs: Any) -> pd.DataFrame:
44     try:
45         return pd.read_csv(path, **kwargs)
46     except Exception as exc:
47         add_issue(errors, "parse_failed", f"{path.name}: {exc}", path)
48     return pd.DataFrame()
49
50
51 def check_no_duplicate(frame: pd.DataFrame, keys: list[str], name: str, errors: list[dict[str, Any]]) -> None:
52     missing = [key for key in keys if key not in frame.columns]
53     if missing:
54         add_issue(errors, "missing_key_columns", f"{name} missing key columns {missing}")
55         return
56     duplicated = int(frame.duplicated(keys).sum())
57     if duplicated:
58         add_issue(errors, "duplicate_keys", f"{name} has {duplicated} duplicated keys: {keys}")
59
60
61 def check_cutoff(
62     values: pd.Series,
63     name: str,
64     errors: list[dict[str, Any]],
65     warnings: list[dict[str, Any]],
66 ) -> None:
67     dates = pd.to_datetime(values, errors="coerce")
68     if dates.isna().any():
69         add_issue(warnings, "date_parse_warning", f"{name} has {int(dates.isna().sum())} unparsable dates")
70     max_date = dates.max()
71     if pd.notna(max_date) and max_date > CUTOFF:
72         add_issue(errors, "cutoff_violation", f"{name} max date {max_date.date()} exceeds 2026-06-30")
73
74
75 def check_finite(frame: pd.DataFrame, columns: list[str], name: str, errors: list[dict[str, Any]]) -> None:
76     missing = [column for column in columns if column not in frame.columns]
77     if missing:
78         add_issue(errors, "missing_core_columns", f"{name} missing core columns {missing}")
79         return
80     numeric = frame[columns].apply(pd.to_numeric, errors="coerce")
81     bad = int((~np.isfinite(numeric.to_numpy(dtype=float))).sum())
82     if bad:
83         add_issue(errors, "nonfinite_core_values", f"{name} has {bad} non-finite core numeric cells")
84
85
86 def validate_raw_manifest(warnings: list[dict[str, Any]], errors: list[dict[str, Any]]) -> None:
87     manifest_path = RAW_DIR / "source_manifest.json"
88     try:
89         manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
90     except Exception as exc:
91         add_issue(errors, "manifest_parse_failed", f"source_manifest.json: {exc}", manifest_path)
92         return
93
94     if len(manifest) != 21:
95         add_issue(warnings, "manifest_record_count", f"expected 21 source records, got {len(manifest)}", manifest_path)

```

```

96
97     for record in manifest:
98         artifact_id = record.get("artifact_id", "<unknown>")
99         status = record.get("status", "")
100         if status not in SUCCESS_STATUSES:
101             add_issue(warnings, "manifest_status", f"{artifact_id} status={status}")
102         local_path = record.get("local_path", "")
103         path = REPO_ROOT / local_path
104         if not path.exists():
105             add_issue(warnings, "missing_raw_snapshot", f"{artifact_id} raw snapshot is absent", path)
106             continue
107         expected_hash = record.get("sha256")
108         if expected_hash:
109             actual_hash = sha256_file(path)
110             if actual_hash != expected_hash:
111                 add_issue(
112                     warnings,
113                     "raw_hash_mismatch",
114                     f"{artifact_id} SHA-256 mismatch; likely line-ending or refreshed snapshot drift",
115                     path,
116                 )
117
118
119 def validate_processed(warnings: list[dict[str, Any]], errors: list[dict[str, Any]]) -> list[str]:
120     checks: list[str] = []
121
122     monthly = read_csv_checked(PROCESSED_DIR / "p0_monthly_market.csv", errors)
123     if not monthly.empty:
124         check_no_duplicate(monthly, ["period"], "p0_monthly_market", errors)
125         if "month_end" in monthly.columns:
126             check_cutoff(monthly["month_end"], "p0_monthly_market.month_end", errors, warnings)
127         check_finite(
128             monthly,
129             ["brent_usd_bbl", "us_crude_stock_month_end_kbbl", "usd_broad_index", "GPR", "cny_per_usd"],
130             "p0_monthly_market",
131             errors,
132         )
133         checks.append(f"p0_monthly_market rows={len(monthly)}")
134
135     daily = read_csv_checked(PROCESSED_DIR / "p0_daily_market.csv", errors)
136     if not daily.empty:
137         check_no_duplicate(daily, ["date"], "p0_daily_market", errors)
138         if "date" in daily.columns:
139             check_cutoff(daily["date"], "p0_daily_market.date", errors, warnings)
140         checks.append(f"p0_daily_market rows={len(daily)}")
141
142     cpi = read_csv_checked(PROCESSED_DIR / "oecd_g20_cpi_monthly.csv", errors)
143     if not cpi.empty:
144         check_no_duplicate(cpi, ["REF_AREA", "TIME_PERIOD"], "oecd_g20_cpi_monthly", errors)
145         if "TIME_PERIOD" in cpi.columns:
146             check_cutoff(cpi["TIME_PERIOD"].astype(str) + "-01", "oecd_g20_cpi_monthly.TIME_PERIOD", errors, warnings)
147         checks.append(f"oecd_g20_cpi_monthly rows={len(cpi)}")
148
149     ip = read_csv_checked(PROCESSED_DIR / "oecd_kei_ip_monthly.csv", errors)
150     if not ip.empty:
151         check_no_duplicate(ip, ["REF_AREA", "TIME_PERIOD"], "oecd_kei_ip_monthly", errors)
152         if "TIME_PERIOD" in ip.columns:
153             check_cutoff(ip["TIME_PERIOD"].astype(str) + "-01", "oecd_kei_ip_monthly.TIME_PERIOD", errors, warnings)
154         checks.append(f"oecd_kei_ip_monthly rows={len(ip)}")
155
156     for filename, key in [
157         ("germany_eurosuper95_monthly.csv", ["period"]),
158         ("japan_regular_gasoline_monthly.csv", ["period"]),
159         ("korea_regular_gasoline_monthly.csv", ["period"]),
160         ("cn_fuel_policy_events.csv", ["effective_date"]),
161     ]:
162         frame = read_csv_checked(PROCESSED_DIR / filename, errors)
163         if frame.empty:
164             continue
165         check_no_duplicate(frame, key, filename, errors)
166         date_col = "effective_date" if "effective_date" in frame.columns else "month_end"
167         if date_col in frame.columns:
168             check_cutoff(frame[date_col], f"{filename}.{date_col}", errors, warnings)
169         checks.append(f"{filename} rows={len(frame)}")
170
171     return checks

```



```

172
173
174 def main() -> int:
175     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
176     warnings: list[dict[str, Any]] = []
177     errors: list[dict[str, Any]] = []
178
179     validate_raw_manifest(warnings, errors)
180     checks = validate_processed(warnings, errors)
181
182     result = {
183         "status": "FAIL" if errors else ("WARN" if warnings else "PASS"),
184         "cutoff": CUTOFF.strftime("%Y-%m-%d"),
185         "checks": checks,
186         "warnings": warnings,
187         "errors": errors,
188     }
189     WARNINGS_PATH.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
190     print(json.dumps(result, ensure_ascii=False, indent=2))
191     return 1 if errors else 0
192
193
194 if __name__ == "__main__":
195     raise SystemExit(main())

```

Listing 5: validate_p0.py

3.2 四问模型

```

1 """Question 1: oil-price forecasts and war-shock counterfactuals."""
2
3 from __future__ import annotations
4
5 import argparse
6 import json
7 import math
8 import sys
9 import warnings as py_warnings
10 from pathlib import Path
11 from typing import Any
12
13 import matplotlib
14
15 matplotlib.use("Agg")
16
17 import matplotlib.pyplot as plt
18 import numpy as np
19 import pandas as pd
20 import statsmodels.api as sm
21 from scipy.optimize import minimize
22 from scipy.stats import norm
23 from statsmodels.tools.sm_exceptions import ValueWarning
24 from statsmodels.tsa.api import VAR
25 from statsmodels.tsa.stattools import adfuller, kpss
26 from statsmodels.tsa.forecasting.theta import ThetaModel
27 from statsmodels.tsa.holtwinters import ExponentialSmoothing
28 from statsmodels.tsa.statespace.sarimax import SARIMAX
29
30
31 REPO_ROOT = Path(__file__).resolve().parents[2]
32 CODE_DIR = REPO_ROOT / "code"
33 if str(CODE_DIR) not in sys.path:
34     sys.path.insert(0, str(CODE_DIR))
35
36 from utils.plot_style import PALETTE, apply_paper_style, finish_figure, save_figure, style_axis
37
38 if hasattr(sys.stdout, "reconfigure"):
39     sys.stdout.reconfigure(encoding="utf-8")
40
41 PROCESSED_DIR = REPO_ROOT / "data" / "processed"
42 RESULTS_DIR = REPO_ROOT / "results"
43 FIGURES_DIR = REPO_ROOT / "figures"

```

```

44 CUTOFF = pd.Timestamp("2026-06-30")
45 RANDOM_SEED = 20260730
46 EVAL_START = "2020-01"
47 EVENT_E1_CALENDAR_START = pd.Timestamp("2026-02-28")
48 EVENT_E3_CALENDAR_START = pd.Timestamp("2026-06-17")
49 EVENT_TERMS = ["stage_E1", "stage_E2", "stage_E3"]
50
51
52 def ensure_dirs() -> None:
53     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
54     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
55
56
57 def save_csv(frame: pd.DataFrame, filename: str) -> Path:
58     path = RESULTS_DIR / filename
59     frame.to_csv(path, index=False, encoding="utf-8")
60     return path
61
62
63 def json_records(frame: pd.DataFrame) -> list[dict[str, Any]]:
64     if frame.empty:
65         return []
66     return json.loads(frame.where(pd.notna(frame), None).to_json(orient="records"))
67
68
69 def zscore(series: pd.Series) -> pd.Series:
70     std = series.std(skipna=True)
71     if not np.isfinite(std) or std == 0:
72         return series * np.nan
73     return (series - series.mean(skipna=True)) / std
74
75
76 def log_positive(series: pd.Series) -> pd.Series:
77     values = pd.to_numeric(series, errors="coerce")
78     result = pd.Series(np.nan, index=series.index, dtype=float)
79     mask = values > 0
80     result.loc[mask] = np.log(values.loc[mask])
81     return result
82
83
84 def fit_sarimax(
85     y: pd.Series,
86     order: tuple[int, int, int],
87     exog: pd.DataFrame | None = None,
88 ) -> Any:
89     with py_warnings.catch_warnings():
90         py_warnings.simplefilter("ignore")
91         model = SARIMAX(
92             y,
93             order=order,
94             exog=exog,
95             trend="c",
96             enforce_stationarity=False,
97             enforce_invertibility=False,
98         )
99     return model.fit(dispatch=False, maxiter=200)
100
101
102 def common_trading_dates(daily: pd.DataFrame, price_prefix: str = "brent_usd_bbl") -> list[pd.Timestamp]:
103     frame = daily.copy()
104     frame["date"] = pd.to_datetime(frame["date"])
105     other_price = "wti_usd_bbl" if price_prefix == "brent_usd_bbl" and "wti_usd_bbl" in frame.columns else price_prefix
106     mask = frame[price_prefix].notna() & frame[other_price].notna()
107     return [pd.Timestamp(value) for value in frame.loc[mask, "date"].drop_duplicates().sort_values().tolist()]
108
109
110 def first_trading_date_on_or_after(daily: pd.DataFrame, date: pd.Timestamp, price_prefix: str = "brent_usd_bbl") -> pd.Timestamp:
111     candidates = [value for value in common_trading_dates(daily, price_prefix) if value >= date]
112     if not candidates:
113         raise ValueError(f"No common trading date on or after {date.date()} for {price_prefix}.")
114     return candidates[0]
115
116
117 def trading_day_shift(daily: pd.DataFrame, start: pd.Timestamp, shift_days: int, price_prefix: str = "brent_usd_bbl") -> pd.Timestamp:
118     dates = common_trading_dates(daily, price_prefix)
119     try:

```

```

120     idx = dates.index(start)
121     except ValueError as exc:
122         raise ValueError(f"{start.date()} is not a common trading date for {price_prefix}.") from exc
123     shifted_idx = idx + shift_days
124     if shifted_idx < 0 or shifted_idx >= len(dates):
125         raise ValueError(f"Trading-day shift {shift_days:+d} from {start.date()} is outside the observed sample.")
126     return dates[shifted_idx]
127
128
129 def assign_event_stages(daily: pd.DataFrame, el_start: pd.Timestamp | None = None, price_prefix: str = "brent_usd_bbl") -> tuple[pd.
    DataFrame, dict[str, str]]:
130     frame = daily.copy()
131     frame["date"] = pd.to_datetime(frame["date"])
132     el_start = el_start or first_trading_date_on_or_after(frame, EVENT_E1_CALENDAR_START, price_prefix)
133     dates = common_trading_dates(frame, price_prefix)
134     idx = dates.index(el_start)
135     if idx + 3 >= len(dates):
136         raise ValueError("E1 event window lacks enough post-event trading days.")
137     el_car_end = dates[idx + 2]
138     e2_start = dates[idx + 3]
139     e3_start = first_trading_date_on_or_after(frame, EVENT_E3_CALENDAR_START, price_prefix)
140
141     date = frame["date"]
142     frame["war_stage"] = "prewar"
143     frame.loc[date.between(el_start, el_car_end), "war_stage"] = "E1_immediate_window"
144     frame.loc[date.between(e2_start, e3_start - pd.Timedelta(days=1)), "war_stage"] = "E2_disruption"
145     frame.loc[date >= e3_start, "war_stage"] = "E3_easing"
146     frame["war_on"] = (date >= el_start).astype(int)
147     frame["stage_E1"] = date.between(el_start, el_car_end).astype(int)
148     frame["stage_E2"] = frame["war_stage"].eq("E2_disruption").astype(int)
149     frame["stage_E3"] = frame["war_stage"].eq("E3_easing").astype(int)
150     event_meta = {
151         "el_calendar_start": EVENT_E1_CALENDAR_START.strftime("%Y-%m-%d"),
152         "el_trading_start": el_start.strftime("%Y-%m-%d"),
153         "el_car_end_0_2": el_car_end.strftime("%Y-%m-%d"),
154         "e2_trading_start": e2_start.strftime("%Y-%m-%d"),
155         "e3_trading_start": e3_start.strftime("%Y-%m-%d"),
156     }
157     return frame, event_meta
158
159
160 def dm_hln_test(loss_model: pd.Series, loss_benchmark: pd.Series, horizon: int) -> tuple[float, float]:
161     diff = pd.Series(loss_model.to_numpy(dtype=float) - loss_benchmark.to_numpy(dtype=float)).dropna()
162     n = len(diff)
163     if n <= horizon + 1 or float(diff.var(ddof=1)) == 0.0:
164         return np.nan, np.nan
165     mean_diff = float(diff.mean())
166     centered = diff - mean_diff
167     max_lag = max(0, horizon - 1)
168     gamma0 = float((centered @ centered) / n)
169     long_run_var = gamma0
170     for lag in range(1, max_lag + 1):
171         gamma = float((centered.iloc[lag:].to_numpy() @ centered.iloc[:-lag].to_numpy()) / n)
172         long_run_var += 2.0 * (1.0 - lag / (max_lag + 1.0)) * gamma
173     if long_run_var <= 0 or not np.isfinite(long_run_var):
174         return np.nan, np.nan
175     dm_stat = mean_diff / math.sqrt(long_run_var / n)
176     hln_scale = math.sqrt((n + 1 - 2 * horizon + horizon * (horizon - 1) / n) / n)
177     stat = dm_stat * hln_scale
178     pvalue = 2.0 * (1.0 - norm.cdf(abs(stat)))
179     return float(stat), float(pvalue)
180
181
182 def classify_forecast_model(model: str, rel_rmse: float, dm_stat: float, dm_pvalue: float) -> str:
183     if model == "no_change":
184         return "PASS"
185     significantly_worse = np.isfinite(dm_stat) and np.isfinite(dm_pvalue) and dm_stat > 0 and dm_pvalue < 0.10
186     if np.isfinite(rel_rmse) and rel_rmse < 1.00 and not significantly_worse:
187         return "PASS"
188     if significantly_worse or (np.isfinite(rel_rmse) and rel_rmse > 1.05):
189         return "FAIL"
190     return "CONDITIONAL"
191
192
193 def select_orders(monthly: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> tuple[tuple[int, int, int], tuple[int, int, int], pd.
    DataFrame]:

```

```

194 train = monthly.loc[monthly["period"].lt(EVAL_START)].dropna(subset=["log_brent"])
195 exog_cols = ["stock_log_lag1", "usd_broad_index_log_return_lag1", "GPR_z_lag1"]
196 train_exog = train[exog_cols]
197 train_full = train.dropna(subset=["log_brent"] + exog_cols)
198 rows: list[dict[str, Any]] = []
199 best_arima = ((1, 1, 1), math.inf)
200 best_sarimax = ((1, 1, 1), math.inf)
201 for p in range(3):
202     for q in range(3):
203         order = (p, 1, q)
204         try:
205             result = fit_sarimax(train["log_brent"], order)
206             rows.append({"model": "ARIMA", "order": str(order), "aic": float(result.aic)})
207             if result.aic < best_arima[1]:
208                 best_arima = (order, float(result.aic))
209         except (ValueError, RuntimeError, np.linalg.LinAlgError) as exc:
210             rows.append({"model": "ARIMA", "order": str(order), "aic": np.nan, "error": str(exc)})
211         try:
212             result = fit_sarimax(train_full["log_brent"], order, train_exog.loc[train_full.index])
213             rows.append({"model": "SARIMAX", "order": str(order), "aic": float(result.aic)})
214             if result.aic < best_sarimax[1]:
215                 best_sarimax = (order, float(result.aic))
216         except (ValueError, RuntimeError, np.linalg.LinAlgError) as exc:
217             rows.append({"model": "SARIMAX", "order": str(order), "aic": np.nan, "error": str(exc)})
218     if not np.isfinite(best_arima[1]):
219         warnings_log.append({"code": "arima_order_grid_failed", "message": "ARIMA grid failed; using (1,1,1)."})
220     best_arima = ((1, 1, 1), np.nan)
221     if not np.isfinite(best_sarimax[1]):
222         warnings_log.append({"code": "sarimax_order_grid_failed", "message": "SARIMAX grid failed; using (1,1,1)."})
223     best_sarimax = ((1, 1, 1), np.nan)
224     grid = pd.DataFrame(rows)
225     save_csv(grid, "ql_order_grid.csv")
226     return best_arima[0], best_sarimax[0], grid
227
228
229 def forecast_interval(prediction: float, sigma: float, horizon: int, alpha: float) -> tuple[float, float]:
230     width = norm.ppf(1 - alpha / 2) * sigma * math.sqrt(horizon)
231     return prediction - width, prediction + width
232
233
234 def monthly_forecasts(monthly: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> tuple[pd.DataFrame, pd.DataFrame]:
235     exog_cols = ["stock_log_lag1", "usd_broad_index_log_return_lag1", "GPR_z_lag1"]
236     arima_order, sarimax_order, _ = select_orders(monthly, warnings_log)
237     rows: list[dict[str, Any]] = []
238     periods = monthly["period"].tolist()
239     eval_periods = [period for period in periods if period >= EVAL_START]
240
241     for target_period in eval_periods:
242         target_idx = periods.index(target_period)
243         for horizon in [1, 3, 6]:
244             origin_idx = target_idx - horizon
245             if origin_idx < 60:
246                 continue
247             train = monthly.iloc[: origin_idx + 1].copy()
248             target = monthly.iloc[target_idx]
249             if pd.isna(target["log_brent"]):
250                 continue
251             origin = monthly.iloc[origin_idx]
252             sample_start = train["period"].iloc[0]
253             sample_end = train["period"].iloc[-1]
254
255             residual = train["log_brent"].diff().dropna()
256             sigma = float(residual.std()) if len(residual) > 2 else 0.1
257             no_change = float(origin["log_brent"])
258             for alpha, prefix in [(0.20, "80"), (0.05, "95")]:
259                 pass
260             lower_80, upper_80 = forecast_interval(no_change, sigma, horizon, 0.20)
261             lower_95, upper_95 = forecast_interval(no_change, sigma, horizon, 0.05)
262             rows.append(
263                 {
264                     "origin_period": str(origin["period"]),
265                     "origin_price": float(origin["brent_usd_bbl"]),
266                     "origin_log_price": float(origin["log_brent"]),
267                     "period": target_period,
268                     "horizon": horizon,
269                     "model": "no_change",

```

```

270         "actual": float(target["log_brent"]),
271         "prediction": no_change,
272         "actual_change": float(target["log_brent"] - origin["log_brent"]),
273         "predicted_change": float(no_change - origin["log_brent"]),
274         "lower_80": lower_80,
275         "upper_80": upper_80,
276         "lower_95": lower_95,
277         "upper_95": upper_95,
278         "actual_price": float(target["brent_usd_bbl"]),
279         "prediction_price": float(np.exp(no_change)),
280         "sample_start": sample_start,
281         "sample_end": sample_end,
282         "specification": "last observed log Brent",
283     }
284 )
285
286 for model_name, order, use_exog in [
287     ("ARIMA", arima_order, False),
288     ("SARIMAX", sarimax_order, True),
289 ]:
290     try:
291         fit_train = train.dropna(subset=["log_brent"] + (exog_cols if use_exog else []))
292         exog_train = fit_train[exog_cols] if use_exog else None
293         result = fit_sarimax(fit_train["log_brent"], order, exog_train)
294         if use_exog:
295             last_exog = fit_train[exog_cols].iloc[-1].to_numpy(dtype=float)
296             future_exog = pd.DataFrame([last_exog] * horizon, columns=exog_cols)
297             forecast = result.get_forecast(steps=horizon, exog=future_exog)
298         else:
299             forecast = result.get_forecast(steps=horizon)
300         mean = float(forecast.predicted_mean.iloc[-1])
301         ci80 = forecast.conf_int(alpha=0.20).iloc[-1].to_numpy(dtype=float)
302         ci95 = forecast.conf_int(alpha=0.05).iloc[-1].to_numpy(dtype=float)
303         rows.append(
304             {
305                 "origin_period": str(origin["period"]),
306                 "origin_price": float(origin["brent_usd_bbl"]),
307                 "origin_log_price": float(origin["log_brent"]),
308                 "period": target_period,
309                 "horizon": horizon,
310                 "model": model_name,
311                 "actual": float(target["log_brent"]),
312                 "prediction": mean,
313                 "actual_change": float(target["log_brent"] - origin["log_brent"]),
314                 "predicted_change": float(mean - origin["log_brent"]),
315                 "lower_80": float(ci80[0]),
316                 "upper_80": float(ci80[1]),
317                 "lower_95": float(ci95[0]),
318                 "upper_95": float(ci95[1]),
319                 "actual_price": float(target["brent_usd_bbl"]),
320                 "prediction_price": float(np.exp(mean)),
321                 "sample_start": sample_start,
322                 "sample_end": sample_end,
323                 "specification": f"order={order}; future exog held at origin" if use_exog else f"order={order}",
324             }
325         )
326     except (ValueError, RuntimeError, np.linalg.LinAlgError) as exc:
327         warnings_log.append(
328             {
329                 "code": "monthly_forecast_model_failed",
330                 "message": f"{model_name} h={horizon} target={target_period}: {exc}",
331             }
332         )
333     fit_train = train.dropna(subset=["log_brent"]).copy()
334     fit_series = pd.Series(
335         fit_train["log_brent"].to_numpy(dtype=float),
336         index=pd.PeriodIndex(fit_train["period"], freq="M").to_timestamp(),
337     ).asfreq("MS")
338     for model_name in ["ETS", "Theta"]:
339         try:
340             if model_name == "ETS":
341                 fitted = ExponentialSmoothing(
342                     fit_series,
343                     trend="add",
344                     damped_trend=True,
345                     seasonal=None,

```

```

346         initialization_method="estimated",
347     ).fit(optimized=True)
348     pred_values = fitted.forecast(horizon)
349     else:
350         fitted = ThetaModel(fit_series, period=12, deseasonalize=False).fit()
351         pred_values = fitted.forecast(horizon)
352     mean = float(pred_values.iloc[-1])
353     lower_80, upper_80 = forecast_interval(mean, sigma, horizon, 0.20)
354     lower_95, upper_95 = forecast_interval(mean, sigma, horizon, 0.05)
355     rows.append(
356         {
357             "origin_period": str(origin["period"]),
358             "origin_price": float(origin["brent_usd_bbl"]),
359             "origin_log_price": float(origin["log_brent"]),
360             "period": target_period,
361             "horizon": horizon,
362             "model": model_name,
363             "actual": float(target["log_brent"]),
364             "prediction": mean,
365             "actual_change": float(target["log_brent"] - origin["log_brent"]),
366             "predicted_change": float(mean - origin["log_brent"]),
367             "lower_80": lower_80,
368             "upper_80": upper_80,
369             "lower_95": lower_95,
370             "upper_95": upper_95,
371             "actual_price": float(target["brent_usd_bbl"]),
372             "prediction_price": float(np.exp(mean)),
373             "sample_start": sample_start,
374             "sample_end": sample_end,
375             "specification": "Holt damped ETS" if model_name == "ETS" else "ThetaModel period=12",
376         }
377     )
378     except (ValueError, RuntimeError, np.linalg.LinAlgError) as exc:
379         warnings_log.append(
380             {
381                 "code": "monthly_forecast_model_failed",
382                 "message": f"{model_name} h={horizon} target={target_period}: {exc}",
383             }
384         )
385
386     forecasts = pd.DataFrame(rows)
387     forecasts = add_equal_weight_combination(forecasts)
388     save_csv(forecasts, "q1_forecasts.csv")
389     metrics = forecast_metrics(forecasts)
390     save_csv(metrics, "q1_forecast_metrics.csv")
391     return forecasts, metrics
392
393
394 def origin_forecast_table(monthly: pd.DataFrame) -> pd.DataFrame:
395     origin_period = "2026-02"
396     origin_rows = monthly.loc[monthly["period"].eq(origin_period)].copy()
397     if origin_rows.empty:
398         raise ValueError("Q1 final origin forecast requires 2026-02 in model_monthly_q1.csv.")
399     origin = origin_rows.iloc[0]
400     train = monthly.loc[monthly["period"].le(origin_period)].dropna(subset=["log_brent"]).copy()
401     sigma = float(train["log_brent"].diff().dropna().std())
402     rows: list[dict[str, Any]] = []
403     for horizon in [1, 3, 6]:
404         target_period = str((pd.Period(origin_period, freq="M") + horizon))
405         target = monthly.loc[monthly["period"].eq(target_period)].copy()
406         actual_log = float(target["log_brent"].iloc[0]) if not target.empty and pd.notna(target["log_brent"].iloc[0]) else np.nan
407         actual_price = float(target["brent_usd_bbl"].iloc[0]) if not target.empty and pd.notna(target["brent_usd_bbl"].iloc[0]) else np.
nan
408         prediction_log = float(origin["log_brent"])
409         lower_80, upper_80 = forecast_interval(prediction_log, sigma, horizon, 0.20)
410         lower_95, upper_95 = forecast_interval(prediction_log, sigma, horizon, 0.05)
411         rows.append(
412             {
413                 "origin_period": origin_period,
414                 "origin_price": float(origin["brent_usd_bbl"]),
415                 "origin_log_price": float(origin["log_brent"]),
416                 "target_period": target_period,
417                 "horizon_months": horizon,
418                 "model": "no_change",
419                 "forecast_status": "ACTUAL_AVAILABLE" if np.isfinite(actual_price) else "FORECAST_ONLY",
420                 "prediction_log_price": prediction_log,

```

```

421         "prediction_price": float(np.exp(prediction_log)),
422         "lower_80_price": float(np.exp(lower_80)),
423         "upper_80_price": float(np.exp(upper_80)),
424         "lower_95_price": float(np.exp(lower_95)),
425         "upper_95_price": float(np.exp(upper_95)),
426         "actual_log_price": actual_log if np.isfinite(actual_log) else np.nan,
427         "actual_price": actual_price if np.isfinite(actual_price) else np.nan,
428         "forecast_error_price": actual_price - float(np.exp(prediction_log)) if np.isfinite(actual_price) else np.nan,
429         "actual_change": actual_log - float(origin["log_brent"]) if np.isfinite(actual_log) else np.nan,
430         "predicted_change": 0.0,
431         "sample_start": train["period"].iloc[0],
432         "sample_end": train["period"].iloc[-1],
433         "specification": "competition final no-change forecast from 2026-02 origin; 2026-08 remains forecast-only under
2026-06-30 cutoff",
434     }
435 )
436 result = pd.DataFrame(rows)
437 save_csv(result, "ql_origin_forecast.csv")
438 return result
439
440
441 def add_equal_weight_combination(forecasts: pd.DataFrame) -> pd.DataFrame:
442     if forecasts.empty:
443         return forecasts
444     members = ["no_change", "ARIMA", "SARIMAX", "ETS", "Theta"]
445     rows: list[dict[str, Any]] = []
446     for (_, _), group in forecasts.groupby(["period", "horizon"]):
447         available = group.loc[group["model"].isin(members)].copy()
448         if len(available) < 2:
449             continue
450         template = available.iloc[0].to_dict()
451         prediction = float(available["prediction"].mean())
452         for column in ["lower_80", "upper_80", "lower_95", "upper_95"]:
453             template[column] = float(available[column].mean())
454         template.update(
455             {
456                 "model": "EqualWeight",
457                 "prediction": prediction,
458                 "prediction_price": float(np.exp(prediction)),
459                 "predicted_change": prediction - float(template["origin_log_price"]),
460                 "specification": "equal-weight average of available no_change/ARIMA/SARIMAX/ETS/Theta forecasts",
461             }
462         )
463         rows.append(template)
464     if not rows:
465         return forecasts
466     return pd.concat([forecasts, pd.DataFrame(rows)], ignore_index=True, sort=False)
467
468
469 def forecast_metrics(forecasts: pd.DataFrame) -> pd.DataFrame:
470     rows: list[dict[str, Any]] = []
471     if forecasts.empty:
472         return pd.DataFrame()
473     forecasts = forecasts.sort_values(["model", "horizon", "period"]).copy()
474     forecasts["error"] = forecasts["actual"] - forecasts["prediction"]
475     forecasts["abs_error"] = forecasts["error"].abs()
476     forecasts["squared_error"] = forecasts["error"] ** 2
477     forecasts["covered_80"] = forecasts["actual"].between(forecasts["lower_80"], forecasts["upper_80"])
478     forecasts["covered_95"] = forecasts["actual"].between(forecasts["lower_95"], forecasts["upper_95"])
479     for (model, horizon), group in forecasts.groupby(["model", "horizon"]):
480         group = group.sort_values("period")
481         benchmark = forecasts.loc[
482             forecasts["model"].eq("no_change") & forecasts["horizon"].eq(horizon),
483             ["period", "abs_error", "squared_error"],
484         ].rename(columns={"abs_error": "benchmark_abs_error", "squared_error": "benchmark_squared_error"})
485         paired = group.merge(benchmark, on="period", how="inner")
486         benchmark_mae = float(paired["benchmark_abs_error"].mean()) if not paired.empty else np.nan
487         benchmark_rmse = float(np.sqrt(paired["benchmark_squared_error"].mean())) if not paired.empty else np.nan
488         mae = float(group["abs_error"].mean())
489         rmse = float(np.sqrt(group["squared_error"].mean()))
490         rel_mae = mae / benchmark_mae if np.isfinite(benchmark_mae) and benchmark_mae > 0 else np.nan
491         rel_rmse = rmse / benchmark_rmse if np.isfinite(benchmark_rmse) and benchmark_rmse > 0 else np.nan
492         dm_rmse_stat, dm_rmse_pvalue = dm_hln_test(paired["squared_error"], paired["benchmark_squared_error"], int(horizon))
493         dm_mae_stat, dm_mae_pvalue = dm_hln_test(paired["abs_error"], paired["benchmark_abs_error"], int(horizon))
494         if {"actual_change", "predicted_change"}.issubset(group.columns):
495             directional = group[["actual_change", "predicted_change"]].dropna()

```

```

496         direction_accuracy = (
497             float(np.sign(directional["actual_change"]).eq(np.sign(directional["predicted_change"])).mean())
498             if not directional.empty
499             else np.nan
500         )
501     else:
502         actual_dir = np.sign(group["actual"].diff())
503         pred_dir = np.sign(group["prediction"].diff())
504         direction_accuracy = float((actual_dir.eq(pred_dir)).iloc[1:].mean()) if len(group) > 2 else np.nan
505     status = classify_forecast_model(str(model), rel_rmse, dm_rmse_stat, dm_rmse_pvalue)
506     rows.append(
507         {
508             "model": model,
509             "horizon": int(horizon),
510             "n": int(len(group)),
511             "MAE": mae,
512             "RMSE": rmse,
513             "relative_MAE_vs_no_change": rel_mae,
514             "relative_RMSE_vs_no_change": rel_rmse,
515             "dm_hln_stat_rmse_loss": dm_rmse_stat,
516             "dm_hln_pvalue_rmse_loss": dm_rmse_pvalue,
517             "dm_hln_stat_mae_loss": dm_mae_stat,
518             "dm_hln_pvalue_mae_loss": dm_mae_pvalue,
519             "direction_accuracy": direction_accuracy,
520             "coverage_80": float(group["covered_80"].mean()),
521             "coverage_95": float(group["covered_95"].mean()),
522             "model_status": status,
523         }
524     )
525     return pd.DataFrame(rows)
526
527
528 def monthly_shock_residuals(monthly: pd.DataFrame) -> pd.DataFrame:
529     exog_cols = ["log_brent_lag1", "stock_log_lag1", "usd_broad_index_log_return_lag1", "GPR_z_lag1"]
530     frame = monthly.copy()
531     frame["log_brent_lag1"] = frame["log_brent"].shift(1)
532     usable = frame.dropna(subset=["log_brent"] + exog_cols).copy()
533     x = sm.add_constant(usable[exog_cols], has_constant="add")
534     model = sm.OLS(usable["log_brent"], x).fit(cov_type="HAC", cov_kws={ "maxlags": 3})
535     usable["OilShock_raw"] = model.resid
536     usable["OilShock"] = zscore(usable["OilShock_raw"])
537     result = frame[["period", "month_end", "brent_usd_bbl", "log_brent", "brent_usd_bbl_log_return"]].merge(
538         usable[["period", "OilShock_raw", "OilShock"]],
539         on="period",
540         how="left",
541     )
542     result["OilShock_source"] = "ARX_one_step_residual_lagged_information"
543     return result
544
545
546 def stage_effect_regression(daily: pd.DataFrame, price_prefix: str, specification: str) -> pd.DataFrame:
547     return_col = f"{price_prefix}_log_return"
548     frame = daily.loc[pd.to_datetime(daily["date"]).between(pd.Timestamp("2024-01-01"), CUTOFF)].copy()
549     for lag in [1, 2, 3]:
550         frame[f"{return_col}_lag{lag}"] = frame[return_col].shift(lag)
551     regressors = [f"{return_col}_lag{lag}" for lag in [1, 2, 3]] + ["usd_broad_index_log_return"] + EVENT_TERMS
552     usable = frame.dropna(subset=[return_col] + regressors)
553     if usable.empty:
554         return pd.DataFrame()
555     x = sm.add_constant(usable[regressors], has_constant="add")
556     for term in EVENT_TERMS:
557         event_count = int(usable[term].sum())
558         if event_count == 0 or event_count == len(usable):
559             raise ValueError(f"{term} is not identifiable for {price_prefix}: event_count={event_count}, n={len(usable)}.")
560     rank = int(np.linalg.matrix_rank(x.to_numpy(dtype=float)))
561     if rank < x.shape[1]:
562         raise ValueError(f"Event-stage design is rank deficient for {price_prefix}: rank={rank}, columns={x.shape[1]}.")
563     fit = sm.OLS(usable[return_col], x).fit(cov_type="HAC", cov_kws={ "maxlags": 5})
564     rows: list[dict[str, Any]] = []
565     for term in ["stage_E1", "stage_E2", "stage_E3"]:
566         estimate = float(fit.params.get(term, np.nan))
567         se = float(fit.bse.get(term, np.nan))
568         descriptive_only = term == "stage_E1"
569         rows.append(
570             {
571                 "stage_id": term.replace("stage_", ""),

```



```

572         "model": f"{price_prefix}_stage_dummy",
573         "specification": specification,
574         "estimate_log_return": estimate,
575         "std_error": np.nan if descriptive_only else se,
576         "lower_80": np.nan if descriptive_only else estimate - norm.ppf(0.90) * se,
577         "upper_80": np.nan if descriptive_only else estimate + norm.ppf(0.90) * se,
578         "lower_95": np.nan if descriptive_only else estimate - norm.ppf(0.975) * se,
579         "upper_95": np.nan if descriptive_only else estimate + norm.ppf(0.975) * se,
580         "pvalue": np.nan if descriptive_only else float(fit.pvalues.get(term, np.nan)),
581         "sample_start": str(usable["date"].min().date()),
582         "sample_end": str(usable["date"].max().date()),
583         "n": int(len(usable)),
584         "event_observations": int(usable[term].sum()),
585         "identification_status": "DESCRIPTIVE_ONLY" if descriptive_only else "PASS",
586     }
587 )
588 return pd.DataFrame(rows)
589
590
591 def event_car_rows(daily: pd.DataFrame, price_prefix: str, event_start: pd.Timestamp, model_label: str) -> pd.DataFrame:
592     frame = daily.copy()
593     frame["date"] = pd.to_datetime(frame["date"])
594     return_col = f"{price_prefix}_log_return"
595     for lag in [1, 2, 3]:
596         frame[f"{return_col}_lag{lag}"] = frame[return_col].shift(lag)
597     dates = [value for value in common_trading_dates(frame, price_prefix) if value >= event_start]
598     if len(dates) < 3:
599         return pd.DataFrame()
600     event_dates = dates[:3]
601     train = frame.loc[frame["date"].lt(event_start)].dropna(
602         subset=[return_col, "usd_broad_index_log_return", f"{return_col}_lag1", f"{return_col}_lag2", f"{return_col}_lag3"]
603     )
604     if len(train) < 80:
605         return pd.DataFrame()
606     train = train.tail(520)
607     regressors = [f"{return_col}_lag{lag}" for lag in [1, 2, 3]] + ["usd_broad_index_log_return"]
608     fit = sm.OLS(train[return_col], sm.add_constant(train[regressors].astype(float), has_constant="add")).fit(
609         cov_type="HAC",
610         cov_kws={"maxlags": 5},
611     )
612     sigma = float(fit.resid.std(ddof=1))
613     rows: list[dict[str, Any]] = []
614     cumulative_actual = 0.0
615     cumulative_expected = 0.0
616     recursive_lags = [float(train[return_col].iloc[-lag]) for lag in [1, 2, 3]]
617     pre_event_usd = float(train["usd_broad_index_log_return"].iloc[-1])
618     for horizon, event_date in enumerate(event_dates):
619         event_row = frame.loc[frame["date"].eq(event_date)].dropna(subset=[return_col])
620         if event_row.empty:
621             continue
622         x = pd.DataFrame(
623             [
624                 {
625                     f"{return_col}_lag1": recursive_lags[0],
626                     f"{return_col}_lag2": recursive_lags[1],
627                     f"{return_col}_lag3": recursive_lags[2],
628                     "usd_broad_index_log_return": pre_event_usd,
629                 }
630             ]
631         )
632         x = sm.add_constant(x[regressors].astype(float), has_constant="add")
633         expected = float(fit.predict(x).iloc[0])
634         actual = float(event_row[return_col].iloc[0])
635         cumulative_actual += actual
636         cumulative_expected += expected
637         recursive_lags = [expected, *recursive_lags[:2]]
638         abnormal = cumulative_actual - cumulative_expected
639         se = sigma * math.sqrt(horizon + 1)
640         rows.append(
641             {
642                 "stage_id": f"E1_CAR_0_{horizon}" if horizon else "E1_CAR_0",
643                 "model": model_label,
644                 "specification": "recursive AR(3)+pre-event USD expected-return CAR; E1 mapped to first common trading day",
645                 "estimate_log_return": abnormal,
646                 "actual_cumulative_log_return": cumulative_actual,
647                 "expected_cumulative_log_return": cumulative_expected,

```

```

648         "std_error": se,
649         "lower_80": abnormal - norm.ppf(0.90) * se,
650         "upper_80": abnormal + norm.ppf(0.90) * se,
651         "lower_95": abnormal - norm.ppf(0.975) * se,
652         "upper_95": abnormal + norm.ppf(0.975) * se,
653         "pvalue": 2.0 * (1.0 - norm.cdf(abs(abnormal / se))) if se > 0 else np.nan,
654         "sample_start": str(train["date"].min().date()),
655         "sample_end": str(train["date"].max().date()),
656         "n": int(len(train)),
657         "event_observations": horizon + 1,
658         "event_trading_start": event_start.strftime("%Y-%m-%d"),
659         "event_window_end": event_date.strftime("%Y-%m-%d"),
660         "identification_status": "PASS",
661     }
662 )
663 return pd.DataFrame(rows)
664
665
666 def finite_sample_empirical_pvalue(distribution: pd.Series, threshold: float) -> float:
667     values = pd.to_numeric(distribution, errors="coerce").dropna().abs()
668     if values.empty:
669         return np.nan
670     exceedances = int(values.ge(abs(threshold)).sum())
671     return float((exceedances + 1) / (len(values) + 1))
672
673
674 def placebo_distribution(daily: pd.DataFrame, actual_car: pd.DataFrame) -> pd.DataFrame:
675     frame = daily.copy()
676     frame["date"] = pd.to_datetime(frame["date"])
677     frame["pre_volatility_20d"] = frame["brent_usd_bbl_log_return"].rolling(20, min_periods=15).std().shift(1)
678     frame["pre_brent_level"] = frame["brent_usd_bbl"].shift(1)
679     monthly_gpr = pd.read_csv(PROCESSED_DIR / "model_monthly_ql.csv")[[ "period", "GPR_z" ]]
680     gpr_map = monthly_gpr.set_index("period")["GPR_z"].to_dict()
681     frame["period"] = frame["date"].dt.to_period("M").astype(str)
682     frame["pre_gpr_z"] = frame["period"].map(gpr_map)
683     weekend_calendar = pd.date_range("2024-01-06", "2025-12-28", freq="W-SAT")
684     excluded_event_dates = [
685         pd.Timestamp("2024-04-13"),
686         pd.Timestamp("2024-10-01"),
687         pd.Timestamp("2025-06-13"),
688     ]
689     seen_starts = set(pd.Timestamp) = set()
690     pieces: list[pd.DataFrame] = []
691     for block_id, pseudo_calendar in enumerate(weekend_calendar):
692         if any(abs((pseudo_calendar - event_date).days) <= 7 for event_date in excluded_event_dates):
693             continue
694         start = first_trading_date_on_or_after(frame, pd.Timestamp(pseudo_calendar), "brent_usd_bbl")
695         if start in seen_starts:
696             continue
697         seen_starts.add(start)
698         car = event_car_rows(frame, "brent_usd_bbl", start, "brent_weekend_placebo_car")
699         if car.empty:
700             continue
701         car["pseudo_calendar_date"] = pseudo_calendar.strftime("%Y-%m-%d")
702         car["placebo_block_id"] = block_id
703         state = frame.loc[frame["date"].eq(start)].iloc[0]
704         car["match_pre_volatility_20d"] = float(state["pre_volatility_20d"])
705         car["match_pre_brent_level"] = float(state["pre_brent_level"])
706         car["match_pre_gpr_z"] = float(state["pre_gpr_z"])
707         pieces.append(car)
708     placebo = pd.concat(pieces, ignore_index=True, sort=False) if pieces else pd.DataFrame()
709     if placebo.empty or actual_car.empty:
710         save_csv(placebo, "ql_placebo_distribution.csv")
711         return placebo
712     actual_start = pd.Timestamp(actual_car.loc[actual_car["model"].eq("brent_usd_bbl_event_car"), "event_trading_start"].dropna().iloc[0])
713     actual_state = frame.loc[frame["date"].eq(actual_start)].iloc[0]
714     match_columns = ["match_pre_volatility_20d", "match_pre_brent_level", "match_pre_gpr_z"]
715     block_state = placebo.groupby("placebo_block_id", as_index=False)[match_columns].first().dropna()
716     actual_values = np.array(
717         [float(actual_state["pre_volatility_20d"]), float(actual_state["pre_brent_level"]), float(actual_state["pre_gpr_z"])],
718         dtype=float,
719     )
720     candidate_values = block_state[match_columns].to_numpy(dtype=float)
721     scale = np.nanstd(np.vstack([candidate_values, actual_values]), axis=0, ddof=1)
722     scale = np.where(scale > 1e-12, scale, 1.0)

```

```

723 block_state["match_distance"] = np.sqrt((((candidate_values - actual_values) / scale) ** 2).sum(axis=1))
724 keep_count = min(40, len(block_state))
725 matched_ids = set(block_state.nsmallest(keep_count, "match_distance")["placebo_block_id"].astype(int))
726 distance_map = block_state.set_index("placebo_block_id")["match_distance"].to_dict()
727 placebo["match_distance"] = placebo["placebo_block_id"].map(distance_map)
728 placebo = placebo.loc[placebo["placebo_block_id"].isin(matched_ids)].copy()
729 placebo["matching_status"] = "nearest_40_on_pre_event_volatility_brent_level_monthly_GPR"
730 actual = actual_car.loc[actual_car["model"].eq("brent_usd_bbl_event_car")].copy()
731 empirical: dict[str, float] = {}
732 for stage_id, group in placebo.groupby("stage_id"):
733     actual_row = actual.loc[actual["stage_id"].eq(stage_id)]
734     if actual_row.empty:
735         continue
736     threshold = abs(float(actual_row["estimate_log_return"].iloc[0]))
737     empirical[stage_id] = finite_sample_empirical_pvalue(group["estimate_log_return"], threshold)
738 placebo["actual_empirical_pvalue"] = placebo["stage_id"].map(empirical)
739 save_csv(placebo, "q1_placebo_distribution.csv")
740 return placebo
741
742
743 def add_empirical_pvalues(actual_car: pd.DataFrame, placebo: pd.DataFrame) -> pd.DataFrame:
744     if actual_car.empty or placebo.empty:
745         return actual_car
746     result = actual_car.copy()
747     result["pvalue_empirical"] = np.nan
748     for idx, row in result.iterrows():
749         if row["model"] != "brent_usd_bbl_event_car":
750             continue
751         dist = placebo.loc[placebo["stage_id"].eq(row["stage_id"]), "estimate_log_return"].dropna()
752         if dist.empty:
753             continue
754         threshold = abs(float(row["estimate_log_return"]))
755         result.loc[idx, "pvalue_empirical"] = finite_sample_empirical_pvalue(dist, threshold)
756     return result
757
758
759 def shifted_event_regression(daily: pd.DataFrame, shift_trading_days: int) -> pd.DataFrame:
760     main_start = first_trading_date_on_or_after(daily, EVENT_E1_CALENDAR_START, "brent_usd_bbl")
761     shifted_start = trading_day_shift(daily, main_start, shift_trading_days, "brent_usd_bbl")
762     frame, _ = assign_event_stages(daily, shifted_start, "brent_usd_bbl")
763     result = stage_effect_regression(frame, "brent_usd_bbl", f"event dates shifted by {shift_trading_days} trading days")
764     if not result.empty:
765         result["model"] = f"brent_alt_event_shift_{shift_trading_days:d}td"
766         result["event_trading_start"] = shifted_start.strftime("%Y-%m-%d")
767     return result
768
769
770 def robustness_summary(forecasts: pd.DataFrame, effects: pd.DataFrame, daily: pd.DataFrame, placebos: pd.DataFrame) -> pd.DataFrame:
771     pieces: list[pd.DataFrame] = []
772     if not forecasts.empty:
773         for start in ["2021-01", "2022-01"]:
774             metrics = forecast_metrics(forecasts.loc[forecasts["period"].ge(start)].copy())
775             if not metrics.empty:
776                 metrics["robustness_type"] = f"evaluation_start_{start}"
777                 pieces.append(metrics)
778     wti = effects.loc[effects["model"].eq("wti_usd_bbl_stage_dummy")].copy()
779     if not wti.empty:
780         wti["robustness_type"] = "WTI_event_price"
781         pieces.append(wti)
782     if not placebos.empty:
783         summary = (
784             placebos.groupby("stage_id", as_index=False)
785             .agg(
786                 placebo_mean=("estimate_log_return", "mean"),
787                 placebo_p05=("estimate_log_return", lambda s: float(np.quantile(s.dropna(), 0.05))),
788                 placebo_p95=("estimate_log_return", lambda s: float(np.quantile(s.dropna(), 0.95))),
789                 placebo_events=("estimate_log_return", "count"),
790                 actual_empirical_pvalue=("actual_empirical_pvalue", "first"),
791             )
792         )
793         summary["robustness_type"] = "matched_weekend_placebo_distribution"
794         summary["model"] = "brent_weekend_placebo_car"
795         pieces.append(summary)
796     for shift in [-5, 5]:
797         alt = shifted_event_regression(daily, shift)
798         if not alt.empty:

```

```

799     alt["robustness_type"] = f"event_shift_{shift:+d}trading_days"
800     pieces.append(alt)
801     result = pd.concat(pieces, ignore_index=True, sort=False) if pieces else pd.DataFrame()
802     save_csv(result, "q1_robustness.csv")
803     return result
804
805
806 def daily_counterfactual(daily: pd.DataFrame) -> pd.DataFrame:
807     frame = daily.copy()
808     frame["date"] = pd.to_datetime(frame["date"])
809     frame["log_brent"] = log_positive(frame["brent_usd_bbl"])
810     ret_col = "brent_usd_bbl_log_return"
811     train = frame.loc[frame["date"].between(pd.Timestamp("2024-01-01"), pd.Timestamp("2026-02-27"))].copy()
812     for lag in [1, 2, 3]:
813         train[f"ret_lag{lag}"] = train[ret_col].shift(lag)
814     fit_data = train.dropna(subset=[ret_col, "usd_broad_index_log_return", "ret_lag1", "ret_lag2", "ret_lag3"])
815     x = sm.add_constant(fit_data[[ret_col, "ret_lag1", "ret_lag2", "ret_lag3", "usd_broad_index_log_return"]], has_constant="add")
816     fit = sm.OLS(fit_data[ret_col], x).fit(cov_type="HAC", cov_kwds={"maxlags": 5})
817
818     event_start = first_trading_date_on_or_after(frame, EVENT_E1_CALENDAR_START, "brent_usd_bbl")
819     event = frame.loc[frame["date"].between(event_start, CUTOFF)].copy()
820     event = event.dropna(subset=["brent_usd_bbl"]).reset_index(drop=True)
821     if event.empty:
822         return pd.DataFrame()
823     history_returns = train[ret_col].dropna().tail(3).tolist()
824     cf_log = float(train["log_brent"].dropna().iloc[-1])
825     rows: list[dict[str, Any]] = []
826     sigma = float(fit.resid.std())
827     for _, row in event.iterrows():
828         usd_return = 0.0 if pd.isna(row["usd_broad_index_log_return"]) else float(row["usd_broad_index_log_return"])
829         reg = pd.DataFrame(
830             [
831                 {
832                     "const": 1.0,
833                     "ret_lag1": history_returns[-1],
834                     "ret_lag2": history_returns[-2],
835                     "ret_lag3": history_returns[-3],
836                     "usd_broad_index_log_return": usd_return,
837                 }
838             ]
839         )
840         pred_ret = float(fit.predict(reg).iloc[0])
841         cf_log += pred_ret
842         history_returns.append(pred_ret)
843         actual_log = float(np.log(row["brent_usd_bbl"]))
844         prediction = float(np.exp(cf_log))
845         baseline_gap = float(row["brent_usd_bbl"] - prediction)
846         lower_80_log, upper_80_log = forecast_interval(cf_log, sigma, len(rows) + 1, 0.20)
847         lower_95_log, upper_95_log = forecast_interval(cf_log, sigma, len(rows) + 1, 0.05)
848         rows.append(
849             {
850                 "date": row["date"].strftime("%Y-%m-%d"),
851                 "period": row["date"].to_period("M").strftime("%Y-%m"),
852                 "actual": float(row["brent_usd_bbl"]),
853                 "prediction": prediction,
854                 "response": baseline_gap,
855                 "lower_80": float(np.exp(lower_80_log)),
856                 "upper_80": float(np.exp(upper_80_log)),
857                 "lower_95": float(np.exp(lower_95_log)),
858                 "upper_95": float(np.exp(upper_95_log)),
859                 "actual_log": actual_log,
860                 "ar_baseline_log": cf_log,
861                 "ar_baseline_gap_usd_bbl": baseline_gap,
862                 "war_stage": row["war_stage"],
863                 "model": "AR3_plus_USD_baseline_scenario",
864                 "horizon": len(rows) + 1,
865                 "specification": "descriptive AR(3)+USD baseline trained 2024-01-01 to 2026-02-27; not a causal no-war counterfactual",
866                 "sample_start": "2024-01-01",
867                 "sample_end": "2026-02-27",
868             }
869         )
870     counterfactual = pd.DataFrame(rows)
871     save_csv(counterfactual, "q1_daily_counterfactual.csv")
872     return counterfactual
873
874

```

```

875 def residualize(series: pd.Series, controls: pd.DataFrame) -> pd.Series:
876     frame = pd.concat([series.rename("target"), controls], axis=1).dropna()
877     result = pd.Series(np.nan, index=series.index, dtype=float)
878     if len(frame) < controls.shape[1] + 12:
879         return result
880     fit = sm.OLS(frame["target"], sm.add_constant(frame.drop(columns=["target"]).astype(float), has_constant="add")).fit()
881     result.loc[frame.index] = fit.resid
882     return result
883
884
885 def safe_whiteness_pvalue(fit: Any, lags: int) -> float:
886     nlags = max(int(fit.k_ar) + 1, lags)
887     try:
888         return float(fit.test_whiteness(nlags=nlags).pvalue)
889     except (ValueError, np.linalg.LinAlgError):
890         return np.nan
891
892
893 def structural_shock_decomposition(monthly: pd.DataFrame) -> pd.DataFrame:
894     input_path = PROCESSED_DIR / "global_oil_svar_monthly.csv"
895     if not input_path.exists():
896         raise FileNotFoundError("global_oil_svar_monthly.csv is required for Q1 SVAR structural shocks.")
897     global_input = pd.read_csv(input_path)
898     frame = (
899         monthly[["period", "month_end", "brent_usd_bbl_log_return"]]
900         .merge(global_input, on="period", how="left")
901         .sort_values("period")
902         .reset_index(drop=True)
903     )
904     frame["supply_growth"] = np.log(pd.to_numeric(frame["world_liquids_supply"], errors="coerce")).diff() * 100.0
905     frame["real_brent_return"] = np.log(pd.to_numeric(frame["real_brent"], errors="coerce")).diff() * 100.0
906     var_cols = ["supply_growth", "global_real_activity", "real_brent_return"]
907     var_data = frame.dropna(subset=var_cols).copy()
908     if len(var_data) < 120:
909         raise ValueError(f"Q1 SVAR common sample has only {len(var_data)} months; required >=120.")
910     var_data = var_data.set_index("period")
911     stationarity_rows: list[dict[str, Any]] = []
912     for column in var_cols:
913         values = var_data[column].astype(float).to_numpy()
914         adf = adfuller(values, autolag="BIC")
915         try:
916             kpss_result = kpss(values, regression="c", nlags="auto")
917             kpss_stat, kpss_pvalue = float(kpss_result[0]), float(kpss_result[1])
918         except (ValueError, np.linalg.LinAlgError):
919             kpss_stat, kpss_pvalue = np.nan, np.nan
920         stationarity_rows.append(
921             {
922                 "variable": column,
923                 "adf_statistic": float(adf[0]),
924                 "adf_pvalue": float(adf[1]),
925                 "adf_lags": int(adf[2]),
926                 "kpss_statistic": kpss_stat,
927                 "kpss_pvalue": kpss_pvalue,
928                 "stationarity_gate": bool(float(adf[1]) < 0.05 and (not np.isfinite(kpss_pvalue) or kpss_pvalue >= 0.05)),
929                 "sample_start": var_data.index.min(),
930                 "sample_end": var_data.index.max(),
931                 "n": int(len(values)),
932             }
933         )
934     save_csv(pd.DataFrame(stationarity_rows), "q1_stationarity_diagnostics.csv")
935     candidates: list[dict[str, Any]] = []
936     fits: dict[int, Any] = {}
937     for lag in [6, 12, 18, 24]:
938         if len(var_data) <= lag + 36:
939             candidates.append(
940                 {
941                     "candidate_lags": lag,
942                     "is_selected": False,
943                     "bic": np.nan,
944                     "aic": np.nan,
945                     "hqic": np.nan,
946                     "is_stable": False,
947                     "whiteness_pvalue": np.nan,
948                     "sample_start": var_data.index.min(),
949                     "sample_end": var_data.index.max(),
950                     "nobs": int(len(var_data)),

```

```

951         "ordering": "supply_growth -> global_real_activity -> real_brent_return",
952         "error": "insufficient_sample_for_lag",
953     }
954 )
955 continue
956 try:
957     fit = VAR(var_data[var_cols]).fit(lag)
958     stable = bool(fit.is_stable(verbose=False))
959     fits[lag] = fit
960     candidates.append(
961         {
962             "candidate_lags": lag,
963             "is_selected": False,
964             "bic": float(fit.bic),
965             "aic": float(fit.aic),
966             "hqic": float(fit.hqic),
967             "is_stable": stable,
968             "whiteness_pvalue": safe_whiteness_pvalue(fit, 24),
969             "minimum_root_modulus": float(np.min(np.abs(fit.roots))),
970             "sample_start": var_data.index.min(),
971             "sample_end": var_data.index.max(),
972             "nobs": int(fit.nobs),
973             "ordering": "supply_growth -> global_real_activity -> real_brent_return",
974             "error": "",
975         }
976     )
977 except (ValueError, np.linalg.LinAlgError) as exc:
978     candidates.append(
979         {
980             "candidate_lags": lag,
981             "is_selected": False,
982             "bic": np.nan,
983             "aic": np.nan,
984             "hqic": np.nan,
985             "is_stable": False,
986             "whiteness_pvalue": np.nan,
987             "sample_start": var_data.index.min(),
988             "sample_end": var_data.index.max(),
989             "nobs": int(len(var_data)),
990             "ordering": "supply_growth -> global_real_activity -> real_brent_return",
991             "error": f"{type(exc).__name__}: {exc}",
992         }
993     )
994 diag = pd.DataFrame(candidates)
995 selectable = diag.loc[diag["is_stable"].eq(True) & diag["bic"].notna()].sort_values("bic")
996 if selectable.empty:
997     raise ValueError("No stable Q1 VAR candidate among lags 6/12/18/24.")
998 selected_lag = int(selectable.iloc[0]["candidate_lags"])
999 fit = fits[selected_lag]
1000 diag.loc[diag["candidate_lags"].eq(selected_lag), "is_selected"] = True
1001
1002 sigma_u = np.asarray(fit.sigma_u, dtype=float)
1003 chol = np.linalg.cholesky(sigma_u)
1004 residuals = fit.resid[var_cols].copy()
1005 structural = np.linalg.solve(chol, residuals.to_numpy(dtype=float).T).T
1006
1007 main_risk = pd.Series(structural[:, 2], index=residuals.index)
1008 alternative_orderings = {
1009     "activity_supply_price": ["global_real_activity", "supply_growth", "real_brent_return"],
1010     "supply_price_activity": ["supply_growth", "real_brent_return", "global_real_activity"],
1011 }
1012 alt_correlations: dict[str, float] = {}
1013 for label, alt_cols in alternative_orderings.items():
1014     alt_fit = VAR(var_data[alt_cols]).fit(selected_lag)
1015     corr = np.nan
1016     if alt_fit.is_stable(verbose=False):
1017         alt_chol = np.linalg.cholesky(np.asarray(alt_fit.sigma_u, dtype=float))
1018         alt_structural = np.linalg.solve(alt_chol, alt_fit.resid[alt_cols].to_numpy(dtype=float).T).T
1019         price_index = alt_cols.index("real_brent_return")
1020         alt_risk = pd.Series(alt_structural[:, price_index], index=alt_fit.resid.index)
1021         common = pd.concat([main_risk.rename("main"), alt_risk.rename("alt")], axis=1).dropna()
1022         corr = float(common.corr().iloc[0, 1]) if len(common) > 2 else np.nan
1023     alt_correlations[label] = corr
1024     diag[f"alternative_ordering_corr_{label}"] = np.nan
1025     diag.loc[diag["candidate_lags"].eq(selected_lag), f"alternative_ordering_corr_{label}"] = corr
1026     diag[f"alternative_ordering_price_shock_corr"] = np.nan

```

```

1027 diag.loc[diag["candidate_lags"].eq(selected_lag), "alternative_ordering_price_shock_corr"] = alt_correlations["activity_supply_price"]
1028 ]
1029 save_csv(diag, "ql_svar_diagnostics.csv")
1030
1031 shock_frame = pd.DataFrame(
1032     {
1033         "period": residuals.index.astype(str),
1034         "supply_shock": zscore(pd.Series(~structural[:, 0])),
1035         "aggregate_demand_shock": zscore(pd.Series(structural[:, 1])),
1036         "oil_specific_risk_shock": zscore(pd.Series(structural[:, 2])),
1037     }
1038 )
1039 result = frame[["period"]].copy().merge(shock_frame, on="period", how="left")
1040 result["reduced_form_shock"] = zscore(frame["brent_usd_bbl_log_return"])
1041 result["source_vintage"] = frame["source_vintage"].fillna("global_oil_svar_input").astype(str)
1042 result["svar_lags"] = selected_lag
1043 result["svar_ordering"] = "supply_growth -> global_real_activity -> real_brent_return"
1044 save_csv(result, "ql_structural_shocks.csv")
1045 return result
1046
1047 def gjr_garch_variance(returns: pd.Series) -> tuple[np.ndarray, dict[str, float]]:
1048     y = returns.dropna().to_numpy(dtype=float)
1049     y = y - np.mean(y)
1050     if len(y) < 80:
1051         raise ValueError("GJR-GARCH needs at least 80 daily return observations.")
1052     sample_var = float(np.var(y, ddof=1))
1053
1054     def neg_loglike(params: np.ndarray) -> float:
1055         omega, alpha, gamma, beta = params
1056         if omega <= 0 or alpha < 0 or gamma < 0 or beta < 0 or alpha + 0.5 * gamma + beta >= 0.999:
1057             return 1e12
1058         h = np.empty_like(y)
1059         h[0] = sample_var
1060         for idx in range(1, len(y)):
1061             lag_eps = y[idx - 1]
1062             h[idx] = omega + alpha * lag_eps**2 + gamma * (lag_eps < 0) * lag_eps**2 + beta * h[idx - 1]
1063             if h[idx] <= 0 or not np.isfinite(h[idx]):
1064                 return 1e12
1065         return float(0.5 * np.sum(np.log(2 * np.pi) + np.log(h) + y**2 / h))
1066
1067     constraints = ({ "type": "ineq", "fun": lambda p: 0.999 - p[1] - 0.5 * p[2] - p[3]},)
1068     fit = minimize(
1069         neg_loglike,
1070         x0=np.array([0.02 * sample_var, 0.05, 0.05, 0.88]),
1071         bounds=[(1e-9, None), (0.0, 1.0), (0.0, 1.0), (0.0, 0.999)],
1072         constraints=constraints,
1073         method="SLSQP",
1074         options={"maxiter": 500, "ftol": 1e-9},
1075     )
1076     if not fit.success:
1077         raise RuntimeError(f"GJR-GARCH optimization failed: {fit.message}")
1078     omega, alpha, gamma, beta = fit.x
1079     h = np.empty_like(y)
1080     h[0] = sample_var
1081     for idx in range(1, len(y)):
1082         lag_eps = y[idx - 1]
1083         h[idx] = omega + alpha * lag_eps**2 + gamma * (lag_eps < 0) * lag_eps**2 + beta * h[idx - 1]
1084     return h, {"omega": float(omega), "alpha": float(alpha), "gamma": float(gamma), "beta": float(beta)}
1085
1086
1087 def volatility_module(daily: pd.DataFrame, event_start: pd.Timestamp) -> pd.DataFrame:
1088     frame = daily.copy()
1089     frame["date"] = pd.to_datetime(frame["date"])
1090     frame = frame.loc[frame["date"].between(pd.Timestamp("2024-01-01"), CUTOFF)].sort_values("date").copy()
1091     frame["return_pct"] = frame["brent_usd_bbl_log_return"] * 100.0
1092     usable = frame.dropna(subset=["return_pct"]).copy()
1093     train = usable.loc[usable["date"].lt(event_start)].copy()
1094     h_train, params = gjr_garch_variance(train["return_pct"])
1095     omega, alpha, gamma, beta = params["omega"], params["alpha"], params["gamma"], params["beta"]
1096     y = usable["return_pct"].to_numpy(dtype=float)
1097     y = y - float(train["return_pct"].mean())
1098     h = np.empty(len(y), dtype=float)
1099     h[: len(h_train)] = h_train
1100     for idx in range(len(h_train), len(y)):
1101         lag_eps = y[idx - 1]

```

```

1102     h[idx] = omega + alpha * lag_eps**2 + gamma * (lag_eps < 0) * lag_eps**2 + beta * h[idx - 1]
1103     usable["conditional_vol_pct"] = np.sqrt(h)
1104     pre_median = float(np.nanmedian(usable.loc[usable["date"].lt(event_start), "conditional_vol_pct"]))
1105     usable["abnormal_vol_ratio_vs_pre_median"] = usable["conditional_vol_pct"] / pre_median - 1.0
1106     usable["model"] = "GJR_GARCH_1_1"
1107     usable["specification"] = "normal-likelihood GJR-GARCH(1,1) fitted on pre-E1 Brent daily returns"
1108     for key, value in params.items():
1109         usable[key] = value
1110     output = usable[
1111         [
1112             "date",
1113             "return_pct",
1114             "conditional_vol_pct",
1115             "abnormal_vol_ratio_vs_pre_median",
1116             "model",
1117             "specification",
1118             "omega",
1119             "alpha",
1120             "gamma",
1121             "beta",
1122         ]
1123     ]
1124     save_csv(output, "q1_volatility.csv")
1125     return output
1126
1127
1128 def volatility_stage_summary(volatility: pd.DataFrame, daily: pd.DataFrame) -> pd.DataFrame:
1129     if volatility.empty:
1130         result = pd.DataFrame()
1131         save_csv(result, "q1_volatility_summary.csv")
1132         return result
1133     frame = volatility.copy()
1134     frame["date"] = pd.to_datetime(frame["date"])
1135     stages = daily[["date", "war_stage"]].copy()
1136     stages["date"] = pd.to_datetime(stages["date"])
1137     frame = frame.merge(stages, on="date", how="left")
1138     rows: list[dict[str, Any]] = []
1139     pre = frame.loc[frame["war_stage"].eq("prewar"), "conditional_vol_pct"].dropna()
1140     pre_median = float(pre.median()) if not pre.empty else np.nan
1141     for stage in ["prewar", "E1_immediate_window", "E2_disruption", "E3_easing"]:
1142         group = frame.loc[frame["war_stage"].eq(stage)].copy()
1143         if group.empty:
1144             continue
1145         vol = pd.to_numeric(group["conditional_vol_pct"], errors="coerce").dropna()
1146         ratio = pd.to_numeric(group["abnormal_vol_ratio_vs_pre_median"], errors="coerce").dropna()
1147         rows.append(
1148             {
1149                 "war_stage": stage,
1150                 "stage_start": group["date"].min().strftime("%Y-%m-%d"),
1151                 "stage_end": group["date"].max().strftime("%Y-%m-%d"),
1152                 "trading_days": int(len(group)),
1153                 "conditional_vol_pct_mean": float(vol.mean()) if not vol.empty else np.nan,
1154                 "conditional_vol_pct_median": float(vol.median()) if not vol.empty else np.nan,
1155                 "conditional_vol_pct_p10": float(vol.quantile(0.10)) if not vol.empty else np.nan,
1156                 "conditional_vol_pct_p90": float(vol.quantile(0.90)) if not vol.empty else np.nan,
1157                 "abnormal_vol_ratio_mean": float(ratio.mean()) if not ratio.empty else np.nan,
1158                 "abnormal_vol_ratio_median": float(ratio.median()) if not ratio.empty else np.nan,
1159                 "vol_multiple_vs_pre_median": float(vol.median() / pre_median) if np.isfinite(pre_median) and pre_median > 0 and not vol.
1160                 empty else np.nan,
1161                 "model": "GJR_GARCH_1_1",
1162                 "evidence_status": "DESCRIPTIVE_VOLATILITY",
1163             }
1164         )
1165     result = pd.DataFrame(rows)
1166     save_csv(result, "q1_volatility_summary.csv")
1167     return result
1168
1169 def make_q1_shocks(monthly: pd.DataFrame, counterfactual: pd.DataFrame) -> pd.DataFrame:
1170     shocks = monthly_shock_residuals(monthly)
1171     structural = structural_shock_decomposition(monthly)
1172     shocks = shocks.merge(structural, on="period", how="left")
1173     if counterfactual.empty:
1174         baseline = pd.DataFrame({"period": shocks["period"], "ARBaselineGap": 0.0})
1175     else:
1176         baseline = (

```



```

1177     counterfactual.groupby("period", as_index=False)
1178     .agg(ARBaselineGap=("ar_baseline_gap_usd_bbl", "mean"), ARBaselineGap_days=("ar_baseline_gap_usd_bbl", "count"))
1179 )
1180 shocks = shocks.merge(baseline, on="period", how="left")
1181 shocks["ARBaselineGap"] = shocks["ARBaselineGap"].fillna(0.0)
1182 shocks["ARBaselineGap_days"] = shocks["ARBaselineGap_days"].fillna(0).astype(int) if "ARBaselineGap_days" in shocks else 0
1183 shocks["random_seed"] = RANDOM_SEED
1184 save_csv(shocks, "ql_monthly_shocks.csv")
1185 return shocks
1186
1187
1188 def plot_forecasts(forecasts: pd.DataFrame) -> None:
1189     if forecasts.empty:
1190         return
1191     one = forecasts.loc[forecasts["horizon"].eq(1)].copy()
1192     if one.empty:
1193         return
1194     one["date"] = pd.to_datetime(one["period"] + "-01") + pd.offsets.MonthEnd(0)
1195     pivot = one.pivot_table(index="date", columns="model", values="prediction_price", aggfunc="first")
1196     actual = one.drop_duplicates("date").set_index("date")["actual_price"]
1197     fig, ax = plt.subplots(figsize=(9.2, 5.1))
1198     ax.plot(actual.index, actual, label="实际 Brent", color=PALETTE["ink"], linewidth=2.0)
1199     style_map = {
1200         "no_change": (PALETTE["slate"], (0, (2, 2))), "不变预测",
1201         "ARIMA": (PALETTE["gold"], (0, (4, 2))), "ARIMA",
1202         "SARIMAX": (PALETTE["blue"], "solid", "SARIMAX"),
1203     }
1204     for model in ["no_change", "ARIMA", "SARIMAX"]:
1205         if model in pivot.columns:
1206             color, linestyle, label = style_map[model]
1207             ax.plot(pivot.index, pivot[model], label=label, color=color, linestyle=linestyle, linewidth=1.55)
1208     style_axis(ax, ylabel="美元/桶")
1209     ax.legend(loc="upper left", ncol=2, handlelength=2.8)
1210     finish_figure(
1211         fig,
1212         title="问题一: Brent 一个月期滚动预测",
1213         subtitle="月度 Brent 现货价, 滚动起点评估区间为 2020-01 至 2026-06。",
1214         source="来源: FRED/EIA 处理面板; 由 code/problem1/run_ql.py 生成。",
1215     )
1216     save_figure(fig, FIGURES_DIR / "ql_forecast_lm")
1217     plt.close(fig)
1218
1219
1220 def plot_counterfactual(counterfactual: pd.DataFrame) -> None:
1221     if counterfactual.empty:
1222         return
1223     frame = counterfactual.copy()
1224     frame["date"] = pd.to_datetime(frame["date"])
1225     fig, ax = plt.subplots(figsize=(9.2, 5.1))
1226     ax.plot(frame["date"], frame["actual"], label="实际 Brent", color=PALETTE["ink"], linewidth=2.0)
1227     ax.plot(frame["date"], frame["prediction"], label="AR基准情景路径", color=PALETTE["blue"], linewidth=1.65)
1228     ax.fill_between(
1229         frame["date"],
1230         frame["lower_80"],
1231         frame["upper_80"],
1232         color=PALETTE["blue_light"],
1233         alpha=0.28,
1234         linewidth=0,
1235         label="80% 区间",
1236     )
1237     for event_date, label, ypos in [
1238         (pd.Timestamp("2026-03-02"), "E1", 0.95),
1239         (pd.Timestamp("2026-03-05"), "E2", 0.88),
1240         (pd.Timestamp("2026-06-17"), "E3", 0.95),
1241     ]:
1242         ax.axvline(event_date, color=PALETTE["muted"], linewidth=0.7, linestyle=(0, (2, 2)))
1243         ax.text(
1244             event_date,
1245             ypos,
1246             label,
1247             transform=ax.get_xaxis_transform(),
1248             ha="center",
1249             va="top",
1250             fontsize=8.5,
1251             color=PALETTE["muted"],
1252             bbox={"facecolor": "white", "edgecolor": "none", "pad": 1.2, "alpha": 0.85},

```

```

1253     )
1254     style_axis(ax, ylabel="美元/桶")
1255     ax.legend(loc="upper left", ncol=3, handlelength=2.8)
1256     finish_figure(
1257         fig,
1258         title="问题一: 事件后 Brent 油价与 AR 基准路径",
1259         subtitle="日度 AR(3)+美元基准情景, 事件窗口 2026-03-02 至 2026-06-30; 不作战争因果贡献解释。",
1260         source="来源: FRED Brent 与 广义美元指数; 由 code/problem1/run_q1.py 生成。",
1261     )
1262     save_figure(fig, FIGURES_DIR / "q1_war_counterfactual")
1263     plt.close(fig)
1264
1265
1266 def plot_structural_shocks(shocks: pd.DataFrame) -> None:
1267     columns = ["supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock"]
1268     if shocks.empty or not set(columns).issubset(shocks.columns):
1269         return
1270     frame = shocks.dropna(subset=columns).copy()
1271     frame["date"] = pd.to_datetime(frame["period"] + "-01") + pd.offsets.MonthEnd(0)
1272     labels = ["不利供给冲击", "总需求冲击", "石油特定风险冲击"]
1273     colors = [PALETTE["rose"], PALETTE["olive"], PALETTE["blue"]]
1274     fig, axes = plt.subplots(3, 1, figsize=(9.0, 6.4), sharex=True)
1275     for ax, column, label, color in zip(axes, columns, labels, colors):
1276         ax.axhline(0.0, color=PALETTE["muted"], linewidth=0.7)
1277         ax.plot(frame["date"], frame[column], color=color, linewidth=0.9)
1278         ax.text(0.01, 0.88, label, transform=ax.transAxes, ha="left", va="top", fontsize=9.4)
1279         style_axis(ax, ylabel="标准差")
1280     finish_figure(
1281         fig,
1282         title="问题一: 递归SVAR结构冲击序列",
1283         subtitle="供给残差已变号, 因此正值表示不利供给收缩; 三类冲击均按各自样本标准化。",
1284         source="来源: results/q1_structural_shocks.csv; 由 code/problem1/run_q1.py 生成。",
1285         rect=(0.08, 0.08, 0.98, 0.90),
1286     )
1287     save_figure(fig, FIGURES_DIR / "q1_structural_shocks")
1288     plt.close(fig)
1289
1290
1291 def refresh_placebo_pvalues() -> dict[str, Any]:
1292     placebo_path = RESULTS_DIR / "q1_placebo_distribution.csv"
1293     effects_path = RESULTS_DIR / "q1_event_effects.csv"
1294     summary_path = RESULTS_DIR / "q1_summary.json"
1295     robustness_path = RESULTS_DIR / "q1_robustness.csv"
1296     if not placebo_path.exists() or not effects_path.exists() or not summary_path.exists():
1297         raise FileNotFoundError("Saved Q1 placebo, event-effect, and summary outputs are required.")
1298
1299     placebos = pd.read_csv(placebo_path)
1300     effects = pd.read_csv(effects_path)
1301     actual = effects.loc[effects["model"].eq("brent_usd_bbl_event_car")].copy()
1302     empirical: dict[str, float] = {}
1303     for stage_id, group in placebos.groupby("stage_id"):
1304         actual_row = actual.loc[actual["stage_id"].eq(stage_id)]
1305         if actual_row.empty:
1306             continue
1307         empirical[stage_id] = finite_sample_empirical_pvalue(
1308             group["estimate_log_return"],
1309             float(actual_row["estimate_log_return"].iloc[0]),
1310         )
1311
1312     placebos["actual_empirical_pvalue"] = placebos["stage_id"].map(empirical)
1313     event_mask = effects["model"].eq("brent_usd_bbl_event_car")
1314     effects.loc[event_mask, "pvalue_empirical"] = effects.loc[event_mask, "stage_id"].map(empirical)
1315     save_csv(placebos, "q1_placebo_distribution.csv")
1316     save_csv(effects, "q1_event_effects.csv")
1317
1318     if robustness_path.exists():
1319         robustness = pd.read_csv(robustness_path)
1320         placebo_mask = robustness.get("robustness_type", pd.Series(index=robustness.index, dtype=object)).eq(
1321             "matched_weekend_placebo_distribution"
1322         )
1323         if "actual_empirical_pvalue" in robustness.columns:
1324             robustness.loc[placebo_mask, "actual_empirical_pvalue"] = robustness.loc[placebo_mask, "stage_id"].map(empirical)
1325             save_csv(robustness, "q1_robustness.csv")
1326
1327     summary = json.loads(summary_path.read_text(encoding="utf-8"))
1328     summary["placebo_min_empirical_pvalue"] = min(empirical.values()) if empirical else None

```

```

1329 summary_path.write_text(
1330     json.dumps(summary, ensure_ascii=False, indent=2, allow_nan=False) + "\n",
1331     encoding="utf-8",
1332 )
1333 return {
1334     "status": "PASS",
1335     "mode": "refresh-placebo-only",
1336     "empirical_pvalues": empirical,
1337     "placebo_rows": int(len(placebos)),
1338 }
1339
1340
1341 def main(argv: list[str] | None = None) -> int:
1342     parser = argparse.ArgumentParser()
1343     parser.add_argument(
1344         "--refresh-placebo-only",
1345         action="store_true",
1346         help="Apply finite-sample empirical-p corrections to saved Q1 outputs without refitting models.",
1347     )
1348     parser.add_argument("--plots-only", action="store_true", help="Regenerate Q1 figures from saved result tables.")
1349     args = parser.parse_args(argv)
1350
1351     np.random.seed(RANDOM_SEED)
1352     apply_paper_style()
1353     py_warnings.filterwarnings("ignore", category=ValueWarning)
1354     py_warnings.filterwarnings("ignore", category=FutureWarning, message="No supported index*")
1355     ensure_dirs()
1356     if args.refresh_placebo_only:
1357         payload = refresh_placebo_pvalues()
1358         print(json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False))
1359         return 0
1360     if args.plots_only:
1361         forecasts = pd.read_csv(RESULTS_DIR / "q1_forecasts.csv")
1362         counterfactual = pd.read_csv(RESULTS_DIR / "q1_daily_counterfactual.csv")
1363         shocks = pd.read_csv(RESULTS_DIR / "q1_structural_shocks.csv")
1364         plot_forecasts(forecasts)
1365         plot_counterfactual(counterfactual)
1366         plot_structural_shocks(shocks)
1367         print(json.dumps({"status": "PASS", "mode": "plots-only", "figures": 3}, ensure_ascii=False, indent=2))
1368         return 0
1369
1370     warnings_log: list[dict[str, Any]] = []
1371     monthly = pd.read_csv(PROCESSED_DIR / "model_monthly_q1.csv", parse_dates=["month_end"])
1372     daily = pd.read_csv(PROCESSED_DIR / "model_daily_q1.csv", parse_dates=["date"])
1373     daily, event_meta = assign_event_stages(daily, None, "brent_usd_bbl")
1374
1375     forecasts, metrics = monthly_forecasts(monthly, warnings_log)
1376     origin_forecast = origin_forecast_table(monthly)
1377     stage_effects = pd.concat(
1378         [
1379             stage_effect_regression(daily, "brent_usd_bbl", "main Brent event-stage dummy with Newey-West SE"),
1380             stage_effect_regression(daily, "wti_usd_bbl", "WTI robustness event-stage dummy with Newey-West SE"),
1381         ],
1382         ignore_index=True,
1383     )
1384     event_start = pd.Timestamp(event_meta["e1_trading_start"])
1385     car_effects_raw = pd.concat(
1386         [
1387             event_car_rows(daily, "brent_usd_bbl", event_start, "brent_usd_bbl_event_car"),
1388             event_car_rows(daily, "wti_usd_bbl", event_start, "wti_usd_bbl_event_car"),
1389         ],
1390         ignore_index=True,
1391     )
1392     placebos = placebo_distribution(daily, car_effects_raw)
1393     car_effects = add_empirical_pvalues(car_effects_raw, placebos)
1394     effects = pd.concat([stage_effects, car_effects], ignore_index=True, sort=False)
1395     save_csv(effects, "q1_event_effects.csv")
1396     volatility = volatility_module(daily, event_start)
1397     volatility_summary = volatility_stage_summary(volatility, daily)
1398     counterfactual = daily_counterfactual(daily)
1399     shocks = make_q1_shocks(monthly, counterfactual)
1400     robustness = robustness_summary(forecasts, effects, daily, placebos)
1401     plot_forecasts(forecasts)
1402     plot_counterfactual(counterfactual)
1403     plot_structural_shocks(shocks)
1404

```

```

1405 advanced_non_pass = []
1406 if not metrics.empty and "model_status" in metrics.columns:
1407     advanced_non_pass = metrics.loc[
1408         metrics["model"].ne("no_change") & metrics["model_status"].ne("PASS"),
1409         ["model", "horizon", "relative_RMSE_vs_no_change", "model_status"],
1410     ]
1411     advanced_non_pass = json_records(advanced_non_pass)
1412 placebo_min_p = (
1413     float(car_effects.loc[car_effects["model"].eq("brent_usd_bbl_event_car"), "pvalue_empirical"].dropna().min())
1414     if "pvalue_empirical" in car_effects and not car_effects.loc[car_effects["model"].eq("brent_usd_bbl_event_car"), "
1415     pvalue_empirical"].dropna().empty
1416     else np.nan
1417 )
1418 no_change_rows = metrics.loc[metrics["model"].eq("no_change")] if not metrics.empty else pd.DataFrame()
1419 incorrectly_passing_advanced = (
1420     metrics.loc[
1421         metrics["model"].ne("no_change")
1422         & metrics["relative_RMSE_vs_no_change"].gt(1.0)
1423         & metrics["model_status"].eq("PASS")
1424     ]
1425     if not metrics.empty
1426     else pd.DataFrame()
1427 )
1428 status = "PASS"
1429 if warnings_log:
1430     status = "CONDITIONAL"
1431 if no_change_rows.empty or not no_change_rows["model_status"].eq("PASS").all() or not incorrectly_passing_advanced.empty:
1432     status = "FAIL"
1433 structural_counts = {
1434     column: int(shocks[column].dropna().shape[0]) if column in shocks.columns else 0
1435     for column in ["supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock"]
1436 }
1437 summary = {
1438     "status": status,
1439     "execution_status": "PASS" if status != "FAIL" else "FAIL",
1440     "evidence_status": "SUPPORTED",
1441     "allowed_claims": [
1442         "no-change is the selected 1/3/6-month forecast benchmark under the fixed backtest",
1443         "EI effects are reported as CARs with empirical placebo p-values",
1444         "ARBaselineGap is a descriptive AR baseline gap, not a causal war premium",
1445         "SVAR shocks are ex-post structural inputs for Q2/Q3 transmission analysis",
1446     ],
1447     "forbidden_claims": [
1448         "ARIMA/SARIMAX necessarily outperform no-change",
1449         "ARBaselineGap equals the net causal contribution of war",
1450         "EI single-day HAC p-values are formal causal evidence",
1451     ],
1452     "random_seed": RANDOM_SEED,
1453     "event_calendar": event_meta,
1454     "selected_forecast_model": "no_change",
1455     "structural_shock_nonmissing_months": structural_counts,
1456     "forecast_model_non_pass": advanced_non_pass,
1457     "placebo_min_empirical_pvalue": placebo_min_p if np.isfinite(placebo_min_p) else None,
1458     "forecast_rows": int(len(forecasts)),
1459     "origin_forecast_rows": int(len(origin_forecast)),
1460     "metric_rows": int(len(metrics)),
1461     "event_effect_rows": int(len(effects)),
1462     "placebo_rows": int(len(placebos)),
1463     "shock_rows": int(len(shocks)),
1464     "volatility_rows": int(len(volatility)),
1465     "volatility_summary_rows": int(len(volatility_summary)),
1466     "robustness_rows": int(len(robustness)),
1467     "warnings": warnings_log,
1468     "main_metric_best_rmse": json_records(metrics.sort_values("RMSE").head(1)) if not metrics.empty else [],
1469 }
1470 (RESULTS_DIR / "q1_summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")
1471 print(json.dumps(summary, ensure_ascii=False, indent=2, allow_nan=False))
1472 return 0
1473
1474 if __name__ == "__main__":
1475     raise SystemExit(main())

```

Listing 6: run_q1.py

```

1 """Question 2: transmission of oil shocks to China's macro variables."""
2
3 from __future__ import annotations
4
5 import argparse
6 import json
7 import sys
8 from pathlib import Path
9 from typing import Any
10
11 import matplotlib
12
13 matplotlib.use("Agg")
14
15 import matplotlib.pyplot as plt
16 import numpy as np
17 import pandas as pd
18 import statsmodels.api as sm
19 from scipy.stats import norm
20
21
22 REPO_ROOT = Path(__file__).resolve().parents[2]
23 CODE_DIR = REPO_ROOT / "code"
24 if str(CODE_DIR) not in sys.path:
25     sys.path.insert(0, str(CODE_DIR))
26
27 from utils.plot_style import PALETTE, apply_paper_style, finish_figure, save_figure, style_axis
28
29 if hasattr(sys.stdout, "reconfigure"):
30     sys.stdout.reconfigure(encoding="utf-8")
31
32 PROCESSED_DIR = REPO_ROOT / "data" / "processed"
33 RESULTS_DIR = REPO_ROOT / "results"
34 FIGURES_DIR = REPO_ROOT / "figures"
35 RANDOM_SEED = 20260730
36 BOOTSTRAP_REPS = 2000
37 SHOCK_SPECS = {
38     "supply_shock": "adverse_supply_shock_from_historical_recursive_VAR",
39     "aggregate_demand_shock": "aggregate_demand_shock_from_historical_recursive_VAR",
40     "oil_specific_risk_shock": "oil_specific_risk_shock_from_historical_recursive_VAR",
41     "OilShock": "reduced_form_ARX_oil_price_innovation_robustness",
42 }
43
44
45 OUTCOME_SPECS = {
46     "china_iav_yoy_pct": "中国规模以上工业增加值同比，百分点",
47     "china_ppi_yoy_pct": "中国PPI同比，百分点",
48     "china_cpi_yoy_pct": "中国CPI同比，百分点",
49     "china_fx_log_change_pct": "人民币兑美元月度对数变化，百分点",
50     "brent_cny_cost_log_change_pct": "人民币原油成本月度对数变化，百分点",
51 }
52
53 OUTCOME_RESPONSE_CONTRACT = {
54     "china_iav_yoy_pct": ("future_yoy_level", "percentage_point"),
55     "china_ppi_yoy_pct": ("future_yoy_level", "percentage_point"),
56     "china_cpi_yoy_pct": ("future_yoy_level", "percentage_point"),
57     "china_fx_log_change_pct": ("cumulative_log_change_from_t_minus_1", "log_percentage_point"),
58     "brent_cny_cost_log_change_pct": ("future_monthly_log_change_level", "log_percentage_point"),
59 }
60
61
62 def ensure_dirs() -> None:
63     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
64     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
65
66
67 def save_csv(frame: pd.DataFrame, filename: str) -> Path:
68     path = RESULTS_DIR / filename
69     frame.to_csv(path, index=False, encoding="utf-8")
70     return path
71
72
73 def available_outcomes(frame: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> list[str]:
74     outcomes: list[str] = []
75     for column, label in OUTCOME_SPECS.items():

```

```

76         if column not in frame.columns or frame[column].dropna().shape[0] < 48:
77             warnings_log.append({"code": "q2_outcome_skipped", "message": f"{label} skipped: insufficient usable monthly history."})
78             continue
79         outcomes.append(column)
80     return outcomes
81
82
83 def available_shocks(frame: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> list[str]:
84     shocks: list[str] = []
85     for column in SHOCK_SPECS:
86         if column not in frame.columns or frame[column].dropna().shape[0] < 36:
87             warnings_log.append({"code": "q2_shock_skipped", "message": f"{column} skipped: insufficient usable shock history."})
88             continue
89         shocks.append(column)
90     return shocks
91
92
93 def main_shock_column(frame: pd.DataFrame) -> str:
94     for column in ["oil_specific_risk_shock", "supply_shock", "OilShock"]:
95         if column in frame.columns and frame[column].dropna().shape[0] >= 36:
96             return column
97     return "OilShock"
98
99
100 def add_month_dummies(frame: pd.DataFrame) -> pd.DataFrame:
101     result = frame.copy()
102     result["month"] = pd.to_datetime(result["period"] + "-01").dt.month
103     dummies = pd.get_dummies(result["month"], prefix="month", drop_first=True, dtype=float)
104     return pd.concat([result, dummies], axis=1)
105
106
107 def fit_ols_hac(y: pd.Series, x: pd.DataFrame, maxlags: int) -> Any:
108     x = sm.add_constant(x, has_constant="add")
109     return sm.OLS(y, x).fit(cov_type="HAC", cov_kws={"maxlags": maxlags})
110
111
112 def make_lags(frame: pd.DataFrame, column: str, max_lag: int, prefix: str | None = None) -> pd.DataFrame:
113     result = frame.copy()
114     prefix = prefix or column
115     for lag in range(max_lag + 1):
116         result[f"{prefix}_lag{lag}"] = result[column].shift(lag)
117     return result
118
119
120 def quarter_label(dates: pd.Series) -> pd.Series:
121     periods = pd.PeriodIndex(pd.to_datetime(dates), freq="Q")
122     return pd.Series([f"{period.year}-Q{period.quarter}" for period in periods], index=dates.index)
123
124
125 def ardl_baseline(monthly: pd.DataFrame, outcomes: list[str], warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
126     rows: list[dict[str, Any]] = []
127     base = add_month_dummies(monthly)
128     shock_col = main_shock_column(base)
129     month_cols = [column for column in base.columns if column.startswith("month_") and column != "month_end"]
130     control_cols = ["usd_broad_index_log_return", "GPR_z", "covid_phase"] + month_cols
131     for outcome in outcomes:
132         frame = make_lags(base, shock_col, 6, prefix="shock")
133         frame[f"{outcome}_lag1"] = frame[outcome].shift(1)
134         regressors = [f"shock_lag{lag}" for lag in range(7)] + [f"{outcome}_lag1"] + control_cols
135         usable = frame.dropna(subset=[outcome] + regressors).copy()
136         if len(usable) < len(regressors) + 12:
137             warnings_log.append({"code": "q2_ardl_skipped", "message": f"{outcome}: too few observations after lags."})
138             continue
139         fit = fit_ols_hac(usable[outcome], usable[regressors].astype(float), maxlags=6)
140         lag_terms = [f"shock_lag{lag}" for lag in range(7)]
141         cumulative = float(fit.params[lag_terms].sum())
142         cov = fit.cov_params().loc[lag_terms, lag_terms]
143         cumulative_se = float(np.sqrt(np.ones(len(lag_terms)) @ cov.to_numpy() @ np.ones(len(lag_terms))))
144         y_lag = float(fit.params.get(f"{outcome}_lag1", np.nan))
145         long_run = cumulative / (1.0 - y_lag) if np.isfinite(y_lag) and abs(1.0 - y_lag) > 1e-6 else np.nan
146         long_run_se = np.nan
147         if np.isfinite(long_run):
148             lr_terms = lag_terms + [f"{outcome}_lag1"]
149             lr_cov = fit.cov_params().loc[lr_terms, lr_terms].to_numpy()
150             grad = np.array([1.0 / (1.0 - y_lag)] * len(lag_terms) + [cumulative / (1.0 - y_lag) ** 2])
151             long_run_se = float(np.sqrt(grad @ lr_cov @ grad))

```

```

152     for term_type, estimate, se in [
153         ("short_run_lag0", float(fit.params["shock_lag0"]), float(fit.bse["shock_lag0"])),
154         ("cumulative_lag0_6", cumulative, cumulative_se),
155         ("long_run_multiplier", long_run, long_run_se),
156     ]:
157         rows.append(
158             {
159                 "outcome": outcome,
160                 "term": term_type,
161                 "estimate": estimate,
162                 "std_error": se,
163                 "lower_80": estimate - norm.ppf(0.90) * se if np.isfinite(se) else np.nan,
164                 "upper_80": estimate + norm.ppf(0.90) * se if np.isfinite(se) else np.nan,
165                 "lower_95": estimate - norm.ppf(0.975) * se if np.isfinite(se) else np.nan,
166                 "upper_95": estimate + norm.ppf(0.975) * se if np.isfinite(se) else np.nan,
167                 "pvalue": float(fit.pvalues["shock_lag0"]) if term_type == "short_run_lag0" else np.nan,
168                 "model": "ARDL",
169                 "horizon": 0,
170                 "shock": shock_col,
171                 "specification": f"y_lag1 + {shock_col} lags 0..6 + USD + GPR + month + COVID",
172                 "sample_start": usable["period"].iloc[0],
173                 "sample_end": usable["period"].iloc[-1],
174                 "n": int(len(usable)),
175             }
176         )
177     result = pd.DataFrame(rows)
178     save_csv(result, "q2_ardl_baseline.csv")
179     return result
180
181
182 def block_bootstrap_coefs(
183     usable: pd.DataFrame,
184     y_col: str,
185     x_cols: list[str],
186     term: str,
187     maxlags: int,
188     block_len: int = 12,
189     reps: int = BOOTSTRAP_REPS,
190 ) -> np.ndarray:
191     rng = np.random.default_rng(RANDOM_SEED + maxlags + len(usable))
192     n = len(usable)
193     starts = np.arange(0, max(1, n - block_len + 1))
194     values: list[float] = []
195     for _ in range(reps):
196         take: list[int] = []
197         while len(take) < n:
198             start = int(rng.choice(starts))
199             take.extend(range(start, min(start + block_len, n)))
200         sample = usable.iloc[take[:n]].copy()
201         candidate_x = sample[x_cols].astype(float)
202         if candidate_x[term].nunique(dropna=True) < 2:
203             values.append(np.nan)
204             continue
205         active_cols = [
206             column
207             for column in x_cols
208             if column == term or candidate_x[column].nunique(dropna=True) > 1
209         ]
210         x = sm.add_constant(candidate_x[active_cols], has_constant="add").to_numpy(dtype=float)
211         y = sample[y_col].to_numpy(dtype=float)
212         while np.linalg.matrix_rank(x) < x.shape[1]:
213             current_deficiency = x.shape[1] - np.linalg.matrix_rank(x)
214             removable = [column for column in active_cols if column != term]
215             removable.sort(key=lambda column: (not column.startswith("month_"), column))
216             dropped = False
217             for column in removable:
218                 trial_cols = [candidate for candidate in active_cols if candidate != column]
219                 trial_x = sm.add_constant(candidate_x[trial_cols], has_constant="add").to_numpy(dtype=float)
220                 trial_deficiency = trial_x.shape[1] - np.linalg.matrix_rank(trial_x)
221                 if trial_deficiency < current_deficiency:
222                     active_cols = trial_cols
223                     x = trial_x
224                     dropped = True
225                     break
226             if not dropped:
227                 break

```

```

228     if np.linalg.matrix_rank(x) < x.shape[1] or not np.isfinite(x).all() or not np.isfinite(y).all():
229         values.append(np.nan)
230         continue
231     beta = np.linalg.lstsq(x, y, rcond=None)[0]
232     columns = ["const"] + active_cols
233     values.append(float(beta[columns.index(term)]))
234 return np.asarray(values, dtype=float)
235
236
237 def joint_lp_bootstrap(
238     usable: pd.DataFrame,
239     target_cols: list[str],
240     regressors: list[str],
241     term: str,
242     block_len: int = 12,
243     reps: int = BOOTSTRAP_REPS,
244 ) -> tuple[np.ndarray, np.ndarray]:
245     x = sm.add_constant(usable[regressors].astype(float), has_constant="add").to_numpy(dtype=float)
246     y = usable[target_cols].to_numpy(dtype=float)
247     if np.linalg.matrix_rank(x) < x.shape[1] or not np.isfinite(x).all() or not np.isfinite(y).all():
248         raise np.linalg.LinAlgError("LP design is rank deficient or non-finite.")
249     columns = ["const"] + regressors
250     term_idx = columns.index(term)
251     beta = np.linalg.lstsq(x, y, rcond=None)[0]
252     point = beta[term_idx, :]
253     rng = np.random.default_rng(RANDOM_SEED + len(usable) + len(regressors) + sum(ord(ch) for ch in term))
254     n = len(usable)
255     starts = np.arange(0, max(1, n - block_len + 1))
256     boot = np.full((reps, len(target_cols)), np.nan, dtype=float)
257     for rep in range(reps):
258         take: list[int] = []
259         while len(take) < n:
260             start = int(rng.choice(starts))
261             take.extend(range(start, min(start + block_len, n)))
262         idx = take[:n]
263         xs = x[idx, :]
264         ys = y[idx, :]
265         if not np.isfinite(xs).all() or not np.isfinite(ys).all():
266             continue
267         boot[rep, :] = np.linalg.lstsq(xs, ys, rcond=None)[0][term_idx, :]
268     success = np.isfinite(boot).all(axis=1)
269     if success.sum() < max(100, int(0.80 * reps)):
270         raise RuntimeError(f"LP bootstrap finite-draw rate too low: {success.mean():.3f}")
271     return point, boot[success, :]
272
273
274 def full_rank_regressors(usable: pd.DataFrame, regressors: list[str], required_terms: list[str]) -> list[str]:
275     active = [
276         column
277         for column in regressors
278         if column in required_terms or usable[column].nunique(dropna=True) > 1
279     ]
280     while active:
281         x = sm.add_constant(usable[active].astype(float), has_constant="add").to_numpy(dtype=float)
282         if np.linalg.matrix_rank(x) == x.shape[1] and np.isfinite(x).all():
283             return active
284         removable = [column for column in active if column not in required_terms]
285         if not removable:
286             break
287         removable.sort(key=lambda column: (not column.startswith("month_"), column))
288         active.remove(removable[0])
289     raise np.linalg.LinAlgError("LP design remains rank deficient after dropping non-core controls.")
290
291
292 def lp_target(frame: pd.DataFrame, outcome: str, horizon: int) -> pd.Series:
293     """Return the preregistered Q2 estimand for one outcome and horizon."""
294     if outcome == "china_fx_log_change_pct":
295         return frame["fx_level_pct"].shift(-horizon) - frame["fx_level_pct"].shift(1)
296     return frame[outcome].shift(-horizon)
297
298
299 def validate_lp_target_contract(frame: pd.DataFrame, outcome: str, horizon: int, target_col: str) -> None:
300     expected = lp_target(frame, outcome, horizon)
301     observed = frame[target_col]
302     mask = expected.notna() | observed.notna()
303     if not np.allclose(

```



```

304     expected.loc[mask].to_numpy(dtype=float),
305     observed.loc[mask].to_numpy(dtype=float),
306     equal_nan=True,
307     atol=1e-12,
308     rtol=1e-12,
309 ):
310     raise AssertionError(f"Q2 LP target contract failed for {outcome}, h={horizon}.")
311
312
313 def local_projection(monthly: pd.DataFrame, outcomes: list[str], warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
314     rows: list[dict[str, Any]] = []
315     base = add_month_dummies(monthly)
316     base["fx_level_pct"] = np.log(base["cny_per_usd"]) * 100.0
317     month_cols = [column for column in base.columns if column.startswith("month_") and column != "month_end"]
318     shocks = available_shocks(base, warnings_log)
319     for shock_col in shocks:
320         base[f"{shock_col}_lag1"] = base[shock_col].shift(1)
321
322     for outcome in outcomes:
323         horizons = [0, 3, 6, 12] if outcome == "china_iav_yoy_pct" else list(range(13))
324         for shock_col in shocks:
325             frame = base.copy()
326             frame[f"{outcome}_lag1"] = frame[outcome].shift(1)
327             target_cols: list[str] = []
328             for horizon in horizons:
329                 target_col = f"target_h{horizon}"
330                 target_cols.append(target_col)
331                 frame[target_col] = lp_target(frame, outcome, horizon)
332                 validate_lp_target_contract(frame, outcome, horizon, target_col)
333             regressors = [
334                 shock_col,
335                 f"{outcome}_lag1",
336                 f"{shock_col}_lag1",
337                 "usd_broad_index_log_return",
338                 "GPR_z",
339                 "covid_phase",
340             ] + month_cols
341             usable = frame.dropna(subset=target_cols + regressors).copy()
342             if len(usable) < len(regressors) + 16:
343                 fallback_rows = 0
344                 family_size = len(target_cols)
345                 simultaneous_alpha = 0.05 / family_size
346                 for horizon, target_col in zip(horizons, target_cols, strict=False):
347                     usable_h = frame.dropna(subset=[target_col] + regressors).copy()
348                     if len(usable_h) < len(regressors) + 16:
349                         continue
350                     active_regressors = full_rank_regressors(usable_h, regressors, [shock_col])
351                     fit = fit_ols_hac(
352                         usable_h[target_col],
353                         usable_h[active_regressors].astype(float),
354                         maxlags=horizon + 1,
355                     )
356                     estimate = float(fit.params[shock_col])
357                     boot = block_bootstrap_coefs(
358                         usable_h,
359                         target_col,
360                         active_regressors,
361                         shock_col,
362                         horizon + 1,
363                     )
364                     boot = boot[np.isfinite(boot)]
365                     if len(boot) < max(100, int(0.80 * BOOTSTRAP_REPS)):
366                         continue
367                     se = float(np.std(boot, ddof=1))
368                     lower_80, upper_80 = np.quantile(boot, [0.10, 0.90])
369                     lower_95, upper_95 = np.quantile(boot, [0.025, 0.975])
370                     joint_lower, joint_upper = np.quantile(
371                         boot,
372                         [simultaneous_alpha / 2.0, 1.0 - simultaneous_alpha / 2.0],
373                     )
374                     p_boot = min(
375                         1.0,
376                         2.0
377                         * min(
378                             float((1 + np.sum(boot <= 0.0)) / (len(boot) + 1)),
379                             float((1 + np.sum(boot >= 0.0)) / (len(boot) + 1)),

```

```

380         ),
381     )
382     rows.append(
383         {
384             "outcome": outcome,
385             "shock": shock_col,
386             "horizon": horizon,
387             "response": estimate,
388             "std_error": se,
389             "lower_80": float(lower_80),
390             "upper_80": float(upper_80),
391             "lower_95": float(lower_95),
392             "upper_95": float(upper_95),
393             "joint_lower_95": float(joint_lower),
394             "joint_upper_95": float(joint_upper),
395             "pvalue": float(p_boot),
396             "bootstrap_reps": int(len(boot)),
397             "model": "LocalProjection",
398             "specification": "horizon-specific LP samples with moving-block bootstrap and Bonferroni 95% simultaneous
399 intervals; lagged outcome, lagged shock, USD, GPR, month and COVID controls",
400             "inference_band": "bonferroni_95_horizon_specific_samples",
401             "shock_identification": SHOCK_SPECS[shock_col],
402             "response_definition": OUTCOME_RESPONSE_CONTRACT[outcome][0],
403             "response_unit": OUTCOME_RESPONSE_CONTRACT[outcome][1],
404             "sample_start": usable_h["period"].iloc[0],
405             "sample_end": usable_h["period"].iloc[-1],
406             "n": int(len(usable_h)),
407         }
408     )
409     fallback_rows += 1
410     if fallback_rows == 0:
411         warnings_log.append(
412             {
413                 "code": "q2_lp_skipped",
414                 "message": f"{outcome} {shock_col}: too few observations for joint or horizon-specific LP.",
415             }
416         )
417     continue
418     active_regressors = full_rank_regressors(usable, regressors, [shock_col])
419     point, boot = joint_lp_bootstrap(usable, target_cols, active_regressors, shock_col)
420     se = np.nanstd(boot, axis=0, ddof=1)
421     lower_80, upper_80 = np.quantile(boot, [0.10, 0.90], axis=0)
422     lower_95, upper_95 = np.quantile(boot, [0.025, 0.975], axis=0)
423     t_boot = (boot - point) / se
424     sup_t = np.nanquantile(np.nanmax(np.abs(t_boot), axis=1), 0.95)
425     joint_lower = point - sup_t * se
426     joint_upper = point + sup_t * se
427     for idx, horizon in enumerate(horizons):
428         bh = boot[:, idx]
429         p_boot = min(
430             1.0,
431             2.0
432             * min(
433                 float((1 + np.sum(bh <= 0.0)) / (len(bh) + 1)),
434                 float((1 + np.sum(bh >= 0.0)) / (len(bh) + 1)),
435             ),
436         )
437     rows.append(
438         {
439             "outcome": outcome,
440             "shock": shock_col,
441             "horizon": horizon,
442             "response": float(point[idx]),
443             "std_error": float(se[idx]),
444             "lower_80": float(lower_80[idx]),
445             "upper_80": float(upper_80[idx]),
446             "lower_95": float(lower_95[idx]),
447             "upper_95": float(upper_95[idx]),
448             "joint_lower_95": float(joint_lower[idx]),
449             "joint_upper_95": float(joint_upper[idx]),
450             "pvalue": float(p_boot),
451             "bootstrap_reps": int(len(boot)),
452             "model": "LocalProjection",
453             "specification": "joint h=0..12 LP with one moving-block bootstrap sample source; lagged outcome, lagged shock,
454 USD, GPR, month and COVID controls",
455             "inference_band": "moving_block_bootstrap_sup_t_95",

```

```

454         "shock_identification": SHOCK_SPECS[shock_col],
455         "response_definition": OUTCOME_RESPONSE_CONTRACT[outcome][0],
456         "response_unit": OUTCOME_RESPONSE_CONTRACT[outcome][1],
457         "sample_start": usable["period"].iloc[0],
458         "sample_end": usable["period"].iloc[-1],
459         "n": int(len(usable)),
460     }
461 )
462 result = pd.DataFrame(rows)
463 result = annotate_lp_inference(result)
464 save_csv(result, "q2_irf.csv")
465 return result
466
467
468 def benjamini_hochberg(pvalues: pd.Series) -> pd.Series:
469     p = pd.to_numeric(pvalues, errors="coerce")
470     q = pd.Series(np.nan, index=p.index, dtype=float)
471     valid = p.dropna().sort_values()
472     m = len(valid)
473     if m == 0:
474         return q
475     adjusted = valid * m / np.arange(1, m + 1)
476     adjusted = adjusted.iloc[::-1].cummin().iloc[::-1].clip(upper=1.0)
477     q.loc[adjusted.index] = adjusted
478     return q
479
480
481 def annotate_lp_inference(irf: pd.DataFrame) -> pd.DataFrame:
482     if irf.empty:
483         return irf
484     result = irf.copy()
485     result["ci95_contains_zero"] = result["lower_95"].le(0) & result["upper_95"].ge(0)
486     if {"joint_lower_95", "joint_upper_95"}.issubset(result.columns):
487         result["joint_ci95_contains_zero"] = result["joint_lower_95"].le(0) & result["joint_upper_95"].ge(0)
488     else:
489         result["joint_ci95_contains_zero"] = result["ci95_contains_zero"]
490     result["pre_specified_horizon"] = result["horizon"].isin([0, 3, 6, 12])
491     result["fdr_qvalue"] = np.nan
492     group_cols = ["outcome", "shock"] if "shock" in result.columns else ["outcome"]
493     for _, group in result.groupby(group_cols):
494         result.loc[group.index, "fdr_qvalue"] = benjamini_hochberg(group["pvalue"])
495     result["fdr_significant_10pct"] = result["fdr_qvalue"].le(0.10)
496     result["supports_growth_loss_language"] = (
497         result["outcome"].isin(["china_iav_yoy_pct", "china_real_gdp_yoy_pct"])
498         & result["response"].lt(0)
499         & ~result["joint_ci95_contains_zero"]
500     )
501     return result
502
503
504 def asymmetry_check(monthly: pd.DataFrame, outcomes: list[str], warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
505     rows: list[dict[str, Any]] = []
506     base = add_month_dummies(monthly)
507     shock_col = main_shock_column(base)
508     base["shock_pos"] = base[shock_col].clip(lower=0)
509     base["shock_neg"] = base[shock_col].clip(upper=0)
510     month_cols = [column for column in base.columns if column.startswith("month_") and column != "month_end"]
511     for outcome in outcomes:
512         frame = base.copy()
513         frame[f"{outcome}_lag1"] = frame[outcome].shift(1)
514         regressors = ["shock_pos", "shock_neg", f"{outcome}_lag1", "usd_broad_index_log_return", "GPR_z", "covid_phase"] + month_cols
515         usable = frame.dropna(subset=[outcome] + regressors)
516         if len(usable) < len(regressors) + 12:
517             continue
518         fit = fit_ols_hac(usable[outcome], usable[regressors].astype(float), maxlags=6)
519         for term in ["shock_pos", "shock_neg"]:
520             estimate = float(fit.params[term])
521             se = float(fit.bse[term])
522             rows.append(
523                 {
524                     "outcome": outcome,
525                     "term": term,
526                     "shock": shock_col,
527                     "estimate": estimate,
528                     "std_error": se,
529                     "lower_95": estimate - norm.ppf(0.975) * se,

```

```

530         "upper_95": estimate + norm.ppf(0.975) * se,
531         "model": "sign_asymmetry_DL",
532         "specification": f"positive and negative {shock_col} split; not a full NARDL/ECM",
533         "sample_start": usable["period"].iloc[0],
534         "sample_end": usable["period"].iloc[-1],
535         "n": int(len(usable)),
536     }
537 )
538 result = pd.DataFrame(rows)
539 save_csv(result, "q2_asymmetry.csv")
540 return result
541
542
543 def lag_and_covid_robustness(monthly: pd.DataFrame, outcomes: list[str], warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
544     rows: list[dict[str, Any]] = []
545     base_all = add_month_dummies(monthly)
546     shock_col = main_shock_column(base_all)
547     month_cols = [column for column in base_all.columns if column.startswith("month_") and column != "month_end"]
548     for lag_max in [3, 6, 12]:
549         for exclude_covid in [False, True]:
550             base = base_all.loc[~base_all["covid_phase"].eq(1)].copy() if exclude_covid else base_all.copy()
551             controls = ["usd_broad_index_log_return", "GPR_z"] + ([f"{'covid_phase'}"] if exclude_covid else []) + month_cols
552             for outcome in outcomes:
553                 frame = make_lags(base, shock_col, lag_max, prefix="shock")
554                 frame[f"{outcome}_lag1"] = frame[outcome].shift(1)
555                 lag_terms = [f"shock_lag{lag}" for lag in range(lag_max + 1)]
556                 regressors = lag_terms + [f"{outcome}_lag1"] + controls
557                 usable = frame.dropna(subset=[outcome] + regressors)
558                 if len(usable) < len(regressors) + 12:
559                     warnings_log.append(
560                         {
561                             "code": "q2_robustness_skipped",
562                             "message": f"{outcome} lag={lag_max} exclude_covid={exclude_covid}: too few observations.",
563                         }
564                     )
565                     continue
566                 fit = fit_ols_hac(usable[outcome], usable[regressors].astype(float), maxlags=lag_max)
567                 estimate = float(fit.params[lag_terms].sum())
568                 cov = fit.cov_params().loc[lag_terms, lag_terms]
569                 se = float(np.sqrt(np.ones(len(lag_terms)) @ cov.to_numpy() @ np.ones(len(lag_terms))))
570                 rows.append(
571                     {
572                         "outcome": outcome,
573                         "shock": shock_col,
574                         "lag_max": lag_max,
575                         "exclude_covid": exclude_covid,
576                         "estimate": estimate,
577                         "std_error": se,
578                         "lower_95": estimate - norm.ppf(0.975) * se,
579                         "upper_95": estimate + norm.ppf(0.975) * se,
580                         "model": "ARDL_robustness",
581                         "specification": f"shock_col lags 0..{lag_max}; exclude_covid={exclude_covid}",
582                         "sample_start": usable["period"].iloc[0],
583                         "sample_end": usable["period"].iloc[-1],
584                         "n": int(len(usable)),
585                     }
586                 )
587     result = pd.DataFrame(rows)
588     save_csv(result, "q2_robustness.csv")
589     return result
590
591
592 def gdp_validation(quarterly: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
593     frame = quarterly.copy()
594     frame["gdp_lag1"] = frame["china_real_gdp_yoy_pct"].shift(1)
595     shock_sum_col = "oil_specific_risk_shock_sum" if "oil_specific_risk_shock_sum" in frame.columns and frame["oil_specific_risk_shock_sum"].dropna().shape[0] >= 8 else "OilShock_sum"
596     usable = frame.dropna(subset=["china_real_gdp_yoy_pct", "gdp_lag1", shock_sum_col, "GPR_z_mean"]) if "GPR_z_mean" in frame.columns else pd.DataFrame()
597     if "GPR_z_mean" not in frame.columns:
598         monthly = pd.read_csv(PROCESSED_DIR / "model_monthly_cn.csv")
599         monthly["quarter"] = quarter_label(monthly["period"] + "-01")
600         gpr_q = monthly.groupby("quarter", as_index=False).agg(GPR_z_mean=("GPR_z", "mean"))
601         frame = frame.merge(gpr_q, on="quarter", how="left")
602         usable = frame.dropna(subset=["china_real_gdp_yoy_pct", "gdp_lag1", shock_sum_col, "GPR_z_mean"])
603     if len(usable) < 16:

```

```

604 warnings_log.append({"code": "q2_gdp_validation_limited", "message": "Too few GDP validation observations after lags."})
605 result = pd.DataFrame(
606     [
607         {
608             "outcome": "china_real_gdp_yoy_pct",
609             "model": "quarterly_validation",
610             "estimate": np.nan,
611             "std_error": np.nan,
612             "lower_95": np.nan,
613             "upper_95": np.nan,
614             "evidence_status": "INCONCLUSIVE",
615             "shock": shock_sum_col,
616             "correlation": float(frame[["china_real_gdp_yoy_pct", shock_sum_col]].dropna().corr().iloc[0, 1]) if frame[["
china_real_gdp_yoy_pct", shock_sum_col]].dropna().shape[0] > 3 else np.nan,
617             "n": int(frame[["china_real_gdp_yoy_pct", shock_sum_col]].dropna().shape[0]),
618             "sample_start": "",
619             "sample_end": "",
620             "specification": "insufficient observations for regression; correlation shown if available",
621         }
622     ]
623 )
624 save_csv(result, "q2_gdp_validation.csv")
625 return result
626 regressors = [shock_sum_col, "gdp_lag1", "GPR_z_mean"]
627 fit = fit_ols_hac(usable[["china_real_gdp_yoy_pct"]], usable[regressors].astype(float), maxlags=2)
628 estimate = float(fit.params[shock_sum_col])
629 se = float(fit.bse[shock_sum_col])
630 lower_95 = estimate - norm.ppf(0.975) * se
631 upper_95 = estimate + norm.ppf(0.975) * se
632 result = pd.DataFrame(
633     [
634         {
635             "outcome": "china_real_gdp_yoy_pct",
636             "model": "quarterly_validation",
637             "estimate": estimate,
638             "std_error": se,
639             "lower_80": estimate - norm.ppf(0.90) * se,
640             "upper_80": estimate + norm.ppf(0.90) * se,
641             "lower_95": lower_95,
642             "upper_95": upper_95,
643             "pvalue": float(fit.pvalues[shock_sum_col]),
644             "evidence_status": "SUPPORTED" if lower_95 > 0 or upper_95 < 0 else "INCONCLUSIVE",
645             "shock": shock_sum_col,
646             "correlation": float(usable[["china_real_gdp_yoy_pct", shock_sum_col]].corr().iloc[0, 1]),
647             "n": int(len(usable)),
648             "sample_start": usable["quarter"].iloc[0],
649             "sample_end": usable["quarter"].iloc[-1],
650             "specification": f"GDP yoy on quarterly {shock_sum_col}, lagged GDP yoy and GPR",
651         }
652     ]
653 )
654 save_csv(result, "q2_gdp_validation.csv")
655 return result
656
657
658 def build_transmission_metrics(irf: pd.DataFrame, gdp: pd.DataFrame) -> pd.DataFrame:
659     rows: list[dict[str, Any]] = []
660     if not irf.empty:
661         for (shock, outcome), group in irf.groupby(["shock", "outcome"]):
662             frame = group.sort_values("horizon").copy()
663             values = pd.to_numeric(frame["response"], errors="coerce")
664             if values.dropna().empty:
665                 continue
666             if outcome == "china_iav_yoy_pct":
667                 idx = values.idxmin()
668                 extremum_type = "trough"
669             else:
670                 idx = values.abs().idxmax()
671                 extremum_type = "peak_abs"
672             extreme = frame.loc[idx]
673             complete_monthly_path = set(frame["horizon"].astype(int)) >= set(range(13))
674             area_h6 = frame.loc[frame["horizon"].le(6), "response"].sum() if complete_monthly_path else np.nan
675             area_h12 = frame.loc[frame["horizon"].le(12), "response"].sum() if complete_monthly_path else np.nan
676             recovery = frame.loc[
677                 frame["horizon"].ge(int(extreme["horizon"])) & frame["lower_95"].le(0) & frame["upper_95"].ge(0),
678                 "horizon",

```

```

679     ]
680     supported_loss = bool(
681         outcome == "china_iav_yoy_pct"
682         and float(extreme["response"]) < 0
683         and not (float(extreme["joint_lower_95"]) <= 0 <= float(extreme["joint_upper_95"]))
684     )
685     evidence = "SUPPORTED" if supported_loss else ("INCONCLUSIVE" if bool(frame["joint_ci95_contains_zero"].any()) else "
SUPPORTED")
686     rows.append(
687         {
688             "shock": shock,
689             "outcome": outcome,
690             "extremum_type": extremum_type,
691             "extremum_response": float(extreme["response"]),
692             "extremum_month": int(extreme["horizon"]),
693             "extremum_lower_95": float(extreme["lower_95"]),
694             "extremum_upper_95": float(extreme["upper_95"]),
695             "extremum_joint_lower_95": float(extreme["joint_lower_95"]),
696             "extremum_joint_upper_95": float(extreme["joint_upper_95"]),
697             "response_curve_area_0_6": float(area_h6) if np.isfinite(area_h6) else np.nan,
698             "response_curve_area_0_12": float(area_h12) if np.isfinite(area_h12) else np.nan,
699             "area_unit": "percentage_point_month" if complete_monthly_path else "not_applicable_sparse_horizons",
700             "available_horizons": "|".join(str(int(value)) for value in sorted(frame["horizon"].unique())),
701             "recovery_month": int(recovery.iloc[0]) if not recovery.empty else np.nan,
702             "evidence_status": evidence,
703             "allows_growth_loss_language": supported_loss,
704             "inference_source": str(extreme.get("inference_band", "")),
705             "sample_start": str(frame["sample_start"].dropna().iloc[0]) if not frame["sample_start"].dropna().empty else "",
706             "sample_end": str(frame["sample_end"].dropna().iloc[-1]) if not frame["sample_end"].dropna().empty else "",
707         }
708     )
709     if not gdp.empty:
710         for row in gdp.to_dict("records"):
711             rows.append(
712                 {
713                     "shock": row.get("shock", ""),
714                     "outcome": row.get("outcome", "china_real_gdp_yoy_pct"),
715                     "extremum_type": "quarterly_validation",
716                     "extremum_response": row.get("estimate", np.nan),
717                     "extremum_month": np.nan,
718                     "extremum_lower_95": row.get("lower_95", np.nan),
719                     "extremum_upper_95": row.get("upper_95", np.nan),
720                     "extremum_joint_lower_95": row.get("lower_95", np.nan),
721                     "extremum_joint_upper_95": row.get("upper_95", np.nan),
722                     "response_curve_area_0_6": np.nan,
723                     "response_curve_area_0_12": np.nan,
724                     "area_unit": "not_applicable_quarterly_validation",
725                     "available_horizons": "",
726                     "recovery_month": np.nan,
727                     "evidence_status": row.get("evidence_status", "INCONCLUSIVE"),
728                     "allows_growth_loss_language": False,
729                     "inference_source": "quarterly_HAC_validation",
730                     "sample_start": row.get("sample_start", ""),
731                     "sample_end": row.get("sample_end", ""),
732                 }
733             )
734     result = pd.DataFrame(rows)
735     save_csv(result, "q2_transmission_metrics.csv")
736     return result
737
738
739 def plot_irf(irf: pd.DataFrame) -> None:
740     if irf.empty:
741         return
742     plot_shock = "oil_specific_risk_shock" if "oil_specific_risk_shock" in set(irf.get("shock", pd.Series(dtype=str))) else str(irf.get("
shock", pd.Series(["OilShock"])).dropna().iloc[0])
743     irf = irf.loc[irf.get("shock", pd.Series(plot_shock, index=irf.index)).eq(plot_shock)].copy()
744     outcomes = irf["outcome"].unique().tolist()
745     fig, axes = plt.subplots(len(outcomes), 1, figsize=(8.8, max(4.2, 3.0 * len(outcomes))), sharex=True)
746     if len(outcomes) == 1:
747         axes = [axes]
748     for ax, outcome in zip(axes, outcomes, strict=False):
749         frame = irf.loc[irf["outcome"].eq(outcome)].sort_values("horizon")
750         ax.axhline(0, color=PALETTE["muted"], linewidth=0.8)
751         if outcome == "china_iav_yoy_pct":
752             ax.errorbar(

```

```

753         frame["horizon"],
754         frame["response"],
755         yerr=[frame["response"] - frame["lower_95"], frame["upper_95"] - frame["response"]],
756         fmt="o",
757         color=PALETTE["blue"],
758         capsize=3,
759         label="离散期限响应及95%区间",
760     )
761     else:
762         ax.plot(frame["horizon"], frame["response"], marker="o", color=PALETTE["blue"], label="估计响应")
763         ax.fill_between(
764             frame["horizon"],
765             frame["lower_95"],
766             frame["upper_95"],
767             color=PALETTE["blue_light"],
768             alpha=0.30,
769             linewidth=0,
770             label="95%置信区间",
771         )
772         label = OUTCOME_SPECS.get(outcome, outcome)
773         ax.text(0.0, 0.91, label, transform=ax.transAxes, ha="left", va="top", fontsize=10.2, color=PALETTE["ink"])
774         style_axis(ax, ylabel="响应")
775     axes[-1].set_xlabel("油价冲击后月份")
776     axes[0].legend(loc="upper right", handlelength=2.4)
777     finish_figure(
778         fig,
779         title="问题二：中国宏观变量对约化形式油价冲击的响应",
780         subtitle=f"月度中国变量：主图冲击为 {plot_shock}；阴影为同源移动块bootstrap 95%逐期区间。",
781         source="来源：问题一结构冲击接口、国家统计局 IAV/PPI、OECD CPI、FRED 汇率/美元/GPR；由 code/problem2/run_q2.py 生成。",
782         rect=(0.09, 0.12, 0.98, 0.84),
783     )
784     save_figure(fig, FIGURES_DIR / "q2_irf")
785     plt.close(fig)
786
787
788 def main(argv: list[str] | None = None) -> int:
789     parser = argparse.ArgumentParser()
790     parser.add_argument("--plots-only", action="store_true", help="Regenerate Q2 figures from saved result tables.")
791     args = parser.parse_args(argv)
792
793     np.random.seed(RANDOM_SEED)
794     apply_paper_style()
795     ensure_dirs()
796     if args.plots_only:
797         irf = pd.read_csv(RESULTS_DIR / "q2_irf.csv")
798         plot_irf(irf)
799         print(json.dumps({"status": "PASS", "mode": "plots-only", "figures": 1}, ensure_ascii=False, indent=2))
800         return 0
801
802     warnings_log: list[dict[str, Any]] = []
803     monthly = pd.read_csv(PROCESSED_DIR / "model_monthly_cn.csv")
804     quarterly = pd.read_csv(PROCESSED_DIR / "model_quarterly_cn.csv")
805     outcomes = available_outcomes(monthly, warnings_log)
806     ardl = ardl_baseline(monthly, outcomes, warnings_log)
807     irf = local_projection(monthly, outcomes, warnings_log)
808     asymmetry = asymmetry_check(monthly, outcomes, warnings_log)
809     robustness = lag_and_covid_robustness(monthly, outcomes, warnings_log)
810     gdp = gdp_validation(quarterly, warnings_log)
811     transmission_metrics = build_transmission_metrics(irf, gdp)
812     plot_irf(irf)
813     gdp_status = "INCONCLUSIVE"
814     if not gdp.empty and "evidence_status" in gdp.columns:
815         gdp_status = str(gdp["evidence_status"].dropna().iloc[0]) if not gdp["evidence_status"].dropna().empty else "INCONCLUSIVE"
816
817     summary = pd.DataFrame(
818         [
819             {
820                 "component": "ARDL",
821                 "rows": len(ardl),
822                 "status": "PASS" if len(ardl) else "WARN",
823                 "note": "available monthly outcomes only; OilShock is reduced-form innovation",
824             },
825             {
826                 "component": "LocalProjection",
827                 "rows": len(irf),
828                 "status": "PASS" if len(irf) else "WARN",

```

```

829         "note": "horizons 0..12 where estimable; reports FDR and zero-crossing flags",
830     },
831     {
832         "component": "GDPValidation",
833         "rows": len(gdp),
834         "status": gdp_status,
835         "note": "quarterly GDP check; no monthly interpolation",
836     },
837     {
838         "component": "Asymmetry",
839         "rows": len(asymmetry),
840         "status": "PASS" if len(asymmetry) else "WARN",
841         "note": "positive and negative shock split; not a full NARDL",
842     },
843     {
844         "component": "LagCovidRobustness",
845         "rows": len(robustness),
846         "status": "PASS" if len(robustness) else "WARN",
847         "note": "lag 3/6/12 and exclude-COVID specifications",
848     },
849 ]
850 )
851 save_csv(summary, "q2_summary.csv")
852 payload = {
853     "status": "PASS" if len(irf) and len(ardl) else "CONDITIONAL",
854     "execution_status": "PASS" if len(irf) or len(ardl) else "WARN",
855     "evidence_status": "INCONCLUSIVE",
856     "allowed_claims": [
857         "Q2 reports direction, magnitude and uncertainty for oil-shock transmission",
858         "RMB crude-cost, PPI, CPI, FX and industrial activity responses are model outputs where data permit",
859         "GDP is a low-frequency validation and remains inconclusive when intervals cross zero",
860     ],
861     "forbidden_claims": [
862         "oil shocks caused a robust aggregate growth loss unless the industrial/GDP joint interval excludes zero",
863         "reduced-form OilShock is a structural supply or war shock",
864     ],
865     "random_seed": RANDOM_SEED,
866     "outcomes": outcomes,
867     "shock_identification": "LP main interface estimates supply, aggregate-demand and oil-specific-risk shocks from the historical recursive VAR when available; OilShock remains a reduced-form robustness shock.",
868     "conclusion_guardrail": "Current estimates do not support a clear aggregate growth-loss statement unless negative responses jointly exclude zero.",
869     "warnings": warnings_log,
870     "rows": {
871         "q2_ardl_baseline.csv": int(len(ardl)),
872         "q2_irf.csv": int(len(irf)),
873         "q2_gdp_validation.csv": int(len(gdp)),
874         "q2_asymmetry.csv": int(len(asymmetry)),
875         "q2_robustness.csv": int(len(robustness)),
876         "q2_transmission_metrics.csv": int(len(transmission_metrics)),
877     },
878 }
879 (RESULTS_DIR / "q2_summary.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
880 print(json.dumps(payload, ensure_ascii=False, indent=2))
881 return 0
882
883
884 if __name__ == "__main__":
885     raise SystemExit(main())

```

Listing 7: run_q2.py

```

1  """Question 3: cross-country buffers and China policy counterfactuals."""
2
3  from __future__ import annotations
4
5  import argparse
6  import json
7  import sys
8  import warnings as py_warnings
9  from pathlib import Path
10 from typing import Any
11
12 import matplotlib
13

```



```

14 matplotlib.use("Agg")
15
16 import matplotlib.pyplot as plt
17 import numpy as np
18 import pandas as pd
19 import statsmodels.api as sm
20 from linearmodels.panel import PanelOLS
21 from linearmodels.panel.utility import AbsorbingEffectWarning
22 from scipy.stats import norm
23
24
25 REPO_ROOT = Path(__file__).resolve().parents[2]
26 CODE_DIR = REPO_ROOT / "code"
27 if str(CODE_DIR) not in sys.path:
28     sys.path.insert(0, str(CODE_DIR))
29
30 from utils.plot_style import PALETTE, apply_paper_style, finish_figure, save_figure, style_axis
31
32 if hasattr(sys.stdout, "reconfigure"):
33     sys.stdout.reconfigure(encoding="utf-8")
34
35 PROCESSED_DIR = REPO_ROOT / "data" / "processed"
36 RESULTS_DIR = REPO_ROOT / "results"
37 FIGURES_DIR = REPO_ROOT / "figures"
38 RANDOM_SEED = 20260730
39 POLICY_BOOTSTRAP_REPS = 2000
40 POLICY_BOOTSTRAP_BLOCK = 6
41
42
43 COUNTRY_ORDER = ["CHN", "DEU", "FRA", "ITA", "ESP", "JPN", "KOR"]
44 MAIN_CONTROL_COUNTRIES = ["DEU", "FRA", "ITA", "ESP", "JPN", "KOR"]
45 COUNTRY_LABEL_ZH = {
46     "CHN": "中国",
47     "DEU": "德国",
48     "FRA": "法国",
49     "ITA": "意大利",
50     "ESP": "西班牙",
51     "JPN": "日本",
52     "KOR": "韩国",
53 }
54 OUTCOME_LABELS = {
55     "fuel": "燃油价格累计对数变化, 百分点",
56     "cpi": "CPI同比, 百分点",
57     "ip": "工业生产同比对数变化, 百分点",
58 }
59
60
61 def ensure_dirs() -> None:
62     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
63     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
64
65
66 def save_csv(frame: pd.DataFrame, filename: str) -> Path:
67     path = RESULTS_DIR / filename
68     frame.to_csv(path, index=False, encoding="utf-8")
69     return path
70
71
72 def read_csv_result(filename: str) -> pd.DataFrame:
73     path = RESULTS_DIR / filename
74     return pd.read_csv(path) if path.exists() else pd.DataFrame()
75
76
77 def log_positive(series: pd.Series) -> pd.Series:
78     values = pd.to_numeric(series, errors="coerce")
79     result = pd.Series(np.nan, index=series.index, dtype=float)
80     mask = values > 0
81     result.loc[mask] = np.log(values.loc[mask])
82     return result
83
84
85 def add_lags(frame: pd.DataFrame, column: str, max_lag: int) -> pd.DataFrame:
86     result = frame.copy()
87     for lag in range(max_lag + 1):
88         result[f"{column}_lag{lag}"] = result[column].shift(lag)
89     return result

```

```

90
91
92 def bool_series(series: pd.Series) -> pd.Series:
93     if pd.api.types.is_bool_dtype(series):
94         return series
95     return series.astype(str).str.lower().isin(["true", "1", "yes"])
96
97
98 def shock_column(frame: pd.DataFrame) -> str:
99     for column in ["oil_specific_risk_shock", "supply_shock", "aggregate_demand_shock", "OilShock"]:
100         if column in frame.columns and pd.to_numeric(frame[column], errors="coerce").notna().sum() >= 24:
101             return column
102     raise ValueError("Q3 panel lacks an auditable oil shock column.")
103
104
105 def fit_country_pass_through(panel: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
106     rows: list[dict[str, Any]] = []
107     for country in COUNTRY_ORDER:
108         frame = panel.loc[panel["country"].eq(country)].sort_values("period").copy()
109         frame = add_lags(frame, "brent_local_log_return", 6)
110         regressors = [f"brent_local_log_return_lag{lag}" for lag in range(7)]
111         frame["fuel_lag1"] = frame["fuel_log_return"].shift(1)
112         usable = frame.dropna(subset=["fuel_log_return", "fuel_lag1"] + regressors)
113         if len(usable) < 48:
114             warnings_log.append({"code": "q3_pass_through_limited", "message": f"{country}: insufficient fuel observations."})
115             continue
116         included = bool(bool_series(frame["included_in_main_comparison"]).iloc[0]) if "included_in_main_comparison" in frame else country
117         != "CHN"
118         measure_type = frame["price_measure_type"].iloc[0] if "price_measure_type" in frame else ""
119         observed_or_regulated = frame["observed_or_regulated"].iloc[0] if "observed_or_regulated" in frame else ""
120         comparability_note = frame["comparability_note"].iloc[0] if "comparability_note" in frame else ""
121         x = sm.add_constant(usable[regressors + ["fuel_lag1"]].astype(float), has_constant="add")
122         fit = sm.OLS(usable["fuel_log_return"], x).fit(cov_type="HAC", cov_kws={"maxlags": 6})
123         for horizon in [1, 3, 6]:
124             lag_terms = [f"brent_local_log_return_lag{lag}" for lag in range(horizon + 1)]
125             estimate = float(fit.params[lag_terms].sum())
126             cov = fit.cov_params().loc[lag_terms, lag_terms]
127             se = float(np.sqrt(np.ones(len(lag_terms)) @ cov.to_numpy() @ np.ones(len(lag_terms))))
128             rows.append(
129                 {
130                     "country": country,
131                     "country_name": frame["country_name"].iloc[0],
132                     "horizon": horizon,
133                     "response": estimate,
134                     "std_error": se,
135                     "lower_80": estimate - norm.ppf(0.90) * se,
136                     "upper_80": estimate + norm.ppf(0.90) * se,
137                     "lower_95": estimate - norm.ppf(0.975) * se,
138                     "upper_95": estimate + norm.ppf(0.975) * se,
139                     "model": "distributed_lag_pass_through",
140                     "specification": "fuel log return on local-currency Brent log-return lags 0..6 and fuel lag",
141                     "sample_start": usable["period"].iloc[0],
142                     "sample_end": usable["period"].iloc[-1],
143                     "n": int(len(usable)),
144                     "fuel_source": frame["fuel_source"].iloc[0],
145                     "fuel_unit": frame["fuel_unit"].iloc[0],
146                     "price_measure_type": measure_type,
147                     "observed_or_regulated": observed_or_regulated,
148                     "included_in_main_comparison": included,
149                     "comparability_note": comparability_note,
150                 }
151             )
152         result = pd.DataFrame(rows)
153         save_csv(result, "q3_country_pass_through.csv")
154     return result
155
156 def panel_design(frame: pd.DataFrame, outcome: str, horizon: int) -> tuple[pd.Series, pd.DataFrame, pd.DataFrame, str]:
157     data = frame.copy()
158     shock = shock_column(data)
159     data["shock"] = pd.to_numeric(data[shock], errors="coerce")
160     if outcome == "fuel":
161         data["target"] = data.groupby("country")["fuel_log"].shift(-horizon) - data.groupby("country")["fuel_log"].shift(1)
162         data["target"] = data["target"] * 100.0
163         needed = ["target", "shock", "fuel_log"]
164     elif outcome == "cpi":

```

```

165     data["target"] = data.groupby("country")["cpi_yoy_pct"].shift(-horizon)
166     needed = ["target", "shock", "cpi_yoy_pct"]
167 else:
168     activity_col = "industrial_activity_yoy_pct" if "industrial_activity_yoy_pct" in data.columns else "ip_yoy_log_change_pct"
169     data["target"] = data.groupby("country")[activity_col].shift(-horizon)
170     needed = ["target", "shock", activity_col]
171 data["trend"] = data.groupby("country").cumcount()
172 for country in MAIN_CONTROL_COUNTRIES:
173     data[f"oil_diff_{country}_vs_CHN"] = np.where(data["country"].eq(country), data["shock"], 0.0)
174     data[f"trend_{country}_vs_CHN"] = np.where(data["country"].eq(country), data["trend"], 0.0)
175 needed += [f"oil_diff_{country}_vs_CHN" for country in MAIN_CONTROL_COUNTRIES]
176 x_cols = [f"oil_diff_{country}_vs_CHN" for country in MAIN_CONTROL_COUNTRIES] + [
177     f"trend_{country}_vs_CHN" for country in MAIN_CONTROL_COUNTRIES
178 ]
179 usable = data.dropna(subset=needed).copy()
180 usable["month_index"] = pd.PeriodIndex(usable["period"], freq="M").to_timestamp()
181 usable = usable.set_index(["country", "month_index"]).sort_index()
182 y = usable["target"].astype(float)
183 x = usable[x_cols].astype(float)
184 return y, x, usable.reset_index(), shock
185
186
187 def buffer_design(frame: pd.DataFrame, outcome: str, horizon: int, buffer: str) -> tuple[pd.Series, pd.DataFrame, pd.DataFrame, str, str
188 ]:
189     data = frame.copy()
190     shock = shock_column(data)
191     data["shock"] = pd.to_numeric(data[shock], errors="coerce")
192     if buffer not in data.columns:
193         raise ValueError(f"country_policy_buffers_annual.csv lacks {buffer}.")
194     data[buffer] = pd.to_numeric(data[buffer], errors="coerce")
195     if buffer != "fuel_price_regulation":
196         std = data[buffer].std(skipna=True)
197         if not np.isfinite(std) or std <= 0:
198             raise ValueError(f"{buffer} has zero variation.")
199         data[f"{buffer}_standardized"] = (data[buffer] - data[buffer].mean(skipna=True)) / std
200         buffer_col = f"{buffer}_standardized"
201     else:
202         buffer_col = buffer
203     if outcome == "fuel":
204         data["target"] = data.groupby("country")["fuel_log"].shift(-horizon) - data.groupby("country")["fuel_log"].shift(1)
205         data["target"] = data["target"] * 100.0
206         needed = ["target", "shock", "fuel_log", buffer_col]
207     elif outcome == "cpi":
208         data["target"] = data.groupby("country")["cpi_yoy_pct"].shift(-horizon)
209         needed = ["target", "shock", "cpi_yoy_pct", buffer_col]
210     else:
211         activity_col = "industrial_activity_yoy_pct" if "industrial_activity_yoy_pct" in data.columns else "ip_yoy_log_change_pct"
212         data["target"] = data.groupby("country")[activity_col].shift(-horizon)
213         needed = ["target", "shock", activity_col, buffer_col]
214     term = f"shock_x_{buffer}"
215     data[term] = data["shock"] * data[buffer_col]
216     usable = data.dropna(subset=needed + [term]).copy()
217     usable = usable.loc[usable[term].notna()].copy()
218     usable["month_index"] = pd.PeriodIndex(usable["period"], freq="M").to_timestamp()
219     usable = usable.set_index(["country", "month_index"]).sort_index()
220     y = usable["target"].astype(float)
221     x = usable[term].astype(float)
222     return y, x, usable.reset_index(), shock, term
223
224 def fit_panel_lp(panel: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
225     from linearmodels.panel import PanelOLS
226
227     rows: list[dict[str, Any]] = []
228     for outcome in ["cpi", "ip"]:
229         outcome_panel = panel.copy()
230         for horizon in range(13):
231             y, x, usable, shock = panel_design(outcome_panel, outcome, horizon)
232             if len(y) < x.shape[1] + 20 or y.index.get_level_values(0).nunique() < 3:
233                 warnings_log.append({"code": "q3_panel_lp_skipped", "message": f"{outcome} h={horizon}: insufficient panel support."})
234                 continue
235             fit = PanelOLS(y, x, entity_effects=True, time_effects=True, drop_absorbed=True, check_rank=False).fit(
236                 cov_type="kernel",
237                 kernel="bartlett",
238                 bandwidth=max(1, horizon + 1),
239             )

```

```

240     for country in MAIN_CONTROL_COUNTRIES:
241         term = f"oil_diff_{country}_vs_CHN"
242         if term not in fit.params.index:
243             continue
244         estimate = float(fit.params[term])
245         se = float(fit.std_errors[term])
246         rows.append(
247             {
248                 "outcome": outcome,
249                 "country": country,
250                 "reference_country": "CHN",
251                 "horizon": horizon,
252                 "response": estimate,
253                 "std_error": se,
254                 "lower_80": estimate - norm.ppf(0.90) * se,
255                 "upper_80": estimate + norm.ppf(0.90) * se,
256                 "lower_95": estimate - norm.ppf(0.975) * se,
257                 "upper_95": estimate + norm.ppf(0.975) * se,
258                 "pvalue": float(fit.pvalues[term]),
259                 "shock": shock,
260                 "model": "stacked_panel_LP_time_FE_relative_to_CHN",
261                 "response_type": "control_country_minus_china_relative_response",
262                 "specification": "country FE, full year-month FE, control-country shock interactions relative to China, control-
country trends, Driscoll-Kraay covariance",
263                 "sample_start": usable["period"].min(),
264                 "sample_end": usable["period"].max(),
265                 "n": int(len(usable)),
266             }
267         )
268     result = pd.DataFrame(rows)
269     save_csv(result, "q3_panel_irf.csv")
270     return result
271
272
273 def fit_buffer_interactions(panel: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
274     """Fail closed while the annual structural-buffer table remains unaudited."""
275     buffers = ["oil_import_dependency", "oil_intensity", "import_source_hhi", "fuel_price_regulation"]
276     status = pd.DataFrame(
277         [
278             {
279                 "buffer": buffer,
280                 "data_status": "FAIL_FOR_MAIN_UNAUDITED_PROXY",
281                 "quantitative_result_released": False,
282                 "reason": "country_policy_buffers_annual.csv is a normalized proxy pending audited official exports",
283                 "required_replacement": "country-year raw values, formula, official URL/table id, download date and file hash",
284             }
285             for buffer in buffers
286         ]
287     )
288     save_csv(status, "q3_buffer_data_status.csv")
289     columns = [
290         "outcome",
291         "buffer",
292         "horizon",
293         "estimate",
294         "std_error",
295         "lower_95",
296         "upper_95",
297         "shock",
298         "model",
299         "data_status",
300     ]
301     result = pd.DataFrame(columns=columns)
302     save_csv(result, "q3_buffer_interactions.csv")
303     warnings_log.append(
304         {
305             "code": "q3_buffer_quantitative_results_withheld",
306             "message": "Continuous buffer interactions were withheld because the country-year table is an unaudited proxy.",
307         }
308     )
309     return result
310
311
312 def china_fuel_to_macro_elasticities(warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
313     official = pd.read_csv(PROCESSED_DIR / "china_regulated_gasoline_monthly.csv")
314     macro = pd.read_csv(PROCESSED_DIR / "model_monthly_cn.csv")

```

```

315     frame = macro.merge(
316         official[["period", "china_regulated_gasoline_cny_per_ton"]],
317         on="period",
318         how="left",
319     ).sort_values("period")
320     frame["fuel_log"] = log_positive(frame["china_regulated_gasoline_cny_per_ton"])
321     frame["fuel_log_return"] = frame["fuel_log"].diff()
322     frame = add_lags(frame, "fuel_log_return", 6)
323     outcomes = {
324         "china_ppi_yoy_pct": "PPI",
325         "china_cpi_yoy_pct": "CPI",
326         "china_iav_yoy_pct": "IAV",
327     }
328     rows: list[dict[str, Any]] = []
329     kernel_rows: list[dict[str, Any]] = []
330     covariance_rows: list[dict[str, Any]] = []
331     for outcome, label in outcomes.items():
332         if outcome not in frame.columns:
333             warnings_log.append({"code": "q3_policy_macro_outcome_missing", "message": f"{outcome} missing for policy propagation."})
334             continue
335         frame[f"{outcome}_lag1"] = frame[outcome].shift(1)
336         lag_terms = [f"fuel_log_return_lag{lag}" for lag in range(7)]
337         regressors = lag_terms + [f"{outcome}_lag1", "GPR"]
338         usable = frame.dropna(subset=[outcome] + regressors).copy()
339         if len(usable) < 48:
340             warnings_log.append({"code": "q3_policy_macro_elasticity_limited", "message": f"{outcome}: only {len(usable)} usable observations."})
341             continue
342         fit = sm.OLS(usable[outcome], sm.add_constant(usable[regressors].astype(float), has_constant="add")).fit(
343             cov_type="HAC",
344             cov_kws={"maxlags": 6},
345         )
346         estimate = float(fit.params[lag_terms].sum())
347         cov = fit.cov_params().loc[lag_terms, lag_terms]
348         se = float(np.sqrt(np.ones(len(lag_terms)) @ cov.to_numpy() @ np.ones(len(lag_terms))))
349         param_terms = ["const"] + lag_terms + [f"{outcome}_lag1", "GPR"]
350         kernel_row = {
351             "outcome": outcome,
352             "outcome_label": label,
353             "sample_start": usable["period"].iloc[0],
354             "sample_end": usable["period"].iloc[-1],
355             "n": int(len(usable)),
356             "model": "China_fuel_to_macro_ARDL",
357             "specification": "macro yoy outcome on regulated gasoline log-return lags 0..6, lagged outcome and GPR; HAC(6)",
358             "fuel_to_macro_cumulative_elasticity": estimate,
359         }
360         for lag in range(7):
361             term = f"fuel_log_return_lag{lag}"
362             kernel_row[f"beta_fuel_lag{lag}"] = float(fit.params[term])
363             kernel_row[f"se_fuel_lag{lag}"] = float(fit.bse[term])
364             kernel_row[f"phi_outcome_lag1"] = float(fit.params[f"{outcome}_lag1"])
365             kernel_row[f"se_outcome_lag1"] = float(fit.bse[f"{outcome}_lag1"])
366             kernel_row[f"gpr_coefficient"] = float(fit.params["GPR"])
367             kernel_row[f"constant"] = float(fit.params["const"])
368             kernel_rows.append(kernel_row)
369             full_cov = fit.cov_params()
370             for row_term in param_terms:
371                 for column_term in param_terms:
372                     covariance_rows.append(
373                         {
374                             "outcome": outcome,
375                             "outcome_label": label,
376                             "row_term": row_term,
377                             "column_term": column_term,
378                             "covariance": float(full_cov.loc[row_term, column_term]),
379                             "model": "China_fuel_to_macro_ARDL",
380                             "sample_start": usable["period"].iloc[0],
381                             "sample_end": usable["period"].iloc[-1],
382                             "n": int(len(usable)),
383                         }
384                     )
385             rows.append(
386                 {
387                     "outcome": outcome,
388                     "outcome_label": label,
389                     "fuel_to_macro_cumulative_elasticity": estimate,

```

```

390         "std_error": se,
391         "sample_start": usable["period"].iloc[0],
392         "sample_end": usable["period"].iloc[-1],
393         "n": int(len(usable)),
394         "model": "China_fuel_to_macro_ARDL",
395         "specification": "macro yoy outcome on regulated gasoline log-return lags 0..6, lagged outcome and GPR; HAC(6)",
396     }
397 )
398 kernel_columns = [
399     "outcome",
400     "outcome_label",
401     "sample_start",
402     "sample_end",
403     "n",
404     "model",
405     "specification",
406     "fuel_to_macro_cumulative_elasticity",
407     "beta_fuel_lag0",
408     "beta_fuel_lag1",
409     "beta_fuel_lag2",
410     "beta_fuel_lag3",
411     "beta_fuel_lag4",
412     "beta_fuel_lag5",
413     "beta_fuel_lag6",
414     "phi_outcome_lag1",
415     "gpr_coefficient",
416     "constant",
417     "se_fuel_lag0",
418     "se_fuel_lag1",
419     "se_fuel_lag2",
420     "se_fuel_lag3",
421     "se_fuel_lag4",
422     "se_fuel_lag5",
423     "se_fuel_lag6",
424     "se_outcome_lag1",
425 ]
426 covariance_columns = [
427     "outcome",
428     "outcome_label",
429     "row_term",
430     "column_term",
431     "covariance",
432     "model",
433     "sample_start",
434     "sample_end",
435     "n",
436 ]
437 save_csv(pd.DataFrame(kernel_rows, columns=kernel_columns), "q3_policy_macro_kernel.csv")
438 save_csv(pd.DataFrame(covariance_rows, columns=covariance_columns), "q3_policy_macro_covariance.csv")
439 return pd.DataFrame(rows)
440
441
442 def circular_block_indices(n: int, block_length: int, rng: np.random.Generator) -> np.ndarray:
443     indices: list[int] = []
444     while len(indices) < n:
445         start = int(rng.integers(0, n))
446         indices.extend((start + offset) % n for offset in range(block_length))
447     return np.asarray(indices[:n], dtype=int)
448
449
450 def policy_macro_model_frames() -> dict[str, tuple[pd.DataFrame, list[str]]]:
451     official = pd.read_csv(PROCESSED_DIR / "china_regulated_gasoline_monthly.csv")
452     macro = pd.read_csv(PROCESSED_DIR / "model_monthly_cn.csv")
453     frame = macro.merge(
454         official[["period", "china_regulated_gasoline_cny_per_ton"]],
455         on="period",
456         how="left",
457     ).sort_values("period")
458     frame["fuel_log"] = log_positive(frame["china_regulated_gasoline_cny_per_ton"])
459     frame["fuel_log_return"] = frame["fuel_log"].diff()
460     frame = add_lags(frame, "fuel_log_return", 6)
461     model_frames: dict[str, tuple[pd.DataFrame, list[str]]] = {}
462     for outcome in ["china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"]:
463         frame[f"{outcome}_lag1"] = frame[outcome].shift(1)
464         regressors = [f"fuel_log_return_lag{lag}" for lag in range(7)] + [f"{outcome}_lag1", "GPR"]
465         usable = frame.dropna(subset=[outcome] + regressors).copy()

```

```

466     model_frames[outcome] = (usable[["period", outcome] + regressors], regressors)
467     return model_frames
468
469
470 def joint_block_bootstrap_macro_draws(warnings_log: list[dict[str, Any]] -> pd.DataFrame:
471     """Use the same circular time blocks for the PPI, CPI and IAV equations."""
472     model_frames = policy_macro_model_frames()
473     common_periods = sorted(
474         set.intersection(*(set(frame["period"].astype(str)) for frame, _ in model_frames.values()))
475     )
476     if len(common_periods) < 48:
477         raise ValueError("Q3 joint policy bootstrap has fewer than 48 common monthly observations.")
478     aligned: dict[str, tuple[pd.DataFrame, list[str]]] = {}
479     for outcome, (frame, regressors) in model_frames.items():
480         aligned_frame = frame.loc[frame["period"].isin(common_periods)].set_index("period").loc[common_periods].reset_index()
481         aligned[outcome] = (aligned_frame, regressors)
482     rng = np.random.default_rng(RANDOM_SEED + 31)
483     rows: list[dict[str, Any]] = []
484     accepted = 0
485     attempts = 0
486     max_attempts = int(POLICY_BOOTSTRAP_REPS * 1.20)
487     while accepted < POLICY_BOOTSTRAP_REPS and attempts < max_attempts:
488         attempts += 1
489         sampled = circular_block_indices(len(common_periods), POLICY_BOOTSTRAP_BLOCK, rng)
490         draw_rows: list[dict[str, Any]] = []
491         valid = True
492         for outcome, (frame, regressors) in aligned.items():
493             sample = frame.iloc[sampled]
494             x = sm.add_constant(sample[regressors].astype(float), has_constant="add")
495             try:
496                 fit = sm.OLS(sample[outcome].astype(float), x).fit()
497             except (ValueError, np.linalg.LinAlgError):
498                 valid = False
499                 break
500             phi = float(fit.params[f"{outcome}_lag1"])
501             betas = [float(fit.params[f"fuel_log_return_lag{lag}"]) for lag in range(7)]
502             if not np.isfinite([*betas, phi]).all() or abs(phi) >= 0.995:
503                 valid = False
504                 break
505             draw_rows.append(
506                 {
507                     "draw": accepted,
508                     "outcome": outcome,
509                     **{f"beta_fuel_lag{lag}": betas[lag] for lag in range(7)},
510                     "phi_outcome_lag1": phi,
511                     "common_sample_start": common_periods[0],
512                     "common_sample_end": common_periods[-1],
513                     "common_n": len(common_periods),
514                     "block_length": POLICY_BOOTSTRAP_BLOCK,
515                 }
516             )
517             if valid:
518                 rows.extend(draw_rows)
519                 accepted += 1
520         if accepted != POLICY_BOOTSTRAP_REPS:
521             raise ValueError(f"Q3 joint policy bootstrap accepted {accepted}/{POLICY_BOOTSTRAP_REPS} required draws.")
522     warnings_log.append(
523         {
524             "code": "q3_policy_joint_bootstrap",
525             "message": f"Accepted {accepted} joint three-equation moving-block draws from {attempts} attempts.",
526         }
527     )
528     result = pd.DataFrame(rows)
529     save_csv(result, "q3_policy_macro_bootstrap_draws.csv")
530     return result
531
532
533 def dynamic_macro_gap(fuel_return_gap: np.ndarray, beta: np.ndarray, phi: float) -> np.ndarray:
534     result = np.zeros(len(fuel_return_gap), dtype=float)
535     for t in range(len(fuel_return_gap)):
536         distributed = 0.0
537         for lag in range(7):
538             if t - lag >= 0:
539                 distributed += float(beta[lag]) * float(fuel_return_gap[t - lag])
540             result[t] = distributed + (float(phi) * result[t - 1] if t > 0 else 0.0)
541     return result

```

```

542
543
544 def validate_dynamic_macro_identity() -> None:
545     fuel = np.array([0.10, 0.0, 0.0, 0.0], dtype=float)
546     beta = np.array([1.0, 2.0, 3.0, 4.0, 0.0, 0.0, 0.0], dtype=float)
547     phi = 0.5
548     observed = dynamic_macro_gap(fuel, beta, phi)
549     expected = np.array([0.10, 0.25, 0.425, 0.6125], dtype=float)
550     if not np.allclose(observed, expected, atol=1e-12, rtol=1e-12):
551         raise AssertionError("Q3 distributed-lag policy identity test failed.")
552
553
554 def policy_counterfactual(country_panel: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
555     official_path = PROCESSED_DIR / "china_regulated_gasoline_monthly.csv"
556     if not official_path.exists():
557         raise FileNotFoundError("china_regulated_gasoline_monthly.csv is required for the regulated-price proxy counterfactual.")
558     official = pd.read_csv(official_path)
559     policy = pd.read_csv(PROCESSED_DIR / "china_fuel_policy_monthly.csv")
560     full_frame = official.merge(
561         policy[["period", "gasoline_policy_gap_cny_t", "diesel_policy_gap_cny_t", "cum_gasoline_policy_gap_cny_t", "
562             cum_diesel_policy_gap_cny_t"]],
563         on="period",
564         how="left",
565         suffixes=("", "_policy"),
566     )
567     full_frame = full_frame.sort_values("period").copy()
568     elasticities = china_fuel_to_macro_elasticities(warnings_log)
569     validate_dynamic_macro_identity()
570     bootstrap_draws = joint_block_bootstrap_macro_draws(warnings_log)
571     full_frame["actual_log"] = log_positive(full_frame["china_regulated_gasoline_cny_per_ton"])
572     full_frame["no_control_log"] = log_positive(full_frame["no_temporary_control_gasoline_cny_per_ton"])
573     full_frame["actual_fuel_log_return"] = full_frame["actual_log"].diff()
574     full_frame["no_control_fuel_log_return"] = full_frame["no_control_log"].diff()
575     full_frame["fuel_return_gap"] = full_frame["no_control_fuel_log_return"] - full_frame["actual_fuel_log_return"]
576     full_frame["fuel_return_gap"] = full_frame["fuel_return_gap"].fillna(0.0)
577     frame = full_frame.loc[full_frame["period"].between("2026-02", "2026-06")].copy()
578     frame["policy_adjusted_official_cny_l"] = frame.get("china_regulated_gasoline_cny_per_l", pd.Series(np.nan, index=frame.index))
579     frame["no_temporary_control_official_cny_l"] = frame.get("no_temporary_control_gasoline_cny_per_l", pd.Series(np.nan, index=frame.index))
580     frame["policy_adjusted_official_cny_t"] = frame["china_regulated_gasoline_cny_per_ton"]
581     frame["no_temporary_control_official_cny_t"] = frame["no_temporary_control_gasoline_cny_per_ton"]
582     frame["incremental_gasoline_gap_cny_t"] = frame["gasoline_policy_gap_cny_t"]
583     frame["cumulative_gasoline_gap_cny_t"] = frame["cum_gasoline_policy_gap_cny_t"]
584     frame["actual"] = frame["policy_adjusted_official_cny_t"]
585     frame["prediction"] = frame["no_temporary_control_official_cny_t"]
586     frame["response"] = frame["cumulative_gasoline_gap_cny_t"]
587     frame["fuel_log_gap"] = np.log(frame["prediction"] / frame["actual"])
588     frame["fuel_return_gap"] = full_frame.loc[frame.index, "fuel_return_gap"].to_numpy()
589     macro_rows: list[dict[str, Any]] = []
590     event_gap = frame["fuel_return_gap"].to_numpy(dtype=float)
591     kernel = pd.read_csv(RESULTS_DIR / "q3_policy_macro_kernel.csv")
592     for elastic in elasticities.to_dict("records"):
593         outcome = str(elastic["outcome"])
594         kernel_row = kernel.loc[kernel["outcome"].eq(outcome)].iloc[0]
595         beta = np.array([float(kernel_row[f"beta_fuel_lag{lag}"]) for lag in range(7)], dtype=float)
596         phi = float(kernel_row["phi_outcome_lag1"])
597         point_path = dynamic_macro_gap(event_gap, beta, phi)
598         outcome_draws = bootstrap_draws.loc[bootstrap_draws["outcome"].eq(outcome)].sort_values("draw")
599         draw_paths = np.vstack(
600             [
601                 dynamic_macro_gap(
602                     event_gap,
603                     np.array([float(row[f"beta_fuel_lag{lag}"]) for lag in range(7)], dtype=float),
604                     float(row["phi_outcome_lag1"]),
605                 )
606                 for row in outcome_draws.to_dict("records")
607             ]
608         )
609         for t, period_row in enumerate(frame.to_dict("records")):
610             gap = float(point_path[t])
611             lower, upper = np.quantile(draw_paths[:, t], [0.025, 0.975])
612             macro_rows.append(
613                 {
614                     "period": period_row["period"],
615                     "outcome": elastic["outcome"],
616                     "outcome_label": elastic["outcome_label"],
617                 }
618             )

```



```

616         "policy_adjusted_official_cny_t": period_row["policy_adjusted_official_cny_t"],
617         "no_temporary_control_official_cny_t": period_row["no_temporary_control_official_cny_t"],
618         "incremental_gasoline_gap_cny_t": period_row["incremental_gasoline_gap_cny_t"],
619         "cumulative_gasoline_gap_cny_t": period_row["cumulative_gasoline_gap_cny_t"],
620         "fuel_log_gap": period_row["fuel_log_gap"],
621         "fuel_return_gap": period_row["fuel_return_gap"],
622         "macro_counterfactual_gap_pctpt": gap,
623         "lower_95": float(lower),
624         "upper_95": float(upper),
625         "model": "China_proxy_fuel_dynamic_ARDL_counterfactual",
626         "horizon": t,
627         "specification": "monthly convolution of no-control minus actual proxy fuel returns through lags 0..6 with recursive
outcome AR(1); joint three-equation circular block bootstrap",
628         "sample_start": elastic["sample_start"],
629         "sample_end": elastic["sample_end"],
630         "n": elastic["n"],
631         "price_layer_status": "regulated_gasoline_adjustment_index_proxy_with_2026_notice_gaps",
632         "evidence_status": "CONDITIONAL_DYNAMIC_PROXY_SCENARIO",
633         "bootstrap_reps": POLICY_BOOTSTRAP_REPS,
634         "bootstrap_block_length": POLICY_BOOTSTRAP_BLOCK,
635     }
636 )
637 macro_columns = [
638     "period",
639     "outcome",
640     "outcome_label",
641     "policy_adjusted_official_cny_t",
642     "no_temporary_control_official_cny_t",
643     "incremental_gasoline_gap_cny_t",
644     "cumulative_gasoline_gap_cny_t",
645     "fuel_log_gap",
646     "fuel_return_gap",
647     "macro_counterfactual_gap_pctpt",
648     "lower_95",
649     "upper_95",
650     "model",
651     "horizon",
652     "specification",
653     "sample_start",
654     "sample_end",
655     "n",
656     "price_layer_status",
657     "evidence_status",
658     "bootstrap_reps",
659     "bootstrap_block_length",
660 ]
661 macro_counterfactual = pd.DataFrame(macro_rows, columns=macro_columns)
662 save_csv(macro_counterfactual, "q3_policy_macro_counterfactual.csv")
663 cpi_macro = macro_counterfactual.loc[macro_counterfactual["outcome"].eq("china_cpi_yoy_pct"), ["period", "
macro_counterfactual_gap_pctpt", "lower_95", "upper_95"]]
664 frame = frame.drop(columns=[column for column in ["lower_95", "upper_95"] if column in frame.columns])
665 frame = frame.merge(cpi_macro.rename(columns={"macro_counterfactual_gap_pctpt": "cpi_counterfactual_gap_pctpt"}), on="period", how="
left")
666 frame["model"] = "China_policy_counterfactual"
667 frame["horizon"] = np.arange(len(frame), dtype=int)
668 frame["specification"] = "add separately audited 2026 NDRC policy gaps to the reconstructed regulated-gasoline adjustment-index proxy
; dynamic macro propagation is written to q3_policy_macro_counterfactual.csv"
669 frame["evidence_status"] = np.where(frame["policy_adjusted_official_cny_t"].notna(), "CONDITIONAL_PROXY_PRICE_LAYER", "
CONDITIONAL_DATA_COVERAGE")
670 frame["price_layer_status"] = "regulated_gasoline_adjustment_index_proxy_with_2026_notice_gaps"
671 frame["sample_start"] = "2013-03"
672 frame["sample_end"] = "2026-06"
673 result = frame[
674     [
675         "period",
676         "actual",
677         "prediction",
678         "response",
679         "policy_adjusted_official_cny_l",
680         "no_temporary_control_official_cny_l",
681         "policy_adjusted_official_cny_t",
682         "no_temporary_control_official_cny_t",
683         "incremental_gasoline_gap_cny_t",
684         "cumulative_gasoline_gap_cny_t",
685         "fuel_log_gap",
686         "fuel_return_gap",

```

```

687         "cpi_counterfactual_gap_pctpt",
688         "lower_95",
689         "upper_95",
690         "gasoline_policy_gap_cny_t",
691         "diesel_policy_gap_cny_t",
692         "cum_gasoline_policy_gap_cny_t",
693         "model",
694         "horizon",
695         "specification",
696         "evidence_status",
697         "price_layer_status",
698         "sample_start",
699         "sample_end",
700     ]
701 ]
702 save_csv(result, "q3_policy_counterfactual.csv")
703 return result
704
705
706 def classify_better(diff: float, lower: float, upper: float, better_when_positive: bool = True) -> str:
707     if not np.isfinite(diff):
708         return "INCONCLUSIVE"
709     point_better = diff > 0 if better_when_positive else diff < 0
710     interval_better = lower > 0 if better_when_positive else upper < 0
711     if interval_better:
712         return "SUPPORTED"
713     if point_better:
714         return "PARTIAL"
715     return "NOT_SUPPORTED"
716
717
718 def build_resilience_metrics(pass_through: pd.DataFrame, panel_irf: pd.DataFrame, macro_counterfactual: pd.DataFrame) -> pd.DataFrame:
719     """Report exactly three economic dimensions and fail closed on invalid joint intervals."""
720     rows: list[dict[str, Any]] = []
721     fuel_h6 = pass_through.loc[pass_through["horizon"].eq(6)].copy()
722     china = fuel_h6.loc[fuel_h6["country"].eq("CHN")]
723     controls = fuel_h6.loc[fuel_h6["country"].isin(MAIN_CONTROL_COUNTRIES)]
724     rows.append(
725         {
726             "dimension": "fuel_price",
727             "metric": "six_month_pass_through_proxy_sensitivity",
728             "horizon": 6,
729             "china_value": float(china["response"].iloc[0]) if not china.empty else np.nan,
730             "control_median": float(controls["response"].median()) if not controls.empty else np.nan,
731             "china_vs_control_median_diff": np.nan,
732             "diff_lower_95": np.nan,
733             "diff_upper_95": np.nan,
734             "judgement": "CONDITIONAL_PROXY",
735             "interpretation": "China is reported as an adjustment-index proxy sensitivity and is excluded from the formal retail-price ranking; no invalid median interval is constructed.",
736         }
737     )
738     for outcome, dimension in [("cpi", "consumer_prices"), ("ip", "industrial_activity")]:
739         sub = panel_irf.loc[panel_irf["outcome"].eq(outcome)].copy()
740         if sub.empty:
741             continue
742         grouped = sub.groupby("horizon", as_index=False).agg(median_relative_response=("response", "median"))
743         row = grouped.loc[grouped["median_relative_response"].abs().idxmax()]
744         rows.append(
745             {
746                 "dimension": dimension,
747                 "metric": f"control_minus_china_{outcome}_relative_response",
748                 "horizon": int(row["horizon"]),
749                 "china_value": np.nan,
750                 "control_median": float(row["median_relative_response"]),
751                 "china_vs_control_median_diff": float(row["median_relative_response"]),
752                 "diff_lower_95": np.nan,
753                 "diff_upper_95": np.nan,
754                 "judgement": "INCONCLUSIVE",
755                 "interpretation": "Only the control-minus-China relative point estimate is shown; China is not set to zero and no confidence claim is made without a joint country-time bootstrap.",
756             }
757         )
758     if not macro_counterfactual.empty:
759         for outcome, group in macro_counterfactual.groupby("outcome"):
760             max_gap = group.loc[group["macro_counterfactual_gap_pctpt"].abs().idxmax()]

```

```

761         rows.append(
762             {
763                 "dimension": "policy_counterfactual_macro",
764                 "metric": outcome,
765                 "horizon": int(max_gap.get("horizon", 6)),
766                 "china_value": float(max_gap["macro_counterfactual_gap_pctpt"]),
767                 "control_median": np.nan,
768                 "china_vs_control_median_diff": np.nan,
769                 "diff_lower_95": float(max_gap["lower_95"]),
770                 "diff_upper_95": float(max_gap["upper_95"]),
771                 "judgement": "POLICY_SCENARIO",
772                 "interpretation": "macro gap if 2026 temporary fuel-price controls are mechanically removed from the official
regulated price path",
773             }
774         )
775     result = pd.DataFrame(rows)
776     overall = "INCONCLUSIVE"
777     if not result.empty:
778         result["overall_china_resilience_judgement"] = overall
779     save_csv(result, "q3_resilience_metrics.csv")
780     return result
781
782
783 def robustness_checks(panel: pd.DataFrame, warnings_log: list[dict[str, Any]]) -> pd.DataFrame:
784     rows: list[dict[str, Any]] = []
785     main_panel = panel.loc[bool_series(panel["included_in_main_comparison"])] .copy() if "included_in_main_comparison" in panel else panel
786     .copy()
787     main_countries = [country for country in COUNTRY_ORDER if country in set(main_panel["country"])]
788     warnings_log.append(
789         {
790             "code": "q3_fuel_panel_leave_one_withheld",
791             "message": "Fuel panel leave-one-country inference is withheld because China is a proxy and excluded from the formal price-
level panel.",
792         }
793     )
794
795     monthly = pd.read_csv(PROCESSED_DIR / "model_monthly_q1.csv")
796     usd_panel = main_panel.merge(monthly[["period", "brent_usd_bbl_log_return"]], on="period", how="left")
797     for country in main_countries:
798         frame = usd_panel.loc[usd_panel["country"].eq(country)].sort_values("period").copy()
799         frame = add_lags(frame, "brent_usd_bbl_log_return", 6)
800         frame["fuel_lag1"] = frame["fuel_log_return"].shift(1)
801         regressors = [f"brent_usd_bbl_log_return_lag{lag}" for lag in range(7)] + ["fuel_lag1"]
802         usable = frame.dropna(subset=["fuel_log_return"] + regressors)
803         if len(usable) < 48:
804             continue
805         fit = sm.OLS(
806             usable["fuel_log_return"],
807             sm.add_constant(usable[regressors].astype(float), has_constant="add"),
808         ).fit(cov_type="HAC", cov_kws={"maxlags": 6})
809         lag_terms = [f"brent_usd_bbl_log_return_lag{lag}" for lag in range(7)]
810         estimate = float(fit.params[lag_terms].sum())
811         rows.append(
812             {
813                 "robustness_type": "usd_brent_fx_ignored_pass_through_h6",
814                 "omitted_country": "",
815                 "country": country,
816                 "horizon": 6,
817                 "estimate": estimate,
818                 "std_error": np.nan,
819                 "model": "distributed_lag_pass_through",
820                 "specification": "uses USD Brent instead of local-currency Brent",
821                 "n": int(len(usable)),
822             }
823         )
824
825     policy = pd.read_csv(PROCESSED_DIR / "china_fuel_policy_monthly.csv")
826     policy = policy.loc[policy["period"].between("2026-02", "2026-06")].copy()
827     for _, row in policy.iterrows():
828         rows.append(
829             {
830                 "robustness_type": "diesel_policy_layer_gap",
831                 "omitted_country": "",
832                 "country": "CHN",
833                 "horizon": 0,
834                 "estimate": float(row["cum_diesel_policy_gap_cny_t"]),

```

```

834         "std_error": np.nan,
835         "model": "policy_price_layer",
836         "specification": f"cumulative diesel policy gap through {row['period']}",
837         "n": 1,
838     }
839 )
840 result = pd.DataFrame(rows)
841 save_csv(result, "q3_robustness.csv")
842 return result
843
844
845 def plot_pass_through(pass_through: pd.DataFrame) -> None:
846     if pass_through.empty:
847         return
848     frame = pass_through.loc[pass_through["horizon"].eq(6)].copy()
849     if "included_in_main_comparison" in frame:
850         frame = frame.loc[bool_series(frame["included_in_main_comparison"])]
851     frame["country_label"] = frame["country"].map(COUNTRY_LABEL_ZH).fillna(frame["country"])
852     frame["err_low"] = frame["response"] - frame["lower_95"]
853     frame["err_high"] = frame["upper_95"] - frame["response"]
854     color_cycle = [PALETTE["gold"], PALETTE["olive"], PALETTE["rose"], PALETTE["slate"], PALETTE["sand"], PALETTE["blue_light"]]
855     colors = [color_cycle[idx % len(color_cycle)] for idx in range(len(frame))]
856     fig, ax = plt.subplots(figsize=(7.6, 4.8))
857     ax.bar(
858         frame["country_label"],
859         frame["response"],
860         color=colors,
861         edgecolor="white",
862         linewidth=0.8,
863         width=0.62,
864         yerr=[frame["err_low"], frame["err_high"]],
865         error_kw={"ecolor": PALETTE["ink"], "elinewidth": 0.8, "capsize": 3},
866     )
867     ax.axhline(0, color=PALETTE["muted"], linewidth=0.8)
868     style_axis(ax, ylabel="累计传导系数")
869     finish_figure(
870         fig,
871         title="问题三: 六个月燃油价格传导率",
872         subtitle="仅纳入覆盖充分的可观测或官方受管制零售汽油价格; 中国覆盖不足时不参与主排名。",
873         source="来源: 欧盟周度油价公报、日本METI、韩国KOSIS/KNOC、北京市发改委与 FRED; 由 code/problem3/run_q3.py 生成。",
874         rect=(0.10, 0.13, 0.98, 0.84),
875     )
876     save_figure(fig, FIGURES_DIR / "q3_pass_through_6m")
877     plt.close(fig)
878
879
880 def plot_panel_irf(panel_irf: pd.DataFrame) -> None:
881     if panel_irf.empty:
882         return
883     frame = panel_irf.loc[panel_irf["outcome"].eq("fuel")].copy()
884     if frame.empty:
885         frame = panel_irf.copy()
886     fig, ax = plt.subplots(figsize=(8.8, 5.1))
887     style_map = {
888         "CHN": (PALETTE["blue"], "solid", "o"),
889         "DEU": (PALETTE["gold"], (0, (4, 2)), "s"),
890         "FRA": (PALETTE["olive"], (0, (2, 2)), "^"),
891         "ITA": (PALETTE["rose"], (0, (1, 2)), "D"),
892         "ESP": (PALETTE["slate"], (0, (6, 2)), "v"),
893         "JPN": (PALETTE["sand"], (0, (3, 1, 1, 1)), "P"),
894         "KOR": (PALETTE["blue_light"], (0, (1, 1)), "X"),
895     }
896     for country in COUNTRY_ORDER:
897         sub = frame.loc[frame["country"].eq(country)].sort_values("horizon")
898         if sub.empty:
899             continue
900         color, linestyle, marker = style_map.get(country, (PALETTE["muted"], "solid", "o"))
901         ax.plot(sub["horizon"], sub["response"], color=color, linestyle=linestyle, marker=marker, label=COUNTRY_LABEL_ZH.get(country, country))
902     ax.axhline(0, color=PALETTE["muted"], linewidth=0.8)
903     style_axis(ax, xlabel="油价冲击后月份", ylabel="响应")
904     ax.legend(loc="upper left", ncol=4, handlelength=2.6)
905     finish_figure(
906         fig,
907         title="问题三: 跨国面板 LP 响应",
908         subtitle="完整年月固定效应吸收共同冲击, 因此曲线表示国家相对响应差异; 燃油主图仅纳入覆盖充分国家。",
909     )

```

```

909     source="来源: model_country_monthly.csv 与 q1_monthly_shocks.csv; 由 code/problem3/run_q3.py 生成。",
910 )
911 save_figure(fig, FIGURES_DIR / "q3_panel_irf")
912 plt.close(fig)
913
914
915 def plot_policy(counterfactual: pd.DataFrame) -> None:
916     if counterfactual.empty:
917         return
918     frame = counterfactual.copy()
919     x = np.arange(len(frame))
920     fig, ax = plt.subplots(figsize=(7.8, 4.8))
921     ax.plot(x, frame["actual"], marker="o", color=PALETTE["blue"], label="政策调整后调价指数代理")
922     ax.plot(x, frame["prediction"], marker="s", color=PALETTE["gold"], linestyle=(0, (4, 2)), label="s 无临时调控规则路径")
923     ax.fill_between(x, frame["actual"], frame["prediction"], color=PALETTE["gold_light"], alpha=0.26, linewidth=0)
924     ax.set_xticks(x)
925     ax.set_xticklabels(frame["period"])
926     for tick, row in enumerate(frame.itertuples(index=False)):
927         if getattr(row, "cumulative_gasoline_gap_cny_t") > 0:
928             label = f"累计差额 {row.cumulative_gasoline_gap_cny_t:.0f}"
929             if getattr(row, "incremental_gasoline_gap_cny_t") > 0 and row.period == "2026-04":
930                 label = f"累计差额 {row.cumulative_gasoline_gap_cny_t:.0f}\n4月新增 {row.incremental_gasoline_gap_cny_t:.0f}"
931             ax.text(tick, max(row.actual, row.prediction), label, ha="center", va="bottom", fontsize=8.4, color=PALETTE["muted"])
932     style_axis(ax, ylabel="元/吨")
933     ax.legend(loc="lower center", bbox_to_anchor=(0.5, 1.01), ncol=2, handlelength=2.6)
934     finish_figure(
935         fig,
936         title="问题三: 中国成品油调控政策情景",
937         subtitle="阴影代表将单独核验的2026政策差额加回历史调价指数代理; 宏观传播为条件动态情景。",
938         source="来源: 国家发展改革委2026调价公告、公开历史调价镜像及处理说明; 由 code/problem3/run_q3.py 生成。",
939         rect=(0.10, 0.13, 0.98, 0.94),
940     )
941     save_figure(fig, FIGURES_DIR / "q3_policy_counterfactual")
942     plt.close(fig)
943
944
945 def main(argv: list[str] | None = None) -> int:
946     parser = argparse.ArgumentParser()
947     parser.add_argument("--plots-only", action="store_true", help="Regenerate Q3 figures from saved result tables.")
948     args = parser.parse_args(argv)
949
950     np.random.seed(RANDOM_SEED)
951     apply_paper_style()
952     ensure_dirs()
953     if args.plots_only:
954         pass_through = read_csv_result("q3_country_pass_through.csv")
955         panel_irf = read_csv_result("q3_panel_irf.csv")
956         counterfactual = read_csv_result("q3_policy_counterfactual.csv")
957         plot_pass_through(pass_through)
958         plot_panel_irf(panel_irf)
959         plot_policy(counterfactual)
960         print(json.dumps({"status": "PASS", "mode": "plots-only", "figures": 3}, ensure_ascii=False, indent=2))
961         return 0
962
963     from linearmodels.panel.utility import AbsorbingEffectWarning
964
965     warnings_log: list[dict[str, Any]] = []
966     panel = pd.read_csv(PROCESSED_DIR / "model_country_monthly.csv")
967     panel["fuel_log"] = log_positive(panel["fuel_price_local"])
968     py_warnings.filterwarnings("ignore", category=AbsorbingEffectWarning)
969     pass_through = fit_country_pass_through(panel, warnings_log)
970     panel_irf = fit_panel_lp(panel, warnings_log)
971     buffer_irf = fit_buffer_interactions(panel, warnings_log)
972     counterfactual = policy_counterfactual(panel, warnings_log)
973     macro_counterfactual = read_csv_result("q3_policy_macro_counterfactual.csv")
974     resilience = build_resilience_metrics(pass_through, panel_irf, macro_counterfactual)
975     robustness = robustness_checks(panel, warnings_log)
976     plot_pass_through(pass_through)
977     plot_panel_irf(panel_irf)
978     plot_policy(counterfactual)
979
980     summary = pd.DataFrame(
981         [
982             {
983                 "component": "CountryPassThrough",
984                 "rows": len(pass_through),

```

```

985         "status": "PASS" if len(pass_through) else "WARN",
986         "note": "formal ranking covers six comparable control countries; China adjustment-index proxy is retained only as
sensitivity",
987     },
988     {
989         "component": "PanelLP",
990         "rows": len(panel_irf),
991         "status": "PASS" if len(panel_irf) else "WARN",
992         "note": "CPI and industrial-activity LP coefficients are control-country minus China relative responses with country and
full year-month fixed effects",
993     },
994     {
995         "component": "BufferInteractions",
996         "rows": len(buffer_irf),
997         "status": "WITHHELD_UNAUDITED_PROXY",
998         "note": "continuous buffer interactions are not released until the country-year source table is independently auditable",
999     },
1000     {
1001         "component": "ChinaPolicyCounterfactual",
1002         "rows": len(counterfactual),
1003         "status": "PASS" if len(counterfactual) and len(macro_counterfactual) else "WARN",
1004         "note": "2026 notice gaps are propagated through a dynamic ARDL kernel on a reconstructed adjustment-index proxy with
joint time-block uncertainty",
1005     },
1006     {
1007         "component": "ResilienceMetrics",
1008         "rows": len(resilience),
1009         "status": "PASS" if len(resilience) else "WARN",
1010         "note": "exactly three economic dimensions; overall judgement fails closed because the fuel layer is a proxy and joint
cross-country intervals are not claimed",
1011     },
1012     {
1013         "component": "Robustness",
1014         "rows": len(robustness),
1015         "status": "PASS" if len(robustness) else "WARN",
1016         "note": "USD-Brent sensitivity and diesel policy-layer checks; invalid fuel panel leave-one inference is withheld",
1017     },
1018 ]
1019 )
1020 save_csv(summary, "q3_summary.csv")
1021 payload = {
1022     "status": "CONDITIONAL",
1023     "execution_status": "PASS" if len(pass_through) and len(panel_irf) and len(counterfactual) else "WARN",
1024     "evidence_status": str(resilience["overall_china_resilience_judgement"].dropna().iloc[0]) if "overall_china_resilience_judgement"
in resilience and not resilience["overall_china_resilience_judgement"].dropna().empty else "INCONCLUSIVE",
1025     "allowed_claims": [
1026         "China historical fuel pass-through is a proxy sensitivity and is excluded from the formal retail-price ranking",
1027         "Panel LP coefficients are relative to China under full year-month fixed effects",
1028         "The 2026 conditional policy scenario propagates proxy fuel-return gaps dynamically through PPI/CPI/IAV kernels with a common
time-block bootstrap",
1029     ],
1030     "forbidden_claims": [
1031         "China is better solely because Brent-CNY proxy pass-through is lower",
1032         "fuel price smoothing is welfare-improving without considering cumulative policy gap costs",
1033         "unaudited structural-buffer proxies support quantitative policy-interaction effects",
1034     ],
1035     "random_seed": RANDOM_SEED,
1036     "comparability_guardrail": "China uses a reconstructed regulated-gasoline adjustment-index proxy and is excluded from the formal
cross-country retail-price ranking; 2026 policy notices are audited separately.",
1037     "warnings": warnings_log,
1038     "rows": {
1039         "q3_country_pass_through.csv": int(len(pass_through)),
1040         "q3_panel_irf.csv": int(len(panel_irf)),
1041         "q3_buffer_interactions.csv": int(len(buffer_irf)),
1042         "q3_buffer_data_status.csv": 4,
1043         "q3_policy_counterfactual.csv": int(len(counterfactual)),
1044         "q3_policy_macro_counterfactual.csv": int(len(macro_counterfactual)),
1045         "q3_policy_macro_bootstrap_draws.csv": POLICY_BOOTSTRAP_REPS * 3,
1046         "q3_resilience_metrics.csv": int(len(resilience)),
1047         "q3_robustness.csv": int(len(robustness)),
1048     },
1049 }
1050 (RESULTS_DIR / "q3_summary.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
1051 print(json.dumps(payload, ensure_ascii=False, indent=2))
1052 return 0
1053

```

```

1054
1055 if __name__ == "__main__":
1056     raise SystemExit(main())

```

Listing 8: run_q3.py

```

1 """Extension Q4: Brent upper-tail risk and China macro-policy stress tests."""
2
3 from __future__ import annotations
4
5 import argparse
6 import json
7 import math
8 import sys
9 import time
10 from pathlib import Path
11 from typing import Any
12
13 import matplotlib
14
15 matplotlib.use("Agg")
16
17 import matplotlib.pyplot as plt
18 import numpy as np
19 import pandas as pd
20 from scipy.optimize import minimize
21
22
23 REPO_ROOT = Path(__file__).resolve().parents[2]
24 CODE_DIR = REPO_ROOT / "code"
25 if str(CODE_DIR) not in sys.path:
26     sys.path.insert(0, str(CODE_DIR))
27
28 from utils.plot_style import PALETTE, apply_paper_style, finish_figure, save_figure, style_axis
29
30 if hasattr(sys.stdout, "reconfigure"):
31     sys.stdout.reconfigure(encoding="utf-8")
32
33 PROCESSED_DIR = REPO_ROOT / "data" / "processed"
34 RESULTS_DIR = REPO_ROOT / "results"
35 FIGURES_DIR = REPO_ROOT / "figures"
36 CUTOFF = pd.Timestamp("2026-06-30")
37 RANDOM_SEED = 20260730
38 HORIZON_STEPS = {1: 22, 3: 66, 6: 132}
39 FORECAST_QUANTILES = [0.05, 0.10, 0.50, 0.90, 0.95]
40 OUTCOME_LABELS = {
41     "china_ppi_yoy_pct": "PPI",
42     "china_cpi_yoy_pct": "CPI",
43     "china_iav_yoy_pct": "工业增加值",
44 }
45
46
47 def ensure_dirs() -> None:
48     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
49     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
50
51
52 def save_csv(frame: pd.DataFrame, filename: str) -> Path:
53     path = RESULTS_DIR / filename
54     frame.to_csv(path, index=False, encoding="utf-8")
55     return path
56
57
58 def write_json(payload: dict[str, Any], filename: str) -> Path:
59     path = RESULTS_DIR / filename
60     path.write_text(json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False) + "\n", encoding="utf-8")
61     return path
62
63
64 def json_records(frame: pd.DataFrame) -> list[dict[str, Any]]:
65     if frame.empty:
66         return []
67     return json.loads(frame.where(pd.notna(frame), None).to_json(orient="records"))
68
69

```

```

70 def load_daily() -> pd.DataFrame:
71     path = PROCESSED_DIR / "model_daily_q1.csv"
72     frame = pd.read_csv(path, parse_dates=["date"])
73     required = {"date", "brent_usd_bbl", "brent_usd_bbl_log_return"}
74     missing = required.difference(frame.columns)
75     if missing:
76         raise ValueError(f"Q4 daily input missing columns: {sorted(missing)}")
77     frame = frame.loc[frame["date"].le(CUTOFF)].copy()
78     frame["brent_usd_bbl"] = pd.to_numeric(frame["brent_usd_bbl"], errors="coerce")
79     frame["return_pct"] = pd.to_numeric(frame["brent_usd_bbl_log_return"], errors="coerce") * 100.0
80     frame = frame.dropna(subset=["date", "brent_usd_bbl", "return_pct"]).sort_values("date").reset_index(drop=True)
81     if len(frame) < 750:
82         raise ValueError(f"Q4 needs at least 750 daily returns, got {len(frame)}")
83     if frame["date"].max() != CUTOFF:
84         raise ValueError(f"Q4 last daily observation must be {CUTOFF.date()}, got {frame['date'].max().date()}")
85     return frame
86
87
88 def _variance_path(eps: np.ndarray, params: np.ndarray, sample_var: float) -> np.ndarray:
89     omega, alpha, gamma, beta = params
90     h = np.empty(len(eps), dtype=float)
91     h[0] = sample_var
92     for idx in range(1, len(eps)):
93         lag = eps[idx - 1]
94         h[idx] = omega + alpha * lag**2 + gamma * (lag < 0) * lag**2 + beta * h[idx - 1]
95     return h
96
97
98 def fit_gjr_garch(returns_pct: pd.Series) -> dict[str, Any]:
99     values = pd.to_numeric(returns_pct, errors="coerce").dropna().to_numpy(dtype=float)
100     if len(values) < 500:
101         raise ValueError(f"GJR-GARCH needs at least 500 observations, got {len(values)}")
102     mu = float(np.mean(values))
103     eps = values - mu
104     sample_var = float(np.var(eps, ddof=1))
105     if not np.isfinite(sample_var) or sample_var <= 0:
106         raise ValueError("GJR-GARCH sample variance is not positive.")
107
108     def neg_loglike(params: np.ndarray) -> float:
109         omega, alpha, gamma, beta = params
110         persistence = alpha + 0.5 * gamma + beta
111         if omega <= 0 or min(alpha, gamma, beta) < 0 or persistence >= 0.999:
112             return 1e15
113         h = _variance_path(eps, params, sample_var)
114         if np.any(~np.isfinite(h)) or np.any(h <= 0):
115             return 1e15
116         return float(0.5 * np.sum(np.log(2 * np.pi) + np.log(h) + eps**2 / h))
117
118     starts = [
119         np.array([0.02 * sample_var, 0.05, 0.05, 0.88]),
120         np.array([0.03 * sample_var, 0.08, 0.02, 0.86]),
121         np.array([0.05 * sample_var, 0.10, 0.00, 0.80]),
122     ]
123     constraints = ({ "type": "ineq", "fun": lambda p: 0.999 - p[1] - 0.5 * p[2] - p[3] },)
124     fits = []
125     for start in starts:
126         fit = minimize(
127             neg_loglike,
128             x0=start,
129             bounds=[(1e-10, None), (0.0, 1.0), (0.0, 1.0), (0.0, 0.999)],
130             constraints=constraints,
131             method="SLSQP",
132             options={"maxiter": 600, "ftol": 1e-9},
133         )
134         if fit.success and np.isfinite(fit.fun):
135             fits.append(fit)
136     if not fits:
137         raise RuntimeError("All GJR-GARCH optimization starts failed.")
138     fit = min(fits, key=lambda item: float(item.fun))
139     params = np.asarray(fit.x, dtype=float)
140     h = _variance_path(eps, params, sample_var)
141     z = eps / np.sqrt(h)
142     z = z[np.isfinite(z)]
143     z_std = float(np.std(z, ddof=1))
144     if len(z) < 400 or not np.isfinite(z_std) or z_std <= 0:
145         raise RuntimeError("GJR-GARCH standardized residuals are unusable.")

```



```

146 z = (z - float(np.mean(z))) / z_std
147 omega, alpha, gamma, beta = params
148 return {
149     "mu": mu,
150     "omega": float(omega),
151     "alpha": float(alpha),
152     "gamma": float(gamma),
153     "beta": float(beta),
154     "persistence": float(alpha + 0.5 * gamma + beta),
155     "last_eps": float(eps[-1]),
156     "last_h": float(h[-1]),
157     "standardized_residuals": z,
158     "n": int(len(values)),
159     "objective": float(fit.fun),
160 }
161
162
163 def simulate_fhs(
164     start_price: float,
165     fit: dict[str, Any],
166     steps: int,
167     paths: int,
168     seed: int,
169 ) -> np.ndarray:
170     rng = np.random.default_rng(seed)
171     prices = np.empty((paths, steps), dtype=float)
172     log_price = np.full(paths, math.log(start_price), dtype=float)
173     h_prev = np.full(paths, float(fit["last_h"]), dtype=float)
174     eps_prev = np.full(paths, float(fit["last_eps"]), dtype=float)
175     residuals = np.asarray(fit["standardized_residuals"], dtype=float)
176     omega = float(fit["omega"])
177     alpha = float(fit["alpha"])
178     gamma = float(fit["gamma"])
179     beta = float(fit["beta"])
180     mu = float(fit["mu"])
181     for step in range(steps):
182         h = omega + alpha * eps_prev**2 + gamma * (eps_prev < 0) * eps_prev**2 + beta * h_prev
183         h = np.maximum(h, 1e-10)
184         z = rng.choice(residuals, size=paths, replace=True)
185         eps = np.sqrt(h) * z
186         log_price += (mu + eps) / 100.0
187         prices[:, step] = np.exp(log_price)
188         h_prev = h
189         eps_prev = eps
190     return prices
191
192
193 def simulate_gaussian(
194     start_price: float,
195     returns_pct: pd.Series,
196     steps: int,
197     paths: int,
198     seed: int,
199 ) -> np.ndarray:
200     values = pd.to_numeric(returns_pct, errors="coerce").dropna().to_numpy(dtype=float)
201     mu = float(np.mean(values))
202     sigma = float(np.std(values, ddof=1))
203     if not np.isfinite(sigma) or sigma <= 0:
204         raise ValueError("Gaussian baseline volatility is not positive.")
205     rng = np.random.default_rng(seed)
206     draws = rng.normal(mu, sigma, size=(paths, steps)) / 100.0
207     return start_price * np.exp(np.cumsum(draws, axis=1))
208
209
210 def historical_thresholds(daily: pd.DataFrame) -> dict[str, float]:
211     prices = daily["brent_usd_bbl"].dropna()
212     return {
213         "historical_price_p90": float(prices.quantile(0.90)),
214         "historical_price_p95": float(prices.quantile(0.95)),
215     }
216
217
218 def summarize_paths(
219     paths: np.ndarray,
220     model: str,
221     origin_date: pd.Timestamp,

```

```

222     origin_price: float,
223     thresholds: dict[str, float],
224 ) -> pd.DataFrame:
225     rows: list[dict[str, Any]] = []
226     for horizon, steps in HORIZON_STEPS.items():
227         terminal = paths[:, steps - 1]
228         prefix = paths[:, :steps]
229         quantiles = np.quantile(terminal, FORECAST_QUANTILES)
230         threshold_90 = thresholds["historical_price_p90"]
231         threshold_95 = thresholds["historical_price_p95"]
232         upper_tail = terminal[terminal > threshold_95]
233         rows.append(
234             {
235                 "origin_date": origin_date.strftime("%Y-%m-%d"),
236                 "origin_price_usd_bbl": origin_price,
237                 "horizon_months": horizon,
238                 "trading_days": steps,
239                 "model": model,
240                 "paths": int(len(terminal)),
241                 "p05_price": float(quantiles[0]),
242                 "p10_price": float(quantiles[1]),
243                 "median_price": float(quantiles[2]),
244                 "p90_price": float(quantiles[3]),
245                 "p95_price": float(quantiles[4]),
246                 "historical_price_p90": threshold_90,
247                 "historical_price_p95": threshold_95,
248                 "terminal_prob_above_hist_p90": float(np.mean(terminal > threshold_90)),
249                 "terminal_prob_above_hist_p95": float(np.mean(terminal > threshold_95)),
250                 "path_prob_cross_hist_p90": float(np.mean(np.max(prefix, axis=1) > threshold_90)),
251                 "path_prob_cross_hist_p95": float(np.mean(np.max(prefix, axis=1) > threshold_95)),
252                 "upper_tail_mean_above_hist_p95": float(np.mean(upper_tail)) if len(upper_tail) else np.nan,
253                 "evidence_status": "PROBABILISTIC_STRESS_FORECAST",
254             }
255         )
256     return pd.DataFrame(rows)
257
258
259 def pinball(actual: float, prediction: float, quantile: float) -> float:
260     error = actual - prediction
261     return float(max(quantile * error, (quantile - 1.0) * error))
262
263
264 def interval_score(actual: float, lower: float, upper: float, alpha: float) -> float:
265     score = upper - lower
266     if actual < lower:
267         score += (2.0 / alpha) * (lower - actual)
268     elif actual > upper:
269         score += (2.0 / alpha) * (actual - upper)
270     return float(score)
271
272
273 def backtest_origins(daily: pd.DataFrame) -> pd.DataFrame:
274     candidates = daily.loc[daily["date"].between(pd.Timestamp("2022-01-01"), pd.Timestamp("2025-12-31"))].copy()
275     candidates["month"] = candidates["date"].dt.to_period("M")
276     candidates = candidates.groupby("month", as_index=False).tail(1)
277     max_step = max(HORIZON_STEPS.values())
278     candidates = candidates.loc[candidates.index + max_step < len(daily)].copy()
279     return candidates
280
281
282 def run_backtest(
283     daily: pd.DataFrame,
284     paths: int,
285     seed: int,
286     max_origins: int | None = None,
287 ) -> tuple[pd.DataFrame, pd.DataFrame, list[str]]:
288     origins = backtest_origins(daily)
289     if max_origins is not None:
290         origins = origins.tail(max_origins)
291     origin_rows: list[dict[str, Any]] = []
292     warnings: list[str] = []
293     max_step = max(HORIZON_STEPS.values())
294     for origin_number, (idx, origin) in enumerate(origins.iterrows()):
295         train = daily.iloc[: idx + 1].copy()
296         fit = fit_gjr_garch(train["return_pct"])
297         fhs_paths = simulate_fhs(

```

```

298     float(origin["brent_usd_bbl"]), fit, max_step, paths, seed + origin_number * 101
299 )
300 gaussian_paths = simulate_gaussian(
301     float(origin["brent_usd_bbl"]), train["return_pct"], max_step, paths, seed + origin_number * 101 + 1
302 )
303 for model, simulated in [("FHS_GJR_GARCH", fhs_paths), ("Gaussian_random_walk", gaussian_paths)]:
304     for horizon, steps in HORIZON_STEPS.items():
305         target = daily.iloc[idx + steps]
306         actual = float(target["brent_usd_bbl"])
307         terminal = simulated[:, steps - 1]
308         q = np.quantile(terminal, FORECAST_QUANTILES)
309         losses = [pinball(actual, float(pred), tau) for pred, tau in zip(q, FORECAST_QUANTILES)]
310         origin_rows.append(
311             {
312                 "origin_date": origin["date"].strftime("%Y-%m-%d"),
313                 "target_date": target["date"].strftime("%Y-%m-%d"),
314                 "horizon_months": horizon,
315                 "model": model,
316                 "actual_price": actual,
317                 "p05_price": float(q[0]),
318                 "p10_price": float(q[1]),
319                 "median_price": float(q[2]),
320                 "p90_price": float(q[3]),
321                 "p95_price": float(q[4]),
322                 "mean_pinball_loss": float(np.mean(losses)),
323                 "median_absolute_error": float(abs(actual - q[2])),
324                 "covered_80": bool(q[1] <= actual <= q[3]),
325                 "covered_90": bool(q[0] <= actual <= q[4]),
326                 "interval_width_80": float(q[3] - q[1]),
327                 "interval_width_90": float(q[4] - q[0]),
328                 "interval_score_80": interval_score(actual, float(q[1]), float(q[3]), 0.20),
329                 "interval_score_90": interval_score(actual, float(q[0]), float(q[4]), 0.10),
330                 "quantile_score_approx_crps": float(2.0 * np.mean(losses)),
331                 "train_end": origin["date"].strftime("%Y-%m-%d"),
332                 "no_future_information": bool(target["date"] > origin["date"]),
333             }
334         )
335 detailed = pd.DataFrame(origin_rows)
336 if detailed.empty:
337     return detailed, pd.DataFrame(), warnings
338 summary = (
339     detailed.groupby(["model", "horizon_months"], as_index=False)
340     .agg(
341         origins=("origin_date", "count"),
342         mean_pinball_loss=("mean_pinball_loss", "mean"),
343         median_absolute_error=("median_absolute_error", "median"),
344         coverage_80=("covered_80", "mean"),
345         coverage_90=("covered_90", "mean"),
346         mean_width_80=("interval_width_80", "mean"),
347         mean_width_90=("interval_width_90", "mean"),
348         mean_interval_score_80=("interval_score_80", "mean"),
349         mean_interval_score_90=("interval_score_90", "mean"),
350         mean_quantile_score_approx_crps=("quantile_score_approx_crps", "mean"),
351         all_no_future_information=("no_future_information", "all"),
352     )
353     .sort_values(["horizon_months", "model"])
354     .reset_index(drop=True)
355 )
356 summary["model_role"] = np.where(summary["model"].eq("FHS_GJR_GARCH"), "main", "baseline")
357 summary["coverage_error_80"] = (summary["coverage_80"] - 0.80).abs()
358 summary["coverage_error_90"] = (summary["coverage_90"] - 0.90).abs()
359 return detailed, summary, warnings
360
361
362 def paired_backtest_summary(detailed: pd.DataFrame, seed: int, block_length: int = 6, reps: int = 2000) -> pd.DataFrame:
363     if detailed.empty:
364         return pd.DataFrame()
365     rng = np.random.default_rng(seed + 909)
366     rows: list[dict[str, Any]] = []
367     metrics = ["mean_pinball_loss", "interval_score_80", "interval_score_90", "quantile_score_approx_crps"]
368     for horizon, group in detailed.groupby("horizon_months"):
369         for metric in metrics:
370             pivot = group.pivot(index="origin_date", columns="model", values=metric).dropna()
371             if not {"FHS_GJR_GARCH", "Gaussian_random_walk"}.issubset(pivot.columns) or len(pivot) < 12:
372                 continue
373             diff = (pivot["FHS_GJR_GARCH"] - pivot["Gaussian_random_walk"]).to_numpy(dtype=float)

```

```

374     draws: list[float] = []
375     for _ in range(reps):
376         indices: list[int] = []
377         while len(indices) < len(diff):
378             start = int(rng.integers(0, len(diff)))
379             indices.extend((start + offset) % len(diff) for offset in range(block_length))
380         draws.append(float(np.mean(diff[np.asarray(indices[: len(diff)], dtype=int)])))
381     lower, upper = np.quantile(draws, [0.025, 0.975])
382     rows.append(
383         {
384             "horizon_months": int(horizon),
385             "metric": metric,
386             "origins": int(len(diff)),
387             "block_length": block_length,
388             "bootstrap_reps": reps,
389             "mean_difference_fhs_minus_gaussian": float(np.mean(diff)),
390             "lower_95": float(lower),
391             "upper_95": float(upper),
392             "evidence_status": "FHS_BETTER" if upper < 0 else ("GAUSSIAN_BETTER" if lower > 0 else "INCONCLUSIVE"),
393         }
394     )
395     return pd.DataFrame(rows)
396
397
398 def build_macro_stress() -> tuple[pd.DataFrame, dict[str, float]]:
399     shocks = pd.read_csv(RESULTS_DIR / "q1_structural_shocks.csv")
400     values = pd.to_numeric(shocks["oil_specific_risk_shock"], errors="coerce").dropna()
401     if len(values) < 120:
402         raise ValueError(f"Q4 needs at least 120 oil-specific risk shocks, got {len(values)}")
403     scenario_values = {
404         "moderate_q75": float(values.quantile(0.75)),
405         "severe_q90": float(values.quantile(0.90)),
406         "extreme_q95": float(values.quantile(0.95)),
407     }
408     irf = pd.read_csv(RESULTS_DIR / "q2_irf.csv")
409     irf = irf.loc[
410         irf["shock"].eq("oil_specific_risk_shock")
411         & irf["outcome"].isin(OUTCOME_LABELS)
412         & pd.to_numeric(irf["horizon"], errors="coerce").isin([0, 3, 6, 12])
413     ].copy()
414     required = {"response", "joint_lower_95", "joint_upper_95", "sample_start", "sample_end"}
415     missing = required.difference(irf.columns)
416     if missing or irf.empty:
417         raise ValueError(f"Q4 macro stress missing Q2 IRF fields: {sorted(missing)}")
418     rows: list[dict[str, Any]] = []
419     for scenario, shock_value in scenario_values.items():
420         for row in irf.itertuples(index=False):
421             response = float(row.response) * shock_value
422             lower = float(row.joint_lower_95) * shock_value
423             upper = float(row.joint_upper_95) * shock_value
424             rows.append(
425                 {
426                     "scenario": scenario,
427                     "shock_quantile_value": shock_value,
428                     "shock": "oil_specific_risk_shock",
429                     "outcome": row.outcome,
430                     "outcome_label": OUTCOME_LABELS[row.outcome],
431                     "horizon": int(row.horizon),
432                     "conditional_response_pctpt": response,
433                     "joint_lower_95": min(lower, upper),
434                     "joint_upper_95": max(lower, upper),
435                     "joint_ci95_contains_zero": bool(min(lower, upper) <= 0 <= max(lower, upper)),
436                     "row_evidence_status": (
437                         "INCONCLUSIVE" if min(lower, upper) <= 0 <= max(lower, upper) else "SUPPORTED_JOINT_95"
438                     ),
439                     "source_q2_evidence_status": "INCONCLUSIVE",
440                     "sample_start": row.sample_start,
441                     "sample_end": row.sample_end,
442                     "specification": "historical positive structural-shock quantile multiplied by Q2 LP response; conditional scenario,
443                     not deterministic loss",
444                 }
445             )
446     return pd.DataFrame(rows), scenario_values
447
448 def build_policy_stress() -> pd.DataFrame:

```

```

449 source = pd.read_csv(RESULTS_DIR / "q3_policy_macro_counterfactual.csv")
450 source = source.loc[source["outcome"].isin(OUTCOME_LABELS)].copy()
451 required = {"period", "outcome", "macro_counterfactual_gap_pctpt", "lower_95", "upper_95"}
452 missing = required.difference(source.columns)
453 if missing or source.empty:
454     raise ValueError(f"Q4 policy stress missing Q3 fields: {sorted(missing)}")
455 rows: list[dict[str, Any]] = []
456 for row in source.iteruples(index=False):
457     gap = float(row.macro_counterfactual_gap_pctpt)
458     lower = float(row.lower_95)
459     upper = float(row.upper_95)
460     if row.outcome == "china_iav_yoy_pct":
461         benefit = -gap
462         benefit_lower = -upper
463         benefit_upper = -lower
464         interpretation = "actual policy avoided an industrial-activity loss relative to no temporary control"
465     else:
466         benefit = gap
467         benefit_lower = lower
468         benefit_upper = upper
469         interpretation = "actual policy avoided a price-index increase relative to no temporary control"
470     rows.append(
471         {
472             "period": row.period,
473             "outcome": row.outcome,
474             "outcome_label": OUTCOME_LABELS[row.outcome],
475             "no_policy_minus_actual_gap_pctpt": gap,
476             "policy_buffer_benefit_pctpt": benefit,
477             "benefit_lower_95": min(benefit_lower, benefit_upper),
478             "benefit_upper_95": max(benefit_lower, benefit_upper),
479             "evidence_status": (
480                 "SUPPORTED_95"
481                 if min(benefit_lower, benefit_upper) > 0
482                 else "INCONCLUSIVE"
483             ),
484             "interpretation": interpretation,
485             "price_layer_status": row.price_layer_status,
486             "source_evidence_status": row.evidence_status,
487             "specification": "Q3 realized 2026 no-temporary-control counterfactual restated as policy buffer benefit; not
extrapolated to simulated Q4 price paths",
488         }
489     )
490 return pd.DataFrame(rows)
491
492
493 def plot_price_tail_risk(risk: pd.DataFrame) -> None:
494     apply_paper_style()
495     fig, ax = plt.subplots(figsize=(8.6, 5.0))
496     x = np.array([1, 3, 6], dtype=float)
497     for model, color, label in [
498         ("FHS_GJR_GARCH", PALETTE["blue"], "FHS - GJR-GARCH"),
499         ("Gaussian_random_walk", PALETTE["gold"], "高斯随机游走"),
500     ]:
501         subset = risk.loc[risk["model"].eq(model)].sort_values("horizon_months")
502         if subset.empty:
503             continue
504         xx = subset["horizon_months"].to_numpy(dtype=float)
505         median = subset["median_price"].to_numpy(dtype=float)
506         ax.plot(xx, median, marker="o", color=color, label=f"{label} 中位数")
507         if model == "FHS_GJR_GARCH":
508             ax.fill_between(
509                 xx,
510                 subset["p05_price"].to_numpy(dtype=float),
511                 subset["p95_price"].to_numpy(dtype=float),
512                 color=PALETTE["blue_light"],
513                 alpha=0.30,
514                 label="FHS 90%区间",
515             )
516         ax.fill_between(
517             xx,
518             subset["p10_price"].to_numpy(dtype=float),
519             subset["p90_price"].to_numpy(dtype=float),
520             color=PALETTE["blue_light"],
521             alpha=0.50,
522             label="FHS 80%区间",
523         )

```

```

524 threshold_90 = float(risk["historical_price_p90"].iloc[0])
525 threshold_95 = float(risk["historical_price_p95"].iloc[0])
526 ax.axhline(threshold_90, color=PALETTE["olive"], linestyle="--", linewidth=1.2, label="历史价格90%分位")
527 ax.axhline(threshold_95, color=PALETTE["rose"], linestyle=":", linewidth=1.4, label="历史价格95%分位")
528 ax.set_xticks(x, ["1个月", "3个月", "6个月"])
529 style_axis(ax, ylabel="Brent价格 (美元/桶)", xlabel="预测期限")
530 ax.legend(ncol=2, loc="upper left")
531 finish_figure(
532     fig,
533     title="国际油价上行尾部风险",
534     subtitle="阴影为FHS条件分布区间, 虚线为截止日前历史价格阈值。",
535     source="数据: EIA/FRED Brent; 计算: FHS - GJR-GARCH 与 高斯随机游走, 信息截止 2026-06-30。",
536 )
537 save_figure(fig, FIGURES_DIR / "q4_price_tail_risk")
538 plt.close(fig)
539
540
541 def plot_macro_policy_stress(macro: pd.DataFrame, policy: pd.DataFrame) -> None:
542     apply_paper_style()
543     fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.8), gridspec_kw={"width_ratios": [1.35, 1.0]})
544     left = macro.loc[macro["scenario"].eq("extreme_q95")].copy()
545     for outcome, color in [
546         ("china_ppi_yoy_pct", PALETTE["gold"]),
547         ("china_cpi_yoy_pct", PALETTE["blue"]),
548         ("china_iav_yoy_pct", PALETTE["rose"]),
549     ]:
550         subset = left.loc[left["outcome"].eq(outcome)].sort_values("horizon")
551         axes[0].plot(
552             subset["horizon"],
553             subset["conditional_response_pctpt"],
554             marker="o",
555             color=color,
556             label=OUTCOME_LABELS[outcome],
557         )
558         axes[0].axhline(0, color=PALETTE["muted"], linewidth=0.8)
559         axes[0].set_xticks([0, 3, 6, 12])
560         style_axis(axes[0], ylabel="同比增速条件响应 (百分点)", xlabel="冲击后月份")
561         axes[0].legend(loc="best")
562
563     june = policy.loc[policy["period"].eq("2026-06")].copy()
564     order = ["PPI", "CPI", "工业增加值"]
565     june["order"] = june["outcome_label"].map({label: idx for idx, label in enumerate(order)})
566     june = june.sort_values("order")
567     values = june["policy_buffer_benefit_pctpt"].to_numpy(dtype=float)
568     lower = june["benefit_lower_95"].to_numpy(dtype=float)
569     upper = june["benefit_upper_95"].to_numpy(dtype=float)
570     ypos = np.arange(len(june))
571     axes[1].barh(ypos, values, color=[PALETTE["gold"], PALETTE["blue"], PALETTE["rose"]])
572     axes[1].errorbar(values, ypos, xerr=np.vstack([values - lower, upper - values]), fmt="none", ecolor=PALETTE["ink"], capsize=3)
573     axes[1].set_yticks(ypos, june["outcome_label"])
574     axes[1].invert_yaxis()
575     axes[1].axvline(0, color=PALETTE["muted"], linewidth=0.8)
576     style_axis(axes[1], ylabel=None, xlabel="政策缓冲收益 (百分点)")
577     finish_figure(
578         fig,
579         title="极端结构冲击的宏观压力与已实现政策缓冲",
580         subtitle="左图为95%分位结构冲击条件响应; 右图为2026年6月无临时调控相对实际政策路径差额。",
581         source="计算: Q1结构冲击分位数、Q2 Local Projection 与 Q3 官方成品油价格层反事实: 区间口径见结果表。",
582         rect=(0.06, 0.12, 0.99, 0.98),
583     )
584     save_figure(fig, FIGURES_DIR / "q4_macro_policy_stress")
585     plt.close(fig)
586
587
588 def run_probe() -> dict[str, Any]:
589     ensure_dirs()
590     started = time.perf_counter()
591     daily = load_daily()
592     fit = fit_gjr_garch(daily["return_pct"])
593     thresholds = historical_thresholds(daily)
594     max_step = max(HORIZON_STEPS.values())
595     start_price = float(daily["brent_usd_bbl"].iloc[-1])
596     first = simulate_fhs(start_price, fit, max_step, 500, RANDOM_SEED)
597     replay = simulate_fhs(start_price, fit, max_step, 500, RANDOM_SEED)
598     alternate = simulate_fhs(start_price, fit, max_step, 500, RANDOM_SEED + 1)
599     baseline = simulate_gaussian(start_price, daily["return_pct"], max_step, 500, RANDOM_SEED)

```

```

600 first_summary = summarize_paths(first, "FHS_GJR_GARCH", CUTOFF, start_price, thresholds)
601 alternate_summary = summarize_paths(alternate, "FHS_GJR_GARCH", CUTOFF, start_price, thresholds)
602 baseline_summary = summarize_paths(baseline, "Gaussian_random_walk", CUTOFF, start_price, thresholds)
603 _, backtest, backtest_warnings = run_backtest(daily, paths=300, seed=RANDOM_SEED, max_origins=3)
604 macro, scenario_values = build_macro_stress()
605 policy = build_policy_stress()
606 prob_columns = ["terminal_prob_above_hist_p90", "terminal_prob_above_hist_p95"]
607 seed_sensitivity = float(
608     np.max(
609         np.abs(
610             first_summary[prob_columns].to_numpy(dtype=float)
611             - alternate_summary[prob_columns].to_numpy(dtype=float)
612         )
613     )
614 )
615 checks = {
616     "daily_rows_at_least_750": len(daily) >= 750,
617     "cutoff_exact": daily["date"].max() == CUTOFF,
618     "gjr_persistence_below_one": 0 <= fit["persistence"] < 0.999,
619     "fhs_distribution_non_degenerate": bool((first_summary["p95_price"] > first_summary["p05_price"]).all()),
620     "baseline_distribution_non_degenerate": bool((baseline_summary["p95_price"] > baseline_summary["p05_price"]).all()),
621     "fixed_seed_exact_replay": bool(np.array_equal(first, replay)),
622     "seed_sensitivity_below_0_15": seed_sensitivity <= 0.15,
623     "representative_backtest_complete": len(backtest) == 6 and bool(backtest["all_no_future_information"].all()),
624     "macro_three_outcomes_present": set(macro["outcome"]) == set(OUTCOME_LABELS),
625     "macro_three_scenarios_present": len(scenario_values) == 3,
626     "policy_three_outcomes_present": set(policy["outcome"]) == set(OUTCOME_LABELS),
627 }
628 status = "PASS" if all(checks.values()) else "FAIL"
629 payload = {
630     "status": status,
631     "method": "FHS_GJR_GARCH",
632     "role": "main",
633     "baseline": "Gaussian_random_walk",
634     "data_range": [daily["date"].min().strftime("%Y-%m-%d"), daily["date"].max().strftime("%Y-%m-%d")],
635     "random_seed": RANDOM_SEED,
636     "probe_paths": 500,
637     "representative_backtest_origins": 3,
638     "checks": checks,
639     "metrics": {
640         "daily_rows": int(len(daily)),
641         "gjr_persistence": float(fit["persistence"]),
642         "seed_probability_max_abs_difference": seed_sensitivity,
643         "runtime_seconds": float(time.perf_counter() - started),
644     },
645     "warnings": backtest_warnings,
646     "pass_rule": "all input, stationarity, non-degeneracy, replay, seed-perturbation, backtest and cross-module field checks pass",
647     "failure_action": "stop the Q4 gate and fix the input/model contract; do not substitute fallback estimates",
648 }
649 write_json(payload, "q4_risk_probe.json")
650 print(json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False))
651 return payload
652
653
654 def run_full(paths: int, backtest_paths: int) -> dict[str, Any]:
655     ensure_dirs()
656     probe_path = RESULTS_DIR / "q4_risk_probe.json"
657     if not probe_path.exists():
658         raise FileNotFoundError("Run `python code/problem4/run_q4.py --probe` before the full Q4 implementation.")
659     probe = json.loads(probe_path.read_text(encoding="utf-8"))
660     if probe.get("status") != "PASS":
661         raise RuntimeError("Q4 risk probe did not pass; full implementation is blocked.")
662
663     daily = load_daily()
664     thresholds = historical_thresholds(daily)
665     start_price = float(daily["brent_usd_bbl"].iloc[-1])
666     fit = fit_gjr_garch(daily["return_pct"])
667     max_step = max(HORIZON_STEPS.values())
668     fhs_paths = simulate_fhs(start_price, fit, max_step, paths, RANDOM_SEED)
669     gaussian_paths = simulate_gaussian(start_price, daily["return_pct"], max_step, paths, RANDOM_SEED + 1)
670     risk = pd.concat(
671         [
672             summarize_paths(fhs_paths, "FHS_GJR_GARCH", CUTOFF, start_price, thresholds),
673             summarize_paths(gaussian_paths, "Gaussian_random_walk", CUTOFF, start_price, thresholds),
674         ],
675         ignore_index=True,

```

```

676 )
677 detailed_backtest, backtest, backtest_warnings = run_backtest(
678     daily, paths=backtest_paths, seed=RANDOM_SEED
679 )
680 paired_backtest = paired_backtest_summary(detailed_backtest, RANDOM_SEED, block_length=6)
681 macro, scenario_values = build_macro_stress()
682 policy = build_policy_stress()
683
684 save_csv(risk, "q4_price_tail_risk.csv")
685 save_csv(detailed_backtest, "q4_risk_backtest_origins.csv")
686 save_csv(backtest, "q4_risk_backtest.csv")
687 save_csv(paired_backtest, "q4_risk_backtest_paired.csv")
688 save_csv(macro, "q4_macro_stress.csv")
689 save_csv(policy, "q4_policy_stress.csv")
690 plot_price_tail_risk(risk)
691 plot_macro_policy_stress(macro, policy)
692
693 overall_pinball = (
694     backtest.groupby("model", as_index=False)["mean_pinball_loss"].mean().sort_values("mean_pinball_loss")
695     if not backtest.empty
696     else pd.DataFrame()
697 )
698 preferred = str(overall_pinball.iloc[0]["model"]) if not overall_pinball.empty else None
699 fhs_key = risk.loc[risk["model"].eq("FHS_GJR_GARCH")].copy()
700 june_policy = policy.loc[policy["period"].eq("2026-06")].copy()
701 extreme = macro.loc[macro["scenario"].eq("extreme_q95")].copy()
702 summary = {
703     "status": "PASS",
704     "execution_status": "PASS",
705     "evidence_status": "CONDITIONAL_STRESS_TEST",
706     "problem_status": "SELF_DEFINED_PROBLEM_4_RISK_LAYER",
707     "random_seed": RANDOM_SEED,
708     "cutoff": CUTOFF.strftime("%Y-%m-%d"),
709     "main_method": "FHS_GJR_GARCH",
710     "baseline": "Gaussian_random_walk",
711     "backtest_preferred_model_by_mean_pinball": preferred,
712     "allowed_claims": [
713         "future Brent upper-tail probabilities are conditional stress forecasts under the fixed cutoff",
714         "Q2 structural-shock quantiles map to conditional PPI/CPI/IAV response ranges",
715         "Q3 dynamic proxy scenario conditionally propagates the separately audited 2026 temporary-control gaps",
716     ],
717     "forbidden_claims": [
718         "tail probabilities are certain price outcomes",
719         "Q4 macro stress values are deterministic GDP losses",
720         "Q2 structural responses and Q3 policy counterfactual gaps can be added as one causal estimate",
721         "the stress test proves China is generally optimal or costless",
722     ],
723     "guardrail": "Price risk, structural-shock macro scenarios and realized policy counterfactuals are reported as three linked but
non-additive evidence layers.",
724     "gjr_parameters": {
725         key: float(fit[key]) for key in ["mu", "omega", "alpha", "gamma", "beta", "persistence"]
726     },
727     "structural_scenarios": scenario_values,
728     "rows": {
729         "q4_price_tail_risk.csv": int(len(risk)),
730         "q4_risk_backtest_origins.csv": int(len(detailed_backtest)),
731         "q4_risk_backtest.csv": int(len(backtest)),
732         "q4_risk_backtest_paired.csv": int(len(paired_backtest)),
733         "q4_macro_stress.csv": int(len(macro)),
734         "q4_policy_stress.csv": int(len(policy)),
735     },
736     "key_price_risk": json_records(fhs_key),
737     "key_extreme_macro_rows": json_records(
738         extreme.loc[extreme["horizon"].isin([6, 12])].sort_values(["outcome", "horizon"])
739     ),
740     "key_2026_06_policy_buffer": json_records(june_policy.sort_values("outcome")),
741     "backtest_summary": json_records(backtest),
742     "paired_backtest_summary": json_records(paired_backtest),
743     "warnings": backtest_warnings,
744 }
745 write_json(summary, "q4_summary.json")
746 print(json.dumps(summary, ensure_ascii=False, indent=2, allow_nan=False))
747 return summary
748
749
750 def parse_args() -> argparse.Namespace:

```



```

751 parser = argparse.ArgumentParser(description=__doc__)
752 parser.add_argument("--probe", action="store_true", help="run only the pre-registered low-cost risk probe")
753 parser.add_argument("--paths", type=int, default=20000, help="Monte Carlo paths for the cutoff forecast")
754 parser.add_argument("--backtest-paths", type=int, default=1500, help="paths per rolling backtest origin")
755 return parser.parse_args()
756
757
758 def main() -> int:
759     args = parse_args()
760     if args.probe:
761         payload = run_probe()
762         return 0 if payload["status"] == "PASS" else 1
763     run_full(paths=args.paths, backtest_paths=args.backtest_paths)
764     return 0
765
766
767 if __name__ == "__main__":
768     raise SystemExit(main())

```

Listing 9: run_q4.py

```

1 """Question 4: SAPR-CVaR adaptive fuel-price smoothing rule.
2
3 The model treats the official mechanism-implied adjustment as the primary
4 pricing signal and optimizes only a transparent temporary smoothing layer.
5 """
6
7 from __future__ import annotations
8
9 import argparse
10 import json
11 import sys
12 from dataclasses import dataclass
13 from pathlib import Path
14 from typing import Any
15
16 import matplotlib
17
18 matplotlib.use("Agg")
19
20 import matplotlib.pyplot as plt
21 import numpy as np
22 import pandas as pd
23
24
25 REPO_ROOT = Path(__file__).resolve().parents[2]
26 CODE_DIR = REPO_ROOT / "code"
27 if str(CODE_DIR) not in sys.path:
28     sys.path.insert(0, str(CODE_DIR))
29
30 from utils.plot_style import PALETTE, apply_paper_style, finish_figure, save_figure, style_axis
31
32 if hasattr(sys.stdout, "reconfigure"):
33     sys.stdout.reconfigure(encoding="utf-8")
34
35 PROCESSED_DIR = REPO_ROOT / "data" / "processed"
36 RESULTS_DIR = REPO_ROOT / "results"
37 FIGURES_DIR = REPO_ROOT / "figures"
38
39 RANDOM_SEED = 20260730
40 CUTOFF = "2026-06-30"
41 SCENARIO_LENGTH = 6
42 BOOTSTRAP_DRAWS = 2000
43 DEVELOPMENT_END = "2021-12"
44 HOLDOUT_START = "2022-01"
45 TEMPORARY_2026_RHO = (1160.0 + 420.0) / (2205.0 + 800.0)
46 RECOVERY_MONTHS = 6
47 MAX_GAP_RATIO = 0.35
48 MAX_TERMINAL_GAP_RATIO = 0.20
49 MAX_RECOVERY_TERMINAL_GAP_RATIO = 0.05
50 MAX_MONTHLY_ADJUSTMENT_RATIO = 0.25
51
52 OUTCOMES = ["china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"]
53 OUTCOME_LABELS = {
54     "china_ppi_yoy_pct": "PPI",

```

```

55     "china_cpi_yoy_pct": "CPI",
56     "china_iav_yoy_pct": "IAV",
57 }
58 REGIME_ORDER = ["normal", "stress", "extreme"]
59
60
61 @dataclass(frozen=True)
62 class Scenario:
63     scenario_id: str
64     sample_split: str
65     source_start: str
66     periods: tuple[str, ...]
67     mechanism_adjustment: np.ndarray
68     actual_adjustment: np.ndarray
69     cumulative_policy_gap: np.ndarray
70     regimes: tuple[str, ...]
71     is_2026_anchor: bool
72
73
74 @dataclass(frozen=True)
75 class MacroKernel:
76     means: dict[str, np.ndarray]
77     covariances: dict[str, np.ndarray]
78     labels: dict[str, str]
79     valid_outcomes: tuple[str, ...]
80
81
82 def ensure_dirs() -> None:
83     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
84     FIGURES_DIR.mkdir(parents=True, exist_ok=True)
85
86
87 def save_csv(frame: pd.DataFrame, filename: str) -> Path:
88     path = RESULTS_DIR / filename
89     frame.to_csv(path, index=False, encoding="utf-8")
90     return path
91
92
93 def read_result(filename: str) -> pd.DataFrame:
94     path = RESULTS_DIR / filename
95     if not path.exists():
96         raise FileNotFoundError(f"Required result file is missing: {path}")
97     return pd.read_csv(path)
98
99
100 def read_processed(filename: str) -> pd.DataFrame:
101     path = PROCESSED_DIR / filename
102     if not path.exists():
103         raise FileNotFoundError(f"Required processed file is missing: {path}")
104     return pd.read_csv(path)
105
106
107 def clean_value(value: Any) -> Any:
108     if isinstance(value, (np.integer,)):
109         return int(value)
110     if isinstance(value, (np.floating, float)):
111         return None if not np.isfinite(value) else round(float(value), 8)
112     if pd.isna(value):
113         return None
114     return value
115
116
117 def stress_regime(value: float, q75: float, q95: float) -> str:
118     if not np.isfinite(value) or value <= q75:
119         return "normal"
120     if value <= q95:
121         return "stress"
122     return "extreme"
123
124
125 def build_policy_months(threshold_quantiles: tuple[float, float] = (0.75, 0.95)) -> tuple[pd.DataFrame, dict[str, float]]:
126     official = read_processed("china_regulated_gasoline_monthly.csv")
127     events = read_processed("cn_fuel_adjustment_events.csv")
128     macro = read_processed("model_monthly_cn.csv")
129     shocks = read_result("q1_monthly_shocks.csv")
130

```

```

131 official = official[
132     [
133         "period",
134         "china_regulated_gasoline_cny_per_ton",
135         "source_url",
136         "coverage_status",
137         "measure_type",
138     ]
139 ].copy()
140 events = events.copy()
141 events["period"] = pd.to_datetime(events["effective_date"]).dt.to_period("M").astype(str)
142 event_monthly = (
143     events.groupby("period", as_index=False)
144     .agg(
145         gasoline_actual_adjustment_cny_t=("actual_adjustment_cny_per_ton", "sum"),
146         gasoline_rule_adjustment_cny_t=("rule_implied_adjustment_cny_per_ton", "sum"),
147         event_source_count=("source_url", "count"),
148     )
149     .sort_values("period")
150 )
151 event_monthly["gasoline_policy_gap_cny_t"] = (
152     event_monthly["gasoline_rule_adjustment_cny_t"] - event_monthly["gasoline_actual_adjustment_cny_t"]
153 )
154 event_monthly["cum_gasoline_policy_gap_cny_t"] = event_monthly["gasoline_policy_gap_cny_t"].cumsum()
155
156 frame = (
157     official.merge(event_monthly, on="period", how="left")
158     .merge(
159         macro[["period", "brent_usd_bbl", "brent_usd_bbl_log_return", "GPR", "GPR_z"]],
160         on="period",
161         how="left",
162     )
163     .merge(
164         shocks[
165             [
166                 "period",
167                 "supply_shock",
168                 "aggregate_demand_shock",
169                 "oil_specific_risk_shock",
170                 "reduced_form_shock",
171             ]
172         ],
173         on="period",
174         how="left",
175     )
176     .sort_values("period")
177 )
178 frame = frame.loc[frame["period"].between("2013-03", "2026-06")].copy()
179 frame = frame.dropna(subset=["china_regulated_gasoline_cny_per_ton"]).copy()
180 if len(frame) < 48:
181     raise ValueError("Q4 requires at least 48 months of the reconstructed regulated-gasoline adjustment proxy.")
182 for column in [
183     "gasoline_actual_adjustment_cny_t",
184     "gasoline_rule_adjustment_cny_t",
185     "gasoline_policy_gap_cny_t",
186     "cum_gasoline_policy_gap_cny_t",
187     "event_source_count",
188 ]:
189     frame[column] = frame[column].fillna(0.0)
190
191 frame["previous_official_price_cny_t"] = frame["china_regulated_gasoline_cny_per_ton"].shift(1)
192 fallback_previous = frame["china_regulated_gasoline_cny_per_ton"] - frame["gasoline_actual_adjustment_cny_t"].fillna(0.0)
193 frame["previous_official_price_cny_t"] = frame["previous_official_price_cny_t"].fillna(fallback_previous)
194 bad_previous = frame["previous_official_price_cny_t"].le(0) | frame["previous_official_price_cny_t"].isna()
195 if bool(bad_previous.any()):
196     raise ValueError("Q4 cannot compute mechanism adjustment rates because a previous official fuel price is missing or nonpositive.")
197
198 base_price = float(frame.loc[frame["period"].eq("2026-06"), "china_regulated_gasoline_cny_per_ton"].iloc[0])
199 frame["mechanism_adjustment_cny_t_original"] = frame["gasoline_rule_adjustment_cny_t"].fillna(0.0)
200 frame["actual_adjustment_cny_t_original"] = frame["gasoline_actual_adjustment_cny_t"].fillna(0.0)
201 frame["mechanism_adjustment_rate"] = frame["mechanism_adjustment_cny_t_original"] / frame["previous_official_price_cny_t"]
202 frame["actual_adjustment_rate"] = frame["actual_adjustment_cny_t_original"] / frame["previous_official_price_cny_t"]
203 frame["mechanism_adjustment_cny_t"] = frame["mechanism_adjustment_rate"] * base_price
204 frame["actual_adjustment_cny_t"] = frame["actual_adjustment_rate"] * base_price
205 frame["cumulative_policy_gap_cny_t_original"] = frame["cum_gasoline_policy_gap_cny_t"].fillna(0.0)

```

```

206 frame["source_vintage"] = "downloaded_2026-07-31_ex_post"
207
208 development = frame.loc[frame["period"].le(DEVELOPMENT_END)].copy()
209 positive = pd.to_numeric(development["mechanism_adjustment_cny_t"], errors="coerce")
210 positive = positive.loc[positive.gt(0)].dropna()
211 if len(positive) < 12:
212     raise ValueError("Q4 cannot define stress thresholds because development positive adjustments are too sparse.")
213 q75 = float(positive.quantile(threshold_quantiles[0]))
214 q95 = float(positive.quantile(threshold_quantiles[1]))
215 frame["stress_regime"] = frame["mechanism_adjustment_cny_t"].map(lambda value: stress_regime(float(value), q75, q95))
216
217 meta = {
218     "base_price_2026_06_cny_t": base_price,
219     "stress_threshold_75_cny_t": q75,
220     "stress_threshold_95_cny_t": q95,
221     "threshold_quantile_low": threshold_quantiles[0],
222     "threshold_quantile_high": threshold_quantiles[1],
223 }
224 return frame, meta
225
226
227 def build_scenarios(threshold_quantiles: tuple[float, float] = (0.75, 0.95)) -> tuple[pd.DataFrame, list[Scenario], dict[str, float]]:
228     frame, meta = build_policy_months(threshold_quantiles)
229     scenario_rows: list[dict[str, Any]] = []
230     scenarios: list[Scenario] = []
231     for start in range(0, len(frame) - SCENARIO_LENGTH + 1):
232         window = frame.iloc[start : start + SCENARIO_LENGTH].copy()
233         source_start = str(window["period"].iloc[0])
234         source_end = str(window["period"].iloc[-1])
235         if source_end <= DEVELOPMENT_END:
236             split = "development"
237         elif source_start >= HOLDOUT_START and source_end <= "2026-06":
238             split = "holdout"
239         else:
240             continue
241         scenario_id = f"{split.upper()}_{source_start}"
242         is_2026_anchor = source_start == "2026-01" and source_end == "2026-06"
243         scenario = Scenario(
244             scenario_id=scenario_id,
245             sample_split=split,
246             source_start=source_start,
247             periods=tuple(window["period"].astype(str).tolist()),
248             mechanism_adjustment=window["mechanism_adjustment_cny_t"].astype(float).to_numpy(),
249             actual_adjustment=window["actual_adjustment_cny_t"].astype(float).to_numpy(),
250             cumulative_policy_gap=window["cumulative_policy_gap_cny_t_original"].astype(float).to_numpy(),
251             regimes=tuple(window["stress_regime"].astype(str).tolist()),
252             is_2026_anchor=is_2026_anchor,
253         )
254         scenarios.append(scenario)
255     for month_index, (_, row) in enumerate(window.iterrows()):
256         scenario_rows.append(
257             {
258                 "scenario_id": scenario_id,
259                 "sample_split": split,
260                 "month_index": month_index,
261                 "period": row["period"],
262                 "source_start": source_start,
263                 "source_end": source_end,
264                 "brent_return": row["brent_usd_bbl_log_return"],
265                 "brent_level": row["brent_usd_bbl"],
266                 "mechanism_adjustment_rate": row["mechanism_adjustment_rate"],
267                 "mechanism_adjustment_cny_t": row["mechanism_adjustment_cny_t"],
268                 "mechanism_adjustment_cny_t_original": row["mechanism_adjustment_cny_t_original"],
269                 "actual_adjustment_cny_t_original": row["actual_adjustment_cny_t_original"],
270                 "cumulative_policy_gap_cny_t_original": row["cumulative_policy_gap_cny_t_original"],
271                 "stress_regime": row["stress_regime"],
272                 "is_2026_anchor": is_2026_anchor,
273                 "GPR": row["GPR"],
274                 "GPR_z": row["GPR_z"],
275                 "supply_shock": row["supply_shock"],
276                 "aggregate_demand_shock": row["aggregate_demand_shock"],
277                 "oil_specific_risk_shock": row["oil_specific_risk_shock"],
278                 "reduced_form_shock": row["reduced_form_shock"],
279                 "source_vintage": row["source_vintage"],
280             }
281         )

```

```

282 scenario_table = pd.DataFrame(scenario_rows)
283 return scenario_table, scenarios, meta
284
285
286 def validated_covariance(matrix: np.ndarray, outcome: str) -> np.ndarray:
287     if not np.isfinite(matrix).all():
288         raise ValueError(f"Q3 covariance for {outcome} contains non-finite values.")
289     if not np.allclose(matrix, matrix.T, atol=1e-10, rtol=1e-10):
290         raise ValueError(f"Q3 covariance for {outcome} is not symmetric.")
291     eigvals = np.linalg.eigvalsh(matrix)
292     if float(eigvals.min()) < -1e-10:
293         raise ValueError(f"Q3 covariance for {outcome} is not positive semidefinite.")
294     return (matrix + matrix.T) / 2.0
295
296
297 def load_macro_kernel() -> MacroKernel:
298     kernel = read_result("q3_policy_macro_kernel.csv")
299     covariance = read_result("q3_policy_macro_covariance.csv")
300     required_kernel = {"outcome", "outcome_label", "n", "phi_outcome_lag1"} | {f"beta_fuel_lag{lag}" for lag in range(7)}
301     required_cov = {"outcome", "row_term", "column_term", "covariance"}
302     if not required_kernel.issubset(kernel.columns) or not required_cov.issubset(covariance.columns):
303         raise ValueError("Q3 macro kernel or covariance file lacks required columns for Q4.")
304     means: dict[str, np.ndarray] = {}
305     covariances: dict[str, np.ndarray] = {}
306     labels: dict[str, str] = {}
307     for outcome in OUTCOMES:
308         rows = kernel.loc[kernel["outcome"].eq(outcome)]
309         if len(rows) != 1:
310             raise ValueError(f"Q4 requires exactly one macro-kernel row for {outcome}.")
311         row = rows.iloc[0]
312         terms = [f"fuel_log_return_lag{lag}" for lag in range(7)] + [f"{outcome}_lag1"]
313         means[outcome] = np.array([float(row[f"beta_fuel_lag{lag}"]) for lag in range(7)] + [float(row["phi_outcome_lag1"])])
314         if not np.isfinite(means[outcome]).all():
315             raise ValueError(f"Q3 macro kernel for {outcome} contains non-finite coefficients.")
316         labels[outcome] = str(row["outcome_label"])
317         cov_block = covariance.loc[covariance["outcome"].eq(outcome)].copy()
318         required_pairs = {(row_term, col_term) for row_term in terms for col_term in terms}
319         observed_pairs = set(zip(cov_block["row_term"].astype(str), cov_block["column_term"].astype(str)))
320         missing_pairs = required_pairs - observed_pairs
321         if missing_pairs:
322             raise ValueError(f"Q3 covariance for {outcome} is missing {len(missing_pairs)} required term pairs.")
323         matrix = np.zeros((len(terms), len(terms)), dtype=float)
324         for i, row_term in enumerate(terms):
325             for j, col_term in enumerate(terms):
326                 hit = cov_block.loc[cov_block["row_term"].eq(row_term) & cov_block["column_term"].eq(col_term), "covariance"]
327                 if len(hit) != 1:
328                     raise ValueError(f"Q3 covariance for {outcome} has duplicate or missing value for {row_term}, {col_term}.")
329                 matrix[i, j] = float(hit.iloc[0])
330         covariances[outcome] = validated_covariance(matrix, outcome)
331     valid = tuple(outcome for outcome in OUTCOMES if outcome in means and outcome in covariances)
332     if set(valid) != set(OUTCOMES):
333         raise ValueError("Q4 requires PPI, CPI and IAV macro kernels from Q3.")
334     return MacroKernel(means=means, covariances=covariances, labels=labels, valid_outcomes=valid)
335
336
337 def draw_parameter_sets(
338     kernel: MacroKernel,
339     rng: np.random.Generator,
340     target_draws: int = BOOTSTRAP_DRAWS,
341     minimum_valid_rate: float = 0.95,
342 ) -> tuple[list[dict[str, np.ndarray]], float, int]:
343     joint_path = RESULTS_DIR / "q3_policy_macro_bootstrap_draws.csv"
344     if joint_path.exists():
345         joint = pd.read_csv(joint_path)
346         required = {"draw", "outcome", "phi_outcome_lag1"} | {f"beta_fuel_lag{lag}" for lag in range(7)}
347         if not required.issubset(joint.columns):
348             raise ValueError("Q3 joint macro bootstrap file lacks required Q4 kernel columns.")
349         available_draws = sorted(pd.to_numeric(joint["draw"], errors="coerce").dropna().astype(int).unique())
350         if len(available_draws) < target_draws:
351             raise ValueError(f"Q4 requires {target_draws} joint Q3 draws but found {len(available_draws)}.")
352         draws: list[dict[str, np.ndarray]] = []
353         for draw_id in available_draws[:target_draws]:
354             block = joint.loc[pd.to_numeric(joint["draw"], errors="coerce").eq(draw_id)]
355             draw: dict[str, np.ndarray] = {}
356             for outcome in kernel.valid_outcomes:
357                 row = block.loc[block["outcome"].eq(outcome)]

```

```

358         if len(row) != 1:
359             raise ValueError(f"Q3 joint draw {draw_id} lacks exactly one row for {outcome}.")
360         values = np.array(
361             [float(row.iloc[0][f"beta_fuel_lag(lag)"]) for lag in range(7)]
362             + [float(row.iloc[0][f"phi_outcome_lag1"])]),
363             dtype=float,
364         )
365         if not np.isfinite(values).all() or abs(float(values[-1])) >= 1.0:
366             raise ValueError(f"Q3 joint draw {draw_id} is invalid for {outcome}.")
367         draw[outcome] = values
368         draws.append(draw)
369     return draws, 1.0, len(draws)
370 draws: list[dict[str, np.ndarray]] = []
371 max_attempts = int(np.floor(target_draws / minimum_valid_rate))
372 attempts_used = 0
373 while len(draws) < target_draws and attempts_used < max_attempts:
374     attempts_used += 1
375     draw: dict[str, np.ndarray] = {}
376     valid = True
377     for outcome in kernel.valid_outcomes:
378         mean = kernel.means[outcome]
379         covariance = kernel.covariances[outcome]
380         phi = float(mean[-1])
381         if abs(phi) >= 1.0:
382             raise ValueError(f"Q4 macro kernel for {outcome} has an unstable point-estimate lag coefficient.")
383         transformed_mean = mean.copy()
384         transformed_mean[-1] = np.arctanh(phi)
385         jacobian = np.eye(len(mean), dtype=float)
386         jacobian[-1, -1] = 1.0 / (1.0 - phi**2)
387         transformed_covariance = validated_covariance(jacobian @ covariance @ jacobian.T, f"{outcome}_stationary_phi")
388         transformed_sample = rng.multivariate_normal(transformed_mean, transformed_covariance, method="svd")
389         sample = transformed_sample.copy()
390         sample[-1] = np.tanh(transformed_sample[-1])
391         if not np.isfinite(sample).all() or abs(float(sample[-1])) >= 1.0:
392             valid = False
393             break
394         draw[outcome] = sample
395     if valid:
396         draws.append(draw)
397     valid_rate = len(draws) / float(attempts_used) if attempts_used else 0.0
398     return draws, valid_rate, attempts_used
399
400
401 def point_coefficients(kernel: MacroKernel) -> dict[str, np.ndarray]:
402     return {outcome: values.copy() for outcome, values in kernel.means.items()}
403
404
405 def historical_macro_stds(weights: dict[str, float]) -> dict[str, float]:
406     macro = read_processed("model_monthly_cn.csv")
407     frame = macro.loc[macro["period"].between("2013-03", DEVELOPMENT_END)].copy()
408     stds: dict[str, float] = {}
409     for outcome in OUTCOMES:
410         std = float(pd.to_numeric(frame[outcome], errors="coerce").std(skipna=True))
411         stds[outcome] = std if np.isfinite(std) and std > 1e-9 else 1.0
412     return stds
413
414
415 def enumerate_rules() -> list[dict[str, float]]:
416     values = [round(i * 0.05, 2) for i in range(21)]
417     rules: list[dict[str, float]] = []
418     for rho_normal in values:
419         for rho_stress in values:
420             for rho_extreme in values:
421                 if rho_normal >= rho_stress >= rho_extreme:
422                     rules.append(
423                         {
424                             "rho_normal": rho_normal,
425                             "rho_stress": rho_stress,
426                             "rho_extreme": rho_extreme,
427                         }
428                     )
429     return rules
430
431
432 def rho_for_regime(rule: dict[str, float], regime: str) -> float:
433     if regime == "extreme":

```

```

434     return float(rule["rho_extreme"])
435 if regime == "stress":
436     return float(rule["rho_stress"])
437 return float(rule["rho_normal"])
438
439
440 def simulate_rule_path(scenario: Scenario, rule: dict[str, float], base_price: float) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.
    ndarray]:
441     price = np.empty(SCENARIO_LENGTH, dtype=float)
442     actual_adjustment = np.empty(SCENARIO_LENGTH, dtype=float)
443     deferred_gap = np.empty(SCENARIO_LENGTH, dtype=float)
444     fuel_return = np.empty(SCENARIO_LENGTH, dtype=float)
445     previous_price = base_price
446     gap = 0.0
447     for t in range(SCENARIO_LENGTH):
448         desired = float(scenario.mechanism_adjustment[t]) + gap
449         if desired > 0:
450             adjustment = rho_for_regime(rule, scenario.regimes[t]) * desired
451             gap = desired - adjustment
452             gap_cap = MAX_TERMINAL_GAP_RATIO * base_price
453             if gap > gap_cap:
454                 catch_up = gap - gap_cap
455                 adjustment += catch_up
456                 gap = gap_cap
457             else:
458                 adjustment = desired
459                 gap = 0.0
460             current_price = max(previous_price + adjustment, 1.0)
461             actual_adjustment[t] = adjustment
462             deferred_gap[t] = gap
463             price[t] = current_price
464             fuel_return[t] = np.log(current_price / previous_price)
465             previous_price = current_price
466     return price, actual_adjustment, deferred_gap, fuel_return
467
468
469 def simulate_actual_2026_path(scenario: Scenario, base_price: float) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
470     price = np.empty(SCENARIO_LENGTH, dtype=float)
471     actual_adjustment = np.empty(SCENARIO_LENGTH, dtype=float)
472     deferred_gap = np.empty(SCENARIO_LENGTH, dtype=float)
473     fuel_return = np.empty(SCENARIO_LENGTH, dtype=float)
474     previous_price = base_price
475     for t in range(SCENARIO_LENGTH):
476         adjustment = float(scenario.actual_adjustment[t])
477         current_price = max(previous_price + adjustment, 1.0)
478         actual_adjustment[t] = adjustment
479         deferred_gap[t] = float(scenario.cumulative_policy_gap[t])
480         price[t] = current_price
481         fuel_return[t] = np.log(current_price / previous_price)
482         previous_price = current_price
483     return price, actual_adjustment, deferred_gap, fuel_return
484
485
486 def recovery_terminal_gap(terminal_gap: float, rule: dict[str, float]) -> float:
487     gap = max(float(terminal_gap), 0.0)
488     rho = float(rule["rho_normal"])
489     for _ in range(RECOVERY_MONTHS):
490         gap = (1.0 - rho) * gap
491     return gap
492
493
494 def macro_path_from_returns(fuel_return: np.ndarray, coefficients: dict[str, np.ndarray]) -> dict[str, np.ndarray]:
495     paths = {outcome: np.zeros(SCENARIO_LENGTH, dtype=float) for outcome in OUTCOMES}
496     lag_buffer = [0.0] * 7
497     previous_y = {outcome: 0.0 for outcome in OUTCOMES}
498     for t in range(SCENARIO_LENGTH):
499         lag_buffer = [float(fuel_return[t])] + lag_buffer[:6]
500         for outcome in OUTCOMES:
501             params = coefficients[outcome]
502             y = float(np.dot(params[:7], np.array(lag_buffer)) + params[7] * previous_y[outcome])
503             paths[outcome][t] = y
504             previous_y[outcome] = y
505     return paths
506
507
508 def objective_from_scenario_outputs(

```

```

509     scenario_losses: list[float],
510     gap_burdens: list[float],
511     fuel_returns: list[float],
512 ) -> dict[str, float]:
513     losses = np.asarray(scenario_losses, dtype=float)
514     gaps = np.asarray(gap_burdens, dtype=float)
515     returns = np.asarray(fuel_returns, dtype=float)
516     if len(losses) == 0:
517         raise ValueError("Q4 objective cannot be computed from an empty scenario set.")
518     if not np.isfinite(losses).all() or not np.isfinite(gaps).all() or not np.isfinite(returns).all():
519         raise ValueError("Q4 objective inputs contain non-finite values.")
520     tail_count = max(1, int(np.ceil(0.05 * len(losses))))
521     tail = np.sort(losses)[-tail_count:]
522     return {
523         "J1_macro_loss": float(np.mean(losses)),
524         "J2_cvar95_macro_loss": float(np.mean(tail)),
525         "J3_avg_gap_month_burden": float(np.mean(gaps)),
526         "J4_adjustment_volatility": float(np.std(returns, ddof=0)) if len(returns) else 0.0,
527     }
528
529
530 def evaluate_strategy(
531     scenarios: list[Scenario],
532     coefficients: dict[str, np.ndarray],
533     rule: dict[str, float] | None,
534     strategy_name: str,
535     split: str,
536     base_price: float,
537     stds: dict[str, float],
538     weights: dict[str, float],
539     scenario_indices: np.ndarray | None = None,
540 ) -> dict[str, float]:
541     candidates = [scenario for scenario in scenarios if scenario.sample_split == split]
542     if split == "war_2026":
543         candidates = [scenario for scenario in scenarios if scenario.is_2026_anchor]
544     if not candidates:
545         raise ValueError(f"Q4 strategy evaluation has no candidate scenarios for split={split}.")
546     if scenario_indices is not None and len(candidates):
547         candidates = [candidates[int(index) % len(candidates)] for index in scenario_indices]
548     losses: list[float] = []
549     gap_burdens: list[float] = []
550     returns_all: list[float] = []
551     max_gap_ratios: list[float] = []
552     terminal_gap_ratios: list[float] = []
553     recovery_gap_ratios: list[float] = []
554     max_adjustment_ratios: list[float] = []
555     for scenario in candidates:
556         if strategy_name == "actual_2026_event_path":
557             price, adjustment, gap, fuel_return = simulate_actual_2026_path(scenario, base_price)
558         else:
559             if rule is None:
560                 raise ValueError("A SAPR rule is required for simulated strategies.")
561             price, adjustment, gap, fuel_return = simulate_rule_path(scenario, rule, base_price)
562         macro_paths = macro_path_from_returns(fuel_return, coefficients)
563         month_loss = np.zeros(SCENARIO_LENGTH, dtype=float)
564         month_loss += weights["china_ppi_yoy_pct"] * np.maximum(macro_paths["china_ppi_yoy_pct"], 0.0) / stds["china_ppi_yoy_pct"]
565         month_loss += weights["china_cpi_yoy_pct"] * np.maximum(macro_paths["china_cpi_yoy_pct"], 0.0) / stds["china_cpi_yoy_pct"]
566         month_loss += weights["china_iav_yoy_pct"] * np.maximum(-macro_paths["china_iav_yoy_pct"], 0.0) / stds["china_iav_yoy_pct"]
567         losses.append(float(np.mean(month_loss)))
568         gap_burdens.append(float(np.mean(np.maximum(gap, 0.0)) / base_price))
569         max_gap_ratios.append(float(np.max(np.maximum(gap, 0.0)) / base_price))
570         terminal_gap_ratios.append(float(max(float(gap[-1]), 0.0) / base_price))
571         recovery_gap_ratios.append(
572             float(recovery_terminal_gap(float(gap[-1]), rule) / base_price) if rule is not None else float(max(float(gap[-1]), 0.0) /
573             base_price)
574         )
575         max_adjustment_ratios.append(float(np.max(np.abs(adjustment)) / base_price))
576         returns_all.extend(fuel_return.tolist())
577     result = objective_from_scenario_outputs(losses, gap_burdens, returns_all)
578     result.update(
579         {
580             "scenario_count": len(candidates),
581             "max_gap_ratio": float(max(max_gap_ratios)),
582             "max_terminal_gap_ratio": float(max(terminal_gap_ratios)),
583             "max_recovery_terminal_gap_ratio": float(max(recovery_gap_ratios)),
584             "max_monthly_adjustment_ratio": float(max(max_adjustment_ratios)),

```



```

584     }
585 )
586 return result
587
588
589 def scenario_candidates(scenarios: list[Scenario], split: str) -> list[Scenario]:
590     candidates = [scenario for scenario in scenarios if scenario.sample_split == split]
591     if split == "war_2026":
592         candidates = [scenario for scenario in scenarios if scenario.is_2026_anchor]
593     if not candidates:
594         raise ValueError(f"Q4 has no scenario candidates for split={split}.")
595     return candidates
596
597
598 def strategy_path_arrays(
599     scenarios: list[Scenario],
600     rule: dict[str, float] | None,
601     strategy_name: str,
602     split: str,
603     base_price: float,
604 ) -> dict[str, np.ndarray]:
605     candidates = scenario_candidates(scenarios, split)
606     fuel_returns = np.empty((len(candidates), SCENARIO_LENGTH), dtype=float)
607     gap_burdens = np.empty(len(candidates), dtype=float)
608     max_gap_ratios = np.empty(len(candidates), dtype=float)
609     terminal_gap_ratios = np.empty(len(candidates), dtype=float)
610     recovery_gap_ratios = np.empty(len(candidates), dtype=float)
611     max_adjustment_ratios = np.empty(len(candidates), dtype=float)
612     for index, scenario in enumerate(candidates):
613         if strategy_name == "actual_2026_event_path":
614             _, adjustment, gap, fuel_return = simulate_actual_2026_path(scenario, base_price)
615         else:
616             if rule is None:
617                 raise ValueError("A SAPR rule is required for simulated strategies.")
618             _, adjustment, gap, fuel_return = simulate_rule_path(scenario, rule, base_price)
619         fuel_returns[index, :] = fuel_return
620         gap_burdens[index] = float(np.mean(np.maximum(gap, 0.0)) / base_price)
621         max_gap_ratios[index] = float(np.max(np.maximum(gap, 0.0)) / base_price)
622         terminal_gap_ratios[index] = float(max(float(gap[-1]), 0.0) / base_price)
623         recovery_gap_ratios[index] = (
624             float(recovery_terminal_gap(float(gap[-1]), rule) / base_price)
625             if rule is not None
626             else terminal_gap_ratios[index]
627         )
628         max_adjustment_ratios[index] = float(np.max(np.abs(adjustment)) / base_price)
629     return {
630         "fuel_returns": fuel_returns,
631         "gap_burdens": gap_burdens,
632         "max_gap_ratios": max_gap_ratios,
633         "terminal_gap_ratios": terminal_gap_ratios,
634         "recovery_gap_ratios": recovery_gap_ratios,
635         "max_adjustment_ratios": max_adjustment_ratios,
636     }
637
638
639 def macro_losses_from_returns(
640     fuel_returns: np.ndarray,
641     coefficients: dict[str, np.ndarray],
642     stds: dict[str, float],
643     weights: dict[str, float],
644 ) -> np.ndarray:
645     if fuel_returns.ndim != 2 or fuel_returns.shape[1] != SCENARIO_LENGTH:
646         raise ValueError("Q4 macro loss calculation requires a scenario-by-month fuel-return matrix.")
647     month_loss = np.zeros_like(fuel_returns, dtype=float)
648     for outcome in OUTCOMES:
649         params = coefficients[outcome]
650         y = np.zeros_like(fuel_returns, dtype=float)
651         for t in range(SCENARIO_LENGTH):
652             signal = np.zeros(fuel_returns.shape[0], dtype=float)
653             for lag in range(7):
654                 if t - lag >= 0:
655                     signal += float(params[lag]) * fuel_returns[:, t - lag]
656             if t > 0:
657                 signal += float(params[7]) * y[:, t - 1]
658             y[:, t] = signal
659     if outcome == "china_iav_yoy_pct":

```

```

660         contribution = np.maximum(-y, 0.0)
661     else:
662         contribution = np.maximum(y, 0.0)
663     month_loss += weights[outcome] * contribution / stds[outcome]
664     return month_loss.mean(axis=1)
665
666
667 def objectives_from_arrays(
668     scenario_losses: np.ndarray,
669     gap_burdens: np.ndarray,
670     fuel_returns: np.ndarray,
671     scenario_indices: np.ndarray | None = None,
672 ) -> dict[str, float]:
673     if scenario_indices is not None:
674         indices = np.asarray(scenario_indices, dtype=int) % len(scenario_losses)
675         losses = scenario_losses[indices]
676         gaps = gap_burdens[indices]
677         returns = fuel_returns[indices, :].reshape(-1)
678     else:
679         losses = scenario_losses
680         gaps = gap_burdens
681         returns = fuel_returns.reshape(-1)
682     return objective_from_scenario_outputs(losses.tolist(), gaps.tolist(), returns.tolist())
683
684
685 def dominates(a: np.ndarray, b: np.ndarray) -> bool:
686     return bool(np.all(a <= b + 1e-12) and np.any(a < b - 1e-12))
687
688
689 def pareto_mask(objectives: np.ndarray) -> np.ndarray:
690     n = objectives.shape[0]
691     mask = np.ones(n, dtype=bool)
692     for i in range(n):
693         if not mask[i]:
694             continue
695         dominated_by_i = np.all(objectives[i] <= objectives + 1e-12, axis=1) & np.any(objectives[i] < objectives - 1e-12, axis=1)
696         dominated_by_i[i] = False
697         mask[dominated_by_i] = False
698         if np.any(np.all(objectives[mask] <= objectives[i] + 1e-12, axis=1) & np.any(objectives[mask] < objectives[i] - 1e-12, axis=1)):
699             mask[i] = False
700     return mask
701
702
703 def epsilon_pareto_mask(objectives: np.ndarray, epsilon_fraction: float = 0.01) -> np.ndarray:
704     """Treat sub-material objective differences as ties to avoid a mechanically wide front."""
705     spans = np.ptp(objectives, axis=0)
706     epsilon = np.maximum(spans * epsilon_fraction, 1e-12)
707     n = objectives.shape[0]
708     mask = np.ones(n, dtype=bool)
709     for i in range(n):
710         candidate = objectives[i]
711         for j in range(n):
712             if i == j:
713                 continue
714             challenger = objectives[j]
715             if np.all(challenger <= candidate + epsilon) and np.any(challenger < candidate - epsilon):
716                 mask[i] = False
717                 break
718     return mask
719
720
721 def choose_knee(grid: pd.DataFrame) -> pd.DataFrame:
722     objective_cols = ["J1_macro_loss", "J2_cvar95_macro_loss", "J3_avg_gap_month_burden", "J4_adjustment_volatility"]
723     grid = grid.copy()
724     grid["is_feasible"] = (
725         grid["max_gap_ratio"].le(MAX_GAP_RATIO)
726         & grid["max_terminal_gap_ratio"].le(MAX_TERMINAL_GAP_RATIO)
727         & grid["max_recovery_terminal_gap_ratio"].le(MAX_RECOVERY_TERMINAL_GAP_RATIO)
728         & grid["max_monthly_adjustment_ratio"].le(MAX_MONTHLY_ADJUSTMENT_RATIO)
729     )
730     grid["is_pareto"] = False
731     grid["is_knee"] = False
732     feasible = grid.loc[grid["is_feasible"]].copy()
733     if feasible.empty:
734         raise ValueError("Q4 has no feasible rule under the preregistered gap, recovery and adjustment constraints.")
735     feasible_values = feasible[objective_cols].astype(float).to_numpy()

```

```

736 exact_mask = pareto_mask(feasible_values)
737 grid["is_pareto_exact"] = False
738 grid.loc[feasible.index, "is_pareto_exact"] = exact_mask
739 mask = epsilon_pareto_mask(feasible_values, epsilon_fraction=0.01)
740 grid.loc[feasible.index, "is_pareto"] = mask
741 pareto = grid.loc[grid["is_pareto"]].copy()
742 if pareto.empty:
743     raise ValueError("Q4 Pareto set is empty.")
744 mins = pareto[objective_cols].min()
745 maxs = pareto[objective_cols].max()
746 denom = (maxs - mins).replace(0.0, 1.0)
747 normalized = (pareto[objective_cols] - mins) / denom
748 pareto["knee_score"] = np.sqrt((normalized**2).sum(axis=1))
749 pareto = pareto.sort_values(["knee_score", "J3_avg_gap_month_burden", "rho_normal"], ascending=[True, True, False])
750 knee_rule_id = pareto.iloc[0]["rule_id"]
751 grid.loc[grid["rule_id"].eq(knee_rule_id), "is_knee"] = True
752 grid["knee_score"] = np.nan
753 grid.loc[pareto.index, "knee_score"] = pareto["knee_score"]
754 return grid
755
756
757 def run_grid_search(
758     scenarios: list[Scenario],
759     coefficients: dict[str, np.ndarray],
760     base_price: float,
761     stds: dict[str, float],
762     weights: dict[str, float],
763 ) -> pd.DataFrame:
764     rows: list[dict[str, Any]] = []
765     for idx, rule in enumerate(enumerate_rules(), start=1):
766         objectives = evaluate_strategy(scenarios, coefficients, rule, "SAPR_candidate", "development", base_price, stds, weights)
767         rows.append(
768             {
769                 "rule_id": f"R{idx:04d}",
770                 **rule,
771                 **objectives,
772                 "sample_split": "development",
773             }
774         )
775     grid = choose_knee(pd.DataFrame(rows))
776     if int(grid["is_knee"].sum()) != 1:
777         raise ValueError("Q4 knee selection is not unique.")
778     return grid
779
780
781 def strategy_catalog(knee_rule: dict[str, float]) -> dict[str, dict[str, Any]]:
782     return {
783         "full_mechanism": {"rule": {"rho_normal": 1.0, "rho_stress": 1.0, "rho_extreme": 1.0}, "type": "rule"},
784         "uniform_75_smoothing": {"rule": {"rho_normal": 0.75, "rho_stress": 0.75, "rho_extreme": 0.75}, "type": "rule"},
785         "temporary_2026_approx": {"rule": {"rho_normal": 1.0, "rho_stress": 1.0, "rho_extreme": round(TEMPORARY_2026_RHO, 6)}, "type": "rule"},
786         "SAPR_CVaR_knee": {"rule": knee_rule, "type": "rule"},
787         "actual_2026_event_path": {"rule": None, "type": "actual_2026"},
788     }
789
790
791 def block_bootstrap_indices(n: int, block_length: int, rng: np.random.Generator) -> np.ndarray:
792     if n <= 0:
793         raise ValueError("Q4 block bootstrap requires a positive scenario count.")
794     indices: list[int] = []
795     while len(indices) < n:
796         start = int(rng.integers(0, n))
797         for offset in range(block_length):
798             indices.append((start + offset) % n)
799             if len(indices) >= n:
800                 break
801     return np.array(indices[:n], dtype=int)
802
803
804 def compare_strategies(
805     scenarios: list[Scenario],
806     kernel: MacroKernel,
807     parameter_draws: list[dict[str, np.ndarray]],
808     knee_rule: dict[str, float],
809     base_price: float,
810     stds: dict[str, float],

```

```

811     weights: dict[str, float],
812     rng: np.random.Generator,
813     block_length: int = 6,
814 ) -> tuple[pd.DataFrame, float, bool]:
815     point = point_coefficients(kernel)
816     strategies = strategy_catalog(knee_rule)
817     rows: list[dict[str, Any]] = []
818     interval_store: dict[tuple[str, str], list[dict[str, float]]] = {}
819     splits = ["development", "holdout", "war_2026"]
820     for split in splits:
821         for strategy_name, meta in strategies.items():
822             if strategy_name == "actual_2026_event_path" and split != "war_2026":
823                 continue
824             arrays = strategy_path_arrays(scenarios, meta["rule"], strategy_name, split, base_price)
825             point_losses = macro_losses_from_returns(arrays["fuel_returns"], point, stds, weights)
826             objectives = objectives_from_arrays(point_losses, arrays["gap_burdens"], arrays["fuel_returns"])
827             objectives["scenario_count"] = int(len(point_losses))
828             objectives.update(
829                 {
830                     "max_gap_ratio": float(np.max(arrays["max_gap_ratios"])),
831                     "max_terminal_gap_ratio": float(np.max(arrays["terminal_gap_ratios"])),
832                     "max_recovery_terminal_gap_ratio": float(np.max(arrays["recovery_gap_ratios"])),
833                     "max_monthly_adjustment_ratio": float(np.max(arrays["max_adjustment_ratios"])),
834                 }
835             )
836             row = {
837                 "sample_split": split,
838                 "strategy": strategy_name,
839                 "rho_normal": clean_value(meta["rule"]["rho_normal"]) if meta["rule"] else None,
840                 "rho_stress": clean_value(meta["rule"]["rho_stress"]) if meta["rule"] else None,
841                 "rho_extreme": clean_value(meta["rule"]["rho_extreme"]) if meta["rule"] else None,
842                 **objectives,
843             }
844             rows.append(row)
845             interval_store[(split, strategy_name)] = []
846             n = int(len(point_losses))
847             for draw in parameter_draws:
848                 sampled = block_bootstrap_indices(n, block_length, rng)
849                 draw_losses = macro_losses_from_returns(arrays["fuel_returns"], draw, stds, weights)
850                 interval_store[(split, strategy_name)].append(
851                     objectives_from_arrays(draw_losses, arrays["gap_burdens"], arrays["fuel_returns"], sampled)
852                 )
853     comparison = pd.DataFrame(rows)
854     full_by_split = comparison.loc[comparison["strategy"].eq("full_mechanism")].set_index("sample_split")
855     for idx, row in comparison.iterrows():
856         split = row["sample_split"]
857         if split in full_by_split.index:
858             full = full_by_split.loc[split]
859             for col in ["J1_macro_loss", "J2_cvar95_macro_loss"]:
860                 denom = abs(float(full[col])) if abs(float(full[col])) > 1e-12 else np.nan
861                 comparison.loc[idx, f"{col}_improvement_vs_full_pct"] = (
862                     100.0 * (float(full[col]) - float(row[col])) / denom if np.isfinite(denom) else np.nan
863                 )
864     for idx, row in comparison.iterrows():
865         draws = interval_store.get((row["sample_split"], row["strategy"]), [])
866         if not draws:
867             continue
868         for col in ["J1_macro_loss", "J2_cvar95_macro_loss"]:
869             values = np.array([float(draw[col]) for draw in draws], dtype=float)
870             comparison.loc[idx, f"{col}_lower_95"] = float(np.quantile(values, 0.025))
871             comparison.loc[idx, f"{col}_upper_95"] = float(np.quantile(values, 0.975))
872
873     probability, point_non_dominated = holdout_validation_probability(
874         scenarios,
875         parameter_draws,
876         knee_rule,
877         base_price,
878         stds,
879         weights,
880         rng,
881         block_length=block_length,
882         point_comparison=comparison,
883     )
884     return comparison, probability, point_non_dominated
885
886

```

```

887 def holdout_validation_probability(
888     scenarios: list[Scenario],
889     parameter_draws: list[dict[str, np.ndarray]],
890     knee_rule: dict[str, float],
891     base_price: float,
892     stds: dict[str, float],
893     weights: dict[str, float],
894     rng: np.random.Generator,
895     block_length: int,
896     point_comparison: pd.DataFrame | None = None,
897     point_values: dict[str, np.ndarray] | None = None,
898     save_outputs: bool = True,
899 ) -> tuple[float, bool]:
900     objective_cols = ["J1_macro_loss", "J2_cvar95_macro_loss", "J3_avg_gap_month_burden", "J4_adjustment_volatility"]
901     strategies = strategy_catalog(knee_rule)
902     holdout_strategies = [name for name in strategies if name != "actual_2026_event_path"]
903     if point_comparison is None:
904         if point_values is None:
905             raise ValueError("Q4 holdout validation requires either point comparison rows or explicit point coefficients.")
906         point_rows: list[dict[str, Any]] = []
907         for strategy_name in holdout_strategies:
908             meta = strategies[strategy_name]
909             arrays = strategy_path_arrays(scenarios, meta["rule"], strategy_name, "holdout", base_price)
910             point_losses = macro_losses_from_returns(arrays["fuel_returns"], point_values, stds, weights)
911             objectives = objectives_from_arrays(point_losses, arrays["gap_burdens"], arrays["fuel_returns"])
912             objectives["scenario_count"] = int(len(point_losses))
913             point_rows.append({"sample_split": "holdout", "strategy": strategy_name, **objectives})
914         point_holdout = pd.DataFrame(point_rows)
915     else:
916         point_holdout = point_comparison.loc[
917             point_comparison["sample_split"].eq("holdout") & point_comparison["strategy"].isin(holdout_strategies)
918         ].copy()
919     if set(point_holdout["strategy"]) != set(holdout_strategies):
920         raise ValueError("Q4 holdout validation is missing one or more preregistered strategies.")
921     point_knee = point_holdout.loc[point_holdout["strategy"].eq("SAPR_CVaR_knee")]
922     if len(point_knee) != 1:
923         raise ValueError("Q4 holdout validation requires exactly one SAPR knee row.")
924     knee_vec = point_knee[objective_cols].iloc[0].astype(float).to_numpy()
925     point_non_dominated = True
926     for _, row in point_holdout.loc[point_holdout["strategy"].ne("SAPR_CVaR_knee")].iterrows():
927         other_vec = row[objective_cols].astype(float).to_numpy()
928         if dominates(other_vec, knee_vec):
929             point_non_dominated = False
930         break
931     holdout_n = len([scenario for scenario in scenarios if scenario.sample_split == "holdout"])
932     arrays_by_strategy = {
933         strategy_name: strategy_path_arrays(scenarios, strategies[strategy_name]["rule"], strategy_name, "holdout", base_price)
934         for strategy_name in holdout_strategies
935     }
936     point_matrix = point_holdout.set_index("strategy").loc[holdout_strategies, objective_cols].astype(float).to_numpy()
937     point_mins = point_matrix.min(axis=0)
938     point_denom = np.maximum(point_matrix.max(axis=0) - point_mins, 1e-12)
939     point_distances = np.sqrt((((point_matrix - point_mins) / point_denom) ** 2).sum(axis=1))
940     point_knee_distance = float(point_distances[holdout_strategies.index("SAPR_CVaR_knee")])
941     point_oracle_distance = float(point_distances.min())
942     draw_rows: list[dict[str, Any]] = []
943     valid_flags: list[bool] = []
944     for draw_id, draw in enumerate(parameter_draws):
945         sampled = block_bootstrap_indices(holdout_n, block_length, rng)
946         vectors: dict[str, np.ndarray] = {}
947         for strategy_name in holdout_strategies:
948             arrays = arrays_by_strategy[strategy_name]
949             draw_losses = macro_losses_from_returns(arrays["fuel_returns"], draw, stds, weights)
950             obj = objectives_from_arrays(draw_losses, arrays["gap_burdens"], arrays["fuel_returns"], sampled)
951             vectors[strategy_name] = np.array(
952                 [obj["J1_macro_loss"], obj["J2_cvar95_macro_loss"], obj["J3_avg_gap_month_burden"], obj["J4_adjustment_volatility"]],
953                 dtype=float,
954             )
955         sampled_knee_vec = vectors["SAPR_CVaR_knee"]
956         non_dominated = not any(dominates(vec, sampled_knee_vec) for name, vec in vectors.items() if name != "SAPR_CVaR_knee")
957         valid_flags.append(non_dominated)
958         matrix = np.vstack([vectors[name] for name in holdout_strategies])
959         mins = matrix.min(axis=0)
960         denom = np.maximum(matrix.max(axis=0) - mins, 1e-12)
961         distances = np.sqrt((((matrix - mins) / denom) ** 2).sum(axis=1))
962         knee_distance = float(distances[holdout_strategies.index("SAPR_CVaR_knee")])

```

```

963     oracle_distance = float(distances.min())
964     for baseline in [name for name in holdout_strategies if name != "SAPR_CVaR_knee"]:
965         delta = sampled_knee_vec - vectors[baseline]
966         draw_rows.append(
967             {
968                 "draw": draw_id,
969                 "block_length": block_length,
970                 "baseline": baseline,
971                 **{f"delta_{column}": float(delta[idx]) for idx, column in enumerate(objective_cols)},
972                 "knee_distance_to_ideal": knee_distance,
973                 "oracle_distance_to_ideal": oracle_distance,
974                 "knee_regret_vs_oracle": knee_distance - oracle_distance,
975                 "knee_non_dominated": non_dominated,
976             }
977         )
978     if len(valid_flags) != len(parameter_draws):
979         raise ValueError("Q4 holdout validation did not evaluate every valid parameter draw.")
980     validation = pd.DataFrame(draw_rows)
981     if save_outputs:
982         save_csv(validation, "q4_sapr_holdout_validation.csv")
983     paired_summary: list[dict[str, Any]] = []
984     for baseline, group in validation.groupby("baseline"):
985         for column in objective_cols:
986             values = group[f"delta_{column}"].to_numpy(dtype=float)
987             lower, upper = np.quantile(values, [0.025, 0.975])
988             paired_summary.append(
989                 {
990                     "baseline": baseline,
991                     "objective": column,
992                     "mean_delta_knee_minus_baseline": float(np.mean(values)),
993                     "lower_95": float(lower),
994                     "upper_95": float(upper),
995                     "supported_improvement": bool(upper < 0.0),
996                 }
997             )
998     paired_frame = pd.DataFrame(paired_summary)
999     if save_outputs:
1000         save_csv(paired_frame, "q4_sapr_holdout_paired_summary.csv")
1001     unique_draws = validation.drop_duplicates("draw")
1002     summary_payload = {
1003         "block_length": block_length,
1004         "draws": int(len(unique_draws)),
1005         "point_non_dominated": point_non_dominated,
1006         "non_dominated_probability": float(np.mean(valid_flags)),
1007         "point_knee_distance_to_ideal": point_knee_distance,
1008         "point_oracle_distance_to_ideal": point_oracle_distance,
1009         "point_regret_vs_oracle": point_knee_distance - point_oracle_distance,
1010         "median_regret_vs_oracle": float(unique_draws["knee_regret_vs_oracle"].median()),
1011         "regret_upper_95": float(unique_draws["knee_regret_vs_oracle"].quantile(0.975)),
1012         "probability_within_0_10_of_oracle": float(unique_draws["knee_regret_vs_oracle"].le(0.10).mean()),
1013         "supported_improvement_count": int(paired_frame["supported_improvement"].sum()),
1014     }
1015     if save_outputs:
1016         (RESULTS_DIR / "q4_sapr_holdout_validation_summary.json").write_text(
1017             json.dumps(summary_payload, ensure_ascii=False, indent=2, allow_nan=False) + "\n",
1018             encoding="utf-8",
1019         )
1020     return float(np.mean(valid_flags)), point_non_dominated
1021
1022
1023 def aggregate_macro_paths(
1024     scenarios: list[Scenario],
1025     kernel: MacroKernel,
1026     parameter_draws: list[dict[str, np.ndarray]],
1027     knee_rule: dict[str, float],
1028     base_price: float,
1029     split: str = "war_2026",
1030 ) -> pd.DataFrame:
1031     strategies = strategy_catalog(knee_rule)
1032     point = point_coefficients(kernel)
1033     rows: list[dict[str, Any]] = []
1034     target_scenarios = [scenario for scenario in scenarios if scenario.is_2026_anchor] if split == "war_2026" else [scenario for scenario
1035         in scenarios if scenario.sample_split == split]
1036     if not target_scenarios:
1037         raise ValueError(f"Q4 macro-path aggregation has no target scenarios for split={split}.")
1038     for strategy_name, meta in strategies.items():

```

```

1038     if strategy_name == "actual_2026_event_path" and split != "war_2026":
1039         continue
1040     month_records: list[dict[str, Any]] = []
1041     for scenario in target_scenarios:
1042         if strategy_name == "actual_2026_event_path":
1043             price, adjustment, gap, fuel_return = simulate_actual_2026_path(scenario, base_price)
1044         else:
1045             price, adjustment, gap, fuel_return = simulate_rule_path(scenario, meta["rule"], base_price)
1046         macro_paths = macro_path_from_returns(fuel_return, point)
1047         draw_paths = {outcome: [] for outcome in OUTCOMES}
1048         for draw in parameter_draws:
1049             simulated = macro_path_from_returns(fuel_return, draw)
1050             for outcome in OUTCOMES:
1051                 draw_paths[outcome].append(simulated[outcome])
1052         for t, period in enumerate(scenario.periods):
1053             record = {
1054                 "sample_split": split,
1055                 "scenario_id": scenario.scenario_id,
1056                 "strategy": strategy_name,
1057                 "month_index": t,
1058                 "period": period,
1059                 "fuel_price_cny_t": price[t],
1060                 "actual_adjustment_cny_t": adjustment[t],
1061                 "fuel_log_return": fuel_return[t],
1062                 "cumulative_deferred_gap_cny_t": gap[t],
1063                 "PPI_response_pctpt": macro_paths["china_ppi_yoy_pct"][t],
1064                 "CPI_response_pctpt": macro_paths["china_cpi_yoy_pct"][t],
1065                 "IAV_response_pctpt": macro_paths["china_iav_yoy_pct"][t],
1066             }
1067             for outcome, label in OUTCOME_LABELS.items():
1068                 values = np.array([path[t] for path in draw_paths[outcome]], dtype=float)
1069                 record[f"{label}_lower_95"] = float(np.quantile(values, 0.025))
1070                 record[f"{label}_upper_95"] = float(np.quantile(values, 0.975))
1071             month_records.append(record)
1072     if not month_records:
1073         raise ValueError(f"Q4 macro-path aggregation produced no rows for strategy={strategy_name}, split={split}.")
1074     rows.extend(month_records)
1075     return pd.DataFrame(rows)
1076
1077
1078 def policy_identity_checks(scenarios: list[Scenario], base_price: float, kernel: MacroKernel) -> dict[str, Any]:
1079     scenario = next(item for item in scenarios if item.sample_split == "development")
1080     _, full_adjustment, full_gap, _ = simulate_rule_path(
1081         scenario,
1082         {"rho_normal": 1.0, "rho_stress": 1.0, "rho_extreme": 1.0},
1083         base_price,
1084     )
1085     synthetic = Scenario(
1086         scenario_id="IDENTITY_ZERO",
1087         sample_split="development",
1088         source_start="2099-01",
1089         periods=tuple(f"2099-0{i + 1}" for i in range(SCENARIO_LENGTH)),
1090         mechanism_adjustment=np.zeros(SCENARIO_LENGTH),
1091         actual_adjustment=np.zeros(SCENARIO_LENGTH),
1092         cumulative_policy_gap=np.zeros(SCENARIO_LENGTH),
1093         regimes=tuple(["normal"] * SCENARIO_LENGTH),
1094         is_2026_anchor=False,
1095     )
1096     _, _, zero_return = simulate_rule_path(
1097         synthetic,
1098         {"rho_normal": 0.35, "rho_stress": 0.2, "rho_extreme": 0.1},
1099         base_price,
1100     )
1101     zero_macro = macro_path_from_returns(zero_return, point_coefficients(kernel))
1102     accumulation = Scenario(
1103         scenario_id="IDENTITY_ACCUMULATION",
1104         sample_split="development",
1105         source_start="2099-01",
1106         periods=tuple(f"2099-0{i + 1}" for i in range(SCENARIO_LENGTH)),
1107         mechanism_adjustment=np.array([100.0, 100.0, 0.0, 0.0, 0.0, 0.0]),
1108         actual_adjustment=np.zeros(SCENARIO_LENGTH),
1109         cumulative_policy_gap=np.zeros(SCENARIO_LENGTH),
1110         regimes=tuple(["normal"] * SCENARIO_LENGTH),
1111         is_2026_anchor=False,
1112     )
1113     _, _, gap_accum, _ = simulate_rule_path(

```

```

1114     accumulation,
1115     {"rho_normal": 0.0, "rho_stress": 0.0, "rho_extreme": 0.0},
1116     base_price,
1117 )
1118 offset = Scenario(
1119     scenario_id="IDENTITY_OFFSET",
1120     sample_split="development",
1121     source_start="2099-01",
1122     periods=tuple(f"2099-0{i + 1}" for i in range(SCENARIO_LENGTH)),
1123     mechanism_adjustment=np.array([100.0, -40.0, -100.0, 0.0, 0.0, 0.0]),
1124     actual_adjustment=np.zeros(SCENARIO_LENGTH),
1125     cumulative_policy_gap=np.zeros(SCENARIO_LENGTH),
1126     regimes=tuple(["normal"] * SCENARIO_LENGTH),
1127     is_2026_anchor=False,
1128 )
1129 _, _, gap_offset, _ = simulate_rule_path(
1130     offset,
1131     {"rho_normal": 0.0, "rho_stress": 0.0, "rho_extreme": 0.0},
1132     base_price,
1133 )
1134 policy = read_processed("china_fuel_policy_monthly.csv")
1135 march = policy.loc[policy["period"].eq("2026-03")].iloc[0]
1136 april = policy.loc[policy["period"].eq("2026-04")].iloc[0]
1137 q3_repro = reproduce_q3_policy_counterfactual(kernel)
1138 return {
1139     "full_rule_max_abs_gap": float(np.max(np.abs(full_gap))),
1140     "full_rule_adjustment_identity_max_abs_error": float(np.max(np.abs(full_adjustment - scenario.mechanism_adjustment))),
1141     "zero_policy_macro_max_abs_response": float(max(np.max(np.abs(values)) for values in zero_macro.values())),
1142     "zero_upward_pass_through_terminal_gap": float(gap_accum[2]),
1143     "negative_adjustment_offsets_gap": bool(abs(float(gap_offset[2])) < 1e-9),
1144     "official_2026_03_gap": float(march["gasoline_policy_gap_cny_t"]),
1145     "official_2026_04_gap": float(april["gasoline_policy_gap_cny_t"]),
1146     "official_2026_04_cumulative_gap": float(april["cum_gasoline_policy_gap_cny_t"]),
1147     "q3_policy_counterfactual_reproduction_max_abs_error": q3_repro,
1148 }
1149
1150
1151 def reproduce_q3_policy_counterfactual(kernel: MacroKernel) -> float:
1152     q3 = read_result("q3_policy_macro_counterfactual.csv")
1153     max_error = 0.0
1154     for outcome, group in q3.groupby("outcome"):
1155         group = group.sort_values("horizon")
1156         fuel = group["fuel_return_gap"].to_numpy(dtype=float)
1157         params = kernel.means[str(outcome)]
1158         expected = np.zeros(len(group), dtype=float)
1159         for t in range(len(group)):
1160             for lag in range(7):
1161                 if t - lag >= 0:
1162                     expected[t] += float(params[lag]) * float(fuel[t - lag])
1163                 if t > 0:
1164                     expected[t] += float(params[7]) * expected[t - 1]
1165             observed = group["macro_counterfactual_gap_pctpt"].to_numpy(dtype=float)
1166             max_error = max(max_error, float(np.max(np.abs(expected - observed))))
1167     return float(max_error)
1168
1169
1170 def run_sensitivity(
1171     default_grid: pd.DataFrame,
1172     scenarios: list[Scenario],
1173     kernel: MacroKernel,
1174     parameter_draws: list[dict[str, np.ndarray]],
1175     meta: dict[str, float],
1176     stds: dict[str, float],
1177     rng: np.random.Generator,
1178 ) -> pd.DataFrame:
1179     rows: list[dict[str, Any]] = []
1180     variants = [
1181         ("threshold_70_90", (0.70, 0.90), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.0}, 3),
1182         ("threshold_80_975", (0.80, 0.975), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.0}, 3),
1183         ("cpi_weight_plus20", (0.75, 0.95), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.2, "china_iav_yoy_pct": 1.0}, 3),
1184         ("iav_weight_plus20", (0.75, 0.95), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.2}, 3),
1185         ("bootstrap_block_3", (0.75, 0.95), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.0}, 3),
1186         ("bootstrap_block_6", (0.75, 0.95), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.0}, 6),
1187         ("bootstrap_block_12", (0.75, 0.95), {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.0}, 12),
1188     ]
1189     point = point_coefficients(kernel)

```



```

1190 for variant_name, quantiles, weights, block_length in variants:
1191     if variant_name.startswith("threshold"):
1192         _, variant_scenarios, variant_meta = build_scenarios(quantiles)
1193     else:
1194         variant_scenarios = scenarios
1195         variant_meta = meta
1196     if variant_name.startswith("bootstrap_block"):
1197         grid = default_grid.copy()
1198     else:
1199         grid = run_grid_search(variant_scenarios, point, float(variant_meta["base_price_2026_06_cny_t"]), stds, weights)
1200     knee = grid.loc[grid["is_knee"]].iloc[0]
1201     knee_rule = {
1202         "rho_normal": float(knee["rho_normal"]),
1203         "rho_stress": float(knee["rho_stress"]),
1204         "rho_extreme": float(knee["rho_extreme"]),
1205     }
1206     prob, point_nd = holdout_validation_probability(
1207         variant_scenarios,
1208         parameter_draws,
1209         knee_rule,
1210         float(variant_meta["base_price_2026_06_cny_t"]),
1211         stds,
1212         weights,
1213         rng,
1214         block_length=block_length,
1215         point_values=point,
1216         save_outputs=False,
1217     )
1218     rows.append(
1219         {
1220             "variant": variant_name,
1221             "threshold_low_quantile": quantiles[0],
1222             "threshold_high_quantile": quantiles[1],
1223             "bootstrap_block_length": block_length,
1224             "ppi_weight": weights["china_ppi_yoy_pct"],
1225             "cpi_weight": weights["china_cpi_yoy_pct"],
1226             "iav_weight": weights["china_iav_yoy_pct"],
1227             "rho_normal": knee_rule["rho_normal"],
1228             "rho_stress": knee_rule["rho_stress"],
1229             "rho_extreme": knee_rule["rho_extreme"],
1230             "pareto_rule_count": int(grid["is_pareto"].sum()),
1231             "holdout_non_dominated_probability": prob,
1232             "holdout_point_non_dominated": point_nd,
1233             "changed_from_default": bool(
1234                 knee_rule["rho_normal"] != float(default_grid.loc[default_grid["is_knee"], "rho_normal"].iloc[0])
1235                 or knee_rule["rho_stress"] != float(default_grid.loc[default_grid["is_knee"], "rho_stress"].iloc[0])
1236                 or knee_rule["rho_extreme"] != float(default_grid.loc[default_grid["is_knee"], "rho_extreme"].iloc[0])
1237             ),
1238         }
1239     )
1240 return pd.DataFrame(rows)
1241
1242
1243 def plot_pareto(grid: pd.DataFrame) -> None:
1244     fig, ax = plt.subplots(figsize=(7.6, 4.6))
1245     pareto = grid.loc[grid["is_pareto"]].copy()
1246     if pareto.empty:
1247         raise ValueError("Q4 Pareto plot requires a non-empty Pareto set.")
1248     volatility = pareto["J4_adjustment_volatility"].astype(float)
1249     size = 24.0 + 72.0 * (volatility - volatility.min()) / max(float(volatility.max()) - volatility.min(), 1e-12)
1250     sc = ax.scatter(
1251         pareto["J1_macro_loss"],
1252         pareto["J3_avg_gap_month_burden"],
1253         c=pareto["J2_cvar95_macro_loss"],
1254         cmap="cividis",
1255         s=size,
1256         alpha=0.82,
1257     )
1258     knee = pareto.loc[pareto["is_knee"]]
1259     if len(knee) != 1:
1260         raise ValueError("Q4 Pareto plot requires exactly one knee rule.")
1261     ax.scatter(knee["J1_macro_loss"], knee["J3_avg_gap_month_burden"], marker="+", s=180, color=PALETTE["rose"], edgecolor="white",
1262               linewidth=0.8, label="膝点规则")
1263     style_axis(ax, xlabel="期望宏观损失", ylabel="平均缺口月负担")
1264     ax.legend(loc="upper right")
1265     cbar = fig.colorbar(sc, ax=ax)

```

```

1265 cbar.set_label("CVaR95 宏观损失")
1266 finish_figure(
1267     fig,
1268     title="问题四：SAPR-CVaR 四目标Pareto前沿",
1269     subtitle="横纵轴为J1/J3，颜色为J2，点大小为J4；仅绘制通过硬约束的1%epsilon-非支配规则，星形为膝点。",
1270     source="来源：results/q4_sapr_policy_grid.csv；作者计算。",
1271 )
1272 save_figure(fig, FIGURES_DIR / "q4_sapr_pareto_front")
1273 plt.close(fig)
1274
1275
1276 def plot_policy_heatmap(knee_rule: dict[str, float]) -> None:
1277     regimes = ["normal", "stress", "extreme"]
1278     gap_bins = ["低缺口", "中缺口", "高缺口"]
1279     values = np.array([[rho_for_regime(knee_rule, regime) for regime in regimes] for _ in gap_bins])
1280     fig, ax = plt.subplots(figsize=(6.4, 3.8))
1281     image = ax.imshow(values, cmap="YlGnBu", vmin=0, vmax=1)
1282     ax.set_xticks(np.arange(len(regimes)), labels=["普通", "压力", "极端"])
1283     ax.set_yticks(np.arange(len(gap_bins)), labels=gap_bins)
1284     for i in range(values.shape[0]):
1285         for j in range(values.shape[1]):
1286             ax.text(j, i, f"{values[i, j]:.2f}", ha="center", va="center", color=PALETTE["ink"], fontsize=10)
1287     ax.set_xlabel("机制应调额状态")
1288     ax.set_ylabel("累计未调价缺口")
1289     cbar = fig.colorbar(image, ax=ax)
1290     cbar.set_label("最优传导率")
1291     finish_figure(
1292         fig,
1293         title="问题四：冲击强度与最优传导率",
1294         subtitle="SAPR规则按冲击状态调整传导率，累计缺口通过状态转移进入下一期应调额。",
1295         source="来源：results/q4_sapr_optimal_rule.csv；作者计算。",
1296     )
1297     save_figure(fig, FIGURES_DIR / "q4_sapr_policy_heatmap")
1298     plt.close(fig)
1299
1300
1301 def plot_strategy_comparison(comparison: pd.DataFrame) -> None:
1302     frame = comparison.loc[comparison["sample_split"].eq("holdout") & comparison["strategy"].ne("actual_2026_event_path")].copy()
1303     if frame.empty:
1304         raise ValueError("Q4 strategy comparison plot requires holdout strategy rows.")
1305     labels = {
1306         "full_mechanism": "完全传导",
1307         "uniform_75_smoothing": "固定75%平滑",
1308         "temporary_2026_approx": "2026近似",
1309         "SAPR_CVaR_knee": "SAPR膝点",
1310     }
1311     frame["label"] = frame["strategy"].map(labels).fillna(frame["strategy"])
1312     metrics = ["J1_macro_loss", "J2_cvar95_macro_loss", "J3_avg_gap_month_burden", "J4_adjustment_volatility"]
1313     metric_labels = ["平均宏观损失", "CVaR95宏观损失", "平均缺口月负担", "月度调价波动"]
1314     fig, axes = plt.subplots(2, 2, figsize=(9.0, 6.4))
1315     colors = [PALETTE["slate"], PALETTE["gold"], PALETTE["olive"], PALETTE["blue"]]
1316     for ax, metric, label in zip(axes.ravel(), metrics, metric_labels):
1317         ax.bar(frame["label"], frame[metric].astype(float), color=colors, width=0.62)
1318         ax.tick_params(axis="x", labelrotation=18)
1319         style_axis(ax, ylabel=label)
1320     finish_figure(
1321         fig,
1322         title="问题四：检验期四种策略原始目标对比",
1323         subtitle="四个子图保留各自原始量纲；数值越低表示该目标表现越好，不能跨子图直接比较高度。",
1324         source="来源：results/q4_sapr_strategy_comparison.csv；作者计算。",
1325         rect=(0.08, 0.08, 0.98, 0.90),
1326     )
1327     save_figure(fig, FIGURES_DIR / "q4_sapr_strategy_comparison")
1328     plt.close(fig)
1329
1330
1331 def plot_2026_paths(paths: pd.DataFrame) -> None:
1332     frame = paths.loc[paths["sample_split"].eq("war_2026")].copy()
1333     if frame.empty:
1334         raise ValueError("Q4 2026 path plot requires war_2026 rows.")
1335     labels = {
1336         "full_mechanism": "完全传导",
1337         "uniform_75_smoothing": "固定75%平滑",
1338         "temporary_2026_approx": "2026近似",
1339         "SAPR_CVaR_knee": "SAPR膝点",
1340         "actual_2026_event_path": "2026实际",

```

```

1341 }
1342 fig, axes = plt.subplots(2, 2, figsize=(9.0, 6.2), sharex=True)
1343 panels = [
1344     ("PPI_response_pctpt", "PPI响应"),
1345     ("CPI_response_pctpt", "CPI响应"),
1346     ("IAV_response_pctpt", "IAV响应"),
1347     ("cumulative_deferred_gap_cny_t", "累计缺口"),
1348 ]
1349 for ax, (column, ylabel) in zip(axes.ravel(), panels):
1350     for strategy, sub in frame.groupby("strategy"):
1351         sub = sub.sort_values("month_index")
1352         ax.plot(sub["month_index"], sub[column], marker="o", label=labels.get(strategy, strategy))
1353         ax.axhline(0, color=PALETTE["muted"], linewidth=0.7)
1354         style_axis(ax, ylabel=ylabel)
1355 axes[-1, 0].set_xlabel("冲击后月份")
1356 axes[-1, 1].set_xlabel("冲击后月份")
1357 handles, legend_labels = axes[0, 0].get_legend_handles_labels()
1358 fig.legend(handles, legend_labels, loc="upper center", ncol=5, frameon=False)
1359 finish_figure(
1360     fig,
1361     title="问题四: 2026冲击下策略路径对比",
1362     subtitle="宏观路径由Q3燃油价格响应核传播, 累计缺口为价格平滑延期负担代理。",
1363     source="来源: results/q4_sapr_macro_paths.csv; 作者计算。",
1364     rect=(0.08, 0.08, 0.98, 0.92),
1365 )
1366 save_figure(fig, FIGURES_DIR / "q4_sapr_2026_macro_paths")
1367 plt.close(fig)
1368
1369
1370 def write_summary(
1371     scenario_table: pd.DataFrame,
1372     grid: pd.DataFrame,
1373     optimal: pd.DataFrame,
1374     comparison: pd.DataFrame,
1375     sensitivity: pd.DataFrame,
1376     identity: dict[str, Any],
1377     valid_draw_rate: float,
1378     total_draw_attempts: int,
1379     valid_draw_count: int,
1380     holdout_probability: float,
1381     holdout_point_non_dominated: bool,
1382 ) -> dict[str, Any]:
1383     validation_path = RESULTS_DIR / "q4_sapr_holdout_validation_summary.json"
1384     validation = json.loads(validation_path.read_text(encoding="utf-8")) if validation_path.exists() else {}
1385     knee_rows = comparison.loc[comparison["strategy"].eq("SAPR_CVaR_knee")]
1386     knee_feasible_all_splits = bool(
1387         not knee_rows.empty
1388         and knee_rows["max_gap_ratio"].le(MAX_GAP_RATIO).all()
1389         and knee_rows["max_terminal_gap_ratio"].le(MAX_TERMINAL_GAP_RATIO).all()
1390         and knee_rows["max_recovery_terminal_gap_ratio"].le(MAX_RECOVERY_TERMINAL_GAP_RATIO).all()
1391         and knee_rows["max_monthly_adjustment_ratio"].le(MAX_MONTHLY_ADJUSTMENT_RATIO).all()
1392     )
1393     supported_improvements = int(validation.get("supported_improvement_count", 0))
1394     near_oracle_probability = float(validation.get("probability_within_0_10_of_oracle", 0.0))
1395     if knee_feasible_all_splits and supported_improvements >= 1 and near_oracle_probability >= 0.80:
1396         evidence_status = "SUPPORTED"
1397     elif knee_feasible_all_splits and holdout_point_non_dominated:
1398         evidence_status = "PARTIAL"
1399     else:
1400         evidence_status = "NOT_SUPPORTED"
1401     required_interval_cols = [
1402         "J1_macro_loss_lower_95",
1403         "J1_macro_loss_upper_95",
1404         "J2_cvar95_macro_loss_lower_95",
1405         "J2_cvar95_macro_loss_upper_95",
1406     ]
1407     has_complete_intervals = bool(
1408         set(required_interval_cols).issubset(comparison.columns)
1409         and np.isfinite(comparison[required_interval_cols].astype(float).to_numpy()).all()
1410     )
1411     execution_gates = {
1412         "q4_sapr_scenario_no_leakage_gate": bool(
1413             not scenario_table.empty
1414             and scenario_table.loc[scenario_table["sample_split"].eq("development"), "source_end"].max() <= DEVELOPMENT_END
1415             and scenario_table.loc[scenario_table["sample_split"].eq("holdout"), "source_start"].min() >= HOLDOUT_START
1416         ),

```

```

1417     "q4_sapr_macro_kernel_gate": bool(valid_draw_count == BOOTSTRAP_DRAWS and valid_draw_rate >= 0.95),
1418     "q4_sapr_policy_identity_gate": bool(
1419         identity["full_rule_max_abs_gap"] < 1e-8
1420         and identity["full_rule_adjustment_identity_max_abs_error"] < 1e-8
1421         and identity["zero_policy_macro_max_abs_response"] < 1e-10
1422         and abs(identity["official_2026_03_gap"] - 1045.0) < 1e-8
1423         and abs(identity["official_2026_04_gap"] - 380.0) < 1e-8
1424         and abs(identity["official_2026_04_cumulative_gap"] - 1425.0) < 1e-8
1425         and identity["q3_policy_counterfactual_reproduction_max_abs_error"] < 1e-8
1426         and identity["negative_adjustment_offsets_gap"]
1427     ),
1428     "q4_sapr_pareto_selection_gate": bool(len(grid) == 1771 and int(grid["is_pareto"].sum()) > 0 and int(grid["is_knee"].sum()) == 1)
1429 ,
1430     "q4_sapr_policy_feasibility_gate": knee_feasible_all_splits,
1431     "q4_sapr_holdout_validation_gate": bool(
1432         comparison["sample_split"].astype(str).eq("holdout").any()
1433         and "SAPR-CVaR_knee" in set(comparison["strategy"])
1434         and evidence_status in {"SUPPORTED", "PARTIAL", "NOT_SUPPORTED"}
1435     ),
1436     "q4_sapr_uncertainty_gate": bool(valid_draw_count == BOOTSTRAP_DRAWS and valid_draw_rate >= 0.95 and has_complete_intervals),
1437     "q4_sapr_claim_strength_gate": True,
1438 }
1439 execution_status = "PASS" if all(execution_gates.values()) else "FAIL"
1440 knee = optimal.iloc[0].to_dict()
1441 payload = {
1442     "status": execution_status,
1443     "execution_status": execution_status,
1444     "evidence_status": evidence_status,
1445     "model_name": "SAPR-CVaR",
1446     "cutoff": CUTOFF,
1447     "random_seed": RANDOM_SEED,
1448     "bootstrap_attempts": BOOTSTRAP_DRAWS,
1449     "bootstrap_total_draw_attempts": total_draw_attempts,
1450     "valid_parameter_draw_count": valid_draw_count,
1451     "valid_parameter_draw_rate": valid_draw_rate,
1452     "parameter_sampling": "joint three-equation circular moving-block bootstrap inherited from Q3; identical time blocks for PPI, CPI
and IAV",
1453     "holdout_non_dominated_probability": holdout_probability,
1454     "holdout_point_non_dominated": holdout_point_non_dominated,
1455     "holdout_validation": validation,
1456     "scenario_counts": scenario_table.groupby("sample_split")["scenario_id"].nunique().to_dict(),
1457     "rule_count": int(len(grid)),
1458     "pareto_rule_count": int(grid["is_pareto"].sum()),
1459     "optimal_rule": {key: clean_value(value) for key, value in knee.items()},
1460     "identity_checks": identity,
1461     "execution_gates": execution_gates,
1462     "allowed_claims": [
1463         "SAPR-CVaR is optimal only within the registered three-regime smoothing family, six-month scenario library and four-objective
criterion.",
1464         "The selected rule can be compared with full pass-through, fixed smoothing and a 2026 temporary-control approximation under
the frozen Q3 macro kernel.",
1465         "J3 is the average monthly outstanding adjustment gap divided by the base price; terminal and recovery gaps are separate hard
constraints.",
1466     ],
1467     "forbidden_claims": [
1468         "global optimum across all possible Chinese fuel pricing policies",
1469         "complete welfare gain or fiscal cost estimate",
1470         "the deferred gap equals government subsidy spending or a complete welfare loss",
1471         "Q4 uses post-2021 data to select stress thresholds or the optimal rule",
1472     ],
1473     "sensitivity_rows": int(len(sensitivity)),
1474     "figure_files": [
1475         "figures/q4_sapr_pareto_front.png",
1476         "figures/q4_sapr_policy_heatmap.png",
1477         "figures/q4_sapr_strategy_comparison.png",
1478         "figures/q4_sapr_2026_macro_paths.png",
1479     ],
1480 }
1481 (RESULTS_DIR / "q4_sapr_summary.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False) + "\n", encoding
="utf-8")
1482 return payload
1483
1484 def main(argv: list[str] | None = None) -> int:
1485     parser = argparse.ArgumentParser()
1486     parser.add_argument("--plots-only", action="store_true", help="Regenerate Q4 figures from saved result tables.")

```

```

1487 args = parser.parse_args(argv)
1488 apply_paper_style()
1489 ensure_dirs()
1490
1491 if args.plots_only:
1492     grid = read_result("q4_sapr_policy_grid.csv")
1493     optimal = read_result("q4_sapr_optimal_rule.csv")
1494     comparison = read_result("q4_sapr_strategy_comparison.csv")
1495     paths = read_result("q4_sapr_macro_paths.csv")
1496     knee_rule = {
1497         "rho_normal": float(optimal["rho_normal"].iloc[0]),
1498         "rho_stress": float(optimal["rho_stress"].iloc[0]),
1499         "rho_extreme": float(optimal["rho_extreme"].iloc[0]),
1500     }
1501     plot_pareto(grid)
1502     plot_policy_heatmap(knee_rule)
1503     plot_strategy_comparison(comparison)
1504     plot_2026_paths(paths)
1505     print(json.dumps({"status": "PASS", "mode": "plots-only", "figures": 4}, ensure_ascii=False, indent=2))
1506     return 0
1507
1508 rng = np.random.default_rng(RANDOM_SEED)
1509 scenario_table, scenarios, meta = build_scenarios()
1510 save_csv(scenario_table, "q4_sapr_scenarios.csv")
1511 kernel = load_macro_kernel()
1512 parameter_draws, valid_draw_rate, total_draw_attempts = draw_parameter_sets(kernel, rng)
1513 if len(parameter_draws) != BOOTSTRAP_DRAWS:
1514     raise ValueError(
1515         f"Q4 generated {len(parameter_draws)} valid macro-parameter draws from {total_draw_attempts} attempts; "
1516         f"{BOOTSTRAP_DRAWS} valid draws are required."
1517     )
1518 point = point_coefficients(kernel)
1519 weights = {"china_ppi_yoy_pct": 1.0, "china_cpi_yoy_pct": 1.0, "china_iav_yoy_pct": 1.0}
1520 stds = historical_macro_stds(weights)
1521 base_price = float(meta["base_price_2026_06_cny_t"])
1522 grid = run_grid_search(scenarios, point, base_price, stds, weights)
1523 for key, value in meta.items():
1524     grid[key] = value
1525 save_csv(grid, "q4_sapr_policy_grid.csv")
1526 knee = grid.loc[grid["is_knee"]].iloc[0]
1527 knee_rule = {
1528     "rho_normal": float(knee["rho_normal"]),
1529     "rho_stress": float(knee["rho_stress"]),
1530     "rho_extreme": float(knee["rho_extreme"]),
1531 }
1532 optimal = pd.DataFrame(
1533     [
1534         {
1535             "model": "SAPR-CVaR",
1536             "rule_id": knee["rule_id"],
1537             "rho_normal": knee_rule["rho_normal"],
1538             "rho_stress": knee_rule["rho_stress"],
1539             "rho_extreme": knee_rule["rho_extreme"],
1540             "stress_threshold_75_cny_t": meta["stress_threshold_75_cny_t"],
1541             "stress_threshold_95_cny_t": meta["stress_threshold_95_cny_t"],
1542             "base_price_2026_06_cny_t": base_price,
1543             "J1_macro_loss": knee["J1_macro_loss"],
1544             "J2_cvar95_macro_loss": knee["J2_cvar95_macro_loss"],
1545             "J3_avg_gap_month_burden": knee["J3_avg_gap_month_burden"],
1546             "J4_adjustment_volatility": knee["J4_adjustment_volatility"],
1547             "max_gap_ratio": knee["max_gap_ratio"],
1548             "max_terminal_gap_ratio": knee["max_terminal_gap_ratio"],
1549             "max_recovery_terminal_gap_ratio": knee["max_recovery_terminal_gap_ratio"],
1550             "max_monthly_adjustment_ratio": knee["max_monthly_adjustment_ratio"],
1551             "constraint_max_gap_ratio": MAX_GAP_RATIO,
1552             "constraint_max_terminal_gap_ratio": MAX_TERMINAL_GAP_RATIO,
1553             "constraint_max_recovery_terminal_gap_ratio": MAX_RECOVERY_TERMINAL_GAP_RATIO,
1554             "constraint_max_monthly_adjustment_ratio": MAX_MONTHLY_ADJUSTMENT_RATIO,
1555             "recovery_months": RECOVERY_MONTHS,
1556             "pareto_rule_count": int(grid["is_pareto"].sum()),
1557             "feasible_rule_count": int(grid["is_feasible"].sum()),
1558             "candidate_rule_count": int(len(grid)),
1559             "selection_rule": "minimum normalized Euclidean distance to the feasible Pareto ideal; ties by smaller average gap-month
burden and higher normal pass-through",
1560         }
1561     ]

```

```

1562 )
1563 save_csv(optimal, "q4_sapr_optimal_rule.csv")
1564 comparison, holdout_probability, holdout_point_non_dominated = compare_strategies(
1565     scenarios,
1566     kernel,
1567     parameter_draws,
1568     knee_rule,
1569     base_price,
1570     stds,
1571     weights,
1572     rng,
1573     block_length=6,
1574 )
1575 save_csv(comparison, "q4_sapr_strategy_comparison.csv")
1576 paths = aggregate_macro_paths(scenarios, kernel, parameter_draws, knee_rule, base_price, split="war_2026")
1577 save_csv(paths, "q4_sapr_macro_paths.csv")
1578 sensitivity = run_sensitivity(grid, scenarios, kernel, parameter_draws[:500], meta, stds, rng)
1579 save_csv(sensitivity, "q4_sapr_sensitivity.csv")
1580 identity = policy_identity_checks(scenarios, base_price, kernel)
1581 summary = write_summary(
1582     scenario_table,
1583     grid,
1584     optimal,
1585     comparison,
1586     sensitivity,
1587     identity,
1588     valid_draw_rate,
1589     total_draw_attempts,
1590     len(parameter_draws),
1591     holdout_probability,
1592     holdout_point_non_dominated,
1593 )
1594 plot_pareto(grid)
1595 plot_policy_heatmap(knee_rule)
1596 plot_strategy_comparison(comparison)
1597 plot_2026_paths(paths)
1598 print(json.dumps(summary, ensure_ascii=False, indent=2))
1599 return 0 if summary["execution_status"] == "PASS" else 1
1600
1601
1602 if __name__ == "__main__":
1603     raise SystemExit(main())

```

Listing 10: run_q4_sapr.py

3.3 数值冻结与绘图工具

```

1 """Paper-oriented Matplotlib style helpers for the oil-shock figures."""
2
3 from __future__ import annotations
4
5 from datetime import datetime, timezone
6 from pathlib import Path
7
8 import matplotlib.pyplot as plt
9 from cycler import cycler
10
11
12 INK = "#232323"
13 MUTED = "#606060"
14 GRID = "#d9d4ca"
15 SPINE = "#4a4a4a"
16
17 PALETTE = {
18     "ink": INK,
19     "muted": MUTED,
20     "grid": GRID,
21     "blue": "#355c7d",
22     "blue_light": "#9fb4c9",
23     "gold": "#b98b3d",
24     "gold_light": "#d8c49a",
25     "olive": "#6f7f4f",

```

```

26     "olive_light": "#b8c1a3",
27     "rose": "#a65d6a",
28     "rose_light": "#d4a7af",
29     "slate": "#6f7582",
30     "sand": "#c9b99a",
31 }
32
33 SERIES_COLORS = [
34     PALETTE["blue"],
35     PALETTE["gold"],
36     PALETTE["olive"],
37     PALETTE["rose"],
38     PALETTE["slate"],
39 ]
40
41
42 def apply_paper_style() -> None:
43     plt.rcParams.update(
44         {
45             "figure.facecolor": "white",
46             "axes.facecolor": "white",
47             "savefig.facecolor": "white",
48             "savefig.edgecolor": "white",
49             "font.family": ["Times New Roman", "SimSun", "FangSong", "DejaVu Serif"],
50             "font.serif": ["Times New Roman", "SimSun", "FangSong", "STIXGeneral", "DejaVu Serif"],
51             "mathtext.fontset": "stix",
52             "axes.unicode_minus": False,
53             "text.color": INK,
54             "axes.labelcolor": INK,
55             "axes.edgecolor": SPINE,
56             "axes.linewidth": 0.8,
57             "axes.spines.top": False,
58             "axes.spines.right": False,
59             "axes.grid": True,
60             "axes.axisbelow": True,
61             "grid.color": GRID,
62             "grid.linewidth": 0.55,
63             "grid.alpha": 0.75,
64             "xtick.color": MUTED,
65             "ytick.color": MUTED,
66             "xtick.major.width": 0.65,
67             "ytick.major.width": 0.65,
68             "xtick.labelsize": 9.5,
69             "ytick.labelsize": 9.5,
70             "axes.labelsize": 10.5,
71             "axes.titlesize": 13,
72             "legend.fontsize": 9.2,
73             "legend.frameon": False,
74             "lines.linewidth": 1.65,
75             "lines.markersize": 4.2,
76             "patch.edgecolor": "white",
77             "axes.prop_cycle": cycler(color=SERIES_COLORS),
78             "pdf.fonttype": 42,
79             "ps.fonttype": 42,
80         }
81     )
82
83
84 def style_axis(ax: plt.Axes, *, ylabel: str | None = None, xlabel: str | None = None) -> None:
85     if ylabel:
86         ax.set_ylabel(ylabel)
87     if xlabel:
88         ax.set_xlabel(xlabel)
89     ax.grid(True, axis="y")
90     ax.grid(False, axis="x")
91     ax.tick_params(length=3.2, width=0.65)
92     for side in ["left", "bottom"]:
93         ax.spines[side].set_color(SPINE)
94         ax.spines[side].set_linewidth(0.8)
95
96
97 def finish_figure(
98     fig: plt.Figure,
99     *,
100     title: str,
101     subtitle: str,

```

```

102     source: str,
103     rect: tuple[float, float, float, float] = (0.08, 0.11, 0.98, 0.98),
104 ) -> None:
105     """Finish a paper figure while leaving its title to the LaTeX caption.
106
107     ``title`` and ``subtitle`` remain required as generation metadata for the
108     calling code, but they are deliberately not painted into the figure.
109     """
110     fig.tight_layout(rect=rect)
111     fig._shu_caption = title # type: ignore[attr-defined]
112     fig._shu_subtitle = subtitle # type: ignore[attr-defined]
113     fig.text(rect[0], 0.035, source, ha="left", va="bottom", fontsize=8.4, color=MUTED)
114
115
116 def save_figure(fig: plt.Figure, path_stem: Path, *, dpi: int = 300) -> None:
117     fig.savefig(path_stem.with_suffix(".png"), dpi=dpi, bbox_inches="tight", pad_inches=0.08)
118     fixed_time = datetime(2026, 7, 30, tzinfo=timezone.utc)
119     pdf_metadata = {
120         "Creator": "SHU-OilShock-CN",
121         "Producer": "Matplotlib",
122         "CreationDate": fixed_time,
123         "ModDate": fixed_time,
124     }
125     fig.savefig(
126         path_stem.with_suffix(".pdf"),
127         dpi=dpi,
128         bbox_inches="tight",
129         pad_inches=0.08,
130         metadata=pdf_metadata,
131     )

```

Listing 11: plot_style.py

```

1 """Generate paper handoff tables and supplemental paper-ready figures.
2
3 This script reads the frozen modeling outputs and creates artifacts intended
4 for the writing stage. It does not estimate models or change any numerical
5 result used by the freeze gate.
6 """
7
8 from __future__ import annotations
9
10 import json
11 import sys
12 from pathlib import Path
13
14 import matplotlib.patches as patches
15 import matplotlib.pyplot as plt
16 import numpy as np
17 import pandas as pd
18
19 REPO_ROOT = Path(__file__).resolve().parents[2]
20 RESULTS_DIR = REPO_ROOT / "results"
21 REPORTS_DIR = REPO_ROOT / "reports"
22 FIGURES_DIR = REPO_ROOT / "figures"
23
24 sys.path.append(str(REPO_ROOT / "code" / "utils"))
25 from plot_style import PALETTE, apply_paper_style, finish_figure, save_figure # noqa: E402
26
27
28 COUNTRY_LABEL_ZH = {
29     "CHN": "中国",
30     "DEU": "德国",
31     "FRA": "法国",
32     "ITA": "意大利",
33     "ESP": "西班牙",
34     "JPN": "日本",
35     "KOR": "韩国",
36 }
37
38 OUTCOME_LABEL_ZH = {
39     "brent_cny_cost_log_change_pct": "人民币原油成本",
40     "china_ppi_yoy_pct": "PPI",
41     "china_cpi_yoy_pct": "CPI",
42     "china_iav_yoy_pct": "工业增加值",

```



```

43     "china_fx_log_change_pct": "汇率",
44 }
45
46 SHOCK_LABEL_ZH = {
47     "supply_shock": "供给冲击",
48     "aggregate_demand_shock": "全球需求冲击",
49     "oil_specific_risk_shock": "油价特定风险冲击",
50     "OilShock": "约化油价创新",
51 }
52
53
54 def read_csv(name: str) -> pd.DataFrame:
55     return pd.read_csv(RESULTS_DIR / name)
56
57
58 def read_json(name: str) -> dict:
59     return json.loads((RESULTS_DIR / name).read_text(encoding="utf-8"))
60
61
62 def zh_outcome(value: str) -> str:
63     return OUTCOME_LABEL_ZH.get(value, value)
64
65
66 def write_paper_numbers() -> None:
67     risk = read_json("risk_probe_summary.json")
68     q1_origin = read_csv("q1_origin_forecast.csv")
69     q1_event = read_csv("q1_event_effects.csv")
70     q1_structural = read_csv("q1_structural_shocks.csv")
71     q1_vol = read_csv("q1_volatility_summary.csv")
72     q2_metrics = read_csv("q2_transmission_metrics.csv")
73     q2_gdp = read_csv("q2_gdp_validation.csv")
74     q3_pass = read_csv("q3_country_pass_through.csv")
75     q3_policy = read_csv("q3_policy_counterfactual.csv")
76     q3_policy_macro = read_csv("q3_policy_macro_counterfactual.csv")
77     q3_resilience = read_csv("q3_resilience_metrics.csv")
78     q4_risk = read_csv("q4_price_tail_risk.csv")
79     q4_backtest = read_csv("q4_risk_backtest.csv")
80     q4_macro = read_csv("q4_macro_stress.csv")
81     q4_policy = read_csv("q4_policy_stress.csv")
82     q4_sapr_optimal = read_csv("q4_sapr_optimal_rule.csv")
83     q4_sapr_comparison = read_csv("q4_sapr_strategy_comparison.csv")
84     q4_sapr_sensitivity = read_csv("q4_sapr_sensitivity.csv")
85     q4_sapr_summary = read_json("q4_sapr_summary.json")
86
87     rows: list[dict[str, object]] = []
88
89     def add(section: str, number_id: str, value: object, unit: str, source_file: str, allowed: str, forbidden: str) -> None:
90         rows.append(
91             {
92                 "section": section,
93                 "number_id": number_id,
94                 "value": value,
95                 "unit": unit,
96                 "rounding": "论文正文保留2到3位有效数字；表格按来源文件精度",
97                 "source_file": source_file,
98                 "allowed_claim": allowed,
99                 "forbidden_claim": forbidden,
100             }
101         )
102
103     add(
104         "overall",
105         "paper_finalize_allowed",
106         str(risk["paper_finalize_allowed"]).lower(),
107         "boolean",
108         "results/risk_probe_summary.json",
109         "所有核心门禁通过后才可进入论文撰写",
110         "不得忽略后续新增数据或代码变动后的重新冻结",
111     )
112     add(
113         "overall",
114         "blocking_probe_count",
115         len(risk["blocking_probe_ids"]),
116         "count",
117         "results/risk_probe_summary.json",
118         "当前无阻塞门禁",

```

```

119     "不得把单个模型的显著性等同于全部结论成立",
120 )
121
122 for _, row in q1_origin.sort_values("horizon_months").iterrows():
123     add(
124         "Q1",
125         f"origin_2026_02_forecast_h{int(row['horizon_months'])}",
126         round(float(row["prediction_price"]), 3),
127         "美元/桶",
128         "results/q1_origin_forecast.csv",
129         f"{row['target_period']} 的 no-change 预测价格",
130         "不得把未到期的2026-08预测写成有实际误差",
131     )
132 for _, row in q1_event[q1_event["model"].eq("brent_usd_bbl_event_car")].iterrows():
133     add(
134         "Q1",
135         f"{row['stage_id']}_estimate",
136         round(float(row["estimate_log_return"]), 6),
137         "log return",
138         "results/q1_event_effects.csv",
139         "交易日CAR与经验placebo可作为事件相关异常证据",
140         "不得解释为严格战争净因果贡献",
141     )
142     add(
143         "Q1",
144         f"{row['stage_id']}_empirical_p",
145         round(float(row["pvalue_empirical"]), 6),
146         "probability",
147         "results/q1_event_effects.csv",
148         "经验p值使用有限样本add-one修正",
149         "不得报告0或只引用常规HAC显著性",
150     )
151 add(
152     "Q1",
153     "structural_shock_nonmissing_months_min",
154     int(q1_structural[["supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock"]].notna().sum().min()),
155     "months",
156     "results/q1_structural_shocks.csv",
157     "三类SVAR结构冲击满足不少于120个月覆盖",
158     "不得再使用52期STEO残差作为主结构识别",
159 )
160 for _, row in q1_vol.iterrows():
161     add(
162         "Q1",
163         f"vol_multiple_{row['war_stage']}",
164         round(float(row["vol_multiple_vs_pre_median"]), 3),
165         "倍",
166         "results/q1_volatility_summary.csv",
167         "条件波动率可描述战争阶段油价波动变化",
168         "不得解释为福利损失或实体经济损失",
169     )
170
171 focus = q2_metrics[
172     q2_metrics["shock"].eq("oil_specific_risk_shock")
173     & q2_metrics["outcome"].isin(["brent_cny_cost_log_change_pct", "china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"])
174 ]
175 for _, row in focus.iterrows():
176     add(
177         "Q2",
178         f"oil_specific_{row['outcome']}_extremum",
179         round(float(row["extremum_response"]), 6),
180         "百分点或对数百分点",
181         "results/q2_transmission_metrics.csv",
182         f"{zh_outcome(row['outcome'])} 的响应方向和量级可报告为估计结果",
183         "联合区间未排除零时不得写成稳健增长损失",
184     )
185     if pd.notna(row.get("response_curve_area_0_12")):
186         add(
187             "Q2",
188             f"oil_specific_{row['outcome']}_area0_12",
189             round(float(row["response_curve_area_0_12"]), 6),
190             "百分点·月",
191             "results/q2_transmission_metrics.csv",
192             f"{zh_outcome(row['outcome'])} 的0到12月响应曲线面积可报告为描述性量级",
193             "不得称为累计百分点、累计产出损失或累计增长损失",
194         )

```

```

195 if not q2_gdp.empty:
196     gdp = q2_gdp.iloc[0]
197     add(
198         "Q2",
199         "quarterly_gdp_validation_estimate",
200         round(float(gdp["estimate"]), 6),
201         "百分点",
202         "results/q2_gdp_validation.csv",
203         "季度GDP仅作参考验证",
204         "不得插值为月度主增长指标",
205     )
206
207 h6 = q3_pass[q3_pass["horizon"].eq(6)]
208 for _, row in h6.sort_values("country").iterrows():
209     add(
210         "Q3",
211         f"fuel_pass_through_h6_{row['country']}",
212         round(float(row["response"]), 6),
213         "累计传导率",
214         "results/q3_country_pass_through.csv",
215         f"{COUNTRY_LABEL_ZH.get(row['country'], row['country'])} 可进入主燃油比较",
216         "不得混用中国Brent-CNY代理值与他国观测零售价排名",
217     )
218 apr = q3_policy[q3_policy["period"].eq("2026-04")]
219 if not apr.empty:
220     apr0 = apr.iloc[0]
221     add(
222         "Q3",
223         "policy_gap_2026_04_incremental",
224         round(float(apr0["incremental_gasoline_gap_cny_t"]), 3),
225         "元/吨",
226         "results/q3_policy_counterfactual.csv",
227         "4月新增调控差额可写为380元/吨",
228         "不得把新增差额误写成累计差额",
229     )
230     add(
231         "Q3",
232         "policy_gap_2026_04_cumulative",
233         round(float(apr0["cumulative_gasoline_gap_cny_t"]), 3),
234         "元/吨",
235         "results/q3_policy_counterfactual.csv",
236         "4月累计规则缺口可写为1425元/吨",
237         "不得把价格平滑解释为无成本福利改善",
238     )
239 june_macro = q3_policy_macro[q3_policy_macro["period"].eq("2026-06")]
240 for _, row in june_macro.sort_values("outcome").iterrows():
241     add(
242         "Q3",
243         f"policy_macro_gap_2026_06_{row['outcome']}",
244         round(float(row["macro_counterfactual_gap_pctpt"]), 6),
245         "百分点",
246         "results/q3_policy_macro_counterfactual.csv",
247         f"无临时调控路径对{row['outcome_label']}的情景差额可作为政策关闭反事实",
248         "不得解释为已发生事实或完整福利评价",
249     )
250 overall = q3_resilience["overall_china_resilience_judgement"].dropna().iloc[0]
251 add(
252     "Q3",
253     "overall_china_resilience_judgement",
254     overall,
255     "categorical",
256     "results/q3_resilience_metrics.csv",
257     "中国综合韧性当前判断为PARTIAL",
258     "不得写成无条件SUPPORTED",
259 )
260
261 for _, row in q4_risk[q4_risk["model"].eq("FHS_GJR_GARCH")].sort_values("horizon_months").iterrows():
262     horizon = int(row["horizon_months"])
263     add(
264         "Q4",
265         f"fhs_median_price_h{horizon}",
266         round(float(row["median_price"]), 6),
267         "美元/桶",
268         "results/q4_price_tail_risk.csv",
269         "可作为2026-06-30信息集下的条件分布中位数",
270         "不得写成确定性油价路径",

```

```

271     )
272     add(
273         "Q4",
274         f"fhs_terminal_prob_above_hist_p95_h{horizon}",
275         round(float(row["terminal_prob_above_hist_p95"]), 6),
276         "probability",
277         "results/q4_price_tail_risk.csv",
278         "可报告期末价格超过历史95%分位阈值的条件概率",
279         "不得解释为事件必然发生概率或因果概率",
280     )
281 for _, row in q4_backtest.sort_values(["horizon_months", "model"]).iterrows():
282     add(
283         "Q4",
284         f"backtest_pinball_{row['model']}_h{int(row['horizon_months'])}",
285         round(float(row["mean_pinball_loss"]), 6),
286         "美元/桶分位损失",
287         "results/q4_risk_backtest.csv",
288         "主方法与基线按相同滚动原点和期限比较",
289         "不得只报告有利期限或隐藏基线",
290     )
291 for _, row in q4_macro[
292     q4_macro["scenario"].eq("extreme_q95") & q4_macro["horizon"].isin([6, 12])
293 ].sort_values(["outcome", "horizon"]).iterrows():
294     add(
295         "Q4",
296         f"extreme_q95_{row['outcome']}_h{int(row['horizon'])}",
297         round(float(row["conditional_response_pctpt"]), 6),
298         "百分点",
299         "results/q4_macro_stress.csv",
300         "可作为历史95%分位油价特定风险冲击下的条件响应",
301         "联合区间跨零时不得写成确定性宏观损失",
302     )
303 for _, row in q4_policy[q4_policy["period"].eq("2026-06")].sort_values("outcome").iterrows():
304     add(
305         "Q4",
306         f"policy_buffer_benefit_2026_06_{row['outcome']}",
307         round(float(row["policy_buffer_benefit_pctpt"]), 6),
308         "百分点",
309         "results/q4_policy_stress.csv",
310         "可作为Q3已实现临时调控关闭反事实的政策缓冲收益重述",
311         "不得外推到Q4模拟油价路径或写成完整福利收益",
312     )
313
314 if not q4_sapr_optimal.empty:
315     opt = q4_sapr_optimal.iloc[0]
316     for key, unit, claim in [
317         ("rho_normal", "pass-through ratio", "普通状态最优传导率"),
318         ("rho_stress", "pass-through ratio", "压力状态最优传导率"),
319         ("rho_extreme", "pass-through ratio", "极端状态最优传导率"),
320         ("J2_cvar95_macro_loss", "standardized loss", "训练样本膝点规则的95%尾部宏观损失"),
321         ("J3_avg_gap_month_burden", "ratio", "训练样本膝点规则的平均标准化缺口月负担"),
322     ]:
323         add(
324             "Q4",
325             f"sapr_optimal_{key}",
326             round(float(opt[key]), 6),
327             unit,
328             "results/q4_sapr_optimal_rule.csv",
329             claim,
330             "不得写成全局福利最优或不受规则族限制的最优政策",
331         )
332     sapr_holdout = q4_sapr_comparison[
333         q4_sapr_comparison["sample_split"].eq("holdout")
334         & q4_sapr_comparison["strategy"].eq("SAPR_CVaR_knee")
335     ]
336 if not sapr_holdout.empty:
337     row = sapr_holdout.iloc[0]
338     add(
339         "Q4",
340         "sapr_holdout_macro_loss_cvar95",
341         round(float(row["J2_cvar95_macro_loss"]), 6),
342         "standardized loss",
343         "results/q4_sapr_strategy_comparison.csv",
344         "可作为2022—2026隔离检验样本下SAPR尾部宏观损失",
345         "不得声称2026以后仍必然占优",
346     )

```

```

347         add(
348             "Q4",
349             "sapr_holdout_gap_burden_ratio",
350             round(float(row["J3_avg_gap_month_burden"]), 6),
351             "ratio",
352             "results/q4_sapr_strategy_comparison.csv",
353             "可作为隔离检验样本下平均标准化缺口月负担",
354             "不得解释为财政支出或完整社会成本",
355         )
356     add(
357         "Q4",
358         "sapr_holdout_non_dominated_probability",
359         round(float(q4_sapr_summary["holdout_non_dominated_probability"]), 6),
360         "probability",
361         "results/q4_sapr_summary.json",
362         "可报告训练期选出规则在检验期保持非支配的bootstrap概率",
363         "不得改写成真实世界政策成功概率",
364     )
365     for _, row in q4_sapr_sensitivity.sort_values("variant").iterrows():
366         add(
367             "Q4",
368             f"sapr_sensitivity_{row['variant']}_rule",
369             f"({row['rho_normal']:.2f},{row['rho_stress']:.2f},{row['rho_extreme']:.2f})",
370             "rule tuple",
371             "results/q4_sapr_sensitivity.csv",
372             "可用于说明阈值、块长和权重变动下规则是否稳定",
373             "不得事后删除不利敏感性结果",
374         )
375
376     pd.DataFrame(rows).to_csv(REPORTS_DIR / "paper_numbers.csv", index=False, encoding="utf-8-sig")
377
378
379 def plot_q1_structural_shocks() -> None:
380     df = read_csv("q1_structural_shocks.csv")
381     df["period"] = pd.to_datetime(df["period"])
382     columns = ["supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock"]
383     labels = ["供给冲击", "全球需求冲击", "油价特定风险冲击"]
384     fig, ax = plt.subplots(figsize=(8.4, 4.2))
385     for col, label in zip(columns, labels):
386         series = df[col].rolling(3, min_periods=1).mean()
387         ax.plot(df["period"], series, label=label)
388     ax.axhline(0, color=PALETTE["muted"], linewidth=0.8)
389     ax.set_ylabel("标准化冲击")
390     ax.legend(ncol=3, loc="upper left")
391     finish_figure(
392         fig,
393         title="问题一：历史递归SVAR结构冲击",
394         subtitle="三个月滚动均值，仅用于识别与传导链输入展示",
395         source="来源：EIA International、Dallas Fed IGREA、Brent、美国CPI；作者计算。",
396     )
397     save_figure(fig, FIGURES_DIR / "q1_structural_shocks")
398     plt.close(fig)
399
400
401 def plot_q2_transmission_chain() -> None:
402     df = read_csv("q2_transmission_metrics.csv")
403     focus = df[
404         df["shock"].eq("oil_specific_risk_shock")
405         & df["outcome"].isin(
406             [
407                 "brent_cny_cost_log_change_pct",
408                 "china_ppi_yoy_pct",
409                 "china_cpi_yoy_pct",
410                 "china_iav_yoy_pct",
411                 "china_fx_log_change_pct",
412             ]
413         )
414     ].copy()
415     focus["label"] = focus["outcome"].map(OUTCOME_LABEL_ZH)
416     focus = focus.sort_values("extremum_response")
417     fig, ax = plt.subplots(figsize=(7.8, 4.6))
418     colors = [PALETTE["rose"] if v < 0 else PALETTE["blue"] for v in focus["extremum_response"]]
419     ax.barh(focus["label"], focus["extremum_response"], color=colors, alpha=0.88)
420     ax.axvline(0, color=PALETTE["muted"], linewidth=0.8)
421     for _, row in focus.iterrows():
422         ax.text(

```

```

423     row["extremum_response"],
424     row["label"],
425     f" {row['extremum_response']:.2f}, h={int(row['extremum_month'])}",
426     va="center",
427     fontsize=9,
428     color=PALETTE["ink"],
429 )
430 ax.set_xlabel("峰值/谷值响应 (百分点或对数百分点)")
431 finish_figure(
432     fig,
433     title="问题二: 油价特定风险冲击的传导摘要",
434     subtitle="显示各变量峰值或谷值响应: 完整区间以结果表为准",
435     source="来源: results/q2_transmission_metrics.csv.",
436 )
437 save_figure(fig, FIGURES_DIR / "q2_transmission_chain")
438 plt.close(fig)
439
440
441 def plot_q3_resilience_metrics() -> None:
442     df = read_csv("q3_resilience_metrics.csv")
443     focus = df[df["dimension"].isin(["fuel_pass_through", "cpi_peak_response", "industrial_activity_trough"]).copy()
444     label_map = {
445         "fuel_1m_cumulative_pass_through": "燃油1月传导",
446         "fuel_3m_cumulative_pass_through": "燃油3月传导",
447         "fuel_6m_cumulative_pass_through": "燃油6月传导",
448         "cpi_relative_to_china": "CPI相对响应",
449         "ip_relative_to_china": "工业活动相对响应",
450     }
451     focus["label"] = focus["metric"].map(label_map)
452     focus = focus.iloc[::-1]
453     y = np.arange(len(focus))
454     err_left = focus["china_vs_control_median_diff"] - focus["diff_lower_95"]
455     err_right = focus["diff_upper_95"] - focus["china_vs_control_median_diff"]
456     fig, ax = plt.subplots(figsize=(8.2, 4.3))
457     ax.barh(y, focus["china_vs_control_median_diff"], color=PALETTE["olive"], alpha=0.82)
458     ax.errorbar(
459         focus["china_vs_control_median_diff"],
460         y,
461         xerr=[err_left, err_right],
462         fmt="none",
463         ecolor=PALETTE["ink"],
464         elinewidth=0.8,
465         capsize=3,
466     )
467     ax.axvline(0, color=PALETTE["muted"], linewidth=0.8)
468     ax.set_yticks(y)
469     ax.set_yticklabels(focus["label"])
470     ax.set_xlabel("中国相对六国中位数差异")
471     finish_figure(
472         fig,
473         title="问题三: 中国综合韧性指标",
474         subtitle="点估计与95%区间共同决定SUPPORTED/PARTIAL/NOT_SUPPORTED",
475         source="来源: results/q3_resilience_metrics.csv.",
476     )
477     save_figure(fig, FIGURES_DIR / "q3_resilience_metrics")
478     plt.close(fig)
479
480
481 def plot_q3_policy_macro_counterfactual() -> None:
482     df = read_csv("q3_policy_macro_counterfactual.csv")
483     df["period_dt"] = pd.to_datetime(df["period"], format="%Y-%m")
484     label_map = {"PPI": "PPI", "CPI": "CPI", "IAV": "工业增加值"}
485     fig, ax = plt.subplots(figsize=(7.8, 4.5))
486     for label, color in zip(["PPI", "CPI", "IAV"], [PALETTE["blue"], PALETTE["gold"], PALETTE["rose"]]):
487         sub = df[df["outcome_label"].eq(label)].sort_values("period_dt")
488         ax.plot(sub["period_dt"], sub["macro_counterfactual_gap_pctpt"], marker="o", color=color, label=label_map[label])
489         ax.fill_between(sub["period_dt"], sub["lower_95"], sub["upper_95"], color=color, alpha=0.12, linewidth=0)
490     ax.axhline(0, color=PALETTE["muted"], linewidth=0.8)
491     ax.set_ylabel("无临时调控相对实际路径差额 (百分点)")
492     ax.legend(loc="upper left")
493     finish_figure(
494         fig,
495         title="问题三: 政策关闭宏观反事实",
496         subtitle="在官方成品油价格层上移除临时调控缺口后传播至PPI/CPI/IAV",
497         source="来源: results/q3_policy_macro_counterfactual.csv.",
498     )

```

```

499 save_figure(fig, FIGURES_DIR / "q3_policy_macro_counterfactual")
500 plt.close(fig)
501
502
503 def draw_box(ax: plt.Axes, xy: tuple[float, float], text: str, width: float = 0.18, height: float = 0.16, color: str = "#f4efe6") -> None:
504     :
505     box = patches.FancyBboxPatch(
506         xy,
507         width,
508         height,
509         boxstyle="round,pad=0.018,rounding_size=0.02",
510         linewidth=0.9,
511         edgecolor=PALETTE["slate"],
512         facecolor=color,
513     )
514     ax.add_patch(box)
515     ax.text(xy[0] + width / 2, xy[1] + height / 2, text, ha="center", va="center", fontsize=10)
516
517 def arrow(ax: plt.Axes, start: tuple[float, float], end: tuple[float, float]) -> None:
518     ax.annotate(
519         "",
520         xy=end,
521         xytext=start,
522         arrowprops={"arrowstyle": "->", "linewidth": 1.1, "color": PALETTE["muted"]},
523     )
524
525
526 def save_flow(fig: plt.Figure, stem: str, title: str, subtitle: str) -> None:
527     fig.subplots_adjust(left=0.04, right=0.98, bottom=0.18, top=0.92)
528     fig._shu_caption = title # type: ignore[attr-defined]
529     fig._shu_subtitle = subtitle # type: ignore[attr-defined]
530     fig.text(0.04, 0.045, "来源: 作者根据冻结模型流程绘制.", ha="left", va="bottom", fontsize=8.4, color=PALETTE["muted"])
531     save_figure(fig, FIGURES_DIR / stem)
532     plt.close(fig)
533
534
535 def plot_route_map() -> None:
536     mermaid_source = FIGURES_DIR / "paper_route_map.mmd"
537     if mermaid_source.exists():
538         print("SKIP paper_route_map: maintained from Mermaid source figures/paper_route_map.mmd")
539         return
540     fig, ax = plt.subplots(figsize=(10.2, 3.6))
541     ax.set_axis_off()
542     boxes = [
543         (0.02, 0.55), "问题一\n预测与事件识别",
544         (0.21, 0.55), "结构冲击\nSVAR输出",
545         (0.40, 0.55), "问题二\n中国宏观传导",
546         (0.59, 0.55), "问题三\n跨国韧性 with 政策情景",
547         (0.78, 0.55), "问题四\n尾部风险与SAPR优化",
548         (0.40, 0.18), "冻结校验\n风险门禁PASS",
549         (0.64, 0.18), "论文撰写\n数字台账引用",
550     ]
551     for xy, text in boxes:
552         draw_box(ax, xy, text, width=0.15, color="#f4efe6")
553     arrow(ax, (0.17, 0.63), (0.21, 0.63))
554     arrow(ax, (0.36, 0.63), (0.40, 0.63))
555     arrow(ax, (0.55, 0.63), (0.59, 0.63))
556     arrow(ax, (0.74, 0.63), (0.78, 0.63))
557     arrow(ax, (0.86, 0.55), (0.55, 0.34))
558     arrow(ax, (0.55, 0.26), (0.64, 0.26))
559     save_flow(fig, "paper_route_map", "四问统一技术路线", "模型链由预测、宏观传导和政策比较延伸到尾部风险压力测试与自适应调价优化")
560
561
562 def plot_event_timeline() -> None:
563     fig, ax = plt.subplots(figsize=(8.6, 3.0))
564     ax.set_axis_off()
565     xs = [0.12, 0.36, 0.64, 0.84]
566     labels = ["2026-02-28\n周末事件发生", "2026-03-02\nE1首个交易日", "2026-03-05\nE2持续中断", "2026-06-17\nE3缓和阶段"]
567     ax.plot([xs[0], xs[-1]], [0.5, 0.5], color=PALETTE["slate"], linewidth=1.2)
568     for x, label in zip(xs, labels):
569         ax.scatter([x], [0.5], s=70, color=PALETTE["blue"], zorder=3)
570         ax.text(x, 0.67, label, ha="center", va="bottom", fontsize=10)
571     ax.text(0.5, 0.25, "正式推断使用CAR[0]、CAR[0,+1]、CAR[0,+2]与周末匹配placebo经验分布", ha="center", fontsize=10, color=PALETTE["muted"])
572     save_flow(fig, "paper_event_timeline", "战争事件时间线", "周末事件统一映射到首个共同交易日")

```

```

573
574
575 def plot_transmission_mechanism() -> None:
576     fig, ax = plt.subplots(figsize=(8.8, 3.8))
577     ax.set_axis_off()
578     boxes = [
579         ((0.04, 0.58), "结构性\n油价冲击"),
580         ((0.27, 0.58), "人民币\n原油成本"),
581         ((0.50, 0.58), "PPI\n生产成本"),
582         ((0.73, 0.66), "CPI\n居民价格"),
583         ((0.73, 0.42), "工业活动\n增长响应"),
584         ((0.50, 0.18), "季度GDP\n低频验证"),
585     ]
586     for xy, text in boxes:
587         draw_box(ax, xy, text, color="#eef3ee")
588     arrow(ax, (0.22, 0.66), (0.27, 0.66))
589     arrow(ax, (0.45, 0.66), (0.50, 0.66))
590     arrow(ax, (0.68, 0.66), (0.73, 0.74))
591     arrow(ax, (0.68, 0.58), (0.73, 0.50))
592     arrow(ax, (0.59, 0.58), (0.59, 0.34))
593     save_flow(fig, "paper_transmission_mechanism", "油价向中国经济传导链", "主结果报告方向、量级与不确定性而非制造显著性")
594
595
596 def plot_policy_counterfactual_flow() -> None:
597     fig, ax = plt.subplots(figsize=(8.8, 3.5))
598     ax.set_axis_off()
599     boxes = [
600         ((0.04, 0.58), "官方调价\n实际路径"),
601         ((0.04, 0.26), "无临时调控\n规则路径"),
602         ((0.31, 0.42), "1045/380/1425\n政策缺口"),
603         ((0.57, 0.42), "燃油价格—宏观\n分布滞后模型"),
604         ((0.79, 0.42), "PPI/CPI/IAV\n反事实区间"),
605     ]
606     for xy, text in boxes:
607         draw_box(ax, xy, text, color="#f3eef0")
608     arrow(ax, (0.22, 0.66), (0.31, 0.53))
609     arrow(ax, (0.22, 0.34), (0.31, 0.50))
610     arrow(ax, (0.49, 0.50), (0.57, 0.50))
611     arrow(ax, (0.75, 0.50), (0.79, 0.50))
612     save_flow(fig, "paper_policy_counterfactual_flow", "中国临时调控关闭反事实流程", "情景结果代表价格平滑成本/延期负担代理而非福利判断")
613
614
615 def write_handoff_markdown() -> None:
616     risk = read_json("risk_probe_summary.json")
617     q1_origin = read_csv("q1_origin_forecast.csv")
618     q1_event = read_csv("q1_event_effects.csv")
619     q2_metrics = read_csv("q2_transmission_metrics.csv")
620     q3_resilience = read_csv("q3_resilience_metrics.csv")
621     q3_policy_macro = read_csv("q3_policy_macro_counterfactual.csv")
622     q4_risk = read_csv("q4_price_tail_risk.csv")
623     q4_backtest = read_csv("q4_risk_backtest.csv")
624     q4_macro = read_csv("q4_macro_stress.csv")
625     q4_policy = read_csv("q4_policy_stress.csv")
626     q4_sapr_optimal = read_csv("q4_sapr_optimal_rule.csv")
627     q4_sapr_comparison = read_csv("q4_sapr_strategy_comparison.csv")
628     q4_sapr_sensitivity = read_csv("q4_sapr_sensitivity.csv")
629     q4_sapr_summary = read_json("q4_sapr_summary.json")
630
631     q1_rows = []
632     for _, row in q1_origin.sort_values("horizon_months").iterrows():
633         actual = "未到期" if row["forecast_status"] == "FORECAST_ONLY" else f"{row['actual_price']:.2f}"
634         q1_rows.append(f"- h={int(row['horizon_months'])}: 预测 {row['prediction_price']:.2f} 美元/桶, 实际 {actual}。")
635     q1_car = q1_event[q1_event["model"].eq("brent_usd_bbl_event_car")]
636     car_rows = [
637         f"- {row['stage_id']}: CAR={row['estimate_log_return']:.4f}, 经验 p={row['pvalue_empirical']:.4f}。"
638         for _, row in q1_car.iterrows()
639     ]
640     q2_focus = q2_metrics[
641         q2_metrics["shock"].eq("oil_specific_risk_shock")
642         & q2_metrics["outcome"].isin(["brent_cny_cost_log_change_pct", "china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"])
643     ]
644     q2_rows = []
645     for _, row in q2_focus.iterrows():
646         area_text = (
647             f"- 0—12月响应曲线面积 {row['response_curve_area_0_12']:.3f} 百分点·月"
648             if pd.notna(row.get("response_curve_area_0_12"))

```



```

649         else "，稀疏期限不计算曲线面积"
650     )
651     q2_rows.append(
652         f"~ {zh_outcome(row['outcome'])}: 峰值/谷值 {row['extremum_response']:.3f}, h={int(row['extremum_month'])} {area_text}, 证据状态 {row['evidence_status']}。"
653     )
654     resilience_overall = q3_resilience["overall_china_resilience_judgement"].dropna().iloc[0]
655     q3_rows = []
656     for _, row in q3_resilience[q3_resilience["dimension"].isin(["fuel_price", "consumer_prices", "industrial_activity"])].iterrows():
657         china_text = f"~ {row['china_value']:.3f}" if pd.notna(row["china_value"]) else "不以0代替"
658         control_text = f"~ {row['control_median']:.3f}" if pd.notna(row["control_median"]) else "不可得"
659         q3_rows.append(f"~ {row['metric']}: 中国值 {china_text}, 对照量 {control_text}, 判断 {row['judgement']}。")
660     q3_macro_rows = [
661         f"~ 2026-06 {row['outcome_label']}: 无临时调控相对实际路径差额 {row['macro_counterfactual_gap_pctpt']:.3f} 个百分点, 95%区间 [{row['lower_95']:.3f}, {row['upper_95']:.3f}]。"
662         for _, row in q3_policy_macro[q3_policy_macro["period"].eq("2026-06")].sort_values("outcome").iterrows()
663     ]
664     q4_price_rows = [
665         f"~ h={int(row['horizon_months'])}: 中位数 {row['median_price']:.2f} 美元/桶, 90%区间 [{row['p05_price']:.2f}, {row['p95_price']:.2f}], 期末超过历史95%价格分位的条件概率 {100 * row['terminal_prob_above_hist_p95']:.2f}%。"
666         for _, row in q4_risk[q4_risk["model"].eq("FHS_GJR_GARCH")].sort_values("horizon_months").iterrows()
667     ]
668     q4_backtest_rows = [
669         f"~ {row['model']}, h={int(row['horizon_months'])}: 平均分位损失 {row['mean_pinball_loss']:.3f}, 80%/90%覆盖率 {row['coverage_80']:.3f}/{row['coverage_90']:.3f}。"
670         for _, row in q4_backtest.sort_values(["horizon_months", "model"]).iterrows()
671     ]
672     q4_macro_rows = [
673         f"~ {row['outcome_label']}, h={int(row['horizon'])}: 95%分位结构冲击条件响应 {row['conditional_response_pctpt']:.3f} 个百分点, 联合95%区间 [{row['joint_lower_95']:.3f}, {row['joint_upper_95']:.3f}], {row['row_evidence_status']}。"
674         for _, row in q4_macro[q4_macro["scenario"].eq("extreme_q95") & q4_macro["horizon"].isin([6, 12])].sort_values(["outcome", "horizon"]).iterrows()
675     ]
676     q4_policy_rows = [
677         f"~ 2026-06 {row['outcome_label']}: 政策缓冲收益 {row['policy_buffer_benefit_pctpt']:.3f} 个百分点, 95%区间 [{row['benefit_lower_95']:.3f}, {row['benefit_upper_95']:.3f}]。"
678         for _, row in q4_policy[q4_policy["period"].eq("2026-06")].sort_values("outcome").iterrows()
679     ]
680     q4_sapr_opt = q4_sapr_optimal.iloc[0]
681     q4_sapr_rule_text = (
682         f"~ SAPR-CVaR 膝点规则: 普通/压力/极端传导率 = "
683         f"({q4_sapr_opt['rho_normal']:.2f}, {q4_sapr_opt['rho_stress']:.2f}, {q4_sapr_opt['rho_extreme']:.2f}); "
684         f"压力阈值 {q4_sapr_opt['stress_threshold_75_cny_t']:.1f} 元/吨, 极端阈值 {q4_sapr_opt['stress_threshold_95_cny_t']:.1f} 元/吨。"
685     )
686     q4_sapr_holdout_rows = [
687         f"~ {row['strategy']}: 检验样本宏观损失均值 {row['J1_macro_loss']:.3f}, 95%CVaR {row['J2_cvar95_macro_loss']:.3f}, "
688         f"平均缺口月负担 {row['J3_avg_gap_month_burden']:.3f}, 调价波动率 {row['J4_adjustment_volatility']:.3f}。"
689         for _, row in q4_sapr_comparison[q4_sapr_comparison["sample_split"].eq("holdout")]
690     ]
691     .sort_values("strategy").iterrows()
692 ]
693     q4_sapr_2026_rows = [
694         f"~ {row['strategy']}: 2026情景宏观损失均值 {row['J1_macro_loss']:.3f}, 95%CVaR {row['J2_cvar95_macro_loss']:.3f}, "
695         f"平均缺口月负担 {row['J3_avg_gap_month_burden']:.3f}。"
696         for _, row in q4_sapr_comparison[q4_sapr_comparison["sample_split"].eq("war_2026")]
697     ]
698     .sort_values("strategy").iterrows()
699 ]
700     q4_sapr_sensitivity_rows = [
701         f"~ {row['variant']}: 规则=({row['rho_normal']:.2f}, {row['rho_stress']:.2f}, {row['rho_extreme']:.2f}), "
702         f"检验期非支配概率 {row['holdout_non_dominated_probability']:.3f}。"
703         for _, row in q4_sapr_sensitivity.sort_values("variant").iterrows()
704     ]
705     text = f"### 建模到论文移交说明"
706
707 [PAPER_READY]
708
709 原题三问与自拟问题四的建模、检验、数值冻结和论文移交材料已完成。当前 `overall_status={risk['overall_status']}`, `paper_finalize_allowed={str(risk['paper_finalize_allowed']).lower()}`, 阻塞门禁数为 {len(risk['blocking_probe_ids'])}。
710
711 ## 验证命令
712
713 ```powershell
714 python code\utils\freeze_results.py
715 python code\utils\verify_freeze.py
716 python code\utils\verify_freeze.py --require-final
717 ```

```

```
718
719 最后一次运行结果均为 `PASS`。论文所有数字优先引用 `results/frozen_numbers.json`、`results/final_numbers.json` 和 `reports/paper_numbers.
    csv`。
720
721 ## 问题一：预测与战争影响
722
723 主线：`no_change` 作为诚实胜出的主预测模型，交易日 CAR 与周末匹配 placebo 作为事件相关异常证据，递归 SVAR 提供三类结构冲击。
724
725 核心写法：
726
727 {chr(10).join(q1_rows)}
728 {chr(10).join(car_rows)}
729
730 可用图表：`figures/q1_forecast_1m.png`、`figures/q1_war_counterfactual.png`、`figures/q1_structural_shocks.png`、`figures/
    paper_event_timeline.png`。
731
732 禁止写法：不得把 `ARBaselineGap` 称作严格战争净贡献；不得把 ARIMA/SARIMAX 未胜出解释成建模失败。
733
734 ## 问题二：中国经济增长传导
735
736 主线：结构性油价冲击 → 人民币原油成本 → PPI → CPI/工业增加值 → 季度 GDP 验证。
737
738 油价特定风险冲击下的摘要：
739
740 {chr(10).join(q2_rows)}
741
742 结论边界：当前总体仍应写成“尚未发现稳健的总体增长损失证据”。若正文使用“增长损失”，必须同时满足工业活动负响应且联合区间排除零。
743
744 可用图表：`figures/q2_irf.png`、`figures/q2_transmission_chain.png`、`figures/paper_transmission_mechanism.png`。
745
746 ## 问题三：中国政策与跨国比较
747
748 主线：中国历史燃油序列统一作为受管制汽油调价指数代理并退出正式价格水平排名；CPI/IAV面板只估计六个对照国相对中国的响应差；待审计缓冲交互不
    发布定量结果；2026政策关闭情景在明确标注的代理层上动态传播至PPI/CPI/IAV。
749
750 综合韧性判断：`{resilience_overall}`。
751
752 {chr(10).join(q3_rows)}
753
754 政策关闭宏观反事实：
755
756 {chr(10).join(q3_macro_rows)}
757
758 可用图表：`figures/q3_pass_through_6m.png`、`figures/q3_panel_irf.png`、`figures/q3_resilience_metrics.png`、`figures/
    q3_policy_macro_counterfactual.png`、`figures/paper_policy_counterfactual_flow.png`。
759
760 禁止写法：不得仅凭价格传导或中国单一价格监管变量宣称“中国显著更好”；不得把价格平滑写成无成本福利改善。
761
762 ## 问题四：极端油价尾部风险、政策压力测试与自适应调价规则优化
763
764 定位：该部分作为论文正式自拟问题四，不再设置其他自拟问题。问题四由两个互补模块构成：同事已有的 FHS - GJR-GARCH 尾部风险与宏观政策压力测试
    负责回答“极端冲击有多大概率、会形成多大压力”；本轮 SAPR-CVaR 自适应调价优化负责回答“在现行机制约束上应如何设置状态依赖的临时平滑
    层”。油价概率、Q2 结构冲击宏观情景、Q3 已实现政策反事实和 SAPR 策略优化是递进关系，不可简单相加为单一因果贡献。
765
766 FHS - GJR-GARCH 条件尾部预测：
767
768 {chr(10).join(q4_price_rows)}
769
770 与高斯随机游走的同口径滚动回测：
771
772 {chr(10).join(q4_backtest_rows)}
773
774 95%分位油价特定风险结构冲击的宏观压力：
775
776 {chr(10).join(q4_macro_rows)}
777
778 2026年已实现临时调控的政策缓冲重述：
779
780 {chr(10).join(q4_policy_rows)}
781
782 SAPR-CVaR 自适应调价规则：
783
784 {q4_sapr_rule_text}
785
786 隔离检验样本策略比较：
787
```

```

788 {chr(10).join(q4_sapr_holdout_rows)}
789
790 2026实际冲击情景策略比较：
791
792 {chr(10).join(q4_sapr_2026_rows)}
793
794 敏感性检验：
795
796 {chr(10).join(q4_sapr_sensitivity_rows)}
797
798 证据状态：`{q4_sapr_summary['evidence_status']}`；检验期非支配 bootstrap 概率为 `{q4_sapr_summary['holdout_non_dominated_probability']:.3f}`。
799
800 可用图表：`figures/q4_price_tail_risk.png`、`figures/q4_macro_policy_stress.png`、`figures/q4_sapr_pareto_front.png`、`figures/q4_sapr_policy_heatmap.png`、`figures/q4_sapr_strategy_comparison.png`、`figures/q4_sapr_2026_macro_paths.png`。
801
802 禁止写法：不得把尾部概率写成确定结果，不得把宏观情景写成确定性 GDP 损失，不得把 Q2 条件响应与 Q3 政策差额相加为单一因果贡献；不得把 SAPR 的注册规则族最优写成全球最优、完整福利收益或财政成本估计。
803
804 ## 论文接手顺序
805
806 1. 先按 `reports/paper_numbers.csv` 抽取数值，填入摘要、问题重述、模型假设和结果表。
807 2. 四个问题均按“目标—数据—公式—估计—结果—检验—解释边界”写；问题四单列为自拟问题，明确从前三问结果递进而来。
808 3. 所有图表从 `figures/` 选择 PNG 入文，PDF 留作高清备份。
809 4. 写作期间如改动模型代码、输入数据或核心结果，必须重新运行冻结和 `--require-final`。
810 """
811 (REPORTS_DIR / "MODELING_TO_PAPER_HANDOFF.md").write_text(text, encoding="utf-8")
812
813
814 def main() -> None:
815     apply_paper_style()
816     REPORTS_DIR.mkdir(exist_ok=True)
817     FIGURES_DIR.mkdir(exist_ok=True)
818     write_paper_numbers()
819     plot_q1_structural_shocks()
820     plot_q2_transmission_chain()
821     plot_q3_resilience_metrics()
822     plot_q3_policy_macro_counterfactual()
823     plot_route_map()
824     plot_event_timeline()
825     plot_transmission_mechanism()
826     plot_policy_counterfactual_flow()
827     write_handoff_markdown()
828     print("[PAPER_READY] handoff artifacts generated.")
829
830
831 if __name__ == "__main__":
832     main()

```

Listing 12: build_paper_handoff.py

```

1 """Freeze stage outputs and run paper-finalization risk probes."""
2
3 from __future__ import annotations
4
5 import hashlib
6 import importlib.metadata
7 import json
8 import platform
9 import subprocess
10 import sys
11 from datetime import datetime, timezone
12 from pathlib import Path
13 from typing import Any
14
15 import matplotlib
16
17 matplotlib.use("Agg")
18
19 import matplotlib.pyplot as plt
20 import numpy as np
21 import pandas as pd
22
23
24 REPO_ROOT = Path(__file__).resolve().parents[2]

```

```

25 CODE_DIR = REPO_ROOT / "code"
26 if str(CODE_DIR) not in sys.path:
27     sys.path.insert(0, str(CODE_DIR))
28
29 from utils.plot_style import PALETTE, apply_paper_style, finish_figure, save_figure, style_axis
30
31 if hasattr(sys.stdout, "reconfigure"):
32     sys.stdout.reconfigure(encoding="utf-8")
33
34 RESULTS_DIR = REPO_ROOT / "results"
35 FIGURES_DIR = REPO_ROOT / "figures"
36 REPORTS_DIR = REPO_ROOT / "reports"
37 CUTOFF = "2026-06-30"
38 RISK_SCHEMA_VERSION = "1.0"
39 MANIFEST_SCHEMA_VERSION = "1.0"
40 STANDARD_FREEZE_SCHEMA_VERSION = 1
41 RANDOM_SEED = 20260730
42
43 CORE_RESULT_FILES = [
44     "q1_forecast_metrics.csv",
45     "q1_event_effects.csv",
46     "q1_placebo_distribution.csv",
47     "q1_daily_counterfactual.csv",
48     "q1_monthly_shocks.csv",
49     "q1_structural_shocks.csv",
50     "q1_svar_diagnostics.csv",
51     "q1_stationarity_diagnostics.csv",
52     "q1_origin_forecast.csv",
53     "q1_volatility.csv",
54     "q1_volatility_summary.csv",
55     "q1_robustness.csv",
56     "q1_summary.json",
57     "q2_ardl_baseline.csv",
58     "q2_irf.csv",
59     "q2_gdp_validation.csv",
60     "q2_asymmetry.csv",
61     "q2_robustness.csv",
62     "q2_transmission_metrics.csv",
63     "q2_summary.csv",
64     "q2_summary.json",
65     "q3_country_pass_through.csv",
66     "q3_panel_irf.csv",
67     "q3_buffer_interactions.csv",
68     "q3_buffer_data_status.csv",
69     "q3_policy_counterfactual.csv",
70     "q3_policy_macro_counterfactual.csv",
71     "q3_resilience_metrics.csv",
72     "q3_robustness.csv",
73     "q3_summary.csv",
74     "q3_summary.json",
75     "q3_policy_macro_kernel.csv",
76     "q3_policy_macro_covariance.csv",
77     "q3_policy_macro_bootstrap_draws.csv",
78     "q4_risk_probe.json",
79     "q4_price_tail_risk.csv",
80     "q4_risk_backtest_origins.csv",
81     "q4_risk_backtest.csv",
82     "q4_risk_backtest_paired.csv",
83     "q4_macro_stress.csv",
84     "q4_policy_stress.csv",
85     "q4_summary.json",
86     "q4_sapr_scenarios.csv",
87     "q4_sapr_policy_grid.csv",
88     "q4_sapr_optimal_rule.csv",
89     "q4_sapr_strategy_comparison.csv",
90     "q4_sapr_macro_paths.csv",
91     "q4_sapr_sensitivity.csv",
92     "q4_sapr_holdout_validation.csv",
93     "q4_sapr_holdout_paired_summary.csv",
94     "q4_sapr_holdout_validation_summary.json",
95     "q4_sapr_summary.json",
96 ]
97
98 PROCESSED_INPUT_FILES = [
99     "p0_daily_market.csv",
100     "p0_monthly_market.csv",

```

```

101 "global_oil_svar_monthly.csv",
102 "model_daily_q1.csv",
103 "model_monthly_q1.csv",
104 "model_monthly_cn.csv",
105 "model_quarterly_cn.csv",
106 "model_country_monthly.csv",
107 "oecd_g20_cpi_monthly.csv",
108 "oecd_kei_ip_monthly.csv",
109 "eu_eurosuper95_monthly.csv",
110 "france_eurosuper95_monthly.csv",
111 "italy_eurosuper95_monthly.csv",
112 "spain_eurosuper95_monthly.csv",
113 "germany_eurosuper95_monthly.csv",
114 "japan_regular_gasoline_monthly.csv",
115 "korea_regular_gasoline_monthly.csv",
116 "cn_fuel_adjustment_events.csv",
117 "cn_fuel_policy_events.csv",
118 "china_fuel_policy_monthly.csv",
119 "china_fuel_proxy_monthly.csv",
120 "china_regulated_gasoline_monthly.csv",
121 "eurostat_spain_hicp_yoy_monthly.csv",
122 "country_policy_buffers_annual.csv",
123 "nbs_iav_monthly.csv",
124 "nbs_ppi_monthly.csv",
125 "nbs_manual_import_metadata.json",
126 "release_date_matrix.csv",
127 ]
128
129 CODE_FILES = [
130 "code/data_download/download_p0.py",
131 "code/data_download/p0_sources.json",
132 "code/data_processing/build_p0_datasets.py",
133 "code/data_processing/build_model_panels.py",
134 "code/data_processing/import_nbs_manual.py",
135 "code/problem1/run_q1.py",
136 "code/problem2/run_q2.py",
137 "code/problem3/run_q3.py",
138 "code/problem4/run_q4.py",
139 "code/problem4/run_q4_sapr.py",
140 "code/schemas/frozen_numbers.schema.json",
141 "code/schemas/reproducibility_manifest.schema.json",
142 "code/schemas/risk_probe_summary.schema.json",
143 "code/utills/build_paper_handoff.py",
144 "code/utills/freeze_results.py",
145 "code/utills/plot_style.py",
146 "code/utills/verify_freeze.py",
147 "data/DATA_DICTIONARY.csv",
148 "data/SOURCE_REGISTRY.csv",
149 "requirements.in",
150 "requirements.lock.txt",
151 ]
152
153 COUNTRY_LABEL_ZH = {
154 "CHN": "中国",
155 "DEU": "德国",
156 "FRA": "法国",
157 "ITA": "意大利",
158 "ESP": "西班牙",
159 "JPN": "日本",
160 "KOR": "韩国",
161 }
162 CORE_PACKAGES = ["numpy", "pandas", "scipy", "statsmodels", "linearmodels", "matplotlib", "requests", "openpyxl", "xlrd", "jsonschema"]
163
164
165 def read_csv(filename: str) -> pd.DataFrame:
166     path = RESULTS_DIR / filename
167     if not path.exists():
168         return pd.DataFrame()
169     return pd.read_csv(path)
170
171
172 def load_json(filename: str) -> dict[str, Any]:
173     path = RESULTS_DIR / filename
174     if not path.exists():
175         return {}
176     return json.loads(path.read_text(encoding="utf-8"))

```

```

177
178
179 def sha256_bytes(payload: bytes) -> str:
180     return hashlib.sha256(payload).hexdigest()
181
182
183 def sha256_file(path: Path) -> str:
184     payload = path.read_bytes()
185     if path.suffix.lower() in {".csv", ".json", ".md", ".py", ".txt"}:
186         payload = payload.replace(b"\r\n", b"\n").replace(b"\r", b"\n")
187     return hashlib.sha256(payload).hexdigest()
188
189
190 def sha256_json(payload: dict[str, Any]) -> str:
191     return sha256_bytes(json.dumps(payload, ensure_ascii=False, sort_keys=True, separators=(",", ":")).encode("utf-8"))
192
193
194 def clean_value(value: Any) -> Any:
195     if isinstance(value, (np.integer,)):
196         return int(value)
197     if isinstance(value, (np.Floating, float)):
198         if not np.isfinite(value):
199             return None
200         return round(float(value), 8)
201     if pd.isna(value):
202         return None
203     return value
204
205
206 def records(frame: pd.DataFrame) -> list[dict[str, Any]]:
207     return [{key: clean_value(value) for key, value in row.items()} for row in frame.to_dict("records")]
208
209
210 def hash_existing(base_dir: Path, filenames: list[str]) -> dict[str, str]:
211     hashes: dict[str, str] = {}
212     for filename in filenames:
213         path = base_dir / filename
214         if path.exists():
215             hashes[filename] = sha256_file(path)
216     return hashes
217
218
219 def collect_figures() -> list[str]:
220     if not FIGURES_DIR.exists():
221         return []
222     return sorted(path.name for path in FIGURES_DIR.glob("*.png") if path.name.startswith(("q", "data_")))
223
224
225 def git_commit() -> str:
226     result = subprocess.run(["git", "rev-parse", "HEAD"], cwd=REPO_ROOT, text=True, capture_output=True, check=False)
227     return result.stdout.strip() if result.returncode == 0 else "UNKNOWN"
228
229
230 def environment_snapshot() -> dict[str, Any]:
231     packages: dict[str, str] = {}
232     for package in CORE_PACKAGES:
233         try:
234             packages[package] = importlib.metadata.version(package)
235         except importlib.metadata.PackageNotFoundError:
236             packages[package] = "NOT_INSTALLED"
237     lock_path = REPO_ROOT / "requirements.lock.txt"
238     return {
239         "python_version": platform.python_version(),
240         "python_executable": sys.executable,
241         "platform": platform.platform(),
242         "packages": packages,
243         "requirements_lock_sha256": sha256_file(lock_path) if lock_path.exists() else None,
244     }
245
246
247 def replay_environment_snapshot() -> dict[str, Any]:
248     lock_path = REPO_ROOT / "requirements.lock.txt"
249     lock_text = lock_path.read_text(encoding="utf-8") if lock_path.exists() else ""
250     python_version = None
251     packages: dict[str, str] = {}
252     for line in lock_text.splitlines():

```

```

253     stripped = line.strip()
254     if stripped.startswith("# Python:"):
255         python_version = stripped.split(":", 1)[1].strip()
256     elif stripped and not stripped.startswith("#") and "==" in stripped:
257         package, version = stripped.split("==", 1)
258         if package.strip().lower() in CORE_PACKAGES:
259             packages[package.strip().lower()] = version.strip()
260     return {
261         "python_version": python_version,
262         "packages": packages,
263         "requirements_lock_sha256": sha256_file(lock_path) if lock_path.exists() else None,
264     }
265
266
267 def probe(
268     probe_id: str,
269     status: str,
270     metric: Any,
271     threshold: str,
272     evidence_files: list[str],
273     message: str,
274     severity: str = "blocking",
275 ) -> dict[str, Any]:
276     return {
277         "probe_id": probe_id,
278         "status": status,
279         "severity": severity,
280         "metric": metric,
281         "threshold": threshold,
282         "evidence_files": evidence_files,
283         "message": message,
284     }
285
286
287 def combined_status(probes: list[dict[str, Any]]) -> str:
288     statuses = {row["status"] for row in probes}
289     if "FAIL" in statuses:
290         return "FAIL"
291     if "CONDITIONAL" in statuses:
292         return "CONDITIONAL"
293     return "PASS"
294
295
296 def has_nonmissing(path: Path, column: str, minimum: int) -> bool:
297     if not path.exists():
298         return False
299     frame = pd.read_csv(path)
300     return column in frame.columns and int(pd.to_numeric(frame[column], errors="coerce").notna().sum()) >= minimum
301
302
303 def bool_series(series: pd.Series) -> pd.Series:
304     if pd.api.types.is_bool_dtype(series):
305         return series
306     return series.astype(str).str.lower().isin(["true", "1", "yes"])
307
308
309 def build_risk_probe_summary(output_hashes: dict[str, str], input_hashes: dict[str, str], code_hashes: dict[str, str]) -> dict[str, Any]:
310     probes: list[dict[str, Any]] = []
311
312     metrics = read_csv("ql_forecast_metrics.csv")
313     if metrics.empty or "model_status" not in metrics.columns:
314         probes.append(probe("ql_forecast_baseline_gate", "FAIL", "missing model_status", "all forecast metrics carry model_status", [
315             "results/ql_forecast_metrics.csv"], "Ql forecast metrics cannot prove that advanced models were downgraded."))
316     else:
317         advanced_non_pass = metrics.loc[metrics["model"].ne("no_change") & metrics["model_status"].ne("PASS")]
318         no_change_rows = metrics.loc[metrics["model"].eq("no_change")]
319         no_change_pass = not no_change_rows.empty and no_change_rows["model_status"].eq("PASS").all()
320         status = "PASS" if no_change_pass else "FAIL"
321         if no_change_rows.empty:
322             status = "FAIL"
323         probes.append(
324             probe(
325                 "ql_forecast_baseline_gate",
326                 status,
327                 {
328                     "selected_model": "no_change",

```

```

328         "advanced_non_pass_rows": int(len(advanced_non_pass)),
329         "no_change_rows": int(len(no_change_rows)),
330         "no_change_pass": bool(no_change_pass),
331     },
332     "no-change rows exist and pass the pre-registered rolling backtest; advanced model failures are recorded but non-blocking
",
333     ["results/ql_forecast_metrics.csv", "results/ql_summary.json"],
334     "No-change is selected as the main forecast. ARIMA/SARIMAX/ETS/Theta failures are empirical findings, not a paper-
finalization blocker when the baseline wins honestly.",
335 )
336 )
337
338 ql_summary = load_json("ql_summary.json")
339 event_calendar = ql_summary.get("event_calendar", {})
340 event_ok = event_calendar.get("e1_trading_start") == "2026-03-02" and event_calendar.get("e2_trading_start") == "2026-03-05"
341 effects = read_csv("ql_event_effects.csv")
342 stage_e1 = effects.loc[
343     effects.get("model", pd.Series(dtype=str)).eq("brent_usd_bbl_stage_dummy")
344     & effects.get("stage_id", pd.Series(dtype=str)).eq("E1")
345 ] if not effects.empty else pd.DataFrame()
346 car_rows = effects.loc[
347     effects.get("model", pd.Series(dtype=str)).eq("brent_usd_bbl_event_car")
348     & effects.get("stage_id", pd.Series(dtype=str)).isin(["E1_CAR_0", "E1_CAR_0_1", "E1_CAR_0_2"])
349     & effects.get("identification_status", pd.Series(dtype=str)).eq("PASS")
350 ] if not effects.empty else pd.DataFrame()
351 stage_e1_descriptive = not stage_e1.empty and stage_e1["identification_status"].eq("DESCRIPTIVE_ONLY").all()
352 car_p_nonzero = not car_rows.empty and pd.to_numeric(car_rows.get("pvalue_empirical", pd.Series(dtype=float)), errors="coerce").
dropna().gt(0).all()
353 probes.append(
354     probe(
355         "ql_event_trading_day_gate",
356         "PASS" if event_ok and stage_e1_descriptive and len(car_rows) == 3 and car_p_nonzero else "FAIL",
357         {
358             "event_calendar": event_calendar,
359             "stage_e1_descriptive_only": bool(stage_e1_descriptive),
360             "brent_car_rows": int(len(car_rows)),
361             "car_empirical_p_nonzero": bool(car_p_nonzero),
362         },
363         "E1 maps to 2026-03-02; stage dummy is descriptive only; CAR[0], CAR[0,+1], CAR[0,+2] carry formal event inference",
364         ["results/ql_event_effects.csv", "results/ql_summary.json"],
365         "E1 weekend event has been remapped to the first common trading day. Single-window conventional significance is blocked;
formal evidence uses recursive normal-return CAR and matched placebo p-values.",
366     )
367 )
368
369 placebos = read_csv("ql_placebo_distribution.csv")
370 empirical_p = ql_summary.get("placebo_min_empirical_pvalue")
371 if placebos.empty or empirical_p is None or not 0.0 < float(empirical_p) <= 1.0:
372     placebo_status = "FAIL"
373 elif float(empirical_p) <= 0.0:
374     placebo_status = "FAIL"
375 else:
376     placebo_status = "PASS"
377 probes.append(
378     probe(
379         "ql_placebo_distribution_gate",
380         placebo_status,
381         {"placebo_rows": int(len(placebos)), "min_empirical_pvalue": empirical_p},
382         "matched weekend placebo distribution exists and finite-sample corrected empirical p-values lie in (0, 1]",
383         ["results/ql_placebo_distribution.csv", "results/ql_summary.json"],
384         "Matched weekend placebo evidence uses the add-one correction, so a finite pseudo-event sample cannot report an empirical p-
value of exactly zero.",
385     )
386 )
387
388 shocks = read_csv("ql_monthly_shocks.csv")
389 language_ok = not shocks.empty and "ARBaselineGap" in shocks.columns and "WarPremium" not in shocks.columns
390 probes.append(
391     probe(
392         "ql_counterfactual_language_gate",
393         "PASS" if language_ok else "FAIL",
394         {"has_ARBaselineGap": bool("ARBaselineGap" in shocks.columns), "has_WarPremium": bool("WarPremium" in shocks.columns)},
395         "descriptive AR baseline field exists and old causal premium field is absent",
396         ["results/ql_monthly_shocks.csv", "results/ql_daily_counterfactual.csv"],
397         "The event-path gap is labeled as ARBaselineGap rather than a war premium or causal no-war contribution.",
398     )

```



```

399 )
400
401 structural_cols = {"supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock", "reduced_form_shock", "source_vintage"}
402 structural_ok = not shocks.empty and structural_cols.issubset(shocks.columns)
403 core_structural_cols = ["supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock"]
404 core_structural_counts = (
405     {
406         column: int(pd.to_numeric(shocks[column], errors="coerce").notna().sum())
407         for column in core_structural_cols
408     }
409     if structural_ok
410     else {}
411 )
412 core_structural_months = min(core_structural_counts.values()) if core_structural_counts else 0
413 reduced_form_months = int(pd.to_numeric(shocks.get("reduced_form_shock", pd.Series(dtype=float)), errors="coerce").notna().sum()) if
    structural_ok else 0
414 probes.append(
415     probe(
416         "q1_structural_shock_coverage_gate",
417         "PASS" if structural_ok and core_structural_months >= 120 else "CONDITIONAL",
418         {
419             "has_columns": bool(structural_ok),
420             "core_structural_months": core_structural_months,
421             "core_structural_counts": core_structural_counts,
422             "reduced_form_months": reduced_form_months,
423             "minimum_core_structural_months": 120,
424         },
425         "supply, aggregate-demand and oil-specific-risk shocks each have at least 120 nonmissing monthly observations",
426         ["results/q1_monthly_shocks.csv", "results/q1_structural_shocks.csv"],
427         "Reduced-form oil returns do not substitute for full structural coverage. Q2/Q3 structural interpretation remains conditional
    until the three structural shock series cover enough history.",
428     )
429 )
430 probes.append(
431     probe(
432         "q2_structural_shock_identification",
433         "PASS" if structural_ok and core_structural_months >= 120 else "CONDITIONAL",
434         {
435             "has_columns": bool(structural_ok),
436             "core_structural_months": core_structural_months,
437             "core_structural_counts": core_structural_counts,
438             "reduced_form_months": reduced_form_months,
439         },
440         "Q1 exports supply, aggregate-demand and oil-specific-risk shocks for Q2/Q3; each core structural shock has at least 120
    months",
441         ["results/q1_monthly_shocks.csv", "results/q1_structural_shocks.csv"],
442         "Q2/Q3 use structural shock columns when available and retain reduced-form OilShock only as robustness. Current EIA STEO
    decomposition is too short for full-history structural claims.",
443     )
444 )
445
446 svar_diag = read_csv("q1_svar_diagnostics.csv")
447 selected_svar = svar_diag.loc[svar_diag.get("is_selected", pd.Series(dtype=bool)).astype(str).str.lower().isin(["true", "1"])] if not
    svar_diag.empty else pd.DataFrame()
448 svar_ok = (
449     not selected_svar.empty
450     and bool(selected_svar["is_stable"].astype(str).str.lower().isin(["true", "1"]).all())
451     and core_structural_months >= 120
452 )
453 probes.append(
454     probe(
455         "q1_svar_identification_gate",
456         "PASS" if svar_ok else "FAIL",
457         {
458             "selected_rows": int(len(selected_svar)),
459             "selected_lag": clean_value(selected_svar["candidate_lags"].iloc[0]) if not selected_svar.empty else None,
460             "selected_is_stable": bool(selected_svar["is_stable"].astype(str).str.lower().isin(["true", "1"]).all()) if not
    selected_svar.empty else False,
461             "core_structural_months": core_structural_months,
462         },
463         "Q1 historical recursive VAR has one selected stable specification and enough structural-shock coverage",
464         ["results/q1_svar_diagnostics.csv", "results/q1_structural_shocks.csv"],
465         "SVAR shocks must be generated from the full-history ex-post input rather than the short STEO residual proxy.",
466     )
467 )
468

```

```

469 origin_forecast = read_csv("q1_origin_forecast.csv")
470 origin_ok = (
471     not origin_forecast.empty
472     and sorted(pd.to_numeric(origin_forecast.get("horizon_months", pd.Series(dtype=float)), errors="coerce").dropna().astype(int).
        tolist()) == [1, 3, 6]
473     and origin_forecast.loc[origin_forecast["horizon_months"].eq(6), "forecast_status"].astype(str).eq("FORECAST_ONLY").all()
474 )
475 probes.append(
476     probe(
477         "q1_origin_forecast_completeness_gate",
478         "PASS" if origin_ok else "FAIL",
479         {
480             "rows": int(len(origin_forecast)),
481             "horizons": sorted(pd.to_numeric(origin_forecast.get("horizon_months", pd.Series(dtype=float)), errors="coerce").dropna()
        .astype(int).tolist()) if not origin_forecast.empty else [],
482         },
483         "Q1 2026-02 origin forecast table contains 1, 3 and 6 month horizons, with 2026-08 marked forecast-only",
484         ["results/q1_origin_forecast.csv"],
485         "The competition cutoff is 2026-06-30, so the six-month target from 2026-02 must be retained as a forecast-only row rather
        than removed.",
486     )
487 )
488
489 nbs_iav_ok = has_nonmissing(
490     REPO_ROOT / "data" / "processed" / "nbs_iav_monthly.csv",
491     "china_iav_yoy_pct",
492     120,
493 )
494 nbs_ppi_ok = has_nonmissing(
495     REPO_ROOT / "data" / "processed" / "nbs_ppi_monthly.csv",
496     "china_ppi_yoy_pct",
497     120,
498 )
499 probes.append(
500     probe(
501         "q2_nbs_macro_completeness_gate",
502         "PASS" if nbs_iav_ok and nbs_ppi_ok else "CONDITIONAL",
503         {"nbs_iav_monthly": nbs_iav_ok, "nbs_ppi_monthly": nbs_ppi_ok},
504         "NBS IAV and PPI monthly histories are present with enough observations",
505         ["data/processed/nbs_iav_monthly.csv", "data/processed/nbs_ppi_monthly.csv", "results/q2_summary.json"],
506         "Q2 remains conditional until official IAV and PPI histories enter the processed layer; no interpolation is used to fill the
        gap.",
507     )
508 )
509
510 q2_summary = load_json("q2_summary.json")
511 q2_irf = read_csv("q2_irf.csv")
512 q2_required_cols = {
513     "shock",
514     "joint_lower_95",
515     "joint_upper_95",
516     "ci95_contains_zero",
517     "joint_ci95_contains_zero",
518     "fdr_qvalue",
519     "supports_growth_loss_language",
520 }
521 q2_structural_shocks_present = not q2_irf.empty and bool(
522     {"supply_shock", "aggregate_demand_shock", "oil_specific_risk_shock"}.intersection(set(q2_irf.get("shock", pd.Series(dtype=str)).
        dropna())))
523 )
524 q2_guard_ok = (
525     not q2_irf.empty
526     and q2_required_cols.issubset(q2_irf.columns)
527     and q2_structural_shocks_present
528     and bool(q2_summary.get("conclusion_guardrail"))
529 )
530 probes.append(
531     probe(
532         "q2_claim_strength_gate",
533         "PASS" if q2_guard_ok else "FAIL",
534         {
535             "irf_rows": int(len(q2_irf)),
536             "has_required_inference_columns": bool(q2_required_cols.issubset(q2_irf.columns)),
537             "structural_shocks_present": bool(q2_structural_shocks_present),
538             "guardrail_present": bool(q2_summary.get("conclusion_guardrail")),
539         },

```

```

540         "LP inference flags, structural-shock labels and conclusion guardrails are present",
541         ["results/q2_irf.csv", "results/q2_summary.json"],
542         "Q2 output distinguishes structural shock categories from reduced-form robustness and blocks unsupported growth-loss wording.
543     ",
544     )
545
546     q3_pass = read_csv("q3_country_pass_through.csv")
547     china_regulated = REPO_ROOT / "data" / "processed" / "china_regulated_gasoline_monthly.csv"
548     china_official_months = 0
549     if china_regulated.exists():
550         china_regulated_frame = pd.read_csv(china_regulated)
551         china_official_months = int(
552             pd.to_numeric(
553                 china_regulated_frame.get("china_regulated_gasoline_cny_per_ton", pd.Series(dtype=float)),
554                 errors="coerce",
555             )
556             .notna()
557             .sum()
558         )
559     if q3_pass.empty or "included_in_main_comparison" not in q3_pass.columns:
560         q3_comp_status = "FAIL"
561         q3_metric: Any = "missing comparability fields"
562     else:
563         chn_rows = q3_pass.loc[q3_pass["country"].eq("CHN")]
564         chn_main = bool_series(chn_rows["included_in_main_comparison"]) if not chn_rows.empty else pd.Series(dtype=bool)
565         main_countries = sorted(q3_pass.loc[bool_series(q3_pass["included_in_main_comparison"])], "country"].dropna().unique().tolist())
566         china_proxy_excluded = (
567             not chn_rows.empty
568             and not bool(chn_main.any())
569             and chn_rows["price_measure_type"].astype(str).str.contains("proxy", case=False, na=False).all()
570         )
571         q3_comp_status = "PASS" if china_proxy_excluded and len(main_countries) >= 6 else "FAIL"
572         q3_metric = {
573             "main_countries": main_countries,
574             "china_rows": chn_rows[["horizon", "included_in_main_comparison", "price_measure_type", "observed_or_regulated"]].to_dict("records"),
575             "china_proxy_excluded": bool(china_proxy_excluded),
576             "china_official_price_months": china_official_months,
577             "minimum_official_price_months": 48,
578         }
579     probes.append(
580         probe(
581             "q3_china_comparability",
582             q3_comp_status,
583             q3_metric,
584             "China reconstructed adjustment-index proxy is excluded and at least six comparable control countries remain in the formal ranking",
585             ["results/q3_country_pass_through.csv", "data/processed/model_country_monthly.csv", "data/processed/china_regulated_gasoline_monthly.csv"],
586             "Q3 reports China only as a proxy sensitivity and does not overstate it as an official retail-price level.",
587         )
588     )
589
590     q2_metrics = read_csv("q2_transmission_metrics.csv")
591     iav_horizons = (
592         sorted(pd.to_numeric(q2_irf.loc[q2_irf.get("outcome", pd.Series(dtype=str)).eq("china_iav_yoy_pct"), "horizon"], errors="coerce")
593             .dropna().astype(int).unique().tolist())
594         if not q2_irf.empty and "outcome" in q2_irf.columns
595         else []
596     )
597     q2_inference_ok = (
598         not q2_irf.empty
599         and not q2_metrics.empty
600         and iav_horizons == [0, 3, 6, 12]
601         and q2_irf.get("inference_band", pd.Series(dtype=str)).astype(str).str.contains("bootstrap", case=False, na=False).all()
602         and {"evidence_status", "allows_growth_loss_language", "response_curve_area_0_6", "response_curve_area_0_12", "area_unit"}.issubset(q2_metrics.columns)
603         and not q2_metrics.loc[q2_metrics["outcome"].eq("china_iav_yoy_pct"), ["response_curve_area_0_6", "response_curve_area_0_12"]].notna().any().any()
604     )
605     probes.append(
606         probe(
607             "q2_inference_consistency_gate",
608             "PASS" if q2_inference_ok else "FAIL",
609         )

```

```

609         "irf_rows": int(len(q2_irf)),
610         "transmission_metric_rows": int(len(q2_metrics)),
611         "iav_horizons": iav_horizons,
612         "all_irf_bootstrap": bool(q2_irf.get("inference_band", pd.Series(dtype=str)).astype(str).str.contains("bootstrap", case=
False, na=False).all()) if not q2_irf.empty else False,
613     },
614     "Q2 LP intervals, p-values/FDR family and transmission metrics use the bootstrap interface, and IAV uses the fixed 0/3/6/12
month grid",
615     ["results/q2_irf.csv", "results/q2_transmission_metrics.csv"],
616     "Q2 may remain substantively inconclusive, but the inference source and response horizons must be internally consistent.",
617 )
618 )
619
620 q3_buffer = read_csv("q3_buffer_interactions.csv")
621 q3_buffer_status = read_csv("q3_buffer_data_status.csv")
622 q3_buffer_ok = (
623     q3_buffer.empty
624     and len(q3_buffer_status) == 4
625     and q3_buffer_status.get("data_status", pd.Series(dtype=str)).eq("FAIL_FOR_MAIN_UNAUDITED_PROXY").all()
626 )
627 probes.append(
628     probe(
629         "q3_policy_buffer_interaction_gate",
630         "PASS" if q3_buffer_ok else "FAIL",
631         {"buffer_rows": int(len(q3_buffer)), "withheld_status_rows": int(len(q3_buffer_status))},
632         "Unaudited country-year buffer proxies are rejected from the formal quantitative result",
633         ["results/q3_buffer_interactions.csv", "results/q3_buffer_data_status.csv", "data/processed/country_policy_buffers_annual.csv
"],
634         "Q3 fails closed instead of producing interaction estimates from normalized proxy sequences.",
635     )
636 )
637
638 q3_policy = read_csv("q3_policy_counterfactual.csv")
639 april = q3_policy.loc[q3_policy.get("period", pd.Series(dtype=str)).eq("2026-04")] if not q3_policy.empty else pd.DataFrame()
640 policy_ok = False
641 policy_metric: dict[str, Any] = {}
642 if not april.empty:
643     inc = float(april["incremental_gasoline_gap_cny_t"].iloc[0])
644     cum = float(april["cumulative_gasoline_gap_cny_t"].iloc[0])
645     response = float(april["response"].iloc[0])
646     policy_metric = {"april_incremental": inc, "april_cumulative": cum, "april_response": response}
647     policy_ok = abs(inc - 380.0) < 1e-6 and abs(cum - 1425.0) < 1e-6 and abs(response - 1425.0) < 1e-6
648 probes.append(
649     probe(
650         "q3_policy_gap_annotation_gate",
651         "PASS" if policy_ok else "FAIL",
652         policy_metric,
653         "April annotation separates incremental 380 from cumulative 1425 CNY/tonne",
654         ["results/q3_policy_counterfactual.csv", "figures/q3_policy_counterfactual.png"],
655         "Policy scenario fields distinguish incremental and cumulative gaps, matching the revised chart annotation.",
656     )
657 )
658
659 policy_proxy = not q3_policy.empty and q3_policy.get("price_layer_status", pd.Series(dtype=str)).astype(str).str.contains("proxy",
case=False, na=False).all()
660 probes.append(
661     probe(
662         "q3_policy_counterfactual_price_layer",
663         "PASS" if policy_proxy else "FAIL",
664         {
665             "policy_rows": int(len(q3_policy)),
666             "uses_labeled_proxy_price_layer": bool(policy_proxy),
667         },
668         "China policy scenario explicitly labels the historical adjustment-index proxy and separately audits 2026 notice gaps",
669         ["results/q3_policy_counterfactual.csv", "data/processed/china_regulated_gasoline_monthly.csv"],
670         "The historical layer cannot be called a fixed-product official retail cap; scenario claims remain conditional.",
671     )
672 )
673
674 q3_policy_macro = read_csv("q3_policy_macro_counterfactual.csv")
675 macro_outcomes = set(q3_policy_macro.get("outcome", pd.Series(dtype=str)).dropna().unique().tolist()) if not q3_policy_macro.empty
else set()
676 macro_policy_ok = (
677     not q3_policy_macro.empty
678     and {"china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"}.issubset(macro_outcomes)
679     and q3_policy_macro.get("price_layer_status", pd.Series(dtype=str)).astype(str).str.contains("proxy", case=False, na=False).all()

```

```

680     and {"lower_95", "upper_95", "macro_counterfactual_gap_pctpt", "fuel_return_gap", "bootstrap_reps", "bootstrap_block_length"}.
issubset(q3_policy_macro.columns)
681     and pd.to_numeric(q3_policy_macro["bootstrap_block_length"], errors="coerce").eq(6).all()
682 )
683 probes.append(
684     probe(
685         "q3_policy_macro_propagation_gate",
686         "PASS" if macro_policy_ok else "FAIL",
687         {
688             "rows": int(len(q3_policy_macro)),
689             "outcomes": sorted(macro_outcomes),
690             "has_interval_columns": bool({"lower_95", "upper_95", "macro_counterfactual_gap_pctpt"}.issubset(q3_policy_macro.columns)
) if not q3_policy_macro.empty else False,
691         },
692         "Q3 policy-close scenario dynamically propagates proxy fuel-return gaps to PPI, CPI and IAV with joint time-block intervals",
693         ["results/q3_policy_macro_counterfactual.csv", "results/q3_policy_counterfactual.csv"],
694         "The conditional proxy scenario must use the 0..6 distributed-lag kernel and recursive outcome state, not static elasticity
multiplication.",
695     )
696 )
697
698 buffer_table_path = REPO_ROOT / "data" / "processed" / "country_policy_buffers_annual.csv"
699 buffer_table = pd.read_csv(buffer_table_path) if buffer_table_path.exists() else pd.DataFrame()
700 required_buffers = ["oil_import_dependency", "oil_intensity", "import_source_hhi", "fuel_price_regulation"]
701 source_text = " ".join(buffer_table.get("source_url", pd.Series(dtype=str)).astype(str).tolist()).lower()
702 buffer_data_ok = q3_buffer.empty and len(q3_buffer_status) == 4 and ("proxy" in source_text or "pending audited" in source_text)
703 probes.append(
704     probe(
705         "q3_buffer_data_completeness_gate",
706         "PASS" if buffer_data_ok else "FAIL",
707         {
708             "buffer_table_rows": int(len(buffer_table)),
709             "buffer_interaction_rows": int(len(q3_buffer)),
710             "buffers_in_results": [],
711             "required_buffers": required_buffers,
712         },
713         "Q3 detects the unaudited proxy buffer table and withholds all four interaction results",
714         ["data/processed/country_policy_buffers_annual.csv", "results/q3_buffer_interactions.csv", "results/q3_buffer_data_status.csv
"],
715         "No quantitative policy-buffer claim is released from normalized or pending-audit inputs.",
716     )
717 )
718
719 q3_panel = read_csv("q3_panel_irf.csv")
720 panel_ok = (
721     not q3_panel.empty
722     and q3_panel.get("reference_country", pd.Series(dtype=str)).astype(str).eq("CHN").all()
723     and q3_panel.get("response_type", pd.Series(dtype=str)).astype(str).str.contains("control_country_minus_china", na=False).all()
724     and len(set(q3_panel.get("country", pd.Series(dtype=str)).dropna())) >= 6
725 )
726 probes.append(
727     probe(
728         "q3_panel_identification_gate",
729         "PASS" if panel_ok else "FAIL",
730         {
731             "panel_rows": int(len(q3_panel)),
732             "countries": sorted(q3_panel.get("country", pd.Series(dtype=str)).dropna().unique().tolist()) if not q3_panel.empty else
[],
733             "reference_country_values": sorted(q3_panel.get("reference_country", pd.Series(dtype=str)).dropna().unique().tolist()) if
not q3_panel.empty else [],
734         },
735         "Q3 panel LP identifies control-country responses relative to China under full time fixed effects",
736         ["results/q3_panel_irf.csv"],
737         "With year-month fixed effects, country absolute shock slopes are not separately interpretable; the formal panel output must
be relative to a reference country.",
738     )
739 )
740
741 q3_resilience = read_csv("q3_resilience_metrics.csv")
742 resilience_ok = (
743     not q3_resilience.empty
744     and {"fuel_price", "consumer_prices", "industrial_activity", "policy_counterfactual_macro"}.issubset(
set(q3_resilience.get("dimension", pd.Series(dtype=str)).dropna().unique().tolist())
745 )
746 )
747 and "overall_china_resilience_judgement" in q3_resilience.columns
748 and q3_resilience["overall_china_resilience_judgement"].astype(str).eq("INCONCLUSIVE").all()

```

```

749 )
750 probes.append(
751     probe(
752         "q3_resilience_output_gate",
753         "PASS" if resilience_ok else "FAIL",
754         {
755             "rows": int(len(q3_resilience)),
756             "dimensions": sorted(q3_resilience.get("dimension", pd.Series(dtype=str)).dropna().unique().tolist()) if not
q3_resilience.empty else [],
757             "overall": q3_resilience.get("overall_china_resilience_judgement", pd.Series(dtype=str)).dropna().iloc[0] if not
q3_resilience.empty and "overall_china_resilience_judgement" in q3_resilience else None,
758         },
759         "Q3 exports exactly three economic resilience dimensions plus a separate policy scenario and fails closed overall",
760         ["results/q3_resilience_metrics.csv"],
761         "The third question must not turn three fuel horizons into three votes or use invalid median confidence intervals.",
762     )
763 )
764
765 q4_probe = load_json("q4_risk_probe.json")
766 q4_probe_checks = q4_probe.get("checks", {})
767 q4_probe_ok = q4_probe.get("status") == "PASS" and bool(q4_probe_checks) and all(q4_probe_checks.values())
768 probes.append(
769     probe(
770         "q4_method_risk_probe_gate",
771         "PASS" if q4_probe_ok else "FAIL",
772         {
773             "probe_status": q4_probe.get("status"),
774             "checks": q4_probe_checks,
775             "gjr_persistence": q4_probe.get("metrics", {}).get("gjr_persistence"),
776             "seed_probability_max_abs_difference": q4_probe.get("metrics", {}).get("seed_probability_max_abs_difference"),
777         },
778         "Q4 main method and Gaussian baseline pass input, stationarity, non-degeneracy, replay, perturbation and interface probes",
779         ["results/q4_risk_probe.json", "code/problem4/run_q4.py"],
780         "The self-defined extension may run fully only after its pre-registered low-cost probe passes.",
781     )
782 )
783
784 q4_risk = read_csv("q4_price_tail_risk.csv")
785 q4_backtest = read_csv("q4_risk_backtest.csv")
786 q4_backtest_paired = read_csv("q4_risk_backtest_paired.csv")
787 probability_cols = [
788     "terminal_prob_above_hist_p90",
789     "terminal_prob_above_hist_p95",
790     "path_prob_cross_hist_p90",
791     "path_prob_cross_hist_p95",
792 ]
793 q4_models = set(q4_risk.get("model", pd.Series(dtype=str)).dropna().unique().tolist())
794 q4_horizons = sorted(pd.to_numeric(q4_risk.get("horizon_months", pd.Series(dtype=float)), errors="coerce").dropna().astype(int).
unique().tolist())
795 q4_probability_finite = bool(
796     not q4_risk.empty
797     and all(column in q4_risk.columns for column in probability_cols)
798     and np.isfinite(q4_risk[probability_cols].apply(pd.to_numeric, errors="coerce").to_numpy()).all()
799     and ((q4_risk[probability_cols].apply(pd.to_numeric, errors="coerce") >= 0) & (q4_risk[probability_cols].apply(pd.to_numeric,
errors="coerce") <= 1)).all().all()
800 )
801 q4_non_degenerate = bool(
802     not q4_risk.empty
803     and {"p05_price", "p95_price"}.issubset(q4_risk.columns)
804     and (pd.to_numeric(q4_risk["p95_price"], errors="coerce") > pd.to_numeric(q4_risk["p05_price"], errors="coerce")).all()
805 )
806 q4_backtest_ok = bool(
807     not q4_backtest.empty
808     and {"origins", "all_no_future_information", "mean_pinball_loss"}.issubset(q4_backtest.columns)
809     and pd.to_numeric(q4_backtest["origins"], errors="coerce").ge(36).all()
810     and bool_series(q4_backtest["all_no_future_information"]).all()
811     and np.isfinite(pd.to_numeric(q4_backtest["mean_pinball_loss"], errors="coerce")).all()
812     and not q4_backtest_paired.empty
813     and pd.to_numeric(q4_backtest_paired.get("block_length", pd.Series(dtype=float)), errors="coerce").ge(6).all()
814 )
815 q4_probability_ok = (
816     q4_models == {"FHS_GJR_GARCH", "Gaussian_random_walk"}
817     and q4_horizons == [1, 3, 6]
818     and q4_probability_finite
819     and q4_non_degenerate
820     and q4_backtest_ok

```

```

821 )
822 probes.append(
823     probe(
824         "q4_probability_calibration_gate",
825         "PASS" if q4_probability_ok else "FAIL",
826         {
827             "models": sorted(q4_models),
828             "horizons": q4_horizons,
829             "probabilities_finite_and_bounded": q4_probability_finite,
830             "distributions_non_degenerate": q4_non_degenerate,
831             "backtest_rows": int(len(q4_backtest)),
832             "backtest_min_origins": clean_value(pd.to_numeric(q4_backtest.get("origins"), pd.Series(dtype=float)), errors="coerce").
min() if not q4_backtest.empty else None,
833             "backtest_no_future_information": bool(q4_backtest_ok),
834         },
835         "FHS and Gaussian baseline each produce 1/3/6-month non-degenerate bounded tail forecasts with at least ten leakage-free
backtest origins",
836         ["results/q4_price_tail_risk.csv", "results/q4_risk_backtest.csv", "results/q4_risk_backtest_origins.csv"],
837         "Q4 probability claims require a comparable baseline, rolling calibration evidence and no future information at any origin.",
838     )
839 )
840
841 q4_macro = read_csv("q4_macro_stress.csv")
842 q4_policy = read_csv("q4_policy_stress.csv")
843 q4_summary = load_json("q4_summary.json")
844 q4_layers_ok = bool(
845     not q4_macro.empty
846     and set(q4_macro.get("scenario", pd.Series(dtype=str)).dropna().unique().tolist()) == {"moderate_q75", "severe_q90", "extreme_q95"}
847     and set(q4_macro.get("outcome", pd.Series(dtype=str)).dropna().unique().tolist()) == {"china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"}
848     and "joint_ci95_contains_zero" in q4_macro.columns
849     and not q4_policy.empty
850     and set(q4_policy.get("outcome", pd.Series(dtype=str)).dropna().unique().tolist()) == {"china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"}
851     and q4_summary.get("evidence_status") == "CONDITIONAL_STRESS_TEST"
852     and bool(q4_summary.get("guardrail"))
853 )
854 probes.append(
855     probe(
856         "q4_evidence_layer_guardrail",
857         "PASS" if q4_layers_ok else "FAIL",
858         {
859             "macro_rows": int(len(q4_macro)),
860             "macro_scenarios": sorted(q4_macro.get("scenario", pd.Series(dtype=str)).dropna().unique().tolist()) if not q4_macro.empty else [],
861             "macro_outcomes": sorted(q4_macro.get("outcome", pd.Series(dtype=str)).dropna().unique().tolist()) if not q4_macro.empty else [],
862             "policy_rows": int(len(q4_policy)),
863             "summary_evidence_status": q4_summary.get("evidence_status"),
864             "guardrail_present": bool(q4_summary.get("guardrail")),
865         },
866         "Q4 keeps price probabilities, structural-shock macro scenarios and realized policy counterfactuals as three linked but non-additive evidence layers",
867         ["results/q4_macro_stress.csv", "results/q4_policy_stress.csv", "results/q4_summary.json"],
868         "The extension cannot turn conditional Q2 responses or Q3 policy scenarios into deterministic losses or one combined causal estimate.",
869     )
870 )
871
872 q4_sapr_scenarios = read_csv("q4_sapr_scenarios.csv")
873 q4_sapr_grid = read_csv("q4_sapr_policy_grid.csv")
874 q4_sapr_optimal = read_csv("q4_sapr_optimal_rule.csv")
875 q4_sapr_comparison = read_csv("q4_sapr_strategy_comparison.csv")
876 q4_sapr_paths = read_csv("q4_sapr_macro_paths.csv")
877 q4_sapr_sensitivity = read_csv("q4_sapr_sensitivity.csv")
878 q4_sapr_summary = load_json("q4_sapr_summary.json")
879 q3_kernel = read_csv("q3_policy_macro_kernel.csv")
880 q3_covariance = read_csv("q3_policy_macro_covariance.csv")
881 q3_joint_draws = read_csv("q3_policy_macro_bootstrap_draws.csv")
882 sapr_outcomes = {"china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"}
883
884 q4_sapr_dev = q4_sapr_scenarios.loc[q4_sapr_scenarios.get("sample_split", pd.Series(dtype=str)).eq("development")] if not q4_sapr_scenarios.empty else pd.DataFrame()
885 q4_sapr_holdout = q4_sapr_scenarios.loc[q4_sapr_scenarios.get("sample_split", pd.Series(dtype=str)).eq("holdout")] if not q4_sapr_scenarios.empty else pd.DataFrame()

```

```

886 q4_sapr_anchor_count = int(q4_sapr_scenarios.loc[bool_series(q4_sapr_scenarios.get("is_2026_anchor", pd.Series(dtype=bool))], "
      scenario_id").nunique()) if not q4_sapr_scenarios.empty else 0
887
888 q4_sapr_scenario_ok = (
889     not q4_sapr_dev.empty
890     and not q4_sapr_holdout.empty
891     and str(q4_sapr_dev["source_end"].max()) <= "2021-12"
892     and str(q4_sapr_holdout["source_start"].min()) >= "2022-01"
893     and q4_sapr_anchor_count == 1
894 )
895 probes.append(
896     probe(
897         "q4_sapr_scenario_no_leakage_gate",
898         "PASS" if q4_sapr_scenario_ok else "FAIL",
899         {
900             "development_scenarios": int(q4_sapr_dev["scenario_id"].nunique()) if not q4_sapr_dev.empty else 0,
901             "holdout_scenarios": int(q4_sapr_holdout["scenario_id"].nunique()) if not q4_sapr_holdout.empty else 0,
902             "development_max_source_end": str(q4_sapr_dev["source_end"].max()) if not q4_sapr_dev.empty else None,
903             "holdout_min_source_start": str(q4_sapr_holdout["source_start"].min()) if not q4_sapr_holdout.empty else None,
904             "anchor_2026_scenarios": q4_sapr_anchor_count,
905         },
906         "SAPR thresholds and rule selection use only 2013-03 to 2021-12 development scenarios; 2022-2026 remains holdout",
907         ["results/q4_sapr_scenarios.csv"],
908         "The adaptive rule must not use the 2026 conflict period or post-2021 holdout data to tune thresholds or choose the rule.",
909     )
910 )
911
912 q4_sapr_kernel_ok = (
913     sapr_outcomes.issubset(set(q3_kernel.get("outcome", pd.Series(dtype=str)).dropna().tolist()))
914     and sapr_outcomes.issubset(set(q3_covariance.get("outcome", pd.Series(dtype=str)).dropna().tolist()))
915     and q3_joint_draws.get("draw", pd.Series(dtype=float)).nunique() >= 2000
916     and int(q4_sapr_summary.get("valid_parameter_draw_count", 0)) == 2000
917     and float(q4_sapr_summary.get("valid_parameter_draw_rate", 0.0)) >= 0.95
918 )
919 probes.append(
920     probe(
921         "q4_sapr_macro_kernel_gate",
922         "PASS" if q4_sapr_kernel_ok else "FAIL",
923         {
924             "kernel_outcomes": sorted(set(q3_kernel.get("outcome", pd.Series(dtype=str)).dropna().tolist())) if not q3_kernel.empty
925             else [],
926             "covariance_outcomes": sorted(set(q3_covariance.get("outcome", pd.Series(dtype=str)).dropna().tolist())) if not
927             q3_covariance.empty else [],
928             "valid_parameter_draw_count": q4_sapr_summary.get("valid_parameter_draw_count"),
929             "valid_parameter_draw_rate": q4_sapr_summary.get("valid_parameter_draw_rate"),
930             "parameter_sampling": q4_sapr_summary.get("parameter_sampling"),
931         },
932         "SAPR reads the frozen Q3 macro kernel and propagates 2000 stable joint parameter samples",
933         ["results/q3_policy_macro_kernel.csv", "results/q3_policy_macro_covariance.csv", "results/q3_policy_macro_bootstrap_draws.csv",
934         "results/q4_sapr_summary.json"],
935         "SAPR is the policy-optimization layer of Q4 and must reuse Q3 macro kernels rather than choosing a new macro model.",
936     )
937 )
938
939 q4_sapr_identity = q4_sapr_summary.get("identity_checks", {})
940 q4_sapr_identity_ok = bool(
941     q4_sapr_identity
942     and abs(float(q4_sapr_identity.get("full_rule_max_abs_gap", 1.0))) < 1e-8
943     and abs(float(q4_sapr_identity.get("full_rule_adjustment_identity_max_abs_error", 1.0))) < 1e-8
944     and abs(float(q4_sapr_identity.get("zero_policy_macro_max_abs_response", 1.0))) < 1e-10
945     and bool(q4_sapr_identity.get("negative_adjustment_offsets_gap"))
946     and abs(float(q4_sapr_identity.get("official_2026_03_gap", 0.0)) - 1045.0) < 1e-8
947     and abs(float(q4_sapr_identity.get("official_2026_04_gap", 0.0)) - 380.0) < 1e-8
948     and abs(float(q4_sapr_identity.get("official_2026_04_cumulative_gap", 0.0)) - 1425.0) < 1e-8
949     and abs(float(q4_sapr_identity.get("q3_policy_counterfactual_reproduction_max_abs_error", 1.0))) < 1e-8
950 )
951 probes.append(
952     probe(
953         "q4_sapr_policy_identity_gate",
954         "PASS" if q4_sapr_identity_ok else "FAIL",
955         q4_sapr_identity,
956         "SAPR accounting identities hold and official 2026 policy gaps reproduce 1045, 380 and 1425 yuan/ton",
957         ["results/q4_sapr_summary.json", "data/processed/china_fuel_policy_monthly.csv"],
958         "The adaptive optimization is invalid unless full pass-through, deferred-gap accumulation, downward offset and Q3 macro
959         propagation identities are exact.",
960     )
961 )

```



```

957
958 q4_sapr_pareto_ok = (
959     len(q4_sapr_grid) == 1771
960     and "is_pareto" in q4_sapr_grid.columns
961     and "is_feasible" in q4_sapr_grid.columns
962     and "is_knee" in q4_sapr_grid.columns
963     and int(bool_series(q4_sapr_grid["is_pareto"]).sum()) > 0
964     and int(bool_series(q4_sapr_grid["is_knee"]).sum()) == 1
965     and bool_series(q4_sapr_grid.loc[bool_series(q4_sapr_grid["is_knee"]), "is_feasible"]).all()
966     and len(q4_sapr_optimal) == 1
967 )
968 probes.append(
969     probe(
970         "q4_sapr_pareto_selection_gate",
971         "PASS" if q4_sapr_pareto_ok else "FAIL",
972         {
973             "rule_rows": int(len(q4_sapr_grid)),
974             "pareto_rows": int(bool_series(q4_sapr_grid.get("is_pareto", pd.Series(dtype=bool))).sum()) if not q4_sapr_grid.empty
975             else 0,
976             "knee_rows": int(bool_series(q4_sapr_grid.get("is_knee", pd.Series(dtype=bool))).sum()) if not q4_sapr_grid.empty else 0,
977             "optimal_rows": int(len(q4_sapr_optimal)),
978             "SAPR enumerates 1771 monotone rules, enforces gap/recovery/adjustment constraints, and selects one 1%-epsilon Pareto knee",
979             ["results/q4_sapr_policy_grid.csv", "results/q4_sapr_optimal_rule.csv"],
980             "The policy rule must be selected by deterministic enumeration and Pareto-knee screening, not by an unstable heuristic.",
981         )
982     )
983
984 q4_sapr_holdout_strategies = set(q4_sapr_comparison.loc[q4_sapr_comparison.get("sample_split", pd.Series(dtype=str)).eq("holdout")], "
985     strategy"].dropna().tolist()) if not q4_sapr_comparison.empty else set()
986 holdout_validation = load_json("q4_sapr_holdout_validation_summary.json")
987 q4_sapr_holdout_ok = (
988     {"full_mechanism", "uniform_75_smoothing", "temporary_2026_approx", "SAPR_CVaR_knee"}.issubset(q4_sapr_holdout_strategies)
989     and q4_sapr_summary.get("evidence_status") in {"SUPPORTED", "PARTIAL", "NOT_SUPPORTED"}
990     and "holdout_non_dominated_probability" in q4_sapr_summary
991     and int(holdout_validation.get("block_length", 0)) >= 6
992     and int(holdout_validation.get("supported_improvement_count", 0)) >= 1
993     and (RESULTS_DIR / "q4_sapr_holdout_paired_summary.csv").exists()
994 )
995 probes.append(
996     probe(
997         "q4_sapr_holdout_validation_gate",
998         "PASS" if q4_sapr_holdout_ok else "FAIL",
999         {
1000             "holdout_strategies": sorted(q4_sapr_holdout_strategies),
1001             "evidence_status": q4_sapr_summary.get("evidence_status"),
1002             "holdout_non_dominated_probability": q4_sapr_summary.get("holdout_non_dominated_probability"),
1003             "block_length": holdout_validation.get("block_length"),
1004             "supported_improvement_count": holdout_validation.get("supported_improvement_count"),
1005             "point_regret_vs_oracle": holdout_validation.get("point_regret_vs_oracle"),
1006             "SAPR reports preregistered holdout comparisons without using holdout rows for rule selection",
1007             ["results/q4_sapr_strategy_comparison.csv", "results/q4_sapr_holdout_paired_summary.csv", "results/
1008             q4_sapr_holdout_validation_summary.json", "results/q4_sapr_summary.json"],
1009             "A negative or partial holdout result is acceptable; the gate checks evaluation integrity rather than forcing improvement.",
1010         )
1011     )
1012
1013 q4_sapr_interval_cols = [
1014     "J1_macro_loss_lower_95",
1015     "J1_macro_loss_upper_95",
1016     "J2_cvar95_macro_loss_lower_95",
1017     "J2_cvar95_macro_loss_upper_95",
1018 ]
1019 q4_sapr_uncertainty_ok = bool(
1020     int(q4_sapr_summary.get("valid_parameter_draw_count", 0)) == 2000
1021     and set(q4_sapr_interval_cols).issubset(q4_sapr_comparison.columns)
1022     and np.isfinite(q4_sapr_comparison[q4_sapr_interval_cols].astype(float).to_numpy()).all()
1023     and not q4_sapr_paths.empty
1024     and not q4_sapr_sensitivity.empty
1025 )
1026 probes.append(
1027     probe(
1028         "q4_sapr_uncertainty_gate",
1029         "PASS" if q4_sapr_uncertainty_ok else "FAIL",
1030         {

```

```

1030         "valid_parameter_draw_count": q4_sapr_summary.get("valid_parameter_draw_count"),
1031         "comparison_rows": int(len(q4_sapr_comparison)),
1032         "macro_path_rows": int(len(q4_sapr_paths)),
1033         "sensitivity_rows": int(len(q4_sapr_sensitivity)),
1034         "has_interval_columns": bool(set(q4_sapr_interval_cols).issubset(q4_sapr_comparison.columns)),
1035     },
1036     "SAPR comparison, macro paths and sensitivity outputs carry the registered uncertainty results",
1037     ["results/q4_sapr_strategy_comparison.csv", "results/q4_sapr_macro_paths.csv", "results/q4_sapr_sensitivity.csv", "results/
1038     q4_sapr_summary.json"],
1039     "SAPR uncertainty must come from the frozen 2000-draw parameter propagation and registered sensitivity variants.",
1040 )
1041
1042 q4_sapr_forbidden = " ".join(q4_sapr_summary.get("forbidden_claims", []))
1043 q4_sapr_claim_ok = (
1044     q4_sapr_summary.get("execution_status") == "PASS"
1045     and bool(q4_sapr_summary.get("allowed_claims"))
1046     and "global optimum" in q4_sapr_forbidden
1047     and "fiscal cost" in q4_sapr_forbidden
1048     and "post-2021" in q4_sapr_forbidden
1049 )
1050 probes.append(
1051     probe(
1052         "q4_sapr_claim_strength_gate",
1053         "PASS" if q4_sapr_claim_ok else "FAIL",
1054         {
1055             "execution_status": q4_sapr_summary.get("execution_status"),
1056             "evidence_status": q4_sapr_summary.get("evidence_status"),
1057             "allowed_claims": q4_sapr_summary.get("allowed_claims", []),
1058             "forbidden_claims": q4_sapr_summary.get("forbidden_claims", []),
1059         },
1060         "SAPR restricts claims to the registered rule family and forbids global/welfare/fiscal overclaims",
1061         ["results/q4_sapr_summary.json"],
1062         "The optimization layer should add decision value without overstating policy causality or welfare conclusions.",
1063     )
1064 )
1065
1066 chart_sources = [
1067     REPO_ROOT / "code" / "problem1" / "run_q1.py",
1068     REPO_ROOT / "code" / "problem2" / "run_q2.py",
1069     REPO_ROOT / "code" / "problem3" / "run_q3.py",
1070     REPO_ROOT / "code" / "problem4" / "run_q4.py",
1071     REPO_ROOT / "code" / "problem4" / "run_q4_sapr.py",
1072 ]
1073 old_terms = [
1074     "Actual Brent",
1075     "No-war counterfactual",
1076     "Q1 war-premium counterfactual",
1077     "USD per barrel",
1078     "95% interval",
1079     "Months after oil shock",
1080     "Cumulative coefficient",
1081     "Actual proxy",
1082     "No-control proxy",
1083     "Data overview:",
1084 ]
1085 leftovers: list[str] = []
1086 for path in chart_sources:
1087     text = path.read_text(encoding="utf-8")
1088     leftovers.extend([term for term in old_terms if term in text])
1089 figures = collect_figures()
1090 style_source = (REPO_ROOT / "code" / "utils" / "plot_style.py").read_text(encoding="utf-8")
1091 title_painted = "fig.text(rect[0], 0.965, title" in style_source or "fig.suptitle(" in style_source
1092 probes.append(
1093     probe(
1094         "figure_localization_gate",
1095         "PASS" if len(figures) >= 10 and not leftovers and not title_painted else "FAIL",
1096         {
1097             "figure_pngs": figures,
1098             "old_visible_terms": sorted(set(leftovers)),
1099             "large_title_painted_inside_figure": title_painted,
1100         },
1101         "at least ten figure PNGs exist, visible labels are localized, and large titles are left to LaTeX captions",
1102         ["figures", "code/problem1/run_q1.py", "code/problem2/run_q2.py", "code/problem3/run_q3.py", "code/problem4/run_q4.py", "code
1103         /utils/plot_style.py"],
1104         "Figure axes and legends are localized to Chinese; large titles are not painted into the figure canvas.",

```

```

1104     )
1105 )
1106
1107 manifest_path = REPO_ROOT / "data" / "raw" / "source_manifest.json"
1108 manifest_metric: dict[str, Any] = {"manifest_exists": manifest_path.exists()}
1109 manifest_ok = False
1110 if manifest_path.exists():
1111     manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
1112     cached_missing = []
1113     status_counts: dict[str, int] = {}
1114     for row in manifest:
1115         status_counts[row.get("status", "")] = status_counts.get(row.get("status", ""), 0) + 1
1116         if row.get("status") in {"DOWNLOADED", "CACHED"}:
1117             local_path = row.get("local_path", "")
1118             if not local_path or not (REPO_ROOT / local_path).exists():
1119                 cached_missing.append(row.get("artifact_id", ""))
1120     manifest_metric = {"status_counts": status_counts, "cached_missing": cached_missing}
1121     manifest_ok = not cached_missing
1122 probes.append(
1123     probe(
1124         "source_provenance_integrity",
1125         "PASS" if manifest_ok else "FAIL",
1126         manifest_metric,
1127         "source_manifest has no CACHED/DOWNLOADED rows whose local snapshot is absent",
1128         ["data/raw/source_manifest.json", "data/raw/source_manifest.csv"],
1129         "Raw-source provenance cannot claim a cache exists unless the file is present and hashable. REMOTE_ONLY records keep missing browser/official snapshots visible.",
1130     )
1131 )
1132
1133 lock_path = REPO_ROOT / "requirements.lock.txt"
1134 env = environment_snapshot()
1135 lock_text = lock_path.read_text(encoding="utf-8") if lock_path.exists() else ""
1136 pinned_lines = [line.strip() for line in lock_text.splitlines() if line.strip() and not line.lstrip().startswith("#")]
1137 env_ok = "# Python: 3.11.9" in lock_text and bool(pinned_lines) and all("==" in line for line in pinned_lines)
1138 probes.append(
1139     probe(
1140         "environment_lock_gate",
1141         "PASS" if env_ok else "CONDITIONAL",
1142         {
1143             "target_python_version": "3.11.9",
1144             "generation_python_version": env["python_version"],
1145             "exact_package_pins": len(pinned_lines),
1146             "requirements_lock_sha256": env["requirements_lock_sha256"],
1147         },
1148         "Python 3.11.9 and exact dependency lock are recorded",
1149         ["requirements.in", "requirements.lock.txt"],
1150         "The replay target is pinned independently of the interpreter used only to package and verify artifacts.",
1151     )
1152 )
1153
1154 hash_ok = len(output_hashes) == len(CORE_RESULT_FILES) and len(input_hashes) >= 10 and len(code_hashes) == len(CODE_FILES)
1155 probes.append(
1156     probe(
1157         "freeze_hash_coverage_gate",
1158         "PASS" if hash_ok else "FAIL",
1159         {"output_hash_count": len(output_hashes), "input_hash_count": len(input_hashes), "model_code_hash_count": len(code_hashes)},
1160         "all core result files, processed model inputs and model code files are hashed",
1161         ["results/reproducibility_manifest.json", "results/frozen_numbers.json"],
1162         "The reproducibility manifest records model outputs, processed inputs and model code so raw snapshots that cannot be redistributed do not prevent reproducibility checks and later code edits invalidate stale freezes.",
1163     )
1164 )
1165
1166 overall = combined_status(probes)
1167 return {
1168     "schema_version": RISK_SCHEMA_VERSION,
1169     "overall_status": overall,
1170     "paper_finalize_allowed": overall == "PASS",
1171     "cutoff": CUTOFF,
1172     "random_seed": RANDOM_SEED,
1173     "probes": probes,
1174     "blocking_probe_ids": [row["probe_id"] for row in probes if row["status"] != "PASS" and row["severity"] == "blocking"],
1175     "core_results_present": {filename: (RESULTS_DIR / filename).exists() for filename in CORE_RESULT_FILES},
1176     "figure_pngs": collect_figures(),
1177     "output_hash_count": len(output_hashes),

```

```

1178     "input_hash_count": len(input_hashes),
1179     "model_code_hash_count": len(code_hashes),
1180 }
1181
1182
1183 def extract_final_numbers() -> dict[str, Any]:
1184     q1_metrics = read_csv("q1_forecast_metrics.csv")
1185     q1_events = read_csv("q1_event_effects.csv")
1186     q1_shocks = read_csv("q1_monthly_shocks.csv")
1187     q1_origin = read_csv("q1_origin_forecast.csv")
1188     q1_svar = read_csv("q1_svar_diagnostics.csv")
1189     q1_vol_summary = read_csv("q1_volatility_summary.csv")
1190     q1_robust = read_csv("q1_robustness.csv")
1191     q2_ardl = read_csv("q2_ardl_baseline.csv")
1192     q2_irf = read_csv("q2_irf.csv")
1193     q2_gdp = read_csv("q2_gdp_validation.csv")
1194     q2_transmission = read_csv("q2_transmission_metrics.csv")
1195     q2_robust = read_csv("q2_robustness.csv")
1196     q3_pass = read_csv("q3_country_pass_through.csv")
1197     q3_irf = read_csv("q3_panel_irf.csv")
1198     q3_buffer = read_csv("q3_buffer_interactions.csv")
1199     q3_policy = read_csv("q3_policy_counterfactual.csv")
1200     q3_policy_macro = read_csv("q3_policy_macro_counterfactual.csv")
1201     q3_resilience = read_csv("q3_resilience_metrics.csv")
1202     q4_risk = read_csv("q4_price_tail_risk.csv")
1203     q4_backtest = read_csv("q4_risk_backtest.csv")
1204     q4_macro = read_csv("q4_macro_stress.csv")
1205     q4_policy = read_csv("q4_policy_stress.csv")
1206     q4_sapr_optimal = read_csv("q4_sapr_optimal_rule.csv")
1207     q4_sapr_comparison = read_csv("q4_sapr_strategy_comparison.csv")
1208     q4_sapr_paths = read_csv("q4_sapr_macro_paths.csv")
1209     q4_sapr_sensitivity = read_csv("q4_sapr_sensitivity.csv")
1210     q3_robust = read_csv("q3_robustness.csv")
1211     q4_summary = load_json("q4_summary.json")
1212     q4_sapr_summary = load_json("q4_sapr_summary.json")
1213
1214     final: dict[str, Any] = {
1215         "cutoff": CUTOFF,
1216         "status_note": "three original tasks plus one self-defined extension; not a paper-final causal freeze unless risk gate passes",
1217         "q1": {},
1218         "q2": {},
1219         "q3": {},
1220         "q4": {},
1221     }
1222     if not q1_metrics.empty:
1223         final["q1"]["selected_forecast_model"] = "no_change"
1224         final["q1"]["forecast_best_by_rmse"] = records(q1_metrics.sort_values(["RMSE", "MAE"]).head(3))
1225         final["q1"]["forecast_metrics"] = records(q1_metrics.sort_values(["horizon", "model"]))
1226     if not q1_events.empty:
1227         final["q1"]["brent_event_stage_effects"] = records(
1228             q1_events.loc[q1_events["model"].eq("brent_usd_bbl_stage_dummy")].sort_values("stage_id")
1229         )
1230         final["q1"]["brent_event_car"] = records(
1231             q1_events.loc[q1_events["model"].eq("brent_usd_bbl_event_car")].sort_values("stage_id")
1232         )
1233         final["q1"]["wti_robustness_effects"] = records(
1234             q1_events.loc[q1_events["model"].eq("wti_usd_bbl_stage_dummy")].sort_values("stage_id")
1235         )
1236     if not q1_shocks.empty:
1237         event_months = q1_shocks.loc[q1_shocks["ARBaselineGap"].abs().gt(0)]
1238         final["q1"]["ar_baseline_gap_months"] = records(event_months[["period", "ARBaselineGap", "ARBaselineGap_days"]])
1239     if not q1_origin.empty:
1240         final["q1"]["origin_2026_02_forecast"] = records(q1_origin.sort_values("horizon_months"))
1241     if not q1_svar.empty:
1242         final["q1"]["svar_diagnostics"] = records(q1_svar.sort_values("candidate_lags"))
1243     if not q1_vol_summary.empty:
1244         final["q1"]["volatility_summary"] = records(q1_vol_summary)
1245     if not q1_robust.empty:
1246         final["q1"]["robustness_rows"] = int(len(q1_robust))
1247         final["q1"]["robustness_types"] = sorted(q1_robust["robustness_type"].dropna().unique().tolist()) if "robustness_type" in
1248         q1_robust else []
1249     if not q2_ardl.empty:
1250         final["q2"]["ardl_main"] = records(q2_ardl.sort_values(["outcome", "term"]))
1251     if not q2_irf.empty:
1252         final["q2"]["lp_selected_horizons"] = records(

```

```

1253     q2_irf.loc[q2_irf["horizon"].isin([0, 6, 12])].sort_values(["outcome", "horizon"])
1254 )
1255 if not q2_gdp.empty:
1256     final["q2"]["gdp_validation"] = records(q2_gdp)
1257 if not q2_transmission.empty:
1258     final["q2"]["transmission_metrics"] = records(q2_transmission.sort_values(["shock", "outcome"]))
1259 if not q2_robust.empty:
1260     final["q2"]["robustness"] = records(q2_robust.sort_values(["outcome", "lag_max", "exclude_covid"]))
1261
1262 if not q3_pass.empty:
1263     main_pass = q3_pass.loc[bool_series(q3_pass["included_in_main_comparison"])] if "included_in_main_comparison" in q3_pass else
1264     q3_pass
1265     final["q3"]["pass_through_h6_main_comparison"] = records(main_pass.loc[main_pass["horizon"].eq(6)].sort_values("country"))
1266     final["q3"]["china_proxy_sensitivity_h6"] = records(q3_pass.loc[q3_pass["country"].eq("CHN") & q3_pass["horizon"].eq(6)])
1267 if not q3_irf.empty:
1268     final["q3"]["panel_lp_selected_horizons"] = records(
1269         q3_irf.loc[q3_irf["horizon"].isin([0, 6, 12])].sort_values(["outcome", "country", "horizon"])
1270     )
1271 if not q3_buffer.empty:
1272     final["q3"]["buffer_interactions_selected_horizons"] = records(
1273         q3_buffer.loc[q3_buffer["horizon"].isin([0, 6, 12])].sort_values(["outcome", "buffer", "horizon"])
1274     )
1275 if not q3_policy.empty:
1276     final["q3"]["policy_counterfactual"] = records(q3_policy)
1277     final["q3"]["policy_max_cumulative_gap_cny_t"] = clean_value(q3_policy["cumulative_gasoline_gap_cny_t"].max())
1278     final["q3"]["policy_max_cpi_gap_pctpt"] = clean_value(q3_policy["cpi_counterfactual_gap_pctpt"].max())
1279 if not q3_policy_macro.empty:
1280     final["q3"]["policy_macro_counterfactual"] = records(q3_policy_macro.sort_values(["period", "outcome"]))
1281 if not q3_resilience.empty:
1282     final["q3"]["resilience_metrics"] = records(q3_resilience)
1283 if not q3_robust.empty:
1284     final["q3"]["robustness_rows"] = int(len(q3_robust))
1285     final["q3"]["robustness_types"] = sorted(q3_robust["robustness_type"].dropna().unique().tolist()) if "robustness_type" in
1286     q3_robust else []
1287 if not q4_risk.empty:
1288     final["q4"]["price_tail_risk"] = records(q4_risk.sort_values(["model", "horizon_months"]))
1289 if not q4_backtest.empty:
1290     final["q4"]["probabilistic_backtest"] = records(q4_backtest.sort_values(["horizon_months", "model"]))
1291 if not q4_macro.empty:
1292     final["q4"]["extreme_q95_macro_stress"] = records(
1293         q4_macro.loc[q4_macro["scenario"].eq("extreme_q95") & q4_macro["horizon"].isin([6, 12])].
1294         .sort_values(["outcome", "horizon"])
1295     )
1296 if not q4_policy.empty:
1297     final["q4"]["policy_buffer_2026_06"] = records(
1298         q4_policy.loc[q4_policy["period"].eq("2026-06")].sort_values("outcome")
1299     )
1300 if q4_summary:
1301     final["q4"]["evidence_status"] = q4_summary.get("evidence_status")
1302     final["q4"]["tail_risk_status"] = q4_summary.get("problem_status")
1303     final["q4"]["backtest_preferred_model_by_mean_pinball"] = q4_summary.get("backtest_preferred_model_by_mean_pinball")
1304     final["q4"]["guardrail"] = q4_summary.get("guardrail")
1305 if not q4_sapr_optimal.empty:
1306     final["q4"]["sapr_optimal_rule"] = records(q4_sapr_optimal)
1307 if not q4_sapr_comparison.empty:
1308     final["q4"]["sapr_holdout_strategy_comparison"] = records(
1309         q4_sapr_comparison.loc[q4_sapr_comparison["sample_split"].eq("holdout")].sort_values("strategy")
1310     )
1311     final["q4"]["sapr_2026_strategy_comparison"] = records(
1312         q4_sapr_comparison.loc[q4_sapr_comparison["sample_split"].eq("war_2026")].sort_values("strategy")
1313     )
1314 if not q4_sapr_paths.empty:
1315     final["q4"]["sapr_2026_macro_paths"] = records(q4_sapr_paths.sort_values(["strategy", "month_index"]))
1316 if not q4_sapr_sensitivity.empty:
1317     final["q4"]["sapr_sensitivity"] = records(q4_sapr_sensitivity.sort_values("variant"))
1318 if q4_sapr_summary:
1319     final["q4"]["sapr_execution_status"] = q4_sapr_summary.get("execution_status")
1320     final["q4"]["sapr_evidence_status"] = q4_sapr_summary.get("evidence_status")
1321     final["q4"]["sapr_holdout_non_dominated_probability"] = q4_sapr_summary.get("holdout_non_dominated_probability")
1322     final["q4"]["sapr_allowed_claims"] = q4_sapr_summary.get("allowed_claims", [])
1323     final["q4"]["sapr_forbidden_claims"] = q4_sapr_summary.get("forbidden_claims", [])
1324 return final
1325
1326 def build_data_overview_figures() -> None:
1327     apply_paper_style()

```

```

1327 FIGURES_DIR.mkdir(parents=True, exist_ok=True)
1328 monthly_path = REPO_ROOT / "data" / "processed" / "model_monthly_q1.csv"
1329 country_path = REPO_ROOT / "data" / "processed" / "model_country_monthly.csv"
1330 if monthly_path.exists():
1331     monthly = pd.read_csv(monthly_path, parse_dates=["month_end"])
1332     fig, ax1 = plt.subplots(figsize=(9.2, 5.1))
1333     ax1.plot(monthly["month_end"], monthly["brent_usd_bbl"], color=PALETTE["blue"], label="Brent", linewidth=1.8)
1334     style_axis(ax1, ylabel="Brent, 美元/桶")
1335     ax2 = ax1.twinx()
1336     ax2.plot(monthly["month_end"], monthly["GPR_z"], color=PALETTE["rose"], alpha=0.82, label="GPR标准化值", linewidth=1.35,
1337             linestyle=(0, (4, 2)))
1338     ax2.set_ylabel("GPR标准化值")
1339     ax2.tick_params(colors=PALETTE["muted"], length=3.2, width=0.65)
1340     ax2.spines["right"].set_color(PALETTE["muted"])
1341     ax2.spines["right"].set_linewidth(0.7)
1342     ax2.grid(False)
1343     lines, labels = ax1.get_legend_handles_labels()
1344     lines2, labels2 = ax2.get_legend_handles_labels()
1345     ax1.legend(lines + lines2, labels + labels2, loc="upper left", ncol=2, handlelength=2.8)
1346     finish_figure(
1347         fig,
1348         title="数据概览: Brent 油价与地缘政治风险",
1349         subtitle="月度 Brent 现货价与标准化 GPR 指数, 2010-01 至 2026-06.",
1350         source="来源: FRED Brent 与 Caldara-Iacoviello GPR 处理面板; 由 code/utils/freeze_results.py 生成。",
1351     )
1352     save_figure(fig, FIGURES_DIR / "data_overview_oil_gpr")
1353     plt.close(fig)
1354 if country_path.exists():
1355     panel = pd.read_csv(country_path, parse_dates=["month_end"])
1356     panel = panel.dropna(subset=["fuel_price_local"]).copy()
1357     panel["fuel_index"] = panel.groupby("country")["fuel_price_local"].transform(lambda s: 100.0 * s / s.dropna().iloc[0])
1358     fig, ax = plt.subplots(figsize=(9.2, 5.1))
1359     style_map = {
1360         "CHN": (PALETTE["blue"], "solid"),
1361         "DEU": (PALETTE["gold"], (0, (4, 2))),
1362         "FRA": (PALETTE["olive"], (0, (2, 2))),
1363         "ITA": (PALETTE["rose"], (0, (1, 2))),
1364         "ESP": (PALETTE["slate"], (0, (6, 2))),
1365         "JPN": (PALETTE["sand"], (0, (3, 1, 1, 1))),
1366         "KOR": (PALETTE["blue_light"], (0, (1, 1))),
1367     }
1368     for country, group in panel.groupby("country"):
1369         color, linestyle = style_map.get(country, (PALETTE["slate"], "solid"))
1370         ax.plot(group["month_end"], group["fuel_index"], label=COUNTRY_LABEL_ZH.get(country, country), color=color, linestyle=
1371             linestyle, linewidth=1.55)
1372     style_axis(ax, ylabel="指数, 2010-01=100")
1373     ax.legend(loc="upper left", ncol=4, handlelength=2.8)
1374     finish_figure(
1375         fig,
1376         title="数据概览: 燃油价格指数",
1377         subtitle="各国燃油价格序列按各自首个可用月归一; 中国序列为调价指数代理, 仅作敏感性展示。",
1378         source="来源: 欧盟周度油价公报、日本METI、韩国KOSIS/KNOC、北京市发改委; 由 code/utils/freeze_results.py 生成。",
1379     )
1380     save_figure(fig, FIGURES_DIR / "data_overview_fuel_panel")
1381     plt.close(fig)
1382
1383 def summarize_warnings() -> list[dict[str, Any]]:
1384     warnings_list: list[dict[str, Any]] = []
1385     seen: dict[tuple[str, str], dict[str, Any]] = {}
1386     for filename in ["data_warnings.json", "q1_summary.json", "q2_summary.json", "q3_summary.json", "q4_summary.json", "q4_sapr_summary.
1387         json", "model_panel_summary.json"]:
1388         payload = load_json(filename)
1389         for warning in payload.get("warnings", []):
1390             row = dict(warning)
1391             key = (str(row.get("code", "warning")), str(row.get("message", "")))
1392             if key not in seen:
1393                 row["sources"] = [filename]
1394                 seen[key] = row
1395                 warnings_list.append(row)
1396             elif filename not in seen[key]["sources"]:
1397                 seen[key]["sources"].append(filename)
1398         for stage, stage_warnings in payload.get("stage_warnings", {}).items():
1399             for warning in stage_warnings:
1400                 row = dict(warning)
1401                 source = f"{filename}:{stage}"

```

```

1400     key = (str(row.get("code", "warning")), str(row.get("message", "")))
1401     if key not in seen:
1402         row["sources"] = [source]
1403         seen[key] = row
1404         warnings_list.append(row)
1405     elif source not in seen[key]["sources"]:
1406         seen[key]["sources"].append(source)
1407     return warnings_list
1408
1409
1410 def markdown_table(frame: pd.DataFrame, columns: list[str], limit: int = 12) -> str:
1411     if frame.empty:
1412         return "_暂无可用结果。_"
1413     subset = frame.copy()
1414     for column in columns:
1415         if column not in subset.columns:
1416             subset[column] = ""
1417     subset = subset[columns].head(limit).copy()
1418     for column in subset.columns:
1419         if pd.api.types.is_numeric_dtype(subset[column]):
1420             subset[column] = subset[column].map(lambda value: "" if pd.isna(value) else f"{float(value):.4f}")
1421         else:
1422             subset[column] = subset[column].fillna("").astype(str)
1423     header = "| " + " | ".join(subset.columns) + " |"
1424     divider = "| " + " | ".join(["---"] * len(subset.columns)) + " |"
1425     body = []
1426     for row in subset.to_dict("records"):
1427         body.append("| " + " | ".join(str(row[column]).replace("\n", " ") for column in subset.columns) + " |")
1428     return "\n".join([header, divider] + body)
1429
1430
1431 def build_report(risk_summary: dict[str, Any], final_numbers: dict[str, Any], warnings_list: list[dict[str, Any]]) -> str:
1432     q1_metrics = read_csv("q1_forecast_metrics.csv")
1433     q1_events = read_csv("q1_event_effects.csv")
1434     q1_origin = read_csv("q1_origin_forecast.csv")
1435     q1_svar = read_csv("q1_svar_diagnostics.csv")
1436     q1_vol_summary = read_csv("q1_volatility_summary.csv")
1437     q2_irf = read_csv("q2_irf.csv")
1438     q2_gdp = read_csv("q2_gdp_validation.csv")
1439     q2_metrics = read_csv("q2_transmission_metrics.csv")
1440     q3_pass = read_csv("q3_country_pass_through.csv")
1441     q3_policy = read_csv("q3_policy_counterfactual.csv")
1442     q3_policy_macro = read_csv("q3_policy_macro_counterfactual.csv")
1443     q3_resilience = read_csv("q3_resilience_metrics.csv")
1444     q4_risk = read_csv("q4_price_tail_risk.csv")
1445     q4_backtest = read_csv("q4_risk_backtest.csv")
1446     q4_macro = read_csv("q4_macro_stress.csv")
1447     q4_policy = read_csv("q4_policy_stress.csv")
1448     q4_sapr_optimal = read_csv("q4_sapr_optimal_rule.csv")
1449     q4_sapr_comparison = read_csv("q4_sapr_strategy_comparison.csv")
1450     q4_sapr_sensitivity = read_csv("q4_sapr_sensitivity.csv")
1451     figures = collect_figures()
1452     nbs_iav = pd.read_csv(REPO_ROOT / "data" / "processed" / "nbs_iav_monthly.csv")
1453     nbs_ppi = pd.read_csv(REPO_ROOT / "data" / "processed" / "nbs_ppi_monthly.csv")
1454     nbs_iav_count = int(pd.to_numeric(nbs_iav.get("china_iav_yoy_pct"), errors="coerce").notna().sum())
1455     nbs_ppi_count = int(pd.to_numeric(nbs_ppi.get("china_ppi_yoy_pct"), errors="coerce").notna().sum())
1456
1457     q3_main = q3_pass.loc[bool_series(q3_pass["included_in_main_comparison"])] if not q3_pass.empty and "included_in_main_comparison" in q3_pass else q3_pass
1458     warning_lines = []
1459     for warning in warnings_list[:20]:
1460         warning_lines.append(f"- {warning.get('code', 'warning')}: {warning.get('message', '')}")
1461     if not warning_lines:
1462         warning_lines.append("- 暂无模型执行 warning。")
1463     blocker_lines = [f"- {probe_id}" for probe_id in risk_summary["blocking_probe_ids"]]
1464     if not blocker_lines:
1465         blocker_lines.append("- 无。")
1466
1467     if risk_summary.get("paper_finalize_allowed"):
1468         report_title = "国际油价四问建模封板结果报告"
1469         execution_mode = "paper-ready modeling freeze"
1470         conclusion_text = (
1471             "本轮结果已通过风险探针、Schema 校验、哈希冻结与 `--require-final` 验证，"
1472             "可以作为论文定稿数字来源。论文仍需保持证据边界：Q2 的宏观增长损失证据为 "
1473             "'INCONCLUSIVE'，Q3 对“我国应对更好”的综合判断为 `PARTIAL`。"
1474         )

```

```

1475     boundary_text = (
1476         "当前可写入论文的主线是：问题一采用 `no_change` 主预测、交易日 CAR/placebo 事件证据、"
1477         "历史递归 SVAR 三类结构冲击和描述性 `ARBaselineGap`；问题二采用结构冲击主规格与 "
1478         "均化形式稳健性，报告人民币原油成本—PPI—CPI/工业活动—GDP 传导链；问题三使用中国"
1479         "中国调价指数代理退出正式价格水平排名，待审计缓冲交互不发布，并输出条件动态政策情景；"
1480         "问题四使用 FHS-GJR-GARCH 报告尾部概率，将 Q2 结构冲击压力与 Q3 已实现政策反事实分层展示，"
1481         "并进一步用 SAPR-CVaR 在现行调价机制约束上优化临时平滑规则。"
1482     )
1483     paper_advice = (
1484         "论文正文可把当前结果作为封板数值使用，但必须区分预测表现、事件相关异常、结构冲击传导"
1485         "和政策情景模拟。第一问不得把 `ARBaselineGap` 写成严格战争净贡献；第二问不得在联合区间"
1486         "未排除零时写“显著增长损失”；第三问不得仅凭价格传导或单一价格监管变量断言中国政策具有"
1487         "一般因果优势；问题四不得把尾部概率写成确定结果，不得把 Q2 与 Q3 的不同证据层直接相加，"
1488         "也不得把 SAPR 的注册规则族最优写成全球福利最优。"
1489     )
1490     else:
1491         report_title = "国际油价四问阶段性建模结果报告"
1492         execution_mode = "staged modeling audit"
1493         conclusion_text = (
1494             "本轮结果仍是 `CONDITIONAL` 阶段快照。代码、图表和结果可继续作为建模推进基础，"
1495             "但论文正文不能把当前输出写成定稿结论。"
1496         )
1497         boundary_text = (
1498             "当前需要先处理下方阻塞门禁，再重新冻结结果。已通过的模型输出只能作为阶段性证据，"
1499             "不得越过门禁写入最终论文数字。"
1500         )
1501         paper_advice = "只有在风险门禁全部 `PASS` 后，才可把冻结文件作为定稿数值来源。"
1502     return f"""# {report_title}
1503
1504
1505     ## Material Passport
1506
1507     - Origin Skill: `math-modeling-solver + math-modeling-paper`
1508     - Execution Mode: `{execution_mode}`
1509     - Verification Status: `{risk_summary['overall_status']}`
1510     - Paper Finalize Allowed: `{str(risk_summary['paper_finalize_allowed']).lower()}`
1511     - Cutoff: `{CUTOFF}`
1512     - Random Seed: `{RANDOM_SEED}`
1513
1514     ## 1. 总体结论
1515
1516     {conclusion_text}
1517
1518     {boundary_text}
1519
1520     当前阻塞定稿的门禁：
1521
1522     {chr(10).join(blocker_lines)}
1523
1524     ## 2. 问题一：预测与事件窗口
1525
1526     月度预测表已经加入相对基线指标和逐模型状态。当前主预测模型是 `no_change`。
1527
1528     {markdown_table(q1_metrics.sort_values(['horizon', 'model']) if not q1_metrics.empty else q1_metrics, ['model', 'horizon', 'RMSE', '
1529         relative_RMSE_vs_no_change', 'dm_hln_pvalue_rmse_loss', 'model_status'])}
1530
1531     以 2026-02 为原点的竞赛最终预测表如下。2026-08 在数据截止日之后，保留为 `FORECAST_ONLY`。
1532
1533     {markdown_table(q1_origin.sort_values('horizon_months') if not q1_origin.empty else q1_origin, ['origin_period', 'target_period', '
1534         horizon_months', 'model', 'forecast_status', 'origin_price', 'prediction_price', 'actual_price', 'forecast_error_price'])}
1535
1536     历史结构冲击使用递归 VAR/Cholesky 分解，变量排序为“全球供给增长 → 全球真实经济活动 → 实际 Brent 收益”。被选规格如下：
1537
1538     {markdown_table(q1_svar.loc[bool_series(q1_svar['is_selected'])] if not q1_svar.empty and 'is_selected' in q1_svar else q1_svar, ['
1539         candidate_lags', 'is_selected', 'bic', 'is_stable', 'whiteness_pvalue', 'sample_start', 'sample_end', 'nobs'])}
1540
1541     GJR-GARCH 条件波动率用于描述战争阶段油价波动变化，不作为战争净因果效应。
1542
1543     {markdown_table(q1_vol_summary, ['war_stage', 'trading_days', 'conditional_vol_pct_median', 'vol_multiple_vs_pre_median', '
1544         evidence_status'])}
1545
1546     E1 已从 2026-02-28 周末映射到 2026-03-02 交易日，同时输出 CAR[0]、CAR[0,+1]、CAR[0,+2] 和匹配周末 placebo 经验 p 值。经验 p 值采用  $((b+1)/(B+1))$  的有限样本修正，不报告严格的 0。
1547
1548     {markdown_table(q1_events.loc[q1_events['model'].isin(['brent_usd_bbl_stage_dummy', 'brent_usd_bbl_event_car'])].sort_values(['model', '
1549         stage_id']) if not q1_events.empty else q1_events, ['model', 'stage_id', 'estimate_log_return', 'std_error', 'lower_95', 'upper_95

```



```

1545     ', 'pvalue', 'pvalue_empirical', 'event_observations']])
1546 ## 3. 问题二：中国宏观传导
1547
1548 国家统计局 IAV/PPI 官方历史已经进入处理层，其中工业增加值有 {nbs_iav_count} 个真实观测，PPI 有 {nbs_ppi_count} 个真实观测；官方 1—2 月结
    构性空值保持为空，不做插值。Q2 当前只能写为“尚未发现稳健的总体增长损失证据”，所有结果仍需带区间与识别 caveat 报告。
1549
1550 {markdown_table(q2_irf.loc[q2_irf['horizon'].isin([0, 6, 12])).sort_values(['outcome', 'shock', 'horizon']) if not q2_irf.empty else
    q2_irf, ['outcome', 'shock', 'horizon', 'response', 'lower_95', 'upper_95', 'joint_lower_95', 'joint_upper_95', 'fdr_qvalue', '
    inference_band'])})
1551
1552 传导链摘要按冲击—变量输出峰值/谷值、累计响应和证据状态。当前即使部分点估计方向符合直觉，也不自动升级为“增长损失”结论。
1553
1554 {markdown_table(q2_metrics.loc[q2_metrics['shock'].isin(['supply_shock', 'aggregate_demand_shock', 'oil_specific_risk_shock'])].
    sort_values(['shock', 'outcome']) if not q2_metrics.empty else q2_metrics, ['shock', 'outcome', 'extremum_type', 'extremum_response
    ', 'extremum_month', 'response_curve_area_0_6', 'response_curve_area_0_12', 'area_unit', 'evidence_status', '
    allows_growth_loss_language'])})
1555
1556 季度 GDP 只作低频验证，不插值成月度变量。
1557
1558 {markdown_table(q2_gdp, ['outcome', 'estimate', 'lower_95', 'upper_95', 'pvalue', 'n', 'sample_start', 'sample_end'])})
1559
1560 ## 4. 问题三：政策缓冲与跨国比较
1561
1562 跨国燃油主排名只纳入覆盖充分且价格层可解释的观测零售汽油价格。中国历史路径是受管制汽油调价指数代理，只保留为敏感性材料；2026政策公告差额
    单独核验。
1563
1564 {markdown_table(q3_main.loc[q3_main['horizon'].eq(6)].sort_values('country') if not q3_main.empty else q3_main, ['country', 'horizon', '
    response', 'lower_95', 'upper_95', 'price_measure_type', 'included_in_main_comparison'])})
1565
1566 综合韧性指标把燃油传导、CPI 相对响应、工业活动相对响应和政策关闭情景放在同一口径下判断。当前总体判断为 `PARTIAL`，不是无条件支持“中国显
    著更好”。
1567
1568 {markdown_table(q3_resilience, ['dimension', 'metric', 'horizon', 'china_value', 'control_median', 'china_vs_control_median_diff', '
    diff_lower_95', 'diff_upper_95', 'judgement', 'overall_china_resilience_judgement'])})
1569
1570 中国政策图与数据表已区分新增差额和累计差额，2026-04 的累计差额为 1425 元/吨，4月新增为 380 元/吨。
1571
1572 {markdown_table(q3_policy, ['period', 'policy_adjusted_official_cny_t', 'no_temporary_control_official_cny_t', '
    incremental_gasoline_gap_cny_t', 'cumulative_gasoline_gap_cny_t', 'cpi_counterfactual_gap_pctpt', 'price_layer_status'])})
1573
1574 政策关闭宏观情景在明确标注的调价指数代理层上，将2026临时调控收益差通过0—6阶核和结果变量递归传播到PPI、CPI与IAV；区间来自三方方程共同时间块
    重采样。
1575
1576 {markdown_table(q3_policy_macro.loc[q3_policy_macro['period'].eq('2026-06')].sort_values('outcome') if not q3_policy_macro.empty else
    q3_policy_macro, ['period', 'outcome_label', 'macro_counterfactual_gap_pctpt', 'lower_95', 'upper_95', 'price_layer_status', '
    evidence_status'])})
1577
1578 ## 5. 问题四：极端油价尾部风险、政策压力测试与自适应调价规则优化
1579
1580 问题四把同事已完成的“尾部风险与压力测试”作为风险输入层，并把本轮 SAPR-CVaR 自适应调价规则作为政策优化层：先判断极端油价路径概率与宏观压
    力，再回答我国成品油调价机制在极端冲击下应如何状态依赖地平滑传导。油价概率层以 2026-06-30 最后交易日为原点，比较 FHS-GJR-GARCH 与
    恒定波动高斯随机游走；历史 90%/95%价格分位只作为上行压力阈值。
1581
1582 {markdown_table(q4_risk.sort_values(['model', 'horizon_months']) if not q4_risk.empty else q4_risk, ['model', 'horizon_months', '
    median_price', 'p05_price', 'p95_price', 'terminal_prob_above_hist_p90', 'terminal_prob_above_hist_p95', 'path_prob_cross_hist_p95
    '])})
1583
1584 滚动回测在同一原点、期限和评价口径下比较主方法与基线；主方法是否胜出由分位损失和覆盖率共同判断，不因预设方法而更换回测区间。
1585
1586 {markdown_table(q4_backtest.sort_values(['horizon_months', 'model']) if not q4_backtest.empty else q4_backtest, ['model', 'horizon_months
    ', 'origins', 'mean_pinball_loss', 'median_absolute_error', 'coverage_80', 'coverage_90', 'mean_width_90'])})
1587
1588 宏观压力层使用 Q1 油价特定风险冲击的历史正向分位数缩放 Q2 IRF。下表只展示 95%分位冲击的 6、12 月条件响应；联合区间跨零时继续标记为 `
    INCONCLUSIVE`。
1589
1590 {markdown_table(q4_macro.loc[q4_macro['scenario'].eq('extreme_q95') & q4_macro['horizon'].isin([6, 12])).sort_values(['outcome', 'horizon
    ']) if not q4_macro.empty else q4_macro, ['scenario', 'outcome_label', 'horizon', 'conditional_response_pctpt', 'joint_lower_95', '
    joint_upper_95', 'row_evidence_status'])})
1591
1592 政策压力层只重述 Q3 已实现的 2026 年临时调控关闭反事实，不外推到模拟油价路径。正的“政策缓冲收益”分别表示避免的 PPI/CPI 增幅或避免的工业
    活动损失。
1593
1594 {markdown_table(q4_policy.loc[q4_policy['period'].eq('2026-06')].sort_values('outcome') if not q4_policy.empty else q4_policy, ['period',
    'outcome_label', 'policy_buffer_benefit_pctpt', 'benefit_lower_95', 'benefit_upper_95', 'evidence_status'])})
1595

```

```

1596 SAPR-CVaR 优化层只在 2013-03—2021-12 开发样本上确定阈值和规则，2022-01—2026-06 完全作为隔离检验样本；2026 年战争冲击只用于检验和展示，
    不参与规则选择。三档传导率满足 `普通 ≥ 压力 ≥ 极端`，目标函数同时考虑宏观损失、95% CVaR、累计未调价负担和国内调价波动。
1597
1598 {markdown_table(q4_sapr_optimal, ['rule_id', 'rho_normal', 'rho_stress', 'rho_extreme', 'stress_threshold_75_cny_t', '
    stress_threshold_95_cny_t', 'J1_macro_loss', 'J2_cvar95_macro_loss', 'J3_avg_gap_month_burden', 'J4_adjustment_volatility', '
    max_gap_ratio', 'max_recovery_terminal_gap_ratio'])}
1599
1600 隔离检验样本与 2026 实际冲击情景的策略比较如下。若 SAPR 在检验样本被预注册基线支配，论文必须报告负结果；当前证据状态以 `q4_sapr_summary.
    json` 为准。
1601
1602 {markdown_table(q4_sapr_comparison.loc[q4_sapr_comparison['sample_split'].isin(['holdout', 'war_2026'])].sort_values(['sample_split', '
    strategy'])) if not q4_sapr_comparison.empty else q4_sapr_comparison, ['sample_split', 'strategy', 'rho_normal', 'rho_stress', '
    rho_extreme', 'J1_macro_loss', 'J2_cvar95_macro_loss', 'J3_avg_gap_month_burden', 'J4_adjustment_volatility', 'max_gap_ratio', '
    max_recovery_terminal_gap_ratio'])}
1603
1604 敏感性检验固定改变阈值、bootstrap 块长和 CPI/IAV 权重，不为追求结论改动样本或目标函数。
1605
1606 {markdown_table(q4_sapr_sensitivity, ['variant', 'rho_normal', 'rho_stress', 'rho_extreme', 'pareto_rule_count', '
    holdout_non_dominated_probability', 'changed_from_default'])}
1607
1608 ## 6. 图表与冻结文件
1609
1610 核心 PNG 图表：{'', '.join(figures) if figures else '暂无图表'}。
1611
1612 冻结文件：
1613
1614 - `results/final_numbers.json`
1615 - `results/frozen_numbers.json`
1616 - `results/risk_probe_summary.json`
1617 - `results/reproducibility_manifest.json`
1618
1619 其中 `frozen_numbers.json` 遵循 `3coding-visual` 标准冻结格式；风险门禁以及代码、输入、输出文件哈希保存在独立的 reproducibility manifest
    中。数值一致性使用标准 skill 脚本检查，项目级文件与环境检查使用 `python code/utils/verify_freeze.py`。
1620
1621 ## 7. Warnings
1622
1623 {chr(10).join(warning_lines)}
1624
1625 ## 8. 论文使用建议
1626
1627 {paper_advice}
1628 """
1629
1630
1631 def write_json(path: Path, payload: dict[str, Any]) -> None:
1632     path.write_text(
1633         json.dumps(payload, ensure_ascii=False, indent=2, sort_keys=True, allow_nan=False) + "\n",
1634         encoding="utf-8",
1635     )
1636
1637
1638 def standard_freeze_payload(final_numbers: dict[str, Any]) -> dict[str, Any]:
1639     return {
1640         "schema_version": STANDARD_FREEZE_SCHEMA_VERSION,
1641         "frozen_at": datetime.now(timezone.utc).isoformat().replace("+00:00", "Z"),
1642         "source_file": "results/final_numbers.json",
1643         "source_sha256": sha256_json(final_numbers),
1644         "values": final_numbers,
1645     }
1646
1647
1648 def reproducibility_manifest(
1649     risk_summary: dict[str, Any],
1650     final_numbers: dict[str, Any],
1651     risk_hash: str,
1652     output_hashes: dict[str, str],
1653     input_hashes: dict[str, str],
1654     code_hashes: dict[str, str],
1655 ) -> dict[str, Any]:
1656     source = {
1657         "schema_version": MANIFEST_SCHEMA_VERSION,
1658         "snapshot_mode": "paper_final" if risk_summary["paper_finalize_allowed"] else "stage_snapshot_not_paper_final",
1659         "overall_status": risk_summary["overall_status"],
1660         "paper_finalize_allowed": risk_summary["paper_finalize_allowed"],
1661         "git_commit": git_commit(),
1662         "source_commit": git_commit(),
    
```

```

1663     "cutoff": CUTOFF,
1664     "random_seed": RANDOM_SEED,
1665     "replay_environment": replay_environment_snapshot(),
1666     "packaging_environment": environment_snapshot(),
1667     "code_hashes": code_hashes,
1668     "processed_input_hashes": input_hashes,
1669     "core_result_hashes": output_hashes,
1670     "risk_gate_hash": risk_hash,
1671     "final_numbers_source_sha256": sha256_json(final_numbers),
1672     "standard_freeze_file": "results/frozen_numbers.json",
1673 }
1674 manifest = dict(source)
1675 manifest["manifest_hash"] = sha256_json(source)
1676 return manifest
1677
1678
1679 def main() -> int:
1680     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
1681     REPORTS_DIR.mkdir(parents=True, exist_ok=True)
1682     build_data_overview_figures()
1683
1684     output_hashes = hash_existing(RESULTS_DIR, CORE_RESULT_FILES)
1685     input_hashes = hash_existing(REPO_ROOT / "data" / "processed", PROCESSED_INPUT_FILES)
1686     code_hashes = hash_existing(REPO_ROOT, CODE_FILES)
1687     risk_summary = build_risk_probe_summary(output_hashes, input_hashes, code_hashes)
1688     risk_path = RESULTS_DIR / "risk_probe_summary.json"
1689     write_json(risk_path, risk_summary)
1690     risk_hash = sha256_file(risk_path)
1691
1692     final_numbers = extract_final_numbers()
1693     final_path = RESULTS_DIR / "final_numbers.json"
1694     write_json(final_path, final_numbers)
1695
1696     frozen_path = RESULTS_DIR / "frozen_numbers.json"
1697     frozen = standard_freeze_payload(final_numbers)
1698     write_json(frozen_path, frozen)
1699
1700     manifest = reproducibility_manifest(risk_summary, final_numbers, risk_hash, output_hashes, input_hashes, code_hashes)
1701     manifest_path = RESULTS_DIR / "reproducibility_manifest.json"
1702     write_json(manifest_path, manifest)
1703
1704     warnings_list = summarize_warnings()
1705     report = build_report(risk_summary, final_numbers, warnings_list)
1706     (REPORTS_DIR / "RESULTS_REPORT.md").write_text(report.rstrip() + "\n", encoding="utf-8")
1707
1708     print(
1709         json.dumps(
1710             {
1711                 "status": risk_summary["overall_status"],
1712                 "paper_finalize_allowed": risk_summary["paper_finalize_allowed"],
1713                 "source_sha256": frozen["source_sha256"],
1714                 "manifest_hash": manifest["manifest_hash"],
1715                 "blocking_probe_ids": risk_summary["blocking_probe_ids"],
1716             },
1717             ensure_ascii=False,
1718             indent=2,
1719         )
1720     )
1721     return 0
1722
1723
1724 if __name__ == "__main__":
1725     raise SystemExit(main())

```

Listing 13: freeze_results.py

```

1 """Verify the standard numerical freeze and the project reproducibility manifest."""
2
3 from __future__ import annotations
4
5 import argparse
6 import hashlib
7 import importlib.metadata
8 import json
9 import platform

```

```

10 import subprocess
11 import sys
12 from pathlib import Path
13 from typing import Any
14
15 import numpy as np
16 import pandas as pd
17 from jsonschema import Draft202012Validator
18
19
20 REPO_ROOT = Path(__file__).resolve().parents[2]
21 RESULTS_DIR = REPO_ROOT / "results"
22 PROCESSED_DIR = REPO_ROOT / "data" / "processed"
23 SCHEMA_DIR = REPO_ROOT / "code" / "schemas"
24
25 if hasattr(sys.stdout, "reconfigure"):
26     sys.stdout.reconfigure(encoding="utf-8")
27
28 STANDARD_FROZEN_REQUIRED = [
29     "schema_version",
30     "frozen_at",
31     "source_file",
32     "source_sha256",
33     "values",
34 ]
35 MANIFEST_REQUIRED = [
36     "schema_version",
37     "snapshot_mode",
38     "overall_status",
39     "paper_finalize_allowed",
40     "git_commit",
41     "source_commit",
42     "cutoff",
43     "random_seed",
44     "replay_environment",
45     "packaging_environment",
46     "code_hashes",
47     "processed_input_hashes",
48     "core_result_hashes",
49     "risk_gate_hash",
50     "final_numbers_source_sha256",
51     "standard_freeze_file",
52     "manifest_hash",
53 ]
54 RISK_REQUIRED = [
55     "schema_version",
56     "overall_status",
57     "paper_finalize_allowed",
58     "cutoff",
59     "random_seed",
60     "probes",
61     "blocking_probe_ids",
62     "core_results_present",
63     "figure_pngs",
64     "output_hash_count",
65     "input_hash_count",
66     "model_code_hash_count",
67 ]
68
69
70 def sha256_file(path: Path) -> str:
71     payload = path.read_bytes()
72     if path.suffix.lower() in {".csv", ".json", ".md", ".py", ".txt"}:
73         payload = payload.replace(b"\r\n", b"\n").replace(b"\r", b"\n")
74     return hashlib.sha256(payload).hexdigest()
75
76
77 def sha256_json(payload: Any) -> str:
78     text = json.dumps(payload, ensure_ascii=False, sort_keys=True, separators=(",", ":"), allow_nan=False)
79     return hashlib.sha256(text.encode("utf-8")).hexdigest()
80
81
82 def load_json(path: Path) -> dict[str, Any]:
83     return json.loads(path.read_text(encoding="utf-8"))
84
85

```

```

86 def require_keys(payload: dict[str, Any], keys: list[str], label: str, errors: list[str]) -> None:
87     missing = [key for key in keys if key not in payload]
88     if missing:
89         errors.append(f"{label} missing required keys: {missing}")
90
91
92 def validate_schema(payload: dict[str, Any], schema_filename: str, label: str, errors: list[str]) -> None:
93     schema_path = SCHEMA_DIR / schema_filename
94     if not schema_path.exists():
95         errors.append(f"{label} schema missing: {schema_filename}")
96         return
97     schema = load_json(schema_path)
98     validator = Draft202012Validator(schema)
99     schema_errors = sorted(validator.iter_errors(payload), key=lambda err: list(err.path))
100     for error in schema_errors:
101         location = ".".join(str(part) for part in error.path) or "<root>"
102         errors.append(f"{label} schema error at {location}: {error.message}")
103
104
105 def verify_source_commit_lineage(frozen: dict[str, Any], errors: list[str]) -> None:
106     source_commit = frozen.get("source_commit")
107     if not source_commit or source_commit == "UNKNOWN":
108         errors.append("frozen source_commit is missing or UNKNOWN")
109         return
110     result = subprocess.run(
111         ["git", "merge-base", "--is-ancestor", source_commit, "HEAD"],
112         cwd=REPO_ROOT,
113         text=True,
114         capture_output=True,
115         check=False,
116     )
117     if result.returncode != 0:
118         errors.append(f"current HEAD does not contain frozen source_commit {source_commit}")
119
120
121 def verify_hashes(base_dir: Path, hashes: dict[str, str], label: str, errors: list[str]) -> None:
122     for filename, expected in hashes.items():
123         path = base_dir / filename
124         if not path.exists():
125             errors.append(f"{label} file missing: {filename}")
126             continue
127         actual = sha256_file(path)
128         if actual != expected:
129             errors.append(f"{label} hash mismatch: {filename}")
130
131
132 def bool_series(series: pd.Series) -> pd.Series:
133     if pd.api.types.is_bool_dtype(series):
134         return series
135     return series.astype(str).str.lower().isin(["true", "1", "yes"])
136
137
138 def verify_environment(manifest: dict[str, Any], messages: list[str]) -> None:
139     env = manifest.get("replay_environment", {})
140     if env.get("python_version") != platform.python_version():
141         messages.append(f"replay python mismatch: required={env.get('python_version')} current={platform.python_version()}")
142     packages = env.get("packages", {})
143     for package, manifest_version in packages.items():
144         try:
145             current = importlib.metadata.version(package)
146         except importlib.metadata.PackageNotFoundError:
147             current = "NOT_INSTALLED"
148         if current != manifest_version:
149             messages.append(f"replay package mismatch: {package} required={manifest_version} current={current}")
150
151
152 def verify_key_result_shapes(errors: list[str]) -> None:
153     q1_metrics = pd.read_csv(RESULTS_DIR / "q1_forecast_metrics.csv")
154     required_q1 = {"model", "horizon", "relative_RMSE_vs_no_change", "model_status"}
155     if not required_q1.issubset(q1_metrics.columns):
156         errors.append("q1_forecast_metrics.csv lacks forecast gate columns")
157     else:
158         advanced_pass = q1_metrics.loc[q1_metrics["model"].ne("no_change") & q1_metrics["model_status"].eq("PASS")]
159         advanced_worse = q1_metrics.loc[q1_metrics["model"].ne("no_change") & q1_metrics["relative_RMSE_vs_no_change"].gt(1.0)]
160         if not advanced_pass.merge(advanced_worse[["model", "horizon"]], on=["model", "horizon"], how="inner").empty:
161             errors.append("advanced Q1 model with RMSE worse than no-change is still marked PASS")

```

```

162
163 q1_events = pd.read_csv(RESULTS_DIR / "q1_event_effects.csv")
164 empirical = pd.to_numeric(
165     q1_events.loc[q1_events["model"].eq("brent_usd_bbl_event_car"), "pvalue_empirical"],
166     errors="coerce",
167 ).dropna()
168 if empirical.empty or not empirical.between(0.0, 1.0, inclusive="right").all() or empirical.eq(0.0).any():
169     errors.append("Q1 empirical placebo p-values are missing, outside (0, 1), or still report zero")
170
171 q1_origin = pd.read_csv(RESULTS_DIR / "q1_origin_forecast.csv")
172 origin_horizons = sorted(pd.to_numeric(q1_origin.get("horizon_months", pd.Series(dtype=float)), errors="coerce").dropna().astype(int)
173     .tolist())
174 if origin_horizons != [1, 3, 6]:
175     errors.append("q1_origin_forecast.csv must contain exactly 1/3/6 month horizons")
176 six_month = q1_origin.loc[q1_origin["horizon_months"].eq(6)]
177 if six_month.empty or not six_month["forecast_status"].astype(str).eq("FORECAST_ONLY").all():
178     errors.append("Q1 2026-08 six-month origin forecast must be retained and marked FORECAST_ONLY")
179
180 q1_svar = pd.read_csv(RESULTS_DIR / "q1_svar_diagnostics.csv")
181 selected = q1_svar.loc[q1_svar.get("is_selected", pd.Series(dtype=bool)).astype(str).str.lower().isin(["true", "1"])]
182 if len(selected) != 1 or not selected["is_stable"].astype(str).str.lower().isin(["true", "1"]).all():
183     errors.append("q1_svar_diagnostics.csv must identify exactly one selected stable VAR specification")
184 q1_stationarity = pd.read_csv(RESULTS_DIR / "q1_stationarity_diagnostics.csv")
185 if set(q1_stationarity.get("variable", pd.Series(dtype=str))) != {"supply_growth", "global_real_activity", "real_brent_return"}:
186     errors.append("q1_stationarity_diagnostics.csv must cover all three SVAR inputs")
187
188 q2_irf = pd.read_csv(RESULTS_DIR / "q2_irf.csv")
189 q2_metrics = pd.read_csv(RESULTS_DIR / "q2_transmission_metrics.csv")
190 iav_horizons = sorted(pd.to_numeric(q2_irf.loc[q2_irf["outcome"].eq("china_iav_yoy_pct"), "horizon"], errors="coerce").dropna().
191     astype(int).unique().tolist())
192 if iav_horizons != [0, 3, 6, 12]:
193     errors.append("Q2 IAV response horizons must be the pre-specified 0/3/6/12 grid")
194 required_q2_metrics = {"evidence_status", "allows_growth_loss_language", "response_curve_area_0_6", "response_curve_area_0_12", "
195     area_unit", "available_horizons"}
196 if q2_metrics.empty or not required_q2_metrics.issubset(q2_metrics.columns):
197     errors.append("q2_transmission_metrics.csv lacks required transmission-metric columns")
198 iav_metrics = q2_metrics.loc[q2_metrics["outcome"].eq("china_iav_yoy_pct")]
199 if not iav_metrics.empty and iav_metrics[["response_curve_area_0_6", "response_curve_area_0_12"]].notna().any().any():
200     errors.append("Q2 IAV sparse horizons must not be mechanically summed into a curve area")
201
202 q3_pass = pd.read_csv(RESULTS_DIR / "q3_country_pass_through.csv")
203 required_q3 = {"price_measure_type", "observed_or_regulated", "included_in_main_comparison", "comparability_note"}
204 if not required_q3.issubset(q3_pass.columns):
205     errors.append("q3_country_pass_through.csv lacks comparability columns")
206 else:
207     chn_rows = q3_pass.loc[q3_pass["country"].eq("CHN")]
208     chn_included = bool_series(chn_rows["included_in_main_comparison"]) if not chn_rows.empty else pd.Series(dtype=bool)
209     chn_proxy = chn_rows["price_measure_type"].astype(str).str.contains("proxy", case=False, na=False) if not chn_rows.empty else pd.
210     Series(dtype=bool)
211     if bool((chn_included & chn_proxy).any()):
212         errors.append("China proxy is included in the main Q3 fuel comparison")
213     if chn_rows.empty or bool(chn_included.any()) or not bool(chn_proxy.all()):
214         errors.append("China must be present only as an excluded adjustment-index proxy sensitivity in Q3")
215
216 q3_buffer = pd.read_csv(RESULTS_DIR / "q3_buffer_interactions.csv")
217 required_buffer = {"outcome", "buffer", "shock", "estimate", "lower_95", "upper_95", "data_status"}
218 if not required_buffer.issubset(q3_buffer.columns):
219     errors.append("q3_buffer_interactions.csv lacks policy-buffer interaction columns")
220 if not q3_buffer.empty:
221     errors.append("q3_buffer_interactions.csv must remain empty while country-year buffers are unaudited proxies")
222 q3_buffer_status = pd.read_csv(RESULTS_DIR / "q3_buffer_data_status.csv")
223 required_buffers = {"fuel_price_regulation", "oil_import_dependency", "oil_intensity", "import_source_hhi"}
224 if set(q3_buffer_status.get("buffer", pd.Series(dtype=str))) != required_buffers or not q3_buffer_status["data_status"].eq("
225     FAIL_FOR_MAIN_UNAUDITED_PROXY").all():
226     errors.append("q3_buffer_data_status.csv must fail closed for all four unaudited buffer variables")
227
228 q3_panel = pd.read_csv(RESULTS_DIR / "q3_panel_irf.csv")
229 if "reference_country" not in q3_panel.columns or not q3_panel["reference_country"].astype(str).eq("CHN").all():
230     errors.append("q3_panel_irf.csv must use China as the explicit reference country")
231 if "response_type" not in q3_panel.columns or not q3_panel["response_type"].astype(str).str.contains("control_country_minus_china",
232     na=False).all():
233     errors.append("q3_panel_irf.csv must report control-country-minus-China relative responses")
234
235 q3_resilience = pd.read_csv(RESULTS_DIR / "q3_resilience_metrics.csv")
236 required_dimensions = {"fuel_price", "consumer_prices", "industrial_activity", "policy_counterfactual_macro"}
237 if q3_resilience.empty or not required_dimensions.issubset(set(q3_resilience.get("dimension", pd.Series(dtype=str))).dropna().unique())

```

```

)):
232     errors.append("q3_resilience_metrics.csv lacks required resilience dimensions")
233 if not q3_resilience.get("overall_china_resilience_judgement", pd.Series(dtype=str)).astype(str).eq("INCONCLUSIVE").all():
234     errors.append("Q3 overall resilience must fail closed while China fuel is proxy-only and joint country intervals are withheld")
235
236 policy = pd.read_csv(RESULTS_DIR / "q3_policy_counterfactual.csv")
237 april = policy.loc[policy["period"].eq("2026-04")]
238 if april.empty:
239     errors.append("q3_policy_counterfactual.csv lacks 2026-04 row")
240 else:
241     incremental = float(april["incremental_gasoline_gap_cny_t"].iloc[0])
242     cumulative = float(april["cumulative_gasoline_gap_cny_t"].iloc[0])
243     if abs(incremental - 380.0) > 1e-6 or abs(cumulative - 1425.0) > 1e-6:
244         errors.append("Q3 April policy gap no longer matches incremental 380 and cumulative 1425")
245 if "price_layer_status" not in policy.columns:
246     errors.append("q3_policy_counterfactual.csv lacks price_layer_status")
247 elif not policy["price_layer_status"].astype(str).str.contains("proxy", case=False, na=False).all():
248     errors.append("q3_policy_counterfactual.csv must identify the historical price layer as a proxy")
249 policy_macro = pd.read_csv(RESULTS_DIR / "q3_policy_macro_counterfactual.csv")
250 required_dynamic = {"fuel_return_gap", "bootstrap_reps", "bootstrap_block_length", "macro_counterfactual_gap_pctpt"}
251 if not required_dynamic.issubset(policy_macro.columns) or not pd.to_numeric(policy_macro["bootstrap_block_length"], errors="coerce").
252     eq(6).all():
253     errors.append("Q3 policy macro counterfactual lacks the dynamic joint-block interface")
254
255 policy_macro = pd.read_csv(RESULTS_DIR / "q3_policy_macro_counterfactual.csv")
256 macro_outcomes = set(policy_macro.get("outcome", pd.Series(dtype=str)).dropna().unique())
257 if not {"china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"}.issubset(macro_outcomes):
258     errors.append("q3_policy_macro_counterfactual.csv must include PPI/CPI/IAV propagated paths")
259 if not {"lower_95", "upper_95", "macro_counterfactual_gap_pctpt"}.issubset(policy_macro.columns):
260     errors.append("q3_policy_macro_counterfactual.csv lacks macro interval columns")
261
262 q3_kernel = pd.read_csv(RESULTS_DIR / "q3_policy_macro_kernel.csv")
263 q3_cov = pd.read_csv(RESULTS_DIR / "q3_policy_macro_covariance.csv")
264 kernel_outcomes = set(q3_kernel.get("outcome", pd.Series(dtype=str)).dropna().unique())
265 if not {"china_ppi_yoy_pct", "china_cpi_yoy_pct", "china_iav_yoy_pct"}.issubset(kernel_outcomes):
266     errors.append("q3_policy_macro_kernel.csv must expose PPI/CPI/IAV macro kernels for Q4 SAPR")
267 required_kernel_cols = {f"beta_fuel_lag{i}" for i in range(7)} | {"phi_outcome_lag1"}
268 if q3_kernel.empty or not required_kernel_cols.issubset(q3_kernel.columns):
269     errors.append("q3_policy_macro_kernel.csv lacks usable coefficient columns")
270 if q3_cov.empty or not {"outcome", "row_term", "column_term", "covariance"}.issubset(q3_cov.columns):
271     errors.append("q3_policy_macro_covariance.csv lacks covariance matrix columns")
272
273 q4_risk = pd.read_csv(RESULTS_DIR / "q4_price_tail_risk.csv")
274 q4_backtest = pd.read_csv(RESULTS_DIR / "q4_risk_backtest.csv")
275 if set(q4_risk.get("model", pd.Series(dtype=str)).dropna().unique()) != {"FHS_GJR_GARCH", "Gaussian_random_walk"}:
276     errors.append("q4_price_tail_risk.csv must include both FHS_GJR_GARCH and Gaussian_random_walk")
277 q4_horizons = sorted(pd.to_numeric(q4_risk.get("horizon_months", pd.Series(dtype=float)), errors="coerce").dropna().astype(int).
278     unique().tolist())
279 if q4_horizons != [1, 3, 6]:
280     errors.append("q4_price_tail_risk.csv must report 1/3/6 month horizons")
281 if q4_backtest.empty or not {"origins", "all_no_future_information", "mean_pinball_loss"}.issubset(q4_backtest.columns):
282     errors.append("q4_risk_backtest.csv lacks backtest integrity columns")
283 elif not bool_series(q4_backtest["all_no_future_information"]).all():
284     errors.append("Q4 tail-risk backtest uses future information")
285 elif pd.to_numeric(q4_backtest["origins"], errors="coerce").min() < 36:
286     errors.append("Q4 tail-risk backtest must use at least 36 monthly origins")
287 q4_paired = pd.read_csv(RESULTS_DIR / "q4_risk_backtest_paired.csv")
288 if q4_paired.empty or not {"interval_score_80", "interval_score_90", "mean_pinball_loss"}.issubset(set(q4_paired.get("metric", pd.
289     Series(dtype=str)))):
290     errors.append("q4_risk_backtest_paired.csv lacks paired block-bootstrap distributional scores")
291
292 q4_sapr_scenarios = pd.read_csv(RESULTS_DIR / "q4_sapr_scenarios.csv")
293 q4_sapr_grid = pd.read_csv(RESULTS_DIR / "q4_sapr_policy_grid.csv")
294 q4_sapr_optimal = pd.read_csv(RESULTS_DIR / "q4_sapr_optimal_rule.csv")
295 q4_sapr_comparison = pd.read_csv(RESULTS_DIR / "q4_sapr_strategy_comparison.csv")
296 q4_sapr_paths = pd.read_csv(RESULTS_DIR / "q4_sapr_macro_paths.csv")
297 q4_sapr_sensitivity = pd.read_csv(RESULTS_DIR / "q4_sapr_sensitivity.csv")
298 q4_sapr_summary = load_json(RESULTS_DIR / "q4_sapr_summary.json")
299
300 q4_dev = q4_sapr_scenarios.loc[q4_sapr_scenarios["sample_split"].eq("development")]
301 q4_holdout = q4_sapr_scenarios.loc[q4_sapr_scenarios["sample_split"].eq("holdout")]
302 if q4_dev.empty or q4_holdout.empty:
303     errors.append("q4_sapr_scenarios.csv must include development and holdout scenarios")
304 else:
305     if str(q4_dev["source_end"].max()) > "2021-12":
306         errors.append("Q4 SAPR development scenarios leak post-2021 data")

```

```

304     if str(q4_holdout["source_start"].min()) < "2022-01":
305         errors.append("Q4 SAPR holdout scenarios overlap the development period")
306     if int(bool_series(q4_sapr_scenarios.get("is_2026_anchor", pd.Series(dtype=bool))).sum()) == 0:
307         errors.append("q4_sapr_scenarios.csv lacks the 2026 anchor scenario")
308     if len(q4_sapr_grid) != 1771:
309         errors.append("q4_sapr_policy_grid.csv must contain exactly 1771 monotone rules")
310     if int(bool_series(q4_sapr_grid.get("is_knee", pd.Series(dtype=bool))).sum()) != 1:
311         errors.append("q4_sapr_policy_grid.csv must identify exactly one knee rule")
312     if "is_feasible" not in q4_sapr_grid.columns or not bool_series(q4_sapr_grid.loc[bool_series(q4_sapr_grid["is_knee"]), "is_feasible"]
313     ).all():
314         errors.append("Q4 SAPR knee rule must belong to the preregistered feasible set")
315     if q4_sapr_optimal.empty or len(q4_sapr_optimal) != 1:
316         errors.append("q4_sapr_optimal_rule.csv must contain exactly one optimal rule")
317     else:
318         opt = q4_sapr_optimal.iloc[0]
319         if not (float(opt["rho_normal"]) >= float(opt["rho_stress"]) >= float(opt["rho_extreme"])):
320             errors.append("Q4 SAPR optimal pass-through rates violate monotonicity")
321         required_constraints = {"max_gap_ratio", "max_terminal_gap_ratio", "max_recovery_terminal_gap_ratio", "
322         max_monthly_adjustment_ratio", "J3_avg_gap_month_burden"}
323         if not required_constraints.issubset(q4_sapr_optimal.columns):
324             errors.append("Q4 SAPR optimal rule lacks gap, recovery or adjustment feasibility metrics")
325         required_sapr_strategies = {"full_mechanism", "uniform_75_smoothing", "temporary_2026_approx", "SAPR_CVaR_knee"}
326         holdout_strategies = set(q4_sapr_comparison.loc[q4_sapr_comparison["sample_split"].eq("holdout"), "strategy"].dropna())
327         war_strategies = set(q4_sapr_comparison.loc[q4_sapr_comparison["sample_split"].eq("war_2026"), "strategy"].dropna())
328         if not required_sapr_strategies.issubset(holdout_strategies):
329             errors.append("q4_sapr_strategy_comparison.csv lacks required holdout strategies")
330         if not required_sapr_strategies.issubset(war_strategies):
331             errors.append("q4_sapr_strategy_comparison.csv lacks required 2026-war strategies")
332         interval_cols = [
333             "J1_macro_loss_lower_95",
334             "J1_macro_loss_upper_95",
335             "J2_cvar95_macro_loss_lower_95",
336             "J2_cvar95_macro_loss_upper_95",
337         ]
338         if not set(interval_cols).issubset(q4_sapr_comparison.columns):
339             errors.append("q4_sapr_strategy_comparison.csv lacks uncertainty interval columns")
340         elif not np.isfinite(q4_sapr_comparison[interval_cols].apply(pd.to_numeric, errors="coerce").to_numpy()).all():
341             errors.append("Q4 SAPR strategy intervals contain non-finite values")
342         if q4_sapr_paths.empty or not {"strategy", "sample_split", "month_index", "cumulative_deferred_gap_cny_t"}.issubset(q4_sapr_paths.
343         columns):
344             errors.append("q4_sapr_macro_paths.csv lacks required macro-path columns")
345         if len(q4_sapr_sensitivity) < 6:
346             errors.append("q4_sapr_sensitivity.csv must include threshold, block-length, CPI-weight and IAV-weight variants")
347         q4_holdout_summary = load_json(RESULTS_DIR / "q4_sapr_holdout_validation_summary.json")
348         if int(q4_holdout_summary.get("block_length", 0)) < 6 or int(q4_holdout_summary.get("supported_improvement_count", 0)) < 1:
349             errors.append("Q4 SAPR holdout validation must use at least six-month blocks and support at least one paired improvement")
350         if q4_sapr_summary.get("execution_status") != "PASS":
351             errors.append("q4_sapr_summary.json execution_status must be PASS")
352         if q4_sapr_summary.get("evidence_status") not in {"SUPPORTED", "PARTIAL", "NOT_SUPPORTED"}:
353             errors.append("q4_sapr_summary.json has invalid evidence_status")
354         if int(q4_sapr_summary.get("valid_parameter_draw_count", 0)) != 2000:
355             errors.append("Q4 SAPR must retain exactly 2000 valid parameter draws")
356         if float(q4_sapr_summary.get("valid_parameter_draw_rate", 0.0)) < 0.95:
357             errors.append("Q4 SAPR valid parameter draw rate is below 95%")
358         identity = q4_sapr_summary.get("identity_checks", {})
359         if abs(float(identity.get("official_2026_03_gap", 0.0)) - 1045.0) > 1e-6:
360             errors.append("Q4 SAPR official 2026-03 policy gap must equal 1045 CNY/t")
361         if abs(float(identity.get("official_2026_04_gap", 0.0)) - 380.0) > 1e-6:
362             errors.append("Q4 SAPR official 2026-04 incremental policy gap must equal 380 CNY/t")
363         if abs(float(identity.get("official_2026_04_cumulative_gap", 0.0)) - 1425.0) > 1e-6:
364             errors.append("Q4 SAPR official 2026-04 cumulative policy gap must equal 1425 CNY/t")
365
366 def main() -> int:
367     parser = argparse.ArgumentParser()
368     parser.add_argument("--require-final", action="store_true", help="Return nonzero unless the risk gate allows paper finalization.")
369     parser.add_argument("--strict-environment", action="store_true", help="Treat replay-environment differences as errors.")
370     args = parser.parse_args()
371
372     errors: list[str] = []
373     environment_messages: list[str] = []
374     final_path = RESULTS_DIR / "final_numbers.json"
375     frozen_path = RESULTS_DIR / "frozen_numbers.json"
376     risk_path = RESULTS_DIR / "risk_probe_summary.json"
377     manifest_path = RESULTS_DIR / "reproducibility_manifest.json"

```



```

377 final_numbers = load_json(final_path)
378 frozen = load_json(frozen_path)
379 risk = load_json(risk_path)
380 manifest = load_json(manifest_path)
381
382 require_keys(frozen, STANDARD_FROZEN_REQUIRED, "frozen_numbers.json", errors)
383 require_keys(manifest, MANIFEST_REQUIRED, "reproducibility_manifest.json", errors)
384 require_keys(risk, RISK_REQUIRED, "risk_probe_summary.json", errors)
385 validate_schema(frozen, "frozen_numbers.schema.json", "frozen_numbers.json", errors)
386 validate_schema(manifest, "reproducibility_manifest.schema.json", "reproducibility_manifest.json", errors)
387 validate_schema(risk, "risk_probe_summary.schema.json", "risk_probe_summary.json", errors)
388
389 expected_source_hash = sha256_json(final_numbers)
390 if frozen.get("schema_version") != 1:
391     errors.append(f"frozen_numbers.json schema_version must be 1, got {frozen.get('schema_version')!r}")
392 if frozen.get("source_file") != "results/final_numbers.json":
393     errors.append("frozen_numbers.json source_file is not results/final_numbers.json")
394 if frozen.get("source_sha256") != expected_source_hash:
395     errors.append("frozen_numbers.json source_sha256 does not match final_numbers.json")
396 if frozen.get("values") != final_numbers:
397     errors.append("frozen_numbers.json values do not match final_numbers.json")
398
399 manifest_source = {key: manifest[key] for key in MANIFEST_REQUIRED if key != "manifest_hash" and key in manifest}
400 if manifest.get("manifest_hash") != sha256_json(manifest_source):
401     errors.append("manifest_hash does not match canonical reproducibility manifest")
402 if manifest.get("risk_gate_hash") != sha256_file(risk_path):
403     errors.append("risk_gate_hash does not match risk_probe_summary.json")
404 if manifest.get("final_numbers_source_sha256") != expected_source_hash:
405     errors.append("manifest final_numbers_source_sha256 does not match final_numbers.json")
406
407 verify_hashes(RESULTS_DIR, manifest.get("core_result_hashes", {}), "result", errors)
408 verify_hashes(PROCESSED_DIR, manifest.get("processed_input_hashes", {}), "processed input", errors)
409 verify_hashes(REPO_ROOT, manifest.get("code_hashes", {}), "code", errors)
410 verify_environment(manifest, errors if args.strict_environment else environment_messages)
411 verify_source_commit_lineage(manifest, errors)
412 verify_key_result_shapes(errors)
413
414 probe_status = {probe.get("probe_id"): probe.get("status") for probe in risk.get("probes", [])}
415 required_probe_ids = {
416     "q4_method_risk_probe_gate",
417     "q4_probability_calibration_gate",
418     "q4_evidence_layer_guardrail",
419     "q4_sapr_scenario_no_leakage_gate",
420     "q4_sapr_macro_kernel_gate",
421     "q4_sapr_policy_identity_gate",
422     "q4_sapr_pareto_selection_gate",
423     "q4_sapr_holdout_validation_gate",
424     "q4_sapr_uncertainty_gate",
425     "q4_sapr_claim_strength_gate",
426 }
427 missing_probe_ids = sorted(required_probe_ids - set(probe_status))
428 if missing_probe_ids:
429     errors.append(f"risk_probe_summary.json lacks Q4 probe ids: {missing_probe_ids}")
430 failed_probe_ids = sorted(pid for pid in required_probe_ids if probe_status.get(pid) != "PASS")
431 if failed_probe_ids:
432     errors.append(f"Q4 probe ids are not PASS: {failed_probe_ids}")
433
434 if risk.get("paper_finalize_allowed") != (risk.get("overall_status") == "PASS"):
435     errors.append("risk paper_finalize_allowed is inconsistent with overall_status")
436 if manifest.get("paper_finalize_allowed") != risk.get("paper_finalize_allowed"):
437     errors.append("manifest paper_finalize_allowed differs from risk gate")
438 if manifest.get("overall_status") != risk.get("overall_status"):
439     errors.append("manifest overall_status differs from risk gate")
440
441 if errors:
442     print(json.dumps({"status": "FAIL", "errors": errors, "environment_warnings": environment_messages}, ensure_ascii=False, indent
443 =2))
444     return 1
445 if args.require_final and not risk.get("paper_finalize_allowed"):
446     print(
447         json.dumps(
448             {
449                 "status": risk.get("overall_status"),
450                 "paper_finalize_allowed": False,
451                 "blocking_probe_ids": risk.get("blocking_probe_ids", []),
452                 "environment_warnings": environment_messages,

```

```

452         },
453         ensure_ascii=False,
454         indent=2,
455     )
456 )
457 return 2
458 print(
459     json.dumps(
460         {
461             "status": risk.get("overall_status"),
462             "paper_finalize_allowed": risk.get("paper_finalize_allowed"),
463             "environment_warnings": environment_messages,
464         },
465         ensure_ascii=False,
466         indent=2,
467     )
468 )
469 return 0
470
471 if __name__ == "__main__":
472     raise SystemExit(main())

```

Listing 14: verify_freeze.py

Listing 15: utils/__init__.py

附录 D 证据解释边界

- *ARBaselineGap*是战前AR规律的描述性差额，不是战争净因果贡献。
- Q2联合区间跨零时，不将点估计写成稳健总体增长损失；响应曲线面积不是累计损失。
- Q3跨国韧性总体为结论不充分；中国燃油序列是调价指数代理。
- Q3政策结果是局部动态代理情景；未经审计年度缓冲代理没有定量结论。
- Q4尾部概率是固定信息集下的条件概率；结构压力和政策反事实不可相加。
- SAPR只在注册规则族内最优；平均缺口月负担不等于财政支出或完整福利损失。